

mitsubishi

Mitsubishi Industrial Robot

CR1/CR2/CR4/CR7/CR8 Controller

INSTRUCTION MANUAL

Detailed explanations of functions and operations

MELFA

BFP-A5992-F

Caution

Users of the robot given as a "Object Model" in "Table 1: List of origin position joint angles" must observe the details below.



Warning

Do not release the brakes from an external source and forcibly move the robot arm at a high speed.

If the operation is carried out, a warning error indicating positional deviation (error No.: L1820) may occur. If it is confirmed that the position has deviated after carrying out "1. Operation to confirm positional deviation of origin position", the origin data has been lost.

In this case, reset the origin with the ABS method. Refer to section "5.5.1 ABS method" in the separate "Instruction Manual/Robot Arm Setup to Maintenance" for the operation methods.

If operation is carried out without resetting the origin, interference with peripheral devices or unforeseen operation could occur due to the loss of origin data.

1. Operation to confirm positional deviation of origin position
 - (1)Set each axis of the robot to the ABS mark using the teaching box's joint jog operation.
 - (2)Confirm that the joint angle displayed on the teaching box screen matches the value corresponding to the object model given in Table 1. If the values do not match, reset the origin with the ABS method.

Table 1: List of origin position joint angles (Position aligned with ABS mark arrow)

Object Model	Joint angle					
	J1	J2	J3	J4	J5	J6
RH-1000GHDC-SA	0degree	0degree	150mm	0degree		
RH-1000GJDC-SA	0degree	0degree	150mm	0degree	0degree	
RH-1000GHLC-SA	0degree	0degree	0degree	0degree		
RH-1000GJLC-SA	0degree	0degree	0degree	0degree	0degree	
RH-1500GJC-SA/SB	138.7 degree	140 degree	0degree	180 degree	0degree	
RH-1500GC-SA**/SA5* -SB**/SB5*	138.7 degree	140 degree	0degree	180 degree	0degree	0degree
RC-1000GHWDC-SA	0degree	0degree	0degree	180 degree		
RC-1000GHWLC-SA	0degree	0degree	0degree	180 degree		

To the User of the Personal Computer Support Software

Thank you very much for purchasing a Mitsubishi Electric industrial robot. This document provides the version update information for the optional Personal Computer Support Software (Model name 3A-01C-WINJ or 3A-01F-WINJ). Please read the information provided, and contact your Mitsubishi Electric dealer if you are using any of the products mentioned in the following.

Products Requiring Version Update

If you are using the simulation function provided in the standard Personal Computer Support Software for Mitsubishi Electric industrial robots RH-5AH, RH-10AH and RH-15AH, version B10 or later is required. If the version you are using is B7 or B9, please contact your Mitsubishi Electric dealer for version update.

Note) It is not necessary to update the version of the mini Personal Computer Support Software since it does not support the simulation function.

Safety Precautions

Always read the following precautions and the separate "Safety Manual" before starting use of the robot to learn the required measures to be taken.

CAUTION

All teaching work must be carried out by an operator who has received special training. (This also applies to maintenance work with the power source turned ON.)
→ Enforcement of safety training

CAUTION

For teaching work, prepare a work plan related to the methods and procedures of operating the robot, and to the measures to be taken when an error occurs or when restarting. Carry out work following this plan. (This also applies to maintenance work with the power source turned ON.)
→ Preparation of work plan

WARNING

Prepare a device that allows operation to be stopped immediately during teaching work. (This also applies to maintenance work with the power source turned ON.)
→ Setting of emergency stop switch

CAUTION

During teaching work, place a sign indicating that teaching work is in progress on the start switch, etc. (This also applies to maintenance work with the power source turned ON.)
→ Indication of teaching work in progress

WARNING

Provide a fence or enclosure during operation to prevent contact of the operator and robot.
→ Installation of safety fence

CAUTION

Establish a set signaling method to the related operators for starting work, and follow this method.
→ Signaling of operation start

CAUTION

As a principle turn the power OFF during maintenance work. Place a sign indicating that maintenance work is in progress on the start switch, etc.
→ Indication of maintenance work in progress

CAUTION

Before starting work, inspect the robot, emergency stop switch and other related devices, etc., and confirm that there are no errors.
→ Inspection before starting work

The points of the precautions given in the separate "Safety Manual" are given below.
Refer to the actual "Safety Manual" for details.

- | | |
|---|---|
|  CAUTION | Use the robot within the environment given in the specifications. Failure to do so could lead to a drop or reliability or faults. (Temperature, humidity, atmosphere, noise environment, etc.) |
|  CAUTION | Transport the robot with the designated transportation posture. Transporting the robot in a non-designated posture could lead to personal injuries or faults from dropping. |
|  CAUTION | Always use the robot installed on a secure table. Use in an instable posture could lead to positional deviation and vibration. |
|  CAUTION | Wire the cable as far away from noise sources as possible. If placed near a noise source, positional deviation or malfunction could occur. |
|  CAUTION | Do not apply excessive force on the connector or excessively bend the cable. Failure to observe this could lead to contact defects or wire breakage. |
|  CAUTION | Make sure that the workpiece weight, including the hand, does not exceed the rated load or tolerable torque. Exceeding these values could lead to alarms or faults. |
|  WARNING | Securely install the hand and tool, and securely grasp the workpiece. Failure to observe this could lead to personal injuries or damage if the object comes off or flies off during operation. |
|  WARNING | Securely ground the robot and controller. Failure to observe this could lead to malfunctioning by noise or to electric shock accidents. |
|  CAUTION | Indicate the operation state during robot operation. Failure to indicate the state could lead to operators approaching the robot or to incorrect operation. |
|  WARNING | When carrying out teaching work in the robot's movement range, always secure the priority right for the robot control. Failure to observe this could lead to personal injuries or damage if the robot is started with external commands. |
|  CAUTION | Keep the jog speed as low as possible, and always watch the robot. Failure to do so could lead to interference with the workpiece or peripheral devices. |
|  CAUTION | After editing the program, always confirm the operation with step operation before starting automatic operation. Failure to do so could lead to interference with peripheral devices because of programming mistakes, etc. |
|  CAUTION | Make sure that if the safety fence entrance door is opened during automatic operation, the door is locked or that the robot will automatically stop. Failure to do so could lead to personal injuries. |
|  CAUTION | Never carry out modifications based on personal judgments, or use non-designated maintenance parts.
Failure to observe this could lead to faults or failures. |
|  WARNING | When the robot arm has to be moved by hand from an external area, do not place hands or fingers in the openings. Failure to observe this could lead to hands or fingers catching depending on the posture. |
|  CAUTION | Do not stop the robot or apply emergency stop by turning the robot controller's main power OFF.
If the robot controller main power is turned OFF during automatic operation, the robot accuracy could be adversely affected. |
|  CAUTION | Do not turn off the main power to the robot controller while rewriting the internal information of the robot controller such as the program or parameters.
If the main power to the robot controller is turned off while in automatic operation or rewriting the program or parameters, the internal information of the robot controller may be damaged. |

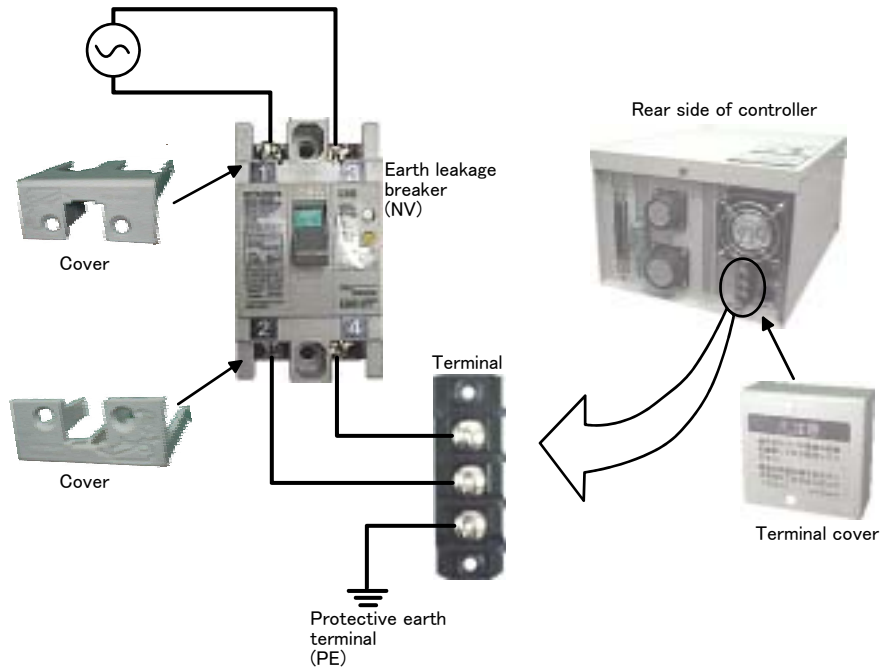
C.Precautions for the basic configuration are shown below.(When CR1-571 is used for the controller.)

⚠ CAUTION

Provide an earth leakage breaker that packed together on the primary power supply of the controller as protection against electric leakage. Confirm the setting connector of the input power supply voltage of the controller, if the type which more than one power supply voltage can be used. Then connect the power supply.

Failure to do so could lead to electric shock accidents.

Power supply *RV-1A/2AJ series and RP-1AH/3AH/5AH series: Single phase 90-132VAC, 180-253VAC.
*Except the above: Single phase 180-253VAC.



⚠ WARNING

For using RH-5AH/10AH/15AH.

While pressing the brake releasing switch on the robot arm, beware of the arm which may drop with its own weight.

Dropping of the hand could lead to a collision with the peripheral equipment or catch the hands or fingers.

Revision history

Date	Specifications No.	Details of revisions
1999-06	BFP-A5992Z-*	First print
1999-09-20	BFP-A5992Z-A	Error in writing correction. The function of RH-1000 was considered.
1999-11-09	BFP-A5992	Error in writing correction.
2000-04-06	BFP-A5992-A	Attention in the power supply connection was added.(CR1 Controller)
2000-06-09	BFP-A5992-B	Parameter CNT was added. Emergency stop input of CR1 controller was added. JRC command was added. The power supply voltage of CR1 controller was corrected.
2000-07-12	BFP-A5992-C	Change title. Error in writing correction.
2001-06-05	BFP-A5992-Da	Major revision. Function list, publication of Q & A, description of system variables, as well as language and similar notation of system functions, and supplementation of various parameter functions.
2001-11-30	BFP-A5992-D	Formal style.
2001-12-12	BFP-A5992-E	Error in writing correction.
2002-11-15	BFP-A5992-F	The explanation and supplementary explanation of the new function corresponding to software version H7 edition were added. The notation of the input-and-output circuit terminal was corrected. Explanation of optimum acceleration/deceleration setting was added. Error in writing correction.

■ Introduction

Thank you for purchasing the Mitsubishi industrial robot.

This instruction manual explains the functions and operation methods of the controller (CR1/CR2/CR4/CR7) and teaching pendant (R28TB), and the functions and specifications of the MELFA-BASIC IV programming language.

Always read through this manual before starting use to ensure correct usage of the robot.

Note that this document is prepared for the following software versions.

Controller : Version H7 or later

T/B : Version A3 or later

- No part of this manual may be reproduced by any means or in any form, without prior consent from Mitsubishi.
- The details of this manual are subject to change without notice.
- An effort has been made to make full descriptions in this manual. However, if any discrepancies or unclear points are found, please contact your dealer.
- The information contained in this document has been written to be accurate as much as possible. Please interpret that items not described in this document "cannot be performed."
Please contact your nearest dealer if you find any doubtful, wrong or skipped point.

Copyright(C) 1999 MITSUBISHI ELECTRIC CORPORATION

Contents

	Page
1 Before starting use	1-1
1.1 Using the instruction manuals	1-1
1.1.1 The details of each instruction manuals	1-1
1.1.2 Symbols used in instruction manual	1-2
1.2 Safety Precautions	1-3
1.2.1 Precautions given in the separate Safety Manual	1-4
2 Explanation of functions	2-5
2.1 Operation panel (O/P) functions	2-5
2.2 Teaching pendant (T/B) functions	2-7
2.2.1 Operation rights	2-10
2.3 Functions Related to Movement and Control	2-11
3 Explanation of operation methods	3-13
3.1 Operation of the teaching pendant menu screens	3-13
(1) Screen tree	3-13
(2) Selecting a menu	3-14
3.2 Jog Feed (Overview)	3-15
3.2.1 Types of jog feed	3-15
3.2.2 Speed of jog feed	3-16
3.2.3 JOINT jog	3-16
3.2.4 TOOL jog	3-17
3.2.5 XYZ jog	3-17
3.2.6 3-axis XYZ jog	3-18
3.2.7 CYLINDER jog	3-18
3.3 Opening/Closing the Hands	3-19
3.4 Aligning the Hand	3-20
3.5 Programming	3-21
3.5.1 Creating a program	3-21
(1) Opening the program edit screen	3-21
(2) Creating a program	3-22
(3) Completion of program creation and saving programs	3-23
(4) Correcting a program	3-24
(5) Registering the current position data	3-26
(6) Confirming the position data (Position jump)	3-27
(7) Correcting the current position data	3-28
(8) Correcting the MDI (Manual Data Input)	3-29
(9) Deleting position data	3-30
(10) Display on the position edit screen	3-31
(11) Saving the program	3-31
3.6 Debugging	3-32
(1) Step feed	3-32
(2) Step return	3-33
(3) Step feed in another slot	3-33
(4) Step jump	3-34
3.7 Automatic operation	3-35
(1) Setting the operation speed	3-35
(2) Selecting the program No.	3-35
(3) Starting automatic operation	3-36
(4) Stopping	3-36
(5) Resuming automatic operation from stopped state	3-37
(6) Resetting the program	3-38
3.8 Turning the servo ON/OFF	3-39
3.9 Error reset operation	3-41
3.10 Operation to Temporarily Reset an Error that Cannot Be Canceled	3-41

	Page
3.11 Operating the program control screen	3-42
(1) Program list display	3-42
(2) Program protection function	3-43
(3) Copying programs	3-44
(4) Changing the program name (Renaming)	3-45
(5) Deleting a program	3-46
3.12 Operating the monitor screen	3-47
(1) Input signal monitor	3-47
(2) Output signal monitor	3-48
(3) Variable monitor	3-49
(4) Error history	3-50
3.13 Operation of maintenance screen	3-51
(1) Setting the parameters	3-51
(2) Initializing the program	3-52
(3) Initializing the battery consumption time	3-53
(4) Releasing the brakes	3-54
(5) Setting the origin	3-55
(6) Displaying the clock data for maintenance	3-56
3.14 Operation of the setting screen	3-57
(1) Setting the time	3-57
4 MELFA-BASIC IV	4-58
4.1 MELFA-BASIC IV functions	4-58
4.1.1 Robot operation control	4-59
(1) Joint interpolation movement	4-59
(2) Linear interpolation movement	4-60
(3) Circular interpolation movement	4-61
(4) Continuous movement	4-63
(5) Acceleration/deceleration time and speed control	4-64
(6) Confirming that the target position is reached	4-66
(7) High path accuracy control	4-67
(8) Hand and tool control	4-68
4.1.2 Pallet operation	4-69
4.1.3 Program control	4-71
(1) Unconditional branching, conditional branching, waiting	4-71
(2) Repetition	4-73
(3) Interrupt	4-74
(4) Subroutine	4-75
(5) Timer	4-76
(6) Stopping	4-77
4.1.4 Inputting and outputting external signals	4-78
(1) Input signals	4-78
(2) Output signals	4-78
4.1.5 Communication	4-79
4.1.6 Expressions and operations	4-80
(1) List of operator	4-80
(2) Relative calculation of position data (multiplication)	4-81
(3) Relative calculation of position data (Addition)	4-81
4.1.7 Appended statement	4-82
4.2 Multitask function	4-83
4.2.1 What is multitasking?	4-83
4.2.2 Executing a multitask	4-84
4.2.3 Operation state of each slot	4-84
4.2.4 Precautions for creating multitask program	4-86
(1) Relationship between number of tasks and processing time	4-86
(2) Specification of the maximum number of programs executed concurrently	4-86
(3) How to pass data between programs via external variables	4-86

Contents

	Page
(4) Confirmation of operating status of programs via robot status variables	4-86
(5) The program that operates the robot is basically executed in slot 1.	4-86
(6) How to perform the initialization processing via constantly executed programs	4-86
4.2.5 Precautions for using a multitask program	4-87
(1) Starting the multitask	4-87
(2) Display of operation status	4-87
4.2.6 Example of using multitask	4-88
(1) Robot work details.	4-88
(2) Procedures to multitask execution	4-89
4.3 Detailed specifications of MELFA-BASIC IV	4-90
(1) Program name	4-90
(2) Command statement	4-90
(3) Variable	4-90
4.3.1 Statement	4-91
4.3.2 Appended statement	4-91
4.3.3 Line	4-91
4.3.4 Line No.	4-91
4.3.5 Label	4-91
4.3.6 Types of characters that can be used in program	4-92
4.3.7 Characters having special meanings	4-93
(1) Uppercase and lowercase identification	4-93
(2) Underscore (_)	4-93
(3) Apostrophe (')	4-93
(4) Asterisk (*)	4-93
(5) Comma (,)	4-93
(6) Period (.)	4-93
(7) Space	4-93
4.3.8 Data type	4-94
4.3.9 Constants	4-94
4.3.10 Numeric value constants	4-94
(1) Decimal number	4-94
(2) Hexadecimal number	4-94
(3) Binary number	4-94
(4) Types of constant	4-94
4.3.11 Character string constants	4-94
4.3.12 Position constants	4-95
(1) Coordinate, posture and additional axis data types and meanings	4-95
(2) Meaning of structure flag data type and meanings	4-95
4.3.13 Joint constants	4-96
(1) Axis data format and meanings	4-96
4.3.14 Angle value	4-96
4.3.15 Variables	4-97
4.3.16 Arithmetic variables	4-98
4.3.17 Character string variables	4-98
4.3.18 Position variables	4-98
4.3.19 Joint variables	4-99
4.3.20 Input/output variables	4-99
4.3.21 Array variables	4-99
4.3.22 External variables	4-100
4.3.23 Program external variables	4-100
4.3.24 User-defined external variables	4-101
4.3.25 Creating User Base Programs	4-101
4.3.26 Robot status variables	4-102
4.4 Logic numbers	4-105
4.5 Functions	4-105
(1) User-defined functions	4-105
(2) Built-in functions	4-105

	Page
4.6 List of Instructions	4-107
4.7 Operators	4-109
4.8 Priority level of operations	4-110
4.9 Depth of program's control structure	4-110
4.10 Reserved words	4-110
4.11 Detailed explanation of command words	4-111
4.11.1 How to read the described items	4-111
4.11.2 Explanation of each command word	4-111
ACCEL	4-112
ACT	4-113
BASE	4-114
CALLP	4-115
CHRSRCH	4-117
CLOSE	4-118
CLR	4-119
CMP JNT	4-120
CMP POS	4-122
CMP TOOL	4-124
CMP OFF	4-126
CMPG	4-127
CNT	4-128
COM ON/COM OFF/COM STOP	4-131
DEF ACT	4-132
DEF ARCH	4-134
DEF FN	4-135
DEF PLT	4-136
DEF INTE/DEF FLOAT/DEF DOUBLE	4-137
DEF IO	4-138
DEF JNT	4-140
DEF POS	4-141
DEF CHAR	4-142
DIM	4-143
DLY	4-144
ERROR	4-145
END	4-146
FINE	4-147
FOR – NEXT	4-148
FPRM	4-149
GETM	4-150
GOSUB	4-151
GOTO	4-152
HLT	4-153
HOPEN / HCLOSE	4-154
IF...THEN...ELSE...ENDIF	4-155
INPUT	4-157
JOVRD	4-158
JRC	4-159
LOADSET	4-161
MOV	4-162
MVA	4-163
MVC	4-164
MVR	4-165
MVR2	4-166
MVR3	4-167
MVS	4-168
OADL	4-170
ON COM GOSUB	4-171

Contents

	Page
ON... GOSUB	4-172
ON...GOTO	4-173
OPEN.....	4-174
OVRD	4-175
PLT	4-176
PREC (Precision)	4-177
PRINT.....	4-178
PRIORITY	4-179
RELM.....	4-180
REM	4-181
RESET ERR.....	4-182
RETURN.....	4-183
SELECT CASE	4-184
SERVO.....	4-185
SKIP	4-186
SPD	4-187
SPDOPT	4-188
TOOL	4-190
TORQ	4-191
WAIT	4-192
WHILE-WEND	4-193
WTH	4-194
WTHIF	4-195
XCLR.....	4-196
XLOAD.....	4-197
XRST	4-198
XRUN.....	4-199
XSTP	4-200
Substitute.....	4-201
(Label)	4-202
4.12 Detailed explanation of Robot Status Variable	4-203
4.12.1 How to Read Described items	4-203
4.12.2 Explanation of Each Robot Status Variable	4-203
C_DATE	4-204
C_MAKER.....	4-204
C_MECHA	4-205
C_PRG.....	4-205
C_TIME.....	4-206
C_USER	4-206
J_CURR	4-207
J_ECURRE	4-207
J_FBC.....	4-208
J_ORIGIN	4-208
M_ACL/M_DACL/M_NACL/M_NDAACL/M_ACLSTS.....	4-209
M_BTME	4-210
M_CMPDST	4-210
M_CMPLMT	4-211
M_CSTP	4-212
M_CYS.....	4-212
M_DIN/M_DOUT	4-213
M_ERR/M_ERRLVL/M_ERRNO	4-214
M_EXP	4-214
M_G	4-215
M_HNDCQ.....	4-215
M_IN/M_INB/M_INW.....	4-216
M_JOVRD/M_NJOVRD/M_OPOVRD/M_OVRD/M_NOVRD	4-217
M_LINE	4-218
M_MODE.....	4-218

	Page
M_ON/M_OFF	4-219
M_OPEN	4-220
M_OUT/M_OUTB/M_OUTW	4-221
M_PI	4-221
M_PSA	4-222
M_RATIO	4-222
M_RDST	4-223
M_RUN	4-223
M_SKIPCQ	4-224
M_SPD/M_NSPD/M_RSPD	4-225
M_SVO	4-225
M_TIMER	4-226
M_UAR	4-226
M_WAI	4-227
P_BASE/P_NBASE	4-227
P_CURR	4-228
P_FBC	4-229
P_SAFE	4-229
P_TOOL/P_NTOOL	4-230
P_ZERO	4-230
4.13 Detailed Explanation of Functions	4-231
4.13.1 How to Read Described items	4-231
4.13.2 Explanation of Each Function	4-231
ABS	4-232
ALIGN	4-233
ASC	4-234
ATN/ATN2	4-234
BIN\$	4-235
CALARC	4-236
CHRSRCH	4-237
CHR\$	4-238
CINT	4-238
CKSUM	4-239
COS	4-239
CVI	4-240
CVS	4-240
CVD	4-241
DEG	4-241
DIST	4-242
EXP	4-242
FIX	4-243
FRAM	4-244
HEX\$	4-245
INT	4-245
INV	4-246
JTOP	4-246
LEFT\$	4-247
LEN	4-247
LN	4-248
LOG	4-248
MAX	4-249
MID\$	4-249
MIN	4-250
MIRROR\$	4-250
MKI\$	4-251
MKS\$	4-251
MKD\$	4-252
POSCQ	4-253

Contents

	Page
POSMID	4-253
PTOJ	4-254
RAD	4-254
RDFL 1	4-255
RDFL 2	4-256
RND	4-257
RIGHT\$	4-257
SETFL 1	4-258
SETFL 2	4-259
SGN	4-260
SIN	4-260
SQR	4-261
STRPOS	4-261
STR\$	4-262
TAN	4-263
VAL	4-263
ZONE	4-264
ZONE 2	4-265
 5 Functions set with parameters	 5-266
5.1 Standard Tool Coordinates	5-275
5.2 About Standard Base Coordinates	5-278
5.3 About user-defined area	5-279
5.4 Free plane limit	5-280
5.5 Automatic return setting after jog feed at pause	5-281
5.6 Automatic execution of program at power up	5-283
5.7 About the hand type	5-284
5.8 About default hand status	5-285
5.9 About the output signal reset pattern	5-286
5.10 About the communication setting	5-288
5.11 About the optimum acceleration/deceleration setting	5-291
5.12 About the singular point adjacent alarm	5-295
5.13 ROM Operation Function	5-296
 6 External input/output functions	 6-304
6.1 Types	6-304
6.2 Connection method	6-305
6.3 Dedicated input/output	6-308
6.4 Enable/disable status of signals	6-313
6.5 External signal timing chart	6-314
6.5.1 Individual timing chart of each signal	6-314
6.5.2 Timing chart example	6-321
(1) External signal operation timing chart (Part 1)	6-321
(2) External signal operation timing chart (Part 2)	6-322
(3) Example of external operation timing chart (Part 3)	6-323
(4) Example of external operation timing chart (Part 4)	6-324
6.6 Emergency stop input	6-325
6.6.1 Robot Behavior upon Emergency Stop Input	6-325
 7 Q & A	 7-326
7.1 Movement	7-326
7.2 Program	7-328
7.3 Operation	7-329
7.4 External input/output signal	7-330

	Page
7.5 Parameter	7-331
8 Appendix	8-332
8.1 Reference Material	8-332
8.1.1 About sink/source type of the standard external input and output	8-332
(1) Electrical specifications of input/output circuit	8-332
(2) Connection example	8-333
(3) Connector pin assignment	8-334

1 Before starting use

This chapter explains the details and usage methods of the instruction manuals, the basic terminology and the safety precautions.

1.1 Using the instruction manuals

1.1.1 The details of each instruction manuals

The contents and purposes of the documents enclosed with this product are shown below. Use these documents according to the application.

For special specifications, a separate instruction manual describing the special section may be enclosed.

Safety Manual	Explains the common precautions and safety measures to be taken for robot handling, system design and manufacture to ensure safety of the operators involved with the robot.
Standard Specifications	Explains the product's standard specifications, factory-set special specifications, option configuration and maintenance parts, etc. Precautions for safety and technology, when incorporating the robot, are also explained.
Robot Arm Setup & Maintenance	Explains the procedures required to operate the robot arm (unpacking, transportation, installation, confirmation of operation), and the maintenance and inspection procedures.
Controller Setup, Basic Operation and Maintenance	Explains the procedures required to operate the controller (unpacking, transportation, installation, confirmation of operation), basic operation from creating the program to automatic operation, and the maintenance and inspection procedures.
Detailed Explanation of Functions and Operations	Explains details on the functions and operations such as each function and operation, commands used in the program, connection with the external input/output device, and parameters, etc.
Explanations of MOVEMASTER COMMANDS	Explains details on the MOVEMASTER commands used in the program. (For RV-1A/2AJ and RV-2A/3AJ series)
Troubleshooting	Explains the causes and remedies to be taken when an error occurs. Explanations are given for each error No.

1.1.2 Symbols used in instruction manual

The symbols and expressions shown in [Table 1-1](#) are used throughout this Instruction Manual. Learn the meaning of these symbols before reading this instruction manual.

Table 1-1 : Symbols in instruction manual

Symbol	Meaning
 DANGER	Precaution indicating cases where there is a risk of operator fatality or serious injury if handling is mistaken. Always observe these precautions to safely use the robot.
 WARNING	Precaution indicating cases where the operator could be subject to fatalities or serious injuries if handling is mistaken. Always observe these precautions to safely use the robot.
 CAUTION	Precaution indicating cases where operator could be subject to injury or physical damage could occur if handling is mistaken. Always observe these precautions to safely use the robot.
[JOINT]	If a word is enclosed in brackets or a box in the text, this refers to a key on the teaching pendant.
[+ / FORWD] + [+ X] (A) (B)	This indicates to press the (B) key while holding down the (A) key. In this example, the [+ / Forward] key is pressed while holding down the [+X / +Y] key.
[STEP / MOVE] + ([COND] → [RPL ↓]) (A) (B) (C)	This indicates to hold down the (A) key, press and release the (B) key, and then press the (C) key. In this example, the [Step/Move] key is held down, the [Condition] key is pressed and released, and the [Replace ↓] key is pressed.
T / B	This indicates the teaching pendant.

1.2 Safety Precautions

Always read the following precautions and the separate "Safety Manual" before starting use of the robot to learn the required measures to be taken.

CAUTION

All teaching work must be carried out by an operator who has received special training. (This also applies to maintenance work with the power source turned ON.)
→ Enforcement of safety training

CAUTION

For teaching work, prepare a work plan related to the methods and procedures of operating the robot, and to the measures to be taken when an error occurs or when restarting. Carry out work following this plan. (This also applies to maintenance work with the power source turned ON.)
→ Preparation of work plan

WARNING

Prepare a device that allows operation to be stopped immediately during teaching work. (This also applies to maintenance work with the power source turned ON.)
→ Setting of emergency stop switch

CAUTION

During teaching work, place a sign indicating that teaching work is in progress on the start switch, etc. (This also applies to maintenance work with the power source turned ON.)
→ Indication of teaching work in progress

DANGER

Provide a fence or enclosure during operation to prevent contact of the operator and robot.
→ Installation of safety fence

CAUTION

Establish a set signaling method to the related operators for starting work, and follow this method.
→ Signaling of operation start

CAUTION

As a principle turn the power OFF during maintenance work. Place a sign indicating that maintenance work is in progress on the start switch, etc.
→ Indication of maintenance work in progress

CAUTION

Before starting work, inspect the robot, emergency stop switch and other related devices, etc., and confirm that there are no errors.
→ Inspection before starting work

1.2.1 Precautions given in the separate Safety Manual

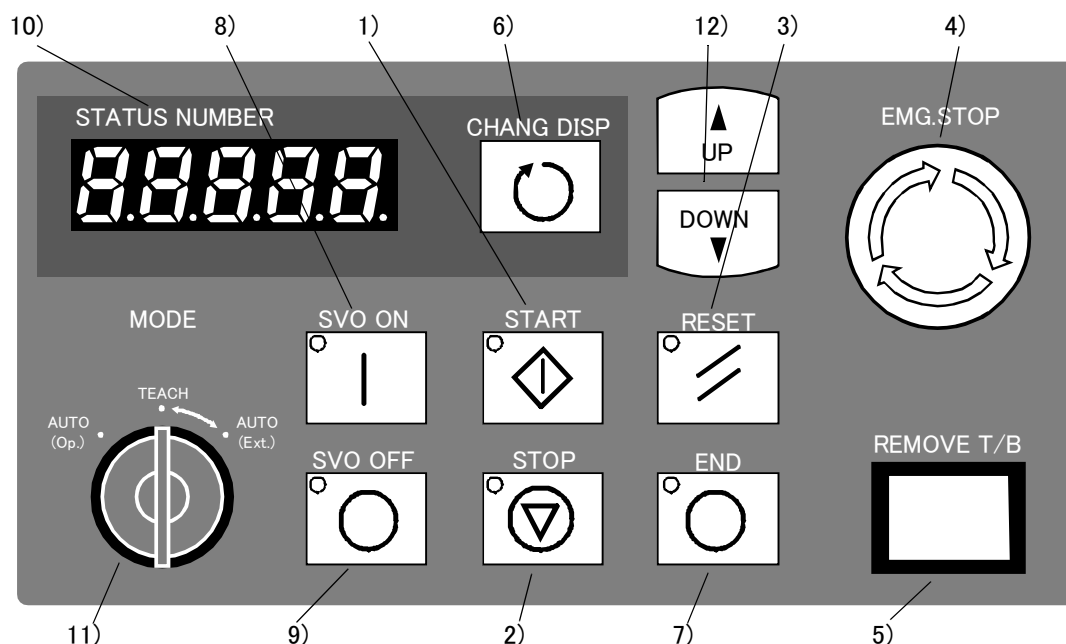
The points of the precautions given in the separate "Safety Manual" are given below.

Refer to the actual "Safety Manual" for details.

- | | |
|--|---|
|  CAUTION | Use the robot within the environment given in the specifications. Failure to do so could lead to a drop or reliability or faults. (Temperature, humidity, atmosphere, noise environment, etc.) |
|  CAUTION | Transport the robot with the designated transportation posture. Transporting the robot in a non-designated posture could lead to personal injuries or faults from dropping. |
|  CAUTION | Always use the robot installed on a secure table. Use in an instable posture could lead to positional deviation and vibration. |
|  CAUTION | Wire the cable as far away from noise sources as possible. If placed near a noise source, positional deviation or malfunction could occur. |
|  CAUTION | Do not apply excessive force on the connector or excessively bend the cable. Failure to observe this could lead to contact defects or wire breakage. |
|  CAUTION | Make sure that the workpiece weight, including the hand, does not exceed the rated load or tolerable torque. Exceeding these values could lead to alarms or faults. |
|  WARNING | Securely install the hand and tool, and securely grasp the workpiece. Failure to observe this could lead to personal injuries or damage if the object comes off or flies off during operation. |
|  WARNING | Securely ground the robot and controller. Failure to observe this could lead to malfunctioning by noise or to electric shock accidents. |
|  CAUTION | Indicate the operation state during robot operation. Failure to indicate the state could lead to operators approaching the robot or to incorrect operation. |
|  WARNING | When carrying out teaching work in the robot's movement range, always secure the priority right for the robot control. Failure to observe this could lead to personal injuries or damage if the robot is started with external commands. |
|  CAUTION | Keep the jog speed as low as possible, and always watch the robot. Failure to do so could lead to interference with the workpiece or peripheral devices. |
|  CAUTION | After editing the program, always confirm the operation with step operation before starting automatic operation. Failure to do so could lead to interference with peripheral devices because of programming mistakes, etc. |
|  CAUTION | Make sure that if the safety fence entrance door is opened during automatic operation, the door is locked or that the robot will automatically stop. Failure to do so could lead to personal injuries. |
|  CAUTION | Never carry out modifications based on personal judgments, or use non-designated maintenance parts.
Failure to observe this could lead to faults or failures. |
|  WARNING | When the robot arm has to be moved by hand from an external area, do not place hands or fingers in the openings. Failure to observe this could lead to hands or fingers catching depending on the posture. |
|  CAUTION | Do not stop the robot or apply emergency stop by turning the robot controller's main power OFF.
If the robot controller main power is turned OFF during automatic operation, the robot accuracy could be adversely affected. |
|  CAUTION | Do not turn off the main power to the robot controller while rewriting the internal information of the robot controller such as the program or parameters.
If the main power to the robot controller is turned off while in automatic operation or rewriting the program or parameters, the internal information of the robot controller may be damaged. |

2 Explanation of functions

2.1 Operation panel (O/P) functions



(1) Explanation of buttons on the operation panel

Table 2-1 : Names of each part on operation panel (Controller)

	Button name	Function
1)	Start button	This executes the program and operates the robot. The program is run continuously. The LED (green) lights during operation. When only executing the program to which "ALWAYS" was set as start conditions, the LED not lights.
2)	Stop button	This stops the robot immediately. The servo does not turn OFF. The LED (red) lights while stopped. (Turns on when the program is interrupted.) However, the program to which "ALWAYS" was set as start conditions does not stop.
3)	Reset button	This resets the error. The LED (red) lights while an error is occurring. This also resets the program's interrupted state and resets the program. (Only when program numbers are displayed.)
4)	Emergency stop button	This stops the robot in an emergency state. The servo turns OFF. Turn to the right to cancel.
5)	T/B connection switch	This is used to connect/disconnect the T/B without turning OFF the controller's control power. The T/B should be removed within five seconds after pressing the switch. An error occurs if more than five seconds elapses after pressing the switch. Similarly, when the T/B should be remounted, it should be connected and the switch returned to the original position within five seconds.
6)	Display changeover switch	This changes the details displayed on the display panel in the order of "Program No." → "Line No." → "Override". When an error is occurring, "Program No." → "Line No." → "Override" appear only when the key is pressed. The error No. will appear when the key is released.
7)	End button	This stops the program being executed at the last line or END statement. (Cycle operation) The LED (red) winks during cycle operation. (Cancels continuous operation.)
8)	SVO.ON switch	This turns ON the servo power. The LED (green) lights during servo ON.
9)	SVO.OFF switch	This turns OFF the servo power. The LED (red) lights during servo OFF.
10)	STATUS.NUMBER	The error No., program No., override value (%), etc., are displayed. The program name is shown with simplified symbols if alphabetic characters are used.
11)	MODE changeover switch Note1)	This changes the robot's operation rights. AUTO (Op.) : Only operations from the controller are valid. Operations for which the operation rights must be at the external device or T/B are not possible. TEACH : When the T/B is valid, only operations from the T/B are valid. Operations for which the operation rights must be at the external device or controller are not possible. AUTO (Ext.) : Only operations from the external device are valid. Operations for which the operation rights must be at the T/B or controller are not possible.
12)	UP/DOWN switch	This scrolls up or down the details displayed on the display panel (Valid for program numbers, override, and error numbers)

⚠ CAUTION

Note) The servo will turn OFF when the controller's [MODE] switch is changed.
Note that axes not provided with brakes could drop with their own weight.

Carry out the following operations to prevent the servo from turning OFF when the [MODE] switch is changed.

*The controller for this function are CR1/CR2A and the CR2/CR4/CR7/CR8/CR9 to which the CPU unit type is B. The CPU unit type is indicated on the unit upside of the right end inside the controller .

The servo on status can be maintained by changing the mode with keeping pressing lightly the deadman switch of T/B. The operating method is shown below.

■ When the mode is changed from TEACH to AUTO.

- 1) While holding down the deadman switch on the T/B, set the [ENABLE/DISABLE] switch to "DISABLE".
- 2) While holding down the deadman switch on the T/B, set the controller [MODE] switch to "AUTO".
- 3) Release the T/B deadman switch.

■ When the mode is changed from AUTO to TEACH.

- 1) While the [ENABLE/DISABLE] switch on the T/B is "DISABLE", hold down the deadman switch.
- 2) While holding down the deadman switch on the T/B, set the controller [MODE] switch to "TEACH".
- 3) While holding down the deadman switch on the T/B, set the [ENABLE/DISABLE] switch to "ENABLE", then do the operation of T/B that you wish.

(2) About the status number display

The following is a description of the simplified symbols shown on the 7-segment LED display when displaying a program name specified with alphabetic characters.

A	A	B	b	C	c	D	d	E	E	F	F	G	G	H	H	I	I
J	J	K	K	L	L	M	M	N	N	O	O	P	P	Q	Q	R	R
S	S	T	T	U	U	V	V	W	W	X	X	Y	Y	Z	Z		

The character "P" is fixed at the beginning of the program name display, which means that the number of characters that can be displayed are four or less. Make sure to use no more than four characters when entering the program name.

It is not possible to select a program name consisting of more than four characters from the operation panel. However, it is allowed to create a program name consisting of more than four characters in the case of a program to be executed as a sub-program by the CALLP instruction of the robot language.

2.2 Teaching pendant (T/B) functions

This chapter explains the functions of R28TB (optional).

(1) Display screens and functions

Table 2-2 shows the functions corresponding to the screens displayed on the T/B, and the pages on which explanations of the operation methods are given.

The screen tree is shown in the [Page 13, "\(1\) Screen tree"](#).

Table 2-2 : Display screens and functions

Screen display	Function	Explanation page
Title screen		
CRn-5xx Ver.A1 RP-1AH Copyright(C)1999 ANY KEY DOWN	Type and software version display	
Menu screen		
<MENU> 1.TEACH 2.RUN 3.FILE 4.MONI 5.MAINT 6.SET	Selection of following screens	Page 13, "3.1 Operation of the teaching pendant menu screens"
Teach screen		
<TEACH> (1) SELECT PROGRAM	Selection and editing of program No.	Page 21, "3.5.1 Creating a program"
Operation menu screen		
<RUN> 1.SERVO 2.CHECK	Servo ON/OFF Step operation	Page 39, "3.8 Turning the servo ON/OFF"
Control menu screen		
<FILE> 1.DIR 2.COPY 3.RENAME 4.DELETE	Program list display	Page 42, "(1) Program list display"
	Program protection	Page 43, "(2) Program protection function"
	Program copy	Page 44, "(3) Copying programs"
	Program name change	Page 45, "(4) Changing the program name (Renaming)"
	Program deletion	Page 46, "(5) Deleting a program"
Monitor menu screen		
<MONI> 1.INPUT 2.OUTPUT 3.VAR 4.ERROR 5.REGISTER	Input signal status display	Page 47, "(1) Input signal monitor"
	Output signal status change and display	Page 48, "(2) Output signal monitor"
	Variable details display	Page 49, "(3) Variable monitor"
	Error history display	Page 50, "(4) Error history"
	Register display	Use when Cc-Link option is used. Separate manual "Cc-Link Interface".
	Parameter setting value display and change	Page 51, "(1) Setting the parameters"
Maintenance screen		
<MAINT> 1.PARAM. 2.INIT 3.BRAKE 4.ORIGIN 5.POWER	Program initialization	Page 52, "(2) Initializing the program"
	Battery timer initialization	Page 53, "(3) Initializing the battery consumption time"
	Brake release	Page 54, "(4) Releasing the brakes"
	Origin setting	Page 55, "(5) Setting the origin"
	Operation time display	Page 56, "(6) Displaying the clock data for maintenance"
Setting menu screen		
<SET> 1.CLOCK	Date and time display and change	Page 57, "(1) Setting the time"

(2) Function of each key

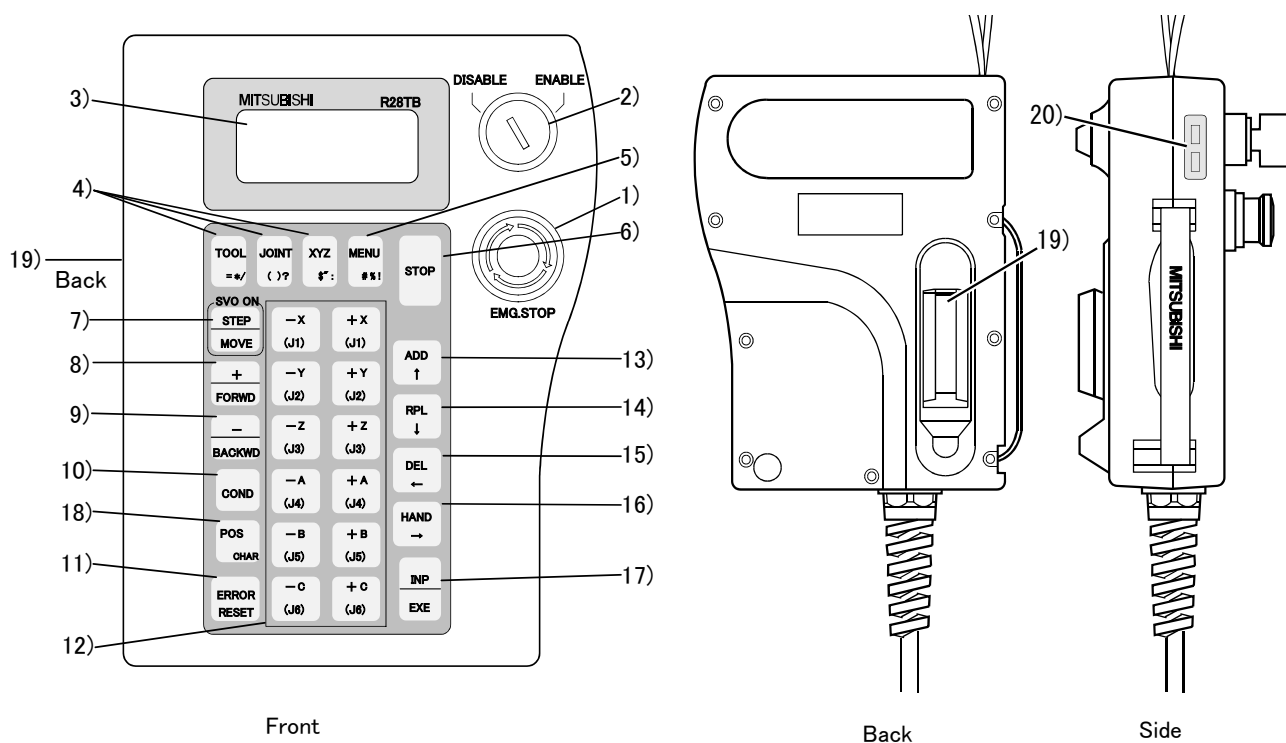


Fig.2-1 : T/B keys

	Key	Explanation
1)	[EMG. STOP] switch	This is a push-button switch with lock function for emergency stop. When this switch is pressed, the servo will turn OFF and the robot will stop immediately regardless of the T/B enable/disable state. To cancel this state, turn the switch clockwise.
2)	[ENABLE/DISABLE] switch	This changeover switch is used to enable or disable the T/B key operations. To carry out operations using the T/B, always set this switch to "ENABLE" (valid). Operations with the T/B will be enabled, and operations from the controller and external sources will be disabled. The T/B will have the operation rights. To operate with the controller or external source, set this switch to "DISABLE" (invalid). It is possible to change modes of operation related to the monitor and the override even in the disabled status. Set this switch to "DISABLE" position while editing in order to save the current program.
3)	Display LCD	The program contents and robot state are displayed with the T/B key operations.
4)	[TOOL] key	This selects the TOOL jog mode
4)	[JOINT] key	This selects the JOINT jog mode. Press twice to select the additional axis jog mode.
4)	[XYZ] key	The XYZ jog mode is selected if the key is pressed while in the TOOL and/or JOINT jog condition, the 3-axis XYZ jog mode is selected if pressed twice, and the cylinder jog mode is selected if pressed three times.
5)	[MENU] key	This returns the display screen to the menu screen. If the key is pressed while editing, the current program is saved.
6)	[STOP] key	This stops the program and decelerates the robot to a stop. This is the same function as the [STOP] switch on the front of the controller, and can be used even when the T/B [ENABLE/DISABLE] switch is set to DISABLE.
7)	[STEP/MOVE] key	Jog operations are possible when this key is pressed simultaneously with the 12) jog operation key. Step jump is carried out when pressed simultaneously with the [INP/EXE] key. Press it when the servo is off to turn the servo on (while the deadman switch is pressed).
8)	[+/FORWD] key	[Step feed is carried out when this key is pressed simultaneously with the [INP/EXE] key. On the edit screen, the next program line is displayed. Press it at the same time as the [STEP/MOVE] key during program operation to increase the override (speed). It is possible to perform this operation even when the T/B is disabled. A "+" is input when characters are entered.

	Key	Explanation
9)	[-/BACKWD] key	On the edit screen, the previous line is displayed. When pressed simultaneously with the [INP/EXE] key, the axis will return along the robot's operation path. When pressed simultaneously with the [STEP/MOVE] key, the override (speed) will decrease. It is possible to perform this operation even when the T/B is disabled. A "/" is input when characters are entered.
10)	[COND] key	Use this key to display the program instruction screen.
11)	[ERROR RESET] key	This key resets an error state that has occurred. When pressed simultaneously with the [INP/EXE] key, the program will be reset.
12)	[Jog operation] key (12 keys from [-X (J1) to +C (J6)])	In this manual, these keys are generically called the "jog operation" keys. When JOINT jog is selected, each axis will rotate, and when XYZ jog is selected, the robot will move along each coordinate system. These keys are also used to input numeric values such as when selecting a menu or inputting a step No.
13)	[ADD ↑] key	Moves the cursor upward (↑). It also, you can add or correct position data by pressing it simultaneously with the [STEP] key on the position data edit screen. (T/B version B1 or later.)
14)	[RPL/ ↓] key	Moves the cursor to the downward (↓). It also, you can display the next screen after the current position display by pressing it simultaneously with the [STEP] key on the position data edit screen. (T/B version B1 or later.)
15)	[DEL/ ←]	This deletes the position data. It also moves the cursor to the left (←).
16)	[HAND/ →] key	The following hand operations When pushed simultaneously with [+C (J6)] or [-C (J6)] key, operate the hand 1. When pushed simultaneously with [+B (J5)] or [-B (J5)] key, operate the hand 2. When pushed simultaneously with [+A (J4)] or [-A (J4)] key, operate the hand 3. When pushed simultaneously with [+Z (J3)] or [-Z (J3)] key, operate the hand 4. This key also moves the cursor to the right (→).
17)	[INP/EXE] key	This inputs the program, and carries out step feed/return
18)	[POS CHAR] key	Use this key to display the position edit screen and to enter characters and symbols.
19)	Deadman switch	When the [ENABLE/DISABLE] switch 2 is enabled, and this switch is released or pressed with force, the servo will turn OFF. Press this switch lightly when carrying out functions with the servo ON, such as jog operations. If emergency stop or servo OFF have been applied, and the servo is OFF, the servo will not turn ON even when this switch is pressed. In this case, carry out the servo ON operation again.
20)	Contrast setting switch (Top: Shade, bottom: light)	This sets the display LCD brightness.

2.2.1 Operation rights

Only one device is allowed to operate the controller (i.e., send commands for operation and servo on, etc.) at the same time, even if several devices, such as T/Bs or PCs, are connected to the controller. This limited device "has the operation rights".

Operations that start the robot, such as program start and error reset, and operations that can cause starting require the operation rights. Conversely, operation that stop the robot, such as stopping and servo OFF, can be used without the operation rights for safety purposes.

Table 2-3 : Relation of setting switches and operation rights

Setting switch	T / B [ENABLE/DISBLE]	DISABLE			ENABLE		
	Controller [MODE]	AUTO(Op.)	AUTO(Ext.)	TEACH	AUTO(Op.)	AUTO(Ext.)	TEACH
Operation rights	T / B	×	×	×	×	×	○
	Controller operation panel	○	×	×	×	×	×
	Personal computer	×	○ Note 1)	×	×	×	×
	External signal	×	○ Note 1)	×	×	×	×

○ : Has operation rights, × : Does not have operation rights

Note 1) When the "operation right input signal (IOENA)" is input from an external device, the external signal has the operation rights, and the personal computer's operation rights are disabled.

Note 2) If the [MODE] switch is set to "AUTO" when the T/B is set to "ENABLE", the error 5000 will occur.

Table 2-4 : Operations requiring operation rights

Operation item: ○ = Requires operation rights, × = Does not require operation rights

Class	Operation rights	Operation
Operation	○	Servo ON
	×	Servo OFF
	○	Program stop/cycle stop
	×	Slot initialization (program reset)
	○	Error reset
	×	Override change. Note this is always possible from the T/B.
	○	Override read
	×	Program No. change
	○	Program No./line No. read
	×	Program stop/cycle stop
Input/output signal	×	Input/output signal read
	×	Output signal write
	○	Dedicated input start/reset/servo ON/brake ON/OFF/manual mode changeover/general-purpose output reset/program No. designation/line No. designation/override designation
	×	Dedicated input stop/servo OFF/continuous cycle/ operation rights input signal/ program No.output request/line No. output request/override output request/error No. request, numeric input
	×	Hand input/output signal read
	○	Hand output signal write
Program editing Note1)	×	Line registration/read/call; Position addition/correction/read; Variable write/read
	○	Step feed/return, execution
	×	Step up/down
	○	Step jump, direct execution, jog
File operation	×	Program list read/protection setting/copy/delete/rename/ initialization
Maintenance operation	×	Parameter read, clock setting/read, operation hour meter read, alarm history read
	○	Origin setting, parameter change

Note1) When one device is being used for editing on-line, editing from other devices is not possible.

2.3 Functions Related to Movement and Control

This controller has the following characteristic functions.

Function	Explanation	Explanation page
Optimum speed control	This function prevents over-speed errors as much as possible by limiting the speed while the robot is tracking a path, if there are postures of the robot that require the speed to be limited while moving between two points. However, the speed of the hand tip of the robot will not be constant if this function is enabled.	Page 187, "SPD"
Optimum acceleration/deceleration control	This function automatically determines the optimum acceleration/deceleration time when the robot starts to move or stops, according to the weight and center of gravity settings of the hand, and the presence of a workpiece. The cycle time improves normally, although the cycle time decreases by the condition..	Page 170, "OADL", Page 161, "LOADSET"
XYZ compliance	With this function, it is possible to control the robot in a pliable manner based on feedback data from the servo. This function is particularly effective for fitting or placing workpieces. However, depending on the workpiece conditions, there are cases where this function may not be used.	Page 124, "CMP TOOL"
Continuous path control	This function is used to operate the robot between multiple positions continuously without acceleration or deceleration. This function is effective to improvement of the cycle time.	Page 63, "(4) Continuous movement", Page 128, "CNT"
Multitask program operation	With this function, it is possible to execute programs concurrently by grouping between programs for the robot movement, programs for communication with external devices, etc. It is effective to shorten input/output processing. In addition, it is possible to construct a PLC-less system by creating a program for controlling peripheral jigs.	Refer to X*** instructions such as Page 83, "4.2.1 What is multitasking?", Page 199, "XRUN".
Program constant execution function	With this function, it is possible to execute a program all the time after the controller's power is turned on. This function is effective when using the multitask functions to make the robot program serve as a PLC.	Refer to "SLTn" parameter start attribute (ALWAYS) in Page 266, "5 Functions set with parameters".
Continuity function	With this function, it is possible to store the status at power off and resume from the same status when the power is turned on again.	Refer to "CTN" parameter in Page 266, "5 Functions set with parameters".
Additional axis control	With this function, it is possible to control up to two axes as additional axes of the robot. Since the positions of these additional axes are stored in the robot's teaching data as well, it is possible to perform completely synchronous control. In addition, arc interpolation while moving additional axes (travelling axes) is also possible. This function is supported as a standard feature in controllers other than CR1.	Separate manual "ADDITIONAL AXIS INTERFACE".
Multi-mechanism control	With this function, it is possible to control up to two (excluding the standard robots) robots (user mechanism) driven by servo motors, besides the standard robots.	Separate manual "ADDITIONAL AXIS INTERFACE".
External device communication function	The following methods are available for communicating with the external devices For controlling the controller and for interlock within a program 1) Via input/output signals (32 points each for standard input/output in the case of CR2) 2) Via CC-Link (optional) As a data link with an external device 3) Communication via RS-232C (1 standard port, and up to four optional ports) 4) Communication via Ethernet The data link refers to a given function in order to exchange data, for instance amount of compensation, with external devices (e.g., vision sensors).	Refer to Page 216, "M.IN/M.INB/M.INW", Page 221, "M.OUT/M.OTB/M.OTW". Page 288, "5.10 About the communication setting" Separate manual "Ethernet Interface".
Interrupt monitoring function	With this function, it is possible to monitor signals, etc. during program operation, and pause the current processing in order to execute an interrupt routine if certain conditions are met. It is effective for monitoring that workpieces are not dropped during transport.	Page 132, "DEF ACT", Page 113, "ACT"
Inter-program jump function	With this function, it is possible to call a program from within another program using the CALLP instruction.	Page 115, "CALLP"
Pallet calculation function	This function calculates the positions of workpieces arranged in the grid and glass circuit boards in the cassette. It helps to reduce the required teaching amount. The positions can be given in row-by-column format, single row format, or arc format.	Page 69, "4.1.2 Pallet operation", Page 136, "DEF PLT", Page 176, "PLT"
User-defined area function	With this function, it is possible to specify an arbitrary space consisting of up to eight areas, monitor whether the robot's hand tip is within these areas in real time, output the status to an external device, and check the status with a program, or use it to generate an error. Moreover, two functions (ZONE and ZONE2) that have a similar function are available for use in a robot program.	Page 279, "5.3 About user-defined area", Page 226, "M.UAR". Page 264, "ZONE", Page 265, "ZONE 2"

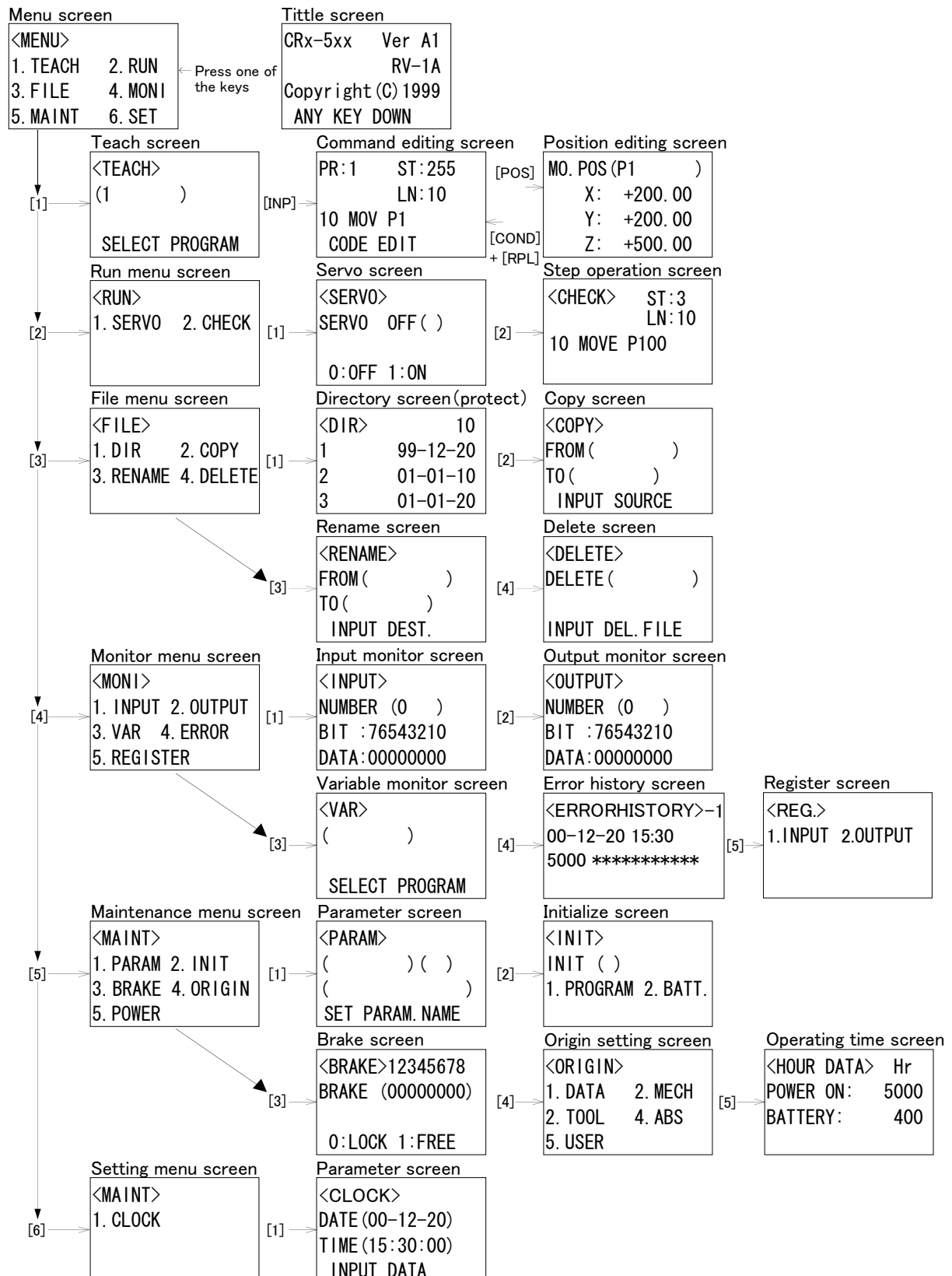
Function	Explanation	Explanation page
JOINT movement range XYZ operation range Free plane limit	It is possible to restrict the robot movement range in the following three ways JOINT movement range: It is possible to restrict the movement range of each axis. XYZ operation range: It is possible to restrict the movement range using the robot's XYZ coordinate system. Free plane limit: It is possible to define an arbitrary plane and restrict the movement range of the robot to be only in front of or only behind the plane.	Refer to "MEJAR" and "MEPAR" parameter in Page 266, "5 Functions set with parameters" Refer to Page 280, "5.4 Free plane limit"

3 Explanation of operation methods

This chapter describes how to operate R28TB (optional)

3.1 Operation of the teaching pendant menu screens

(1) Screen tree

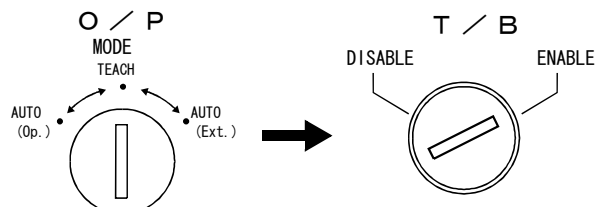


(2) Selecting a menu

A menu can be selected with either of the following two methods.

- Press the number key for the item to be selected.
- Move the cursor to the item to be selected, and press the [INP] key.

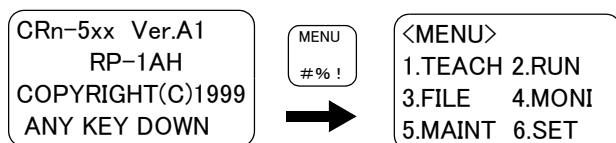
How to select the TEACH screen ("1. TEACH") from the menu screen with each method is shown below.



1) Set the controller [MODE] switch to "TEACH".

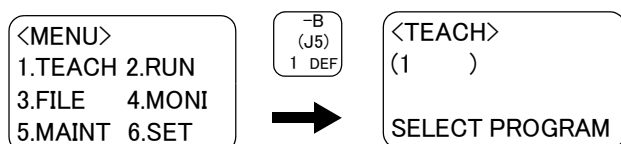
2) Set the T/B to "ENABLE".

Display the MENU screen from the TEACH screen.



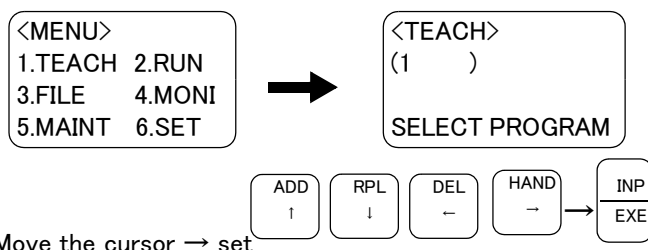
3) Press one of the keys (example, [MENU] key) while the <TITLE> screen is displayed. The <MENU> screen will appear.

■ Press the number key method



1) Press the [1] key. The <TEACH> screen will appear.

■ Use the arrow key method



1) Press the arrow keys ("↑", "↓", "←", "→") and move the cursor to "1. TEACH", and then press the [INP] key. The <TEACH> screen will appear.

The same operations can be carried out on the other menu screens.

Move the cursor → set

The same operations can be used on the other menu screens.

◆◆◆ Using the T/B ◆◆◆

Unless the controller [MODE] switch is set to "TEACH", operations other than specific operations (current position display on JOG screen, changing of override, monitoring of input/output, error history) cannot be carried out from the T/B.

◆◆◆ Inputting numbers and spaces ◆◆◆

To input a number, press the key having a number on the lower left.
To input a space, press the key having "SPACE" on the lower left.

◆◆◆ Correcting incorrect numbers ◆◆◆

Press the [DEL] key while holding down the [CHAR] key to delete the character, and then input it again.
If the cursor is returned by pressing the [←] key, and a character is input, it will be inserted.

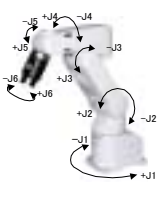
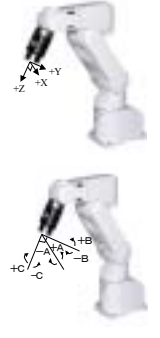
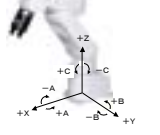
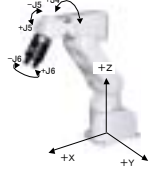
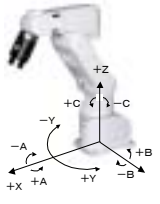
3.2 Jog Feed (Overview)

Jog feed refers to a mode of operation in which the position of the robot is adjusted manually. Here, an overview of this operation is given, using the vertical multi-joint type robot "RV-1A" as an example. The axes are configured differently depending on the type of robot. For each individual type of robot, please refer to separate manual: "ROBOT ARM SETUP & MAINTENANCE," which provides more detailed explanations.

3.2.1 Types of jog feed

The following five types of jog feed are available

Table 3-1 : Types of jog feed

Type	Operation	Explanation
JOINT jog 	1) Set the key switch to the [ENABLE] position. 2) Hold the deadman lightly. 3) Press the [STEP/MOVE] key. (The servo is turned on.) 4) Press the [JOINT] key to change to the JOINT jog mode. 5) Press the key corresponding to each of the axes from J1 to J6. 6) Press the [JOINT] key twice to shift to the additional axis mode.	In this mode, each of the axes can be adjusted independently. It is possible to adjust the coordinates of the axes J1 to J6 as well as the additional axes J7 and J8 independently. Note that the exact number of axes may be different depending on the type of robot, however. The additional axis keys [J1] and [J2] correspond to axes J7 and J8, respectively.
TOOL jog 	Perform steps 1) to 3) above. 4) Press the [TOOL] key to change to the TOOL jog mode. 5) Press the key corresponding to each of the axes from X,Y,Z,A,B,C.	The position can be adjusted forward/backward, left/right, or upward/downward relative to the direction of the hand tip of the robot (the TOOL coordinate system). The tip moves linearly. The posture can be rotated around the X, Y, and Z axes of the TOOL coordinate system of the hand tip by pressing the A, B, and C keys, without changing the actual position of the hand tip. It is necessary to specify the tool length in advance using the MEXTL parameter. The TOOL coordinate system, in which the hand tip position is defined, depends on the type of robot. In the case of a vertical multi-joint type robot, the direction from the mechanical interface plane to the hand tip is +Z. In the case of a horizontal multi-joint type robot, the upward direction from the mechanical interface plane is +Z.
XYZ jog 	Perform steps 1) to 3) above. 4) Press the [XYZ] key to change to the XYZ jog mode.	The axes are adjusted linearly with respect to the robot coordinate system. The posture rotates around the X, Y, and Z axes of the robot coordinate system by pressing the A, B, and C keys, without changing the actual position of the hand tip. It is necessary to specify the tool length in advance using the MEXTL parameter.
3-axis XYZ jog 	Perform steps 1) to 3) above. 4) Press the [XYZ] key twice to switch to the 3-axis XYZ jog mode.	The axes are adjusted linearly with respect to the robot coordinate system. Unlike in the case of XYZ jog, the posture will be the same as in the case of the J4, J5, and J6 axes JOINT jog feed. While the position of the hand tip remains fixed, the posture is interpolated by X, Y, Z, J4, J5, and J6; i.e., a constant posture is not maintained. It is necessary to specify the tool length in advance using the MEXTL parameter.
CYLINDER jog 	Perform steps 1) to 3) above. 4) Press the [XYZ] key twice to switch to the CYLINDER jog mode.	Use the cylindrical jog when moving the hand in the cylindrical direction with respect to the robot's origin. Adjusting the X-axis coordinate moves the hand in the radial direction from the center of the robot. Adjusting the Y-axis coordinate moves the hand in the same way as in JOINT jog feed around the J1 axis. Adjusting the Z-axis coordinate moves the hand in the Z direction in the same way as in XYZ jog feed. Adjusting the coordinates of the A, B, and C axes rotates the hand in the same way as in XYZ jog feed. They may be valid in horizontal 4-axis (or 5-axis) RH type robots.

3.2.2 Speed of jog feed

Press the [STEP/MOVE] key to display the current position and speed (%) on the screen. To change these values, hold the [STEP/MOVE] key down and press either the [+] key or the [-] key. The following types of jog feed speed are available.

← [-] key		[+] key →						
LOW	HIGH	3%	5%	10%	30%	50%	70%	100%

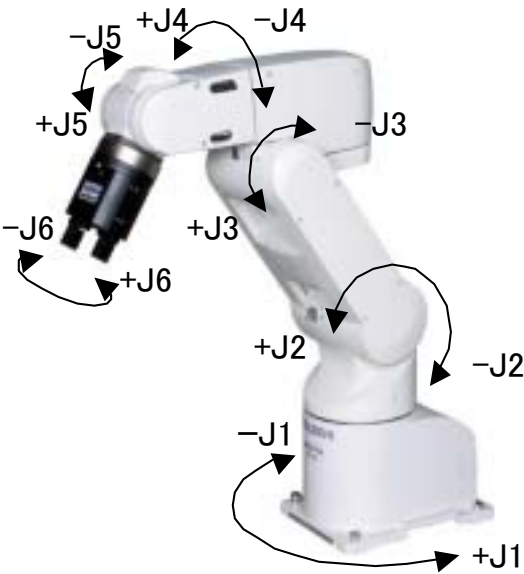
LOW and HIGH are fixed-dimension feed. In fixed-dimension feed, the robot moves a fixed amount every time the key is pressed. The amount of movement depends on the individual robot.

Table 3-2 : Fixed-dimension of RV-1A

	JOINT jog	TOOL, XYZ jog
LOW	0.01 deg.	0.01 mm
HIGH	0.10 deg.	0.10 mm

3.2.3 JOINT jog

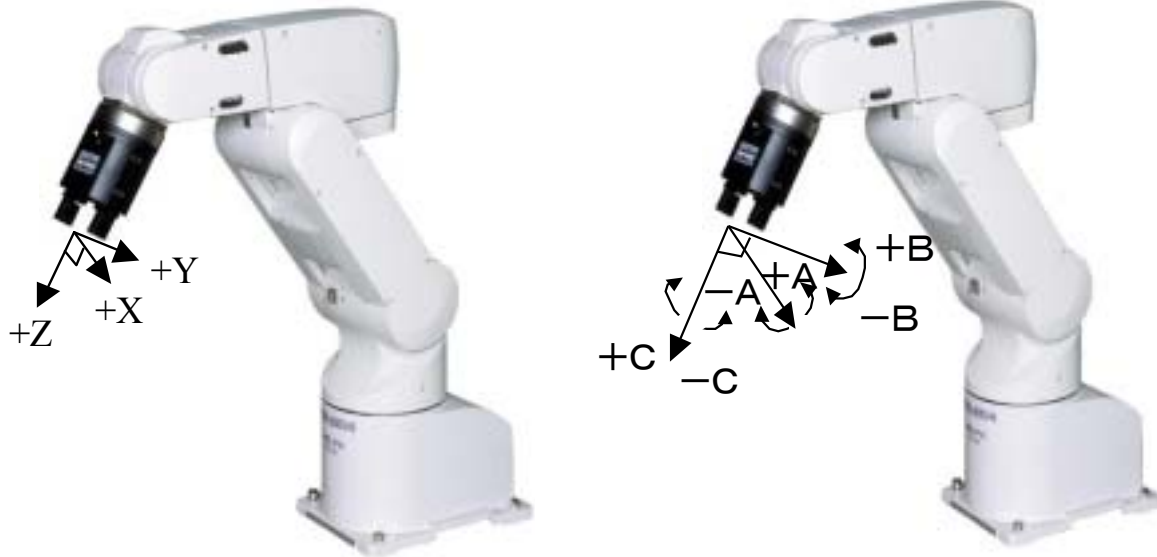
Adjusts the coordinates of each axis independently in angle units.



3.2.4 TOOL jog

Adjusts the coordinates of each axes along the direction of the hand tip.

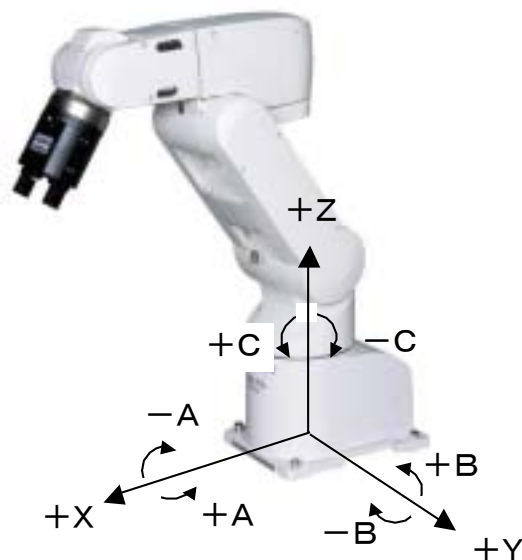
The X, Y, and Z axis coordinates are adjusted in mm units. The A, B, and C axis coordinates are adjusted in angle units.



3.2.5 XYZ jog

Adjusts the axis coordinates along the direction of the robot coordinate system.

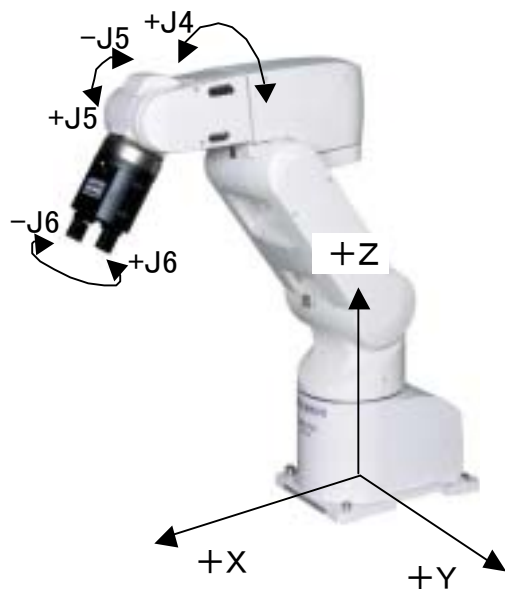
The X, Y, and Z axis coordinates are adjusted in mm units. The A, B, and C axis coordinates are adjusted in angle units.



3.2.6 3-axis XYZ jog

Adjusts the X, Y, and Z axis coordinates along the direction of the robot coordinate system in the same way as in XYZ jog feed. The J4, J5, and J6 axes are adjusted independently in the same way as in JOINT jog feed.

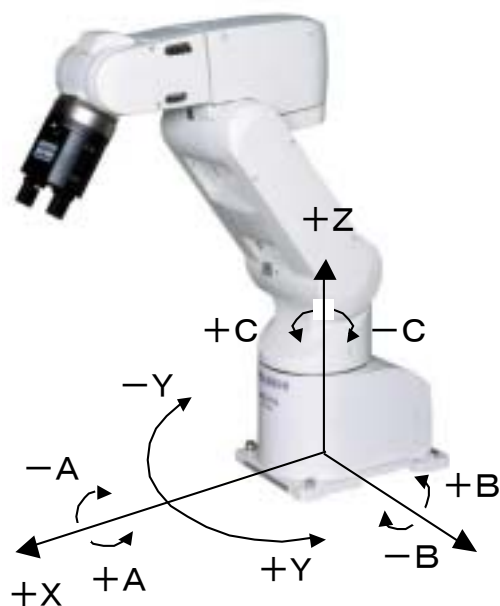
The X, Y, and Z axis coordinates are adjusted in mm units. The J4, J5, and J6 axis coordinates are adjusted in angle units.



3.2.7 CYLINDER jog

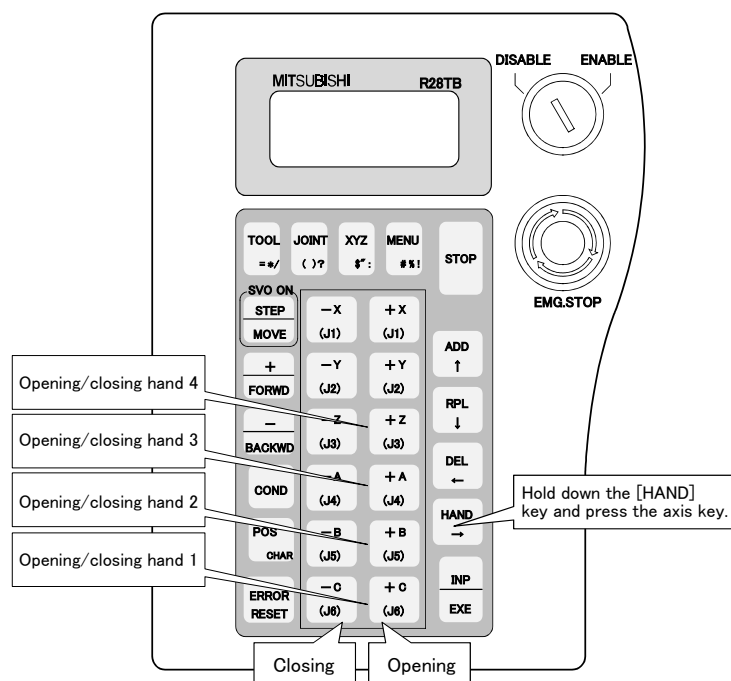
Adjusting the X-axis coordinate moves the hand in the radial direction away from the robot's origin. Adjusting the Y-axis coordinate rotates the arm around the J1 axis. Adjusting the Z-axis coordinate moves the hand in the Z direction of the robot coordinate system. Adjusting coordinates of the A, B, and C axes moves the hand in the same way as in XYZ jog feed.

The X and Z axis coordinates are adjusted in mm units. The Y, A, B, and C axis coordinates are adjusted in angle units.



3.3 Opening/Closing the Hands

The open/close operation of the hands attached to on the robot is explained below.

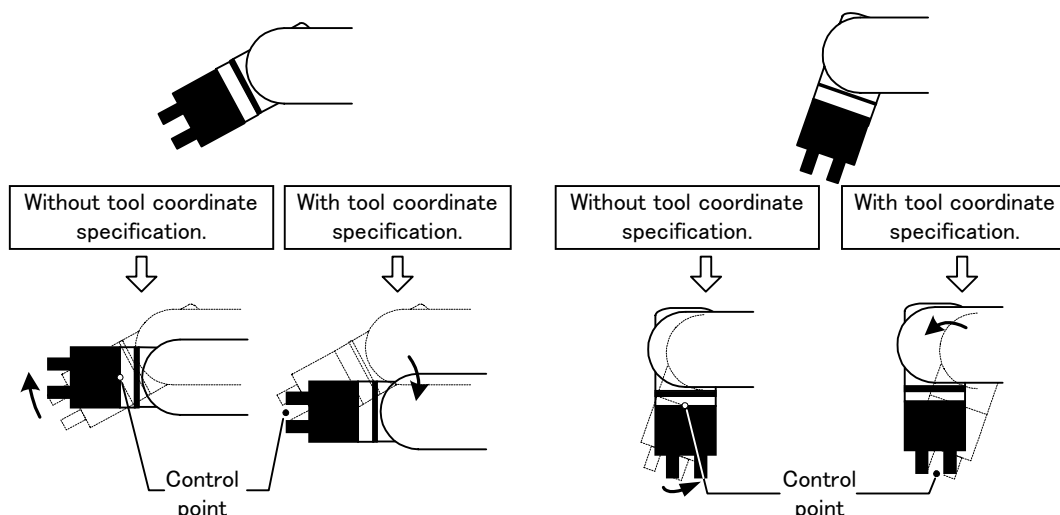


It is possible to mount various tools on the robot's hand area. In the case of pneumatic control, where the solenoid valve (at double solenoid) is used, two bits of the hand signal is controlled by the open/close operation of the hand. For more information about the hand signal, please refer to [Page 284, "5.7 About the hand type"](#) and [Page 285, "5.8 About default hand status"](#).

3.4 Aligning the Hand

The posture of the hand attached to the robot can be aligned in units of 90 degrees.

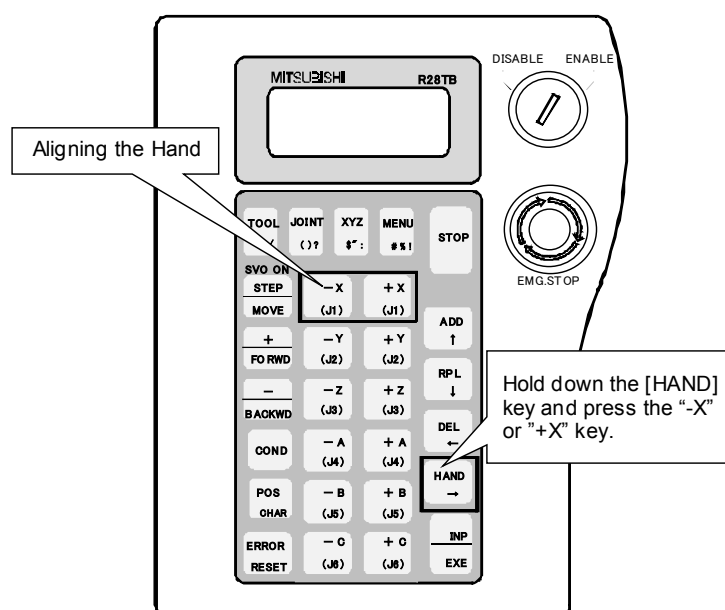
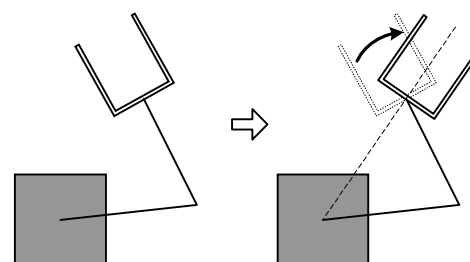
This feature moves the robot to the position where the A, B and C components of the current position are set at the closest values in units of 90 degrees.



If the tool coordinates are specified by the TOOL instruction or parameters, the hand is aligned at the specified tool coordinates. If the tool coordinates are not specified, the hand is aligned at the center of the mechanical interface. The above illustration shows an example of a small vertical robot. [With Tool Coordinate Specification] indicates when the tool coordinates are specified at the tip of the hand. For more information about the tool coordinates, refer to [Page 275, "5.1 Standard Tool Coordinates"](#).

However, in the case of the RH-15UHC robot, the hand aligns radially from the center position of the robot, as shown in the right figure.

The hand alignment procedure is as follows:



- 1) Set the key switch to the [ENABLE] position.
- 2) Hold the deadman switch lightly.
- 3) Press the [STEP/MOVE] key and turn on the servo.
- 4) Hold down the [HAND] key and press the "-X" or "+X" key.

The robot stops when any of the above keys is released while aligning the hand.

3.5 Programming

MELFA-BASIC IV used with this controller allows advanced work to be described with ample operation functions. The programming methods using the T/B are explained in this section. The functions shown in Table 3-3 are used to input one line. (Refer to Page 111, "4.11 Detailed explanation of command words" in this manual for details on the MELFA-BASIC IV commands and description methods.)

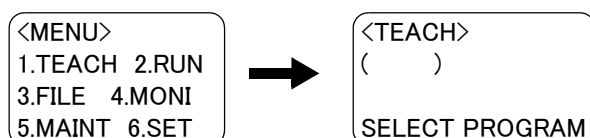
Table 3-3 : Process for inputting one line

Input details	Function
Line No. and command statement (Example: 10 MOV P1)	Input as one line of the program
Only line No. (Example; 10)	Deletes designated line from program
Only command statement (Example: MOV P1)	Immediately executes that command (Direct execution)

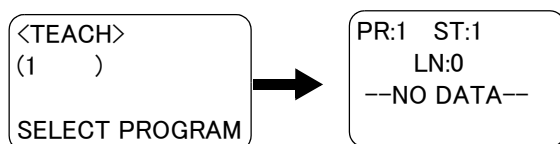
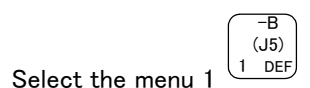
3.5.1 Creating a program

(1) Opening the program edit screen

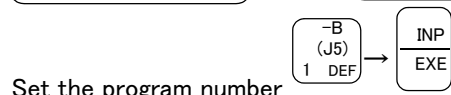
Open the screen for editing the program to be created.



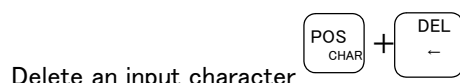
- 1) Press the [1] key.
The PROGRAM SELECTION screen will appear.



- 2) Press the [1] key.
The program name 1 edit screen will appear, and the head line will appear.



Set the program number

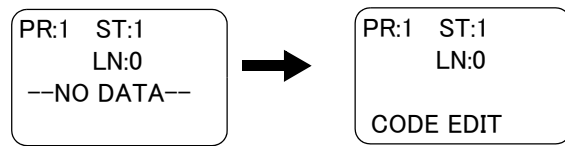


Delete an input character

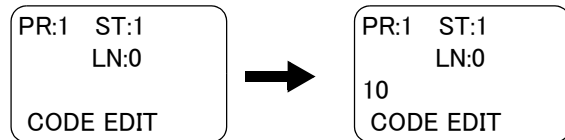
◆◆◆ Editing a program in constant execution mode (ALWAYS attribute) ◆◆◆

In order to edit a program set to be in constant execution mode (the ALWAYS attribute in the SLTn parameter is set), the constant execution attribute must be canceled first. Since programs in constant execution mode are executed continually, they cannot be edited. Change ALWAYS to START in the SLTn parameter, turn the controller's power on again, and stop the constant execution.

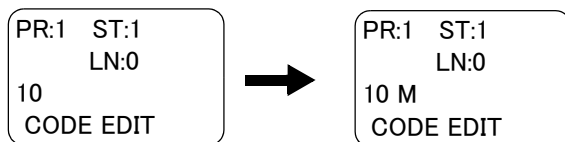
(2) Creating a program



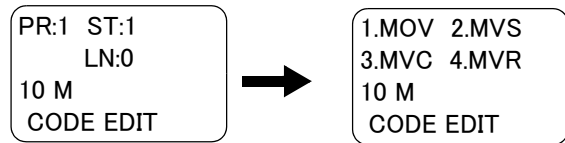
Move the cursor



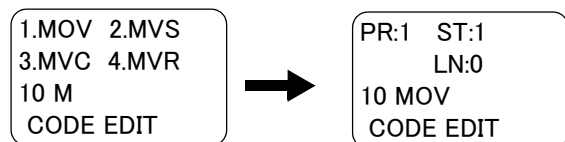
Input "1","0"



Input "M"



Input "M"



Input "MOV"

1) Press the [↓] key three times.
The cursor will move to the command editing line.

2) Press the [1], [0] and [SPACE] keys. The line No. "10" will be input.

3) Press the [-Y/MNO] key once while holding down the [CHAR] key. "M" will appear.

4) Release the [CHAR] key once, and then hold it down. The four commands assigned to "M" will appear.

5) Press the [1] key while holding down the [CHAR] key.
The "MOV" command will be input.

◆◆◆ Inputting characters ◆◆◆

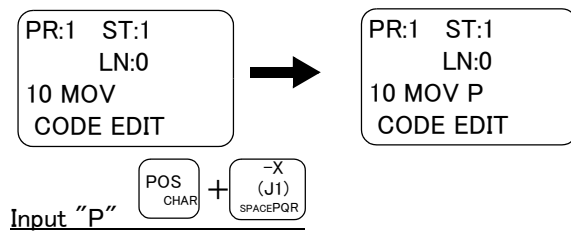
The characters that can be input are indicated, three in a group, on the lower right of each key. To input a character, hold down the [CHAR] key and press the key having the character to be input. Each time the corresponding character key is pressed while the [CHAR] key is pressed, the three characters will appear alternately.

Release the [CHAR] key when the target character appears, and set the character.

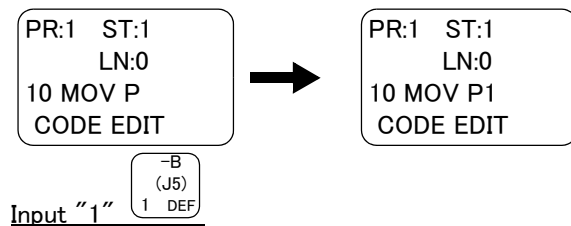
◆◆◆ Inputting commands ◆◆◆

The commands can be input one character at a time (ex., for "M" → "O" → "V" for the MOV command), but if the head character of the command is input, the command can be selected as a number from the list of commands that appears.

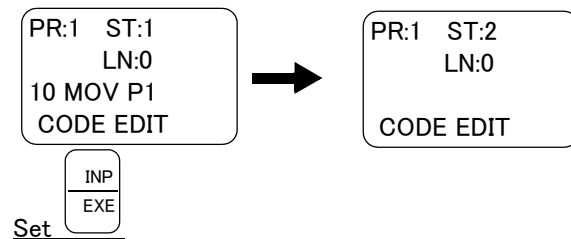
After inputting the head character of the command, press the [CHAR] key. The list of commands will appear. While holding down the [CHAR] key, press the numeral key for the target command No., and select the command. If the target command is not found in the list, press the [CHAR] key again to update the list.



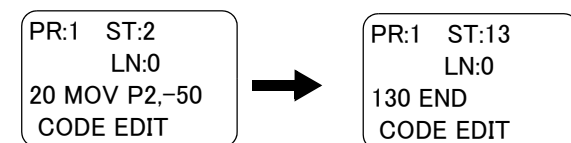
- 6) Press the [-X/PQR] key once while holding down the [CHAR] key, and then release the [CHAR] key.
"P" will be input.



- 7) Press the [1] key. "1" will be input.



- 8) Press the [INP] key.
"10 MOV P1" will be set.



- 9) Input the remaining program in the same manner.

This completes the inputting of the program.

(3) Completion of program creation and saving programs

- Press the [MENU] key, or
 - Set the [ENABLE/DISABLE] switch of the T/B to the "DISABLE" position.
- The program is saved when either of these operations is performed.

◆◆◆ Precautions when saving programs ◆◆◆

Make sure to perform the operation above. The edited data will not be updated if the power is turned off without doing so after modifying a program on the program edit screen. Moreover, as much as possible, try to save programs not only on the controller but also on a PC in order to make backup copies of your work. It is recommended to manage programs using PC support software (optional).

◆◆◆ Displaying the previous and next command line ◆◆◆

To display the previous line, press the [BACKWD] key, and to display the next line, press the [FORWD] key.

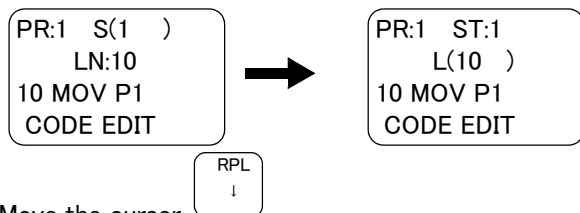
◆◆◆ Displaying a specific line ◆◆◆

Press the [↑] key and move the cursor to LN:. Input the No. of the line to be displayed in the parentheses, and then press the [INP] key. The designated line will appear.

(4) Correcting a program

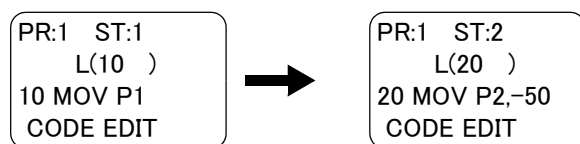
Before correcting a program, refer to [Page 21, "3.5.1 Creating a program"](#) in "(1)Opening the program edit screen", and open the program edit screen.

Call the line No.



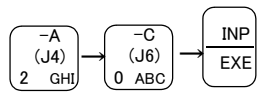
1) Press the [↓] key, and move the cursor to LN:().

Move the cursor



2) Press the [2], [0], [INP] key to display line 20.

Input the line number



◆◆◆ Cursor movement ◆◆◆

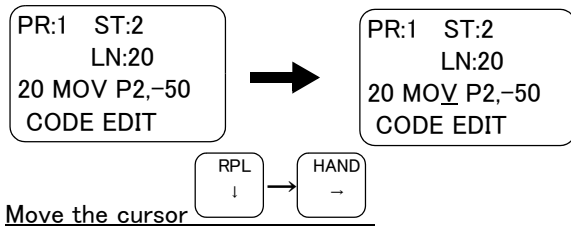
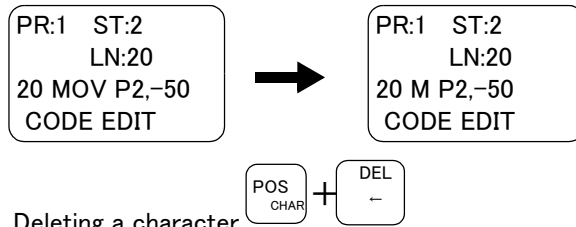
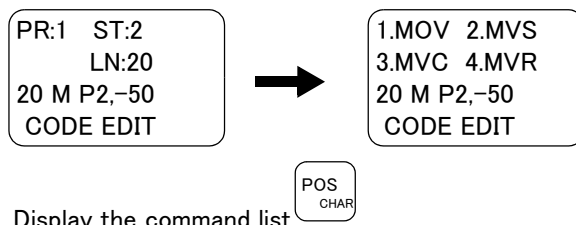
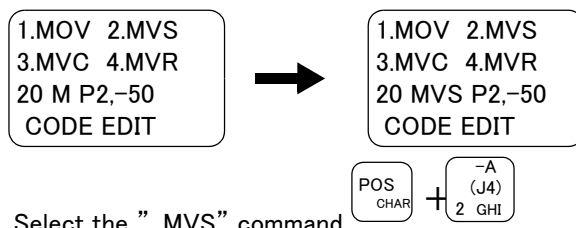
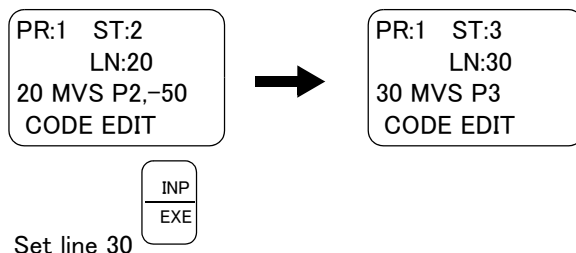
When the cursor is at a command line display, the command can be edited. When at a line No. display (LN:), the line No. is designated.

The cursor is moved with the [↑], [↓], [←] and [→] keys.

◆◆◆ Calling out a line No. ◆◆◆

When designating and calling out a line No., move the cursor to the line No. display (LN:), input the line No., and then press the [INP] key.

The displayed line can be scrolled up or down by pressing the [FORWD] or [BACKWD] key.

Change the commandMove the cursorDeleting a characterDisplay the command listSelect the "MVS" commandSet line 30

3) Press the [↓] key and move the cursor to the command line.
Press the [→] key six times, and move the cursor to the right of "V".

4) Press the [DEL] key two times while holding down the [CHAR] key, and delete "OV". "M" will remain displayed.

5) Hold down the [CHAR] key.
The four commands assigned to "M" will appear.

6) Press the [2] key while holding down the [CHAR] key, and select "MVS". MVS will appear on the screen.

7) Press the [INP] key, and set line No. 70. The next line will appear on the screen.

8) After finishing modifying an instruction, press the [MENU] key to save it.

Line No. 20 has been changed to linear movement with the above operation.

◆◆◆ Correcting a character ◆◆◆

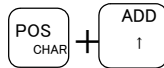
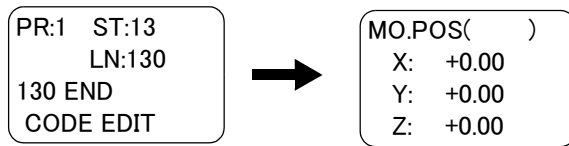
Move the cursor to the right of the incorrect character, and press the [DEL] key to delete in the left direction. Then, input the correct character.

◆◆◆ After correcting a program ◆◆◆

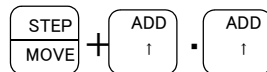
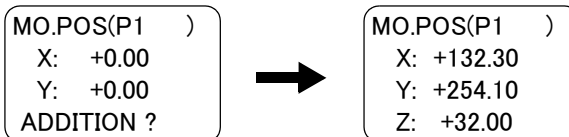
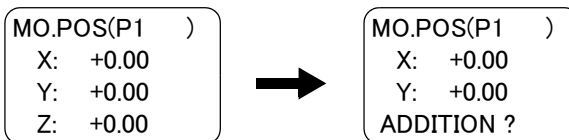
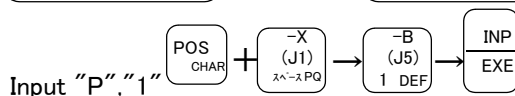
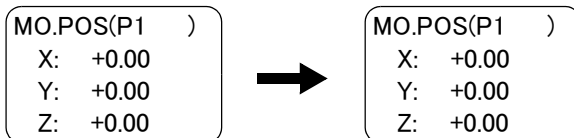
After modifying a program, make sure to perform the save operation (pressing the [MENU] key) and perform a step operation to check that the content of the program is properly changed.

(5) Registering the current position data

Entering position data(Teaching P1)



Change to the position screen



Entering position data

- 1) On the command editing screen, press the [ADD] key or [RPL] while holding down the [POS] key.
The position editing screen will appear.

- 2) Input "P1" in the parentheses at MO.POS, and then press the [INP] key.
The position variable name P1 will be called, and the currently registered coordinate value will appear.

Refer to [Page 22, "Inputting characters ◇◇"](#) for details on inputting characters.

- 3) [Press the [ADD] key or [RPL] key while holding down the [STEP] key, and release only the [ADD] key or [RPL] key.
The buzzer will sound a "beep", and a confirmation message will appear.
While holding down the [STEP] key, press the [ADD] key or [RPL] key again.
The buzzer will sound a "beep", and the message "ADDING" will appear. Then, the current position will be registered.

- 4) After finishing modifying an instruction, press the [MENU] key to save it.

The robot's operation position can be taught with the above operations.

◇◆◇ Changing between the command editing screen and position editing screen. ◇◆◇

The commands are edited on the command editing screen, and the positions are edited on the position editing screen.

To change from the command editing screen to the position editing screen, press [POS] + ([ADD] or [RPL] key).
If the cursor is not displayed, press the [COND] key and then the [POS] key to display the cursor.

(6) Confirming the position data (Position jump)

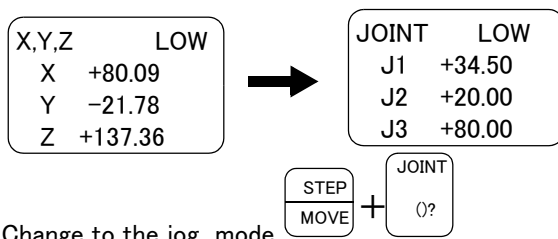
Move the robot to the registered position data place.

The robot can be moved with the "MO position movement" or "MS position movement" method.

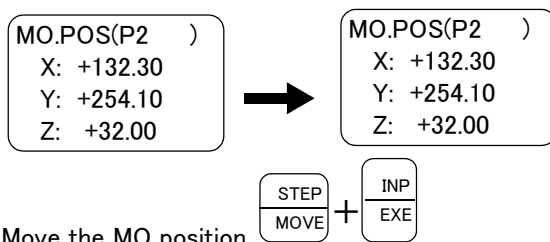
Perform a servo ON operation while lightly holding the deadman switch before moving positions.

Table 3-4 : Moving to designated position data

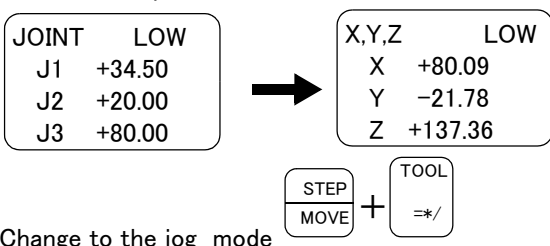
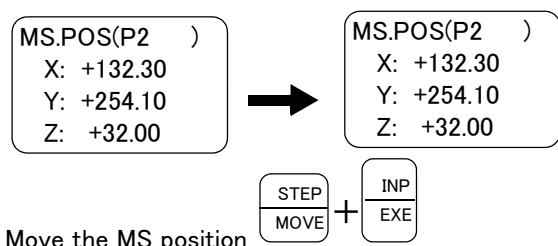
Name	Movement method
MO position movement	The robot moves with joint interpolation to the designated position data place. If the jog mode is JOINT jog feed The axes are adjusted in the same way as with the MOV instruction.
MS position movement	The robot moves with linear interpolation to the designated position data place. Thus, the robot will not move if the structure flag for the current position and designated position differ. If the jog mode is XYZ or TOOL jog feed The axes are adjusted in the same way as with the MVS instruction.

Move the MO position

- 1) MO position movement is possible in jog modes other than the XYZ jog mode and TOOL jog mode. Thus, change the mode if these modes are entered.

Change to the jog mode

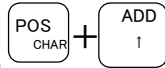
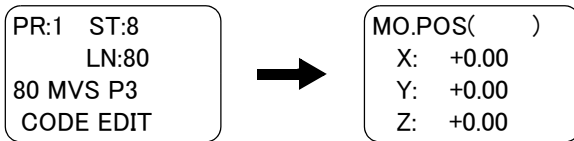
- 2) When the [EXE] key is pressed while holding down the [MOVE] key, the robot will move with joint interpolation to the currently displayed position data place only while the [EXE] key is held down.

Move the MO positionMove the MS positionChange to the jog modeMove the MS position

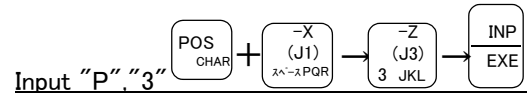
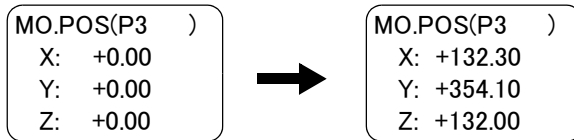
- 3) MS position movement is possible in jog modes other than the XYZ jog mode and TOOL jog mode. Thus, change the mode if these modes are entered.
- 4) When the [EXE] key is pressed while holding down the [MOVE] key, the robot will move with linear interpolation to the currently displayed position data place only while the [EXE] key is held down.
If linear movement from the current position to the designated position is not possible, the robot will not move.

(7) Correcting the current position data

Change the movement position



Change to the position screen



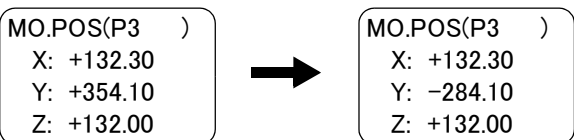
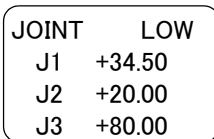
- 1) On the command editing screen, press the [ADD] key or [RPL] key while holding down the [POS] key. The position editing screen will appear.

- 2) Input "P3" in the parentheses at MO.POS, and then press the [INP] key. The position variable name P3 will be called, and the currently registered coordinate value will appear.

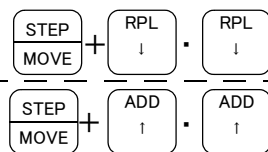
- 3) After finishing modifying an instruction, press the [MENU] key to save it.

◇◆◇ Calling out a position variable ◇◆◇

Input the name of the variable to be called out in the parentheses at MO. POS on the position editing screen. Then, press the [INP] key. The position variable can be scrolled up or down by pressing the [FORWD] or [BACKWD] key.



*) If the software version of T/B is B1 or later, operate with lower stage specified keys.



Position variable compensation

- 4) Move the robot to the new wait position with jog operation.
- 5) Press the [RPL] key while holding down the [STEP] key, and release only the [RPL] key. The buzzer will sound a "beep", and a confirmation message will appear. While holding down the [STEP] key, press the [RPL] key again. The buzzer will sound a "beep", and the message "Replacing" will appear. Then, the current position will be corrected.

*) If the software version of T/B is B1 or later, change the [RPL] key to [ADD], and operate.(like the left figure)

This completes correction of the wait position.

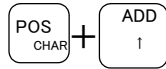
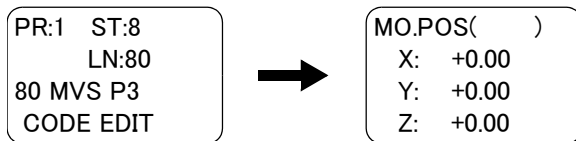
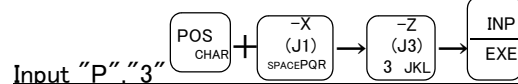
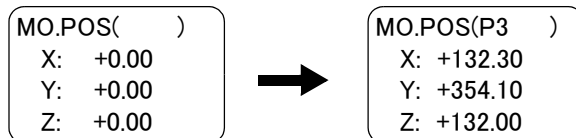
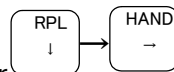
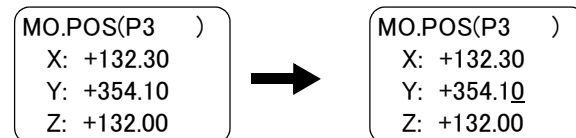
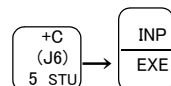
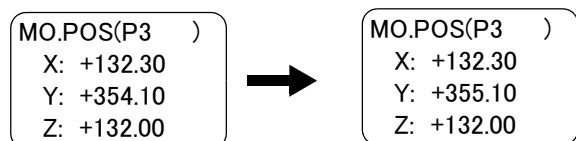
◇◆◇ After correcting a program ◇◆◇

After modifying a program, make sure to perform the save operation (pressing the [MENU] key) and perform a step operation to check that the content of the program is properly changed.

◇◆◇ The check of the software version of T/B ◇◆◇

After turning on the controller power, it can check by the T/B screen before title screen is displayed.

(8) Correcting the MDI (Manual Data Input)

MDI compensation methodChange to the position screenInput "P", "3"Move the cursorChange a setting value

1) On the command editing screen, press the [ADD] key or [RPL] key while holding down the [POS] key. The position editing screen will appear.

2) Input "P3" in the parentheses at MO.POS, and then press the [INP] key. The position variable name P3 will be called, and the currently registered coordinate value will appear.

3) [→] Using the [↓] key, move the cursor to the Y: line, and then using the [→] key, move the cursor to above 4.

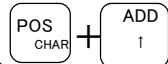
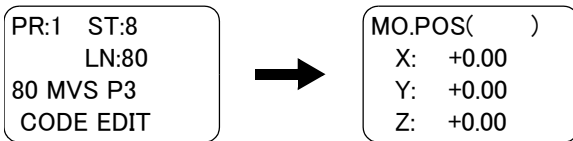
4) Press the [5] key, and then press the [INP] key. 5 will be written over 4, and the Y coordinate value will be changed.

5) After finishing modifying an instruction, press the [MENU] key to save it.

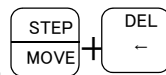
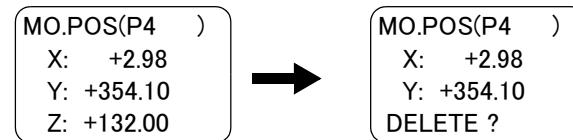
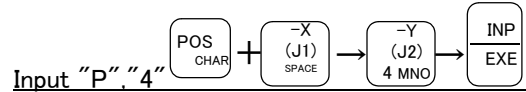
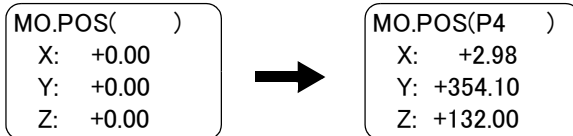
(9) Deleting position data

Only position data that is not currently being used by the program can be deleted. If position data being used in the program is deleted, an error will occur.

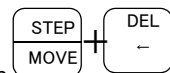
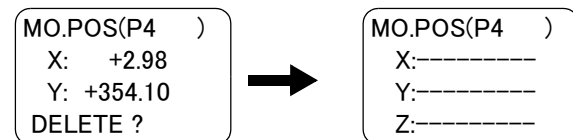
Position data deletion method



Change to the position screen



Delete position data



Confirm the position data deletion

1) On the command editing screen, press the [ADD] key or [RPL] key while holding down the [POS] key.

The position editing screen will appear.

2) Input "P4" in the parentheses at MO.POS, and then press the [INP] key.
The position variable name P4 will be called, and the currently registered coordinate value will appear.

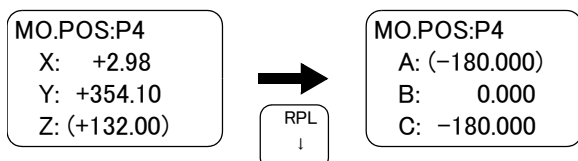
3) Press the [DEL] key while holding down the [STEP] key, and release only the [DEL] key.
The buzzer will sound a "beep", and a confirmation message will appear.

4) While holding down the [STEP] key, press the [DEL] key again.
The buzzer will sound a "beep", and the message indicating the deletion will appear at the bottom line of the screen. The position data will be deleted.

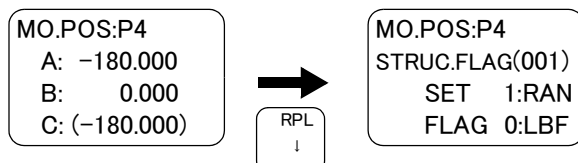
5) After finishing modifying an instruction, press the [MENU] key to save it.

(10) Display on the position edit screen

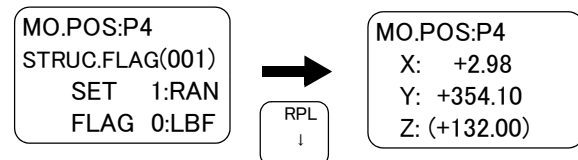
The XYZ coordinate values of the X, Y, and Z axes, the posture data of the A, B, and C axes, the posture structure flags, and the multiple rotation information of each axis, are saved as the robot's position data. Perform the following operations to display each screen.

Position edit screen

- 1) When the position edit screen is displayed, the coordinate values of the X, Y, and Z axes are displayed first. Press the [RPL/ ↓] key to display the data of the A, B, and C axes.

Change to the position screen (A,B,C axis)

- 2) Press the [RPL/ ↓] key again to display the information of structure flag 1.

Change to the position screen (Flg 1)

- 3) Press the [RPL/ ↓] key once again to return to the X, Y, and Z axes display.

Return to the position screen (XYZ)

(11) Saving the program

When completed creating or revising a program, save the edited details with one of the following operations.

- * Press the [MENU] key and display the Menu screen.
- * Set the T/B [ENABLE/DISABLE] switch to "DISABLE"

◆◆◆ Precaution when saving the edited program/data ◆◆◆

Please be aware that any updates to the program or data, including teaching data, will be lost if the power is shut down while in the program edit screen. Please be aware that any updates to the program or data, including teaching data, will be lost if the power is shut down while in the program edit screen. Please be aware that any updates to the program or data, including teaching data, will be lost if the power is shut down while in the program edit screen.

3.6 Debugging

Debugging refers to testing that the created program operates correctly, and to correcting an errors if an abnormality is found. These can be carried out by using the T/B's debugging function. The debugging functions that can be used are shown below. Always carry out debugging after creating a program, and confirm that the program runs without error.

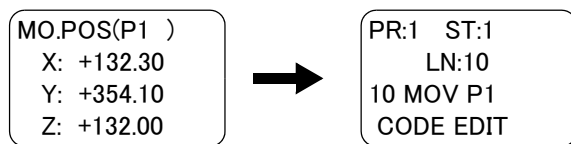
(1) Step feed

The program is run one line at a time in the feed direction. The program is run in line order from the head or the designated line.

Confirm that the program runs correctly with this process.

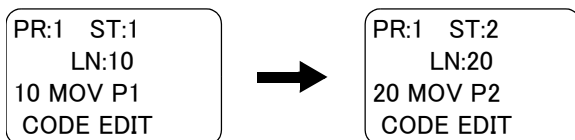
Using the T/B execute the program line by line (step operation), and confirm the operation.

Perform the following operations while pressing lightly on the deadman switch of the T/B after the servo has been turned on.



1) Press the [COND] key, and display the command editing screen.

Change to the command screen



2) While holding down the [FORWD] key or [STEP] key, hold down the [EXE] key. The robot will start moving.

When the execution of one line is completed, the robot will stop, and the next line will appear on the screen. If [EXE] is released during this step, the robot will stop.



⚠ CAUTION

Take special care to the robot movements during operation. If any abnormality occurs, such as interference with the peripheral devices, release the [EXE] key or deadman switch, or press the deadman switch with force and stop the robot.

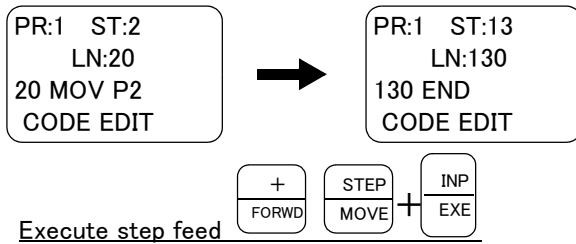
◆◆◆ Step operation ◆◆◆

"Step operation" executes the program line by line. The operation speed is slow, and the robot stops after each line, so the program and operation position can be confirmed.

During execution, the lamp on the controller's [START] switch will light.

◆◆◆ Immediately stopping the robot during operation ◆◆◆

- Press the [EMG. STOP] (emergency stop) switch.
The servo will turn OFF, and the moving robot will immediately stop.
To resume operation, reset the error, turn the servo ON, and start step operation.
- Release or forcibly press the "deadman" switch.
The servo will turn OFF, and the moving robot will immediately stop. Error 2000 will occur.
To resume operation, reset the error, lightly press the [Deadman] switch, press the [SVO ON] key to turn ON the servo, and then start step operation.
- Release the [EXE] key.
The step execution will be stopped. The servo will not turn OFF.
To resume operation, press the [EXE] key.

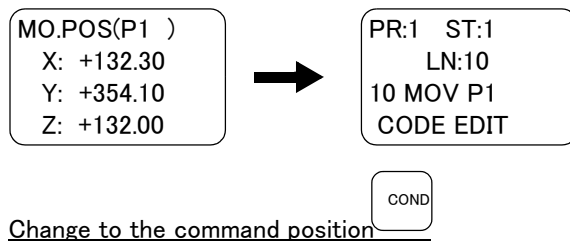


3) Carry out step operation of the entire program, and confirm the operation in the same manner.

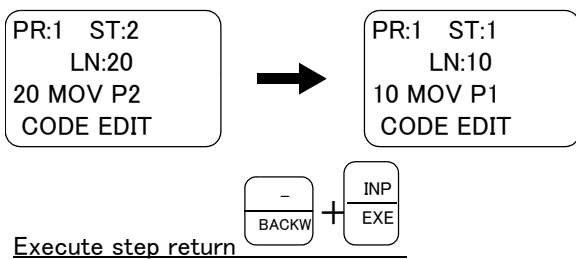
If the robot operation or position is incorrect, refer to the following operations and make corrections.

(2) Step return

The line of a program that has been stopped with step feed or normal operation is returned one line at a time and executed. This can be used only for the interpolation commands. Note that only up to four lines can be returned.



1) Press the [COND] key, and display the command editing screen. After this, carry out step feed referring to the previous page.

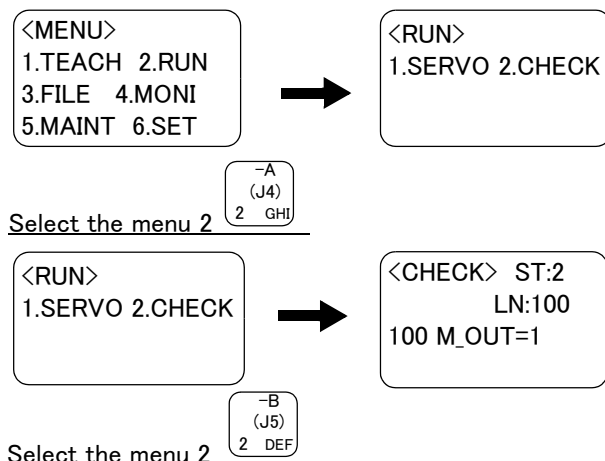


2) While holding down the [BACKWD] key, hold down the [EXE] key. The robot will start step return.

When the execution of one line is completed, the robot will stop, and the previous line will appear on the screen. If [EXE] is released during this step, the robot will stop.

(3) Step feed in another slot

When checking a multitask program, it is possible to perform step feed in the confirmation screen of the operation menu, not in the edit screen.



1) Press the "2" key to display the operation menu screen.

2) Press the "2" key to display the confirmation screen.

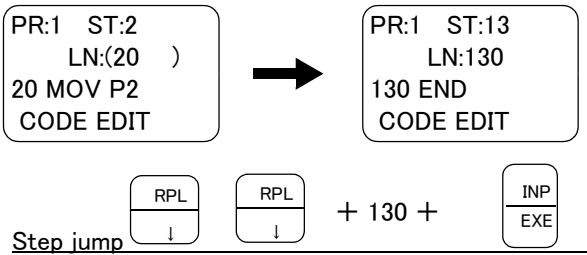
3) Change the slot number to display and allow step feed for another task slot.

4) The operations for step feed are the same as in the edit screen. Hold down "+" (or "-") and press the [INP/EXE] key.

Specify 0 for the slot number in order to perform step feed for all the task slots at the same time.

(4) Step jump

It is possible to change the currently displayed step or line.



- 1) Use the arrow keys to move to the ST column or LN column and enter the step number or line number you want to display, and then press the [INP] key.
- 2) It is possible to perform step feed from the step after the change. However, an undefined error or similar will occur if lines for initialization of variables, etc. are skipped.

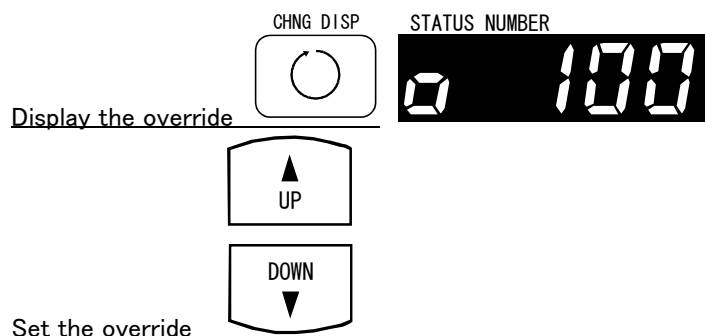
3.7 Automatic operation

(1) Setting the operation speed

The operation speed is set with the controller or T/B.

The actual speed during automatic operation will be the operation speed = (controller (T/B) setting value) × (program setting value).

■ Operating with the controller

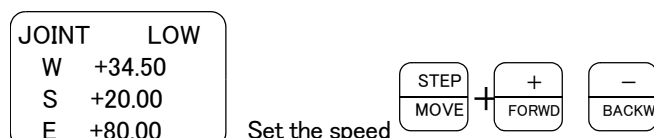


1) Press the controller [CHNG DISP] switch twice, and display the "OVERRIDE" on the STATUS NUMBER display panel.

2) Each time the [UP] key is pressed, the override will increase in the order of (10 → 20 → 30 → 40 → 50 → 60 → 70 → 80 → 90 → 100%).

The speed will decrease in reverse each time the [DOWN] key is pressed.

■ Operating with the T/B

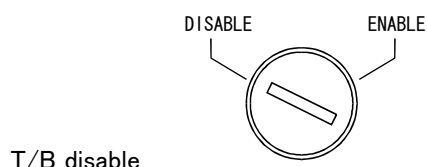


1) Each time the [MOVE] + [+] keys are pressed, the override will increase in the order of (LOW → HIGH → 3 → 5 → 10 → 30 → 50 → 70 → 100%). The speed will decrease in reverse each time the [MOVE] + [-] keys are pressed.

The currently set speed will appear on the upper right of the screen.

(2) Selecting the program No.

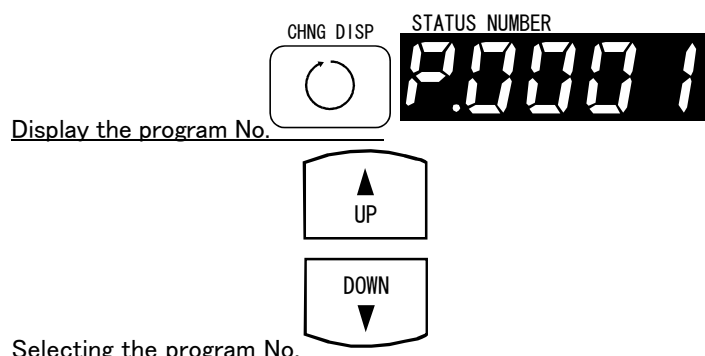
Prepare the control



T/B disable

Controller enable

Selecting the program No.



1) Set the T/B [ENABLE/DISABLE] switch to "DISABLE".

2) Set the controller [MODE] switch to "AUTO (Op.)".

3) Press the [CHNG DISP] switch and display "PROGRAM NO." on the STATUS NUMBER display.

4) When the [UP] switch is pressed, the registered program Nos. will scroll up, and then the [DOWN] switch is pressed, the program Nos. will scroll down.

Display the program No. to be used for automatic operation.

*They are not displayed if a program name consisting of five or more characters is specified. If these are selected from an external device, "P - - - -" is displayed.

(3) Starting automatic operation

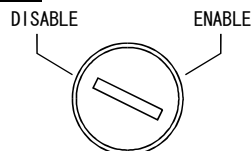


CAUTION

Before starting automatic operation, always confirm the following items. Starting automatic operation without confirming these items could lead to property damage or physical injury.

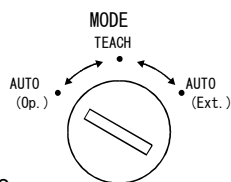
- Make sure that there are no operators near the robot.
- Make sure that the safety fence is locked, and operators cannot enter unintentionally.
- Make sure that there are no unnecessary items, such as tools, inside the robot operation range.
- Make sure that the workpiece is correctly placed at the designated position.
- Confirm that the program operates correctly with step operation.

Prepare the control



T/B disable

- 1) Set the T/B [ENABLE/DISABLE] switch to "DISABLE".



Controller enable

- 2) Set the controller [MODE] switch to "AUTO (Op.)".

Start automatic operation

Start

(Continuous operation)



End with one cycle

- 3) Automatic operation will start when the controller [START] switch is pressed. (Continuous operation)

If the [END] switch is pressed during the continuous operation, the program will stop after one cycle.



CAUTION

Before starting automatic operation, always confirm that the target program No. is selected.



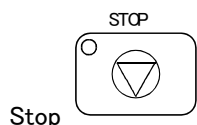
CAUTION

Take special care to the robot movements during automatic operation. If any abnormality occurs, press the [EMG. STOP] switch and immediately stop the robot.

(4) Stopping

The running program is immediately stopped, and the moving robot is decelerated to a stop.

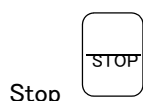
■ Operating with the controller



Stop

- 1) Press the [STOP] switch.

■ Operating with the T/B



Stop

- 1) Press the [STOP] key.



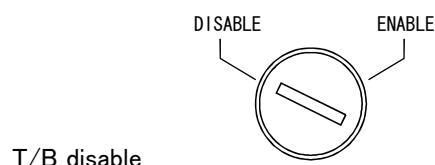
The stopping operation is always valid regardless of the operation rights.

(5) Resuming automatic operation from stopped state

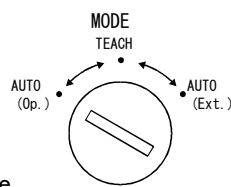
CAUTION

Before starting automatic operation, always confirm the following items. Starting automatic operation without confirming these items could lead to property damage or physical injury.

- Make sure that there are no operators near the robot.
- Make sure that the safety fence is locked, and operators cannot enter unintentionally.
- Make sure that there are no unnecessary items, such as tools, inside the robot operation range.
- Make sure that the workpiece is correctly placed at the designated position.
- Confirm that the program operates correctly with step operation.



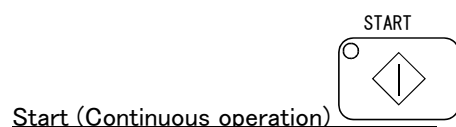
1) Set the T/B [ENABLE/DISABLE] switch to "DISABLE".



2) Set the controller [MODE] switch to "AUTO (Op.)".

Controller enable

Start automatic operation



3) Automatic operation will start when the controller [START] switch is pressed. (Continuous operation)

The continuous operation/one cycle operation state will be the same as before the operation was stopped.

CAUTION

Before starting automatic operation, always confirm that the target program No. is selected.

CAUTION

Take special care to the robot movements during automatic operation. If any abnormality occurs, press the [EMG. STOP] switch and immediately stop the robot.

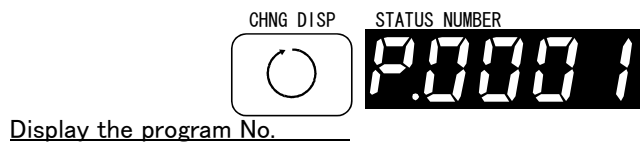
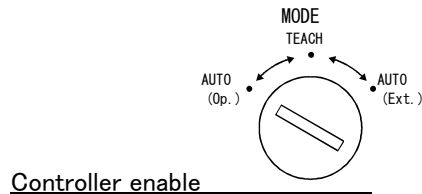
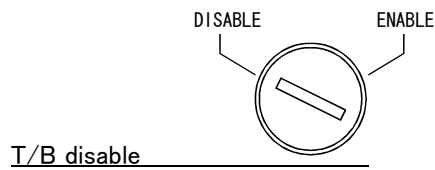
CAUTION

Do not turn the controller's power off during the automatic operation. The memory in the controller may be malfunctioned and programs may be destroyed. Use the emergency stop to stop the robot immediately.

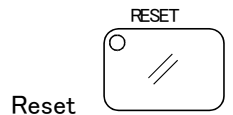
(6) Resetting the program

The program's stopped state is canceled, and the execution line is returned to the head.

■ Operating with the controller



Execute of program reset



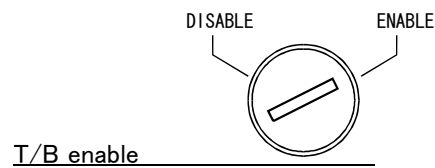
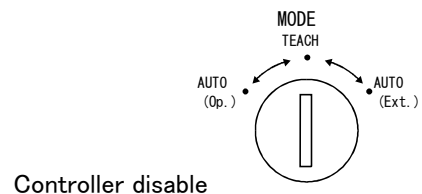
1) Set the T/B [ENABLE/DISABLE] switch to "DISABLE".

2) Set the controller [MODE] switch to "AUTO (Op.)".

3) Press the controller [CHG DISP] switch, and display the program No.

4) Press the controller [RESET] switch.
The STOP lamp will turn OFF, and the program's stopped state will be canceled.

■ Operating with the T/B



Execute of program reset



1) Set the [Mode selection switch] on the front of the controller to "TEACH".

2) Set the T/B [ENABLE/DISABLE] switch to "ENABLE".

3) Press the [EXE] key while holding down the [ERROR RESET] key. The execution line will return to the head, and the program will be reset.

◆◆◆ Valid only while program is stopped ◆◆◆

The program cannot be reset while the program is running. Always carry out this step while the program is stopped.

When resetting the program from the controller operation panel, display the "program No." on the STATUS NUMBER display, and then reset.

◆◆◆ STOP lamp turns OFF ◆◆◆

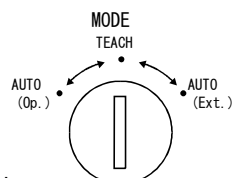
The STOP lamp will turn OFF when the program is reset.

3.8 Turning the servo ON/OFF

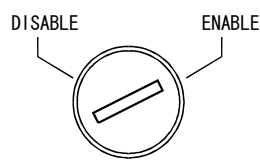
For safety purposes, the servo power can be turned ON during the teaching mode only while the deadman switch on the back of the T/B is lightly pressed. Carry out this operation with the T/B while lightly pressing the deadman switch.

■ Turning servo ON with T/B [SVO ON] key

Prepare the T/B



Controller disable



T/B enable

Execute servo ON



Servo ON operation

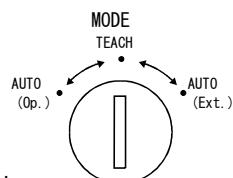
1) Set the [Mode selection switch] on the front of the controller to "TEACH".

2) Set the T/B [ENABLE/DISABLE] switch to "ENABLE".

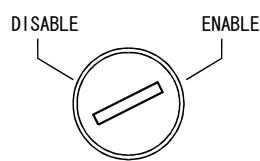
3) The servo will turn ON when the [SVO ON] key ([STEP/MOVE] key) is pressed.

■ Turning servo ON/OFF with T/B Servo screen

Prepare the T/B



Controller disable



T/B enable

<MENU>
1.TEACH 2.RUN
3.FILE 4.MONI
5.MAINT 6.SET



<RUN>
1.SERVO 2.CHECK

Select the RUN screen

-A
(J4)
2 GHI

<RUN>
1.SERVO 2.CHECK



<SERVO>
SERVO ON()
0:OFF 1:ON

Select the SERVO screen

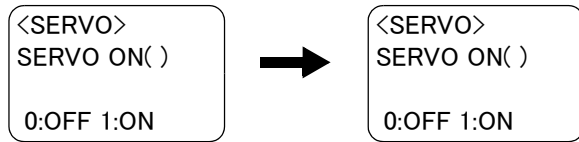
-B
(J5)
1 DEF

1) Set the [Mode selection switch] on the front of the controller to "TEACH".

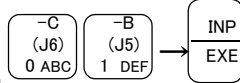
2) Set the T/B [ENABLE/DISABLE] switch to "ENABLE".

3) Press the [2] key, and select the operation screen.

4) Press the [1] key, and select the servo screen.

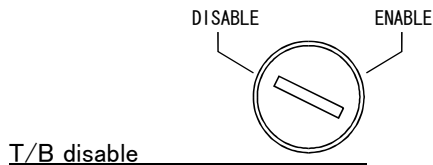


Select the SERVO screen

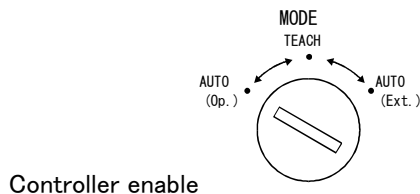


■ Operating with the controller

Prepare the controller

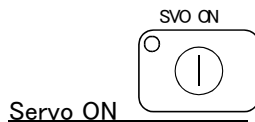


T/B disable



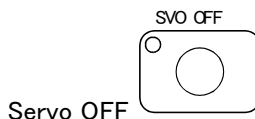
Controller enable

Execute servo ON



Servo ON

Execute servo OFF



Servo OFF

- 5) To turn ON/OFF the servo, press the [0] key when the servo is OFF or the [1] key when the servo is ON while lightly holding the deadman switch, and then press the [INP] key.
(It is also possible to turn the servo on by pressing the [STEP/MOVE] key.)

- 1) Set the T/B [ENABLE/DISABLE] switch to "DIS-ABLE".

- 2) Set the controller [MODE] switch to "AUTO (Op.)".

- 3) When the [SVO ON] switch is pressed, the servo will turn ON, and the SVO ON lamp will light.

- 4) When the [SVO OFF] switch is pressed, the servo will turn OFF, and the SVO OFF lamp will light.

◆◆◆ Brakes will activate ◆◆◆

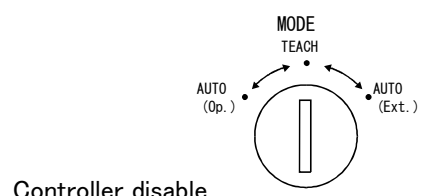
The brakes will automatically activate when the servo is turned OFF. Depending on the type of robot, some axes may not have brakes.

3.9 Error reset operation

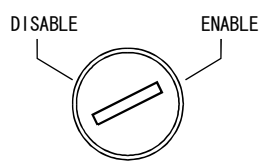
■ Error reset operation from the operation panel

Set the key switch to the AUTO (OP) position, and then press the reset key on the operation panel.

■ Error reset operation from the T/B



Controller disable



T/B enable

Cancel errors

Error reset



1) Set the [Mode selection switch] on the front of the controller to "TEACH".

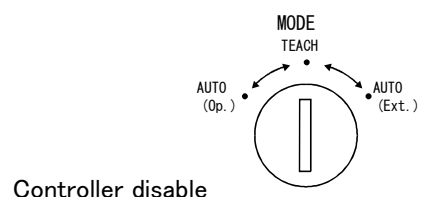
2) Set the T/B [ENABLE/DISABLE] switch to "ENABLE".

3) Press the [ERROR RESET] key.

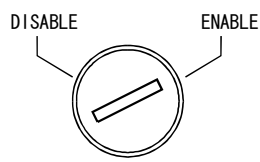
3.10 Operation to Temporarily Reset an Error that Cannot Be Canceled

Depending on the type of robot, errors that cannot be cancelled may occur when axis coordinates are outside the movement range, etc. In this case, it is not possible to turn the servo on and perform jog operations with the normal operations. The following procedure can be used to cancel such errors temporarily. For instance, if the axes are outside the movement range, perform a jog operation to adjust the axes while the error is canceled temporarily.

■ Operation to cancel errors temporarily from the T/B



Controller disable



T/B enable

Cancel errors temporarily



Error reset

1) Set the [Mode selection switch] on the front of the controller to "TEACH".

2) Set the T/B [ENABLE/DISABLE] switch to "ENABLE".

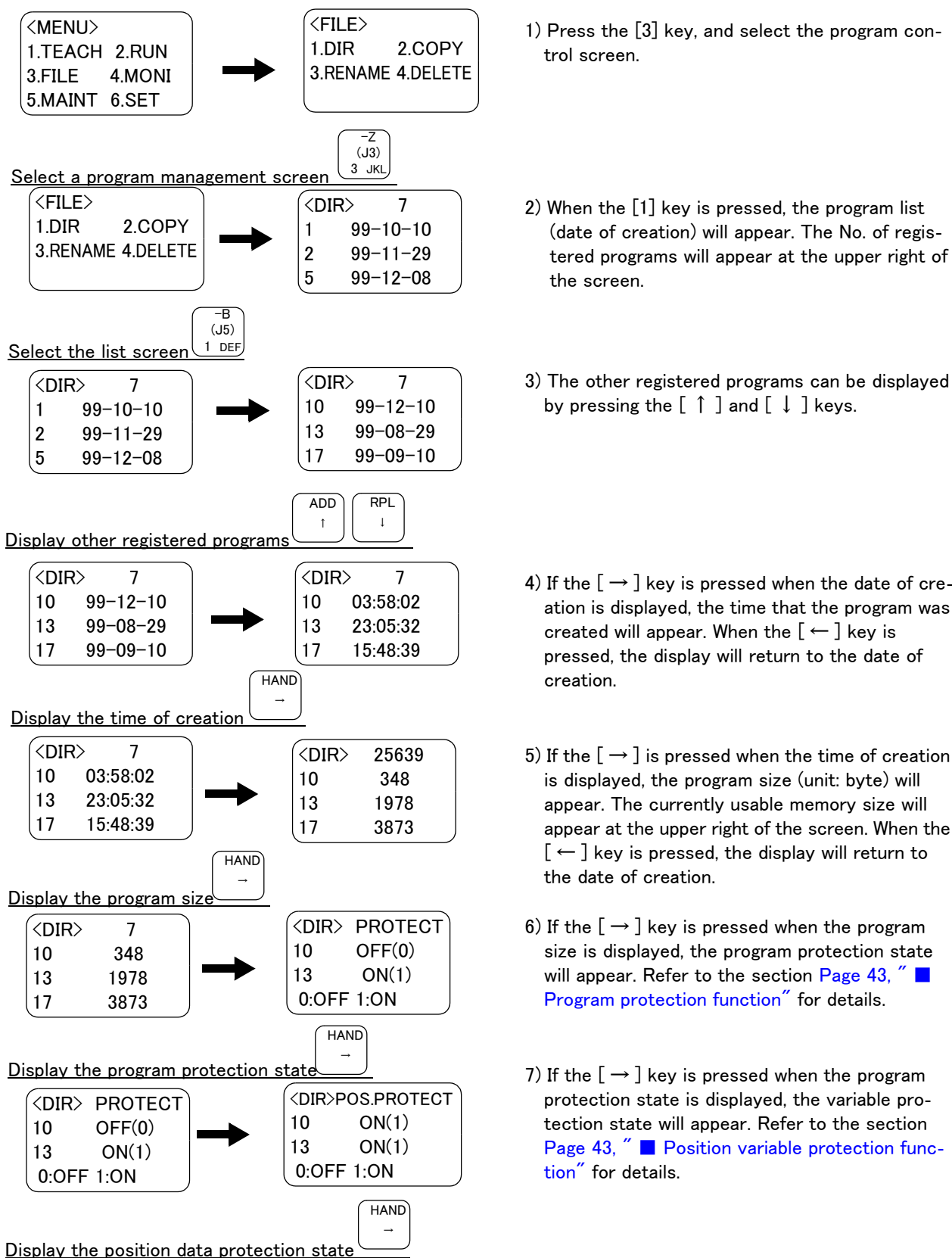
3) Hold the deadman switch lightly, hold down the [SVO] key and keep on pressing the [ERROR RESET] key.

The operation above will reset errors temporarily. Do not release the key; if it is released the error occurs again. Perform a jog operation as well while keeping the [ERROR RESET] key pressed.

3.11 Operating the program control screen

(1) Program list display

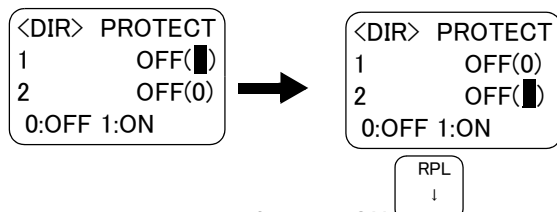
This functions allows the status of the programs registered in the controller to be confirmed.



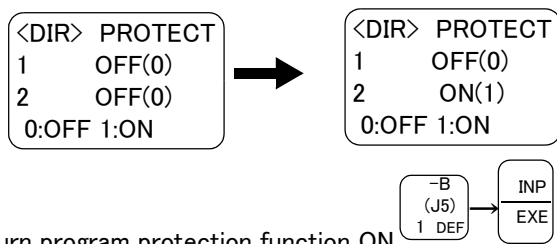
(2) Program protection function

■ Program protection function

This function protects the program from being deleted or changed inadvertently.



Turn program protection function ON



Turn program protection function ON

1) Display the "program list display" explained on the previous page, and then display the program protection state. Move the cursor to the program targeted for protection with the [↓] and [↑] keys.

2) The program protection function for that program will turn ON or OFF when the [INP] key is pressed after the following key:
Program protection function ON ... [1] key
Program protection function OFF ... [0] key

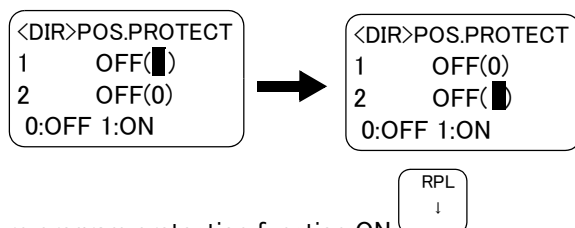
◆◆◆ Program protection ◆◆◆

This function protects the program from inadvertent program deletion, renaming and command changing.

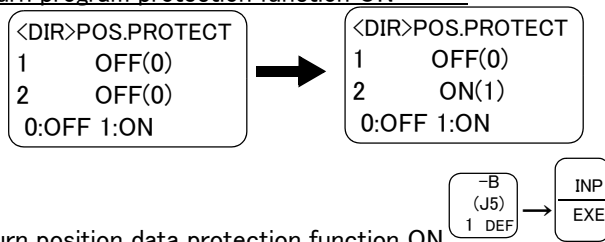
- The protection function is not copied with the copy operation.
- The protected information is ignored during the initialization process.

■ Position variable protection function

This function protects the program from inadvertent variable deletion and changing.



Turn program protection function ON



Turn position data protection function ON

1) Display the "program list display" explained on the previous page, and then display the variable protection state. Move the cursor to the program targeted for protection with the [↓] and [↑] keys.

2) The variable protection function for that program will turn ON or OFF when the [INP] key is pressed after the following key:
Variable protection function ON ... [1] key
Variable protection function OFF ... [0] key

◆◆◆ Variable protection ◆◆◆

This function protects the variable from inadvertent position data registration or changing, and from substitution to each variable during incorrect program execution.

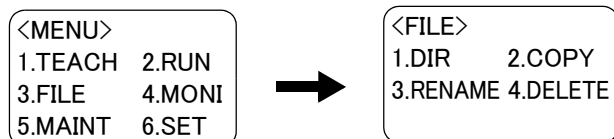
- The protection function is not copied with the copy operation.
- The protected information is ignored during the initialization process.

(3) Copying programs

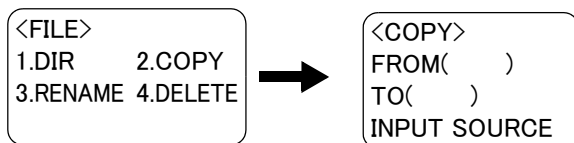
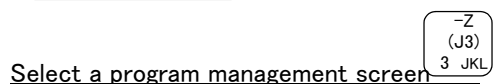
This function copies a registered program to another program.

If an existing program name is designated as the copy destination program, an error will occur.

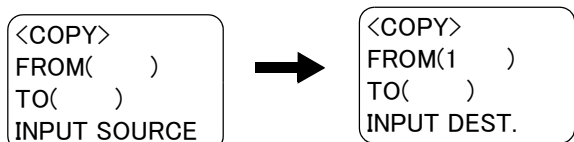
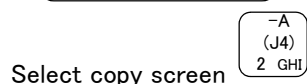
An example for copying program 1 to program 5 is shown below.



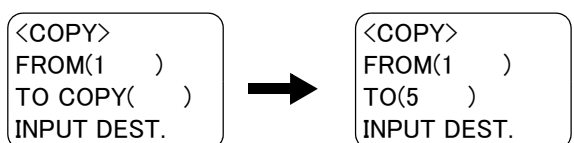
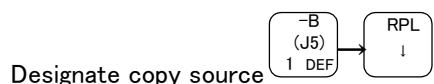
1) Press the [3] key, and select the program control screen.



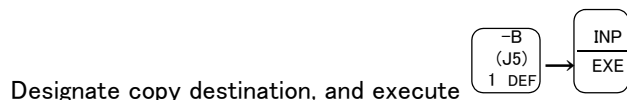
2) Press the [2] key, and select the program copy screen.



3) Press the [1] key and then the [] key to move the cursor to the copy destination program name input line.



4) When the [INP] key is pressed after pressing [5], the program will be copied. The program control screen will appear after execution.



◆◆◆ Protected information is not copied ◆◆◆

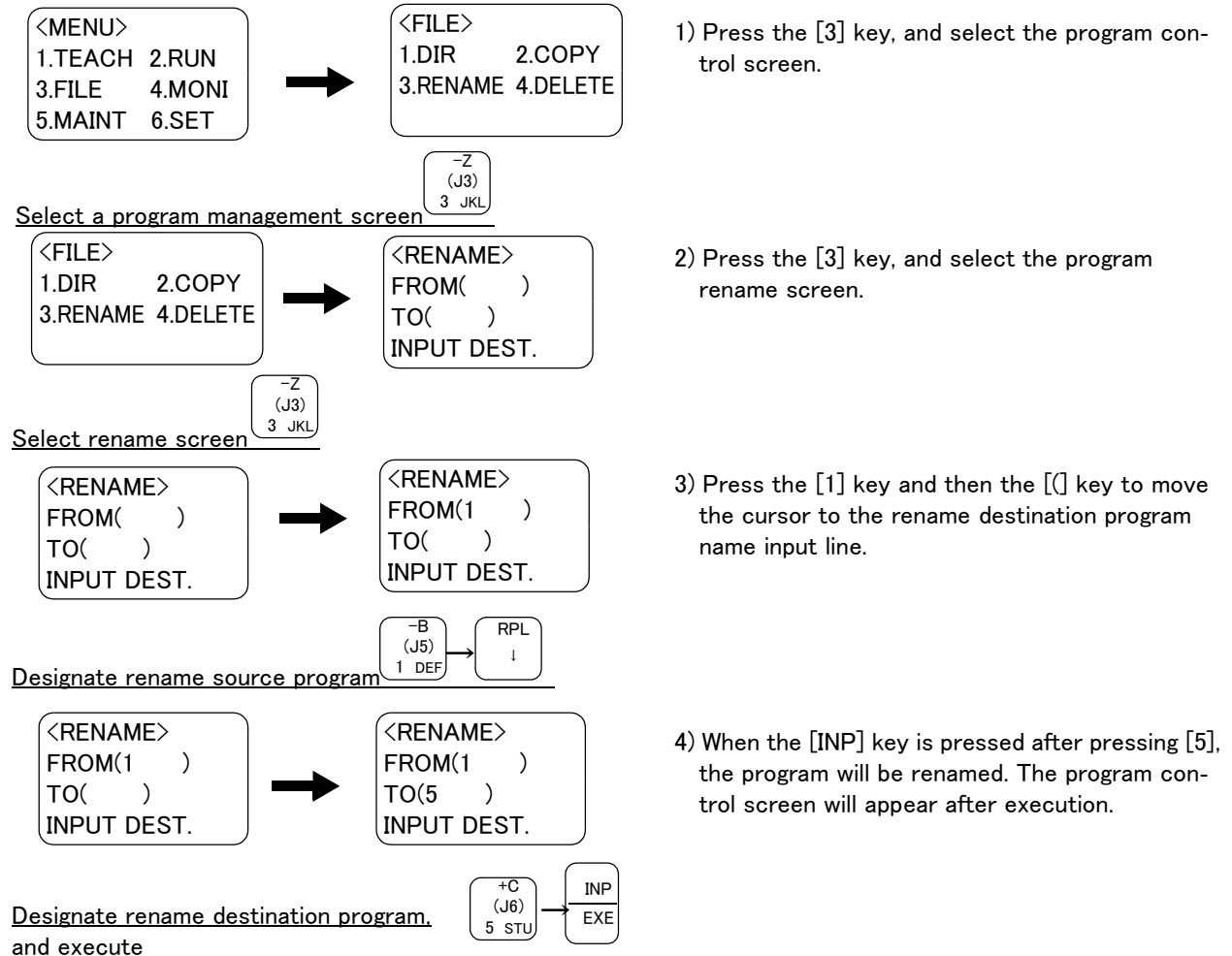
The program protection information and variable protection information is not copied with the copy operation. Reset this information as necessary.

(4) Changing the program name (Renaming)

This function renames a registered program's name.

If an existing program name is designated as the rename destination program, an error will occur.

An example for renaming program 1 to program 5 is shown below.



◆◆◆ Renaming is not possible when protected ◆◆◆

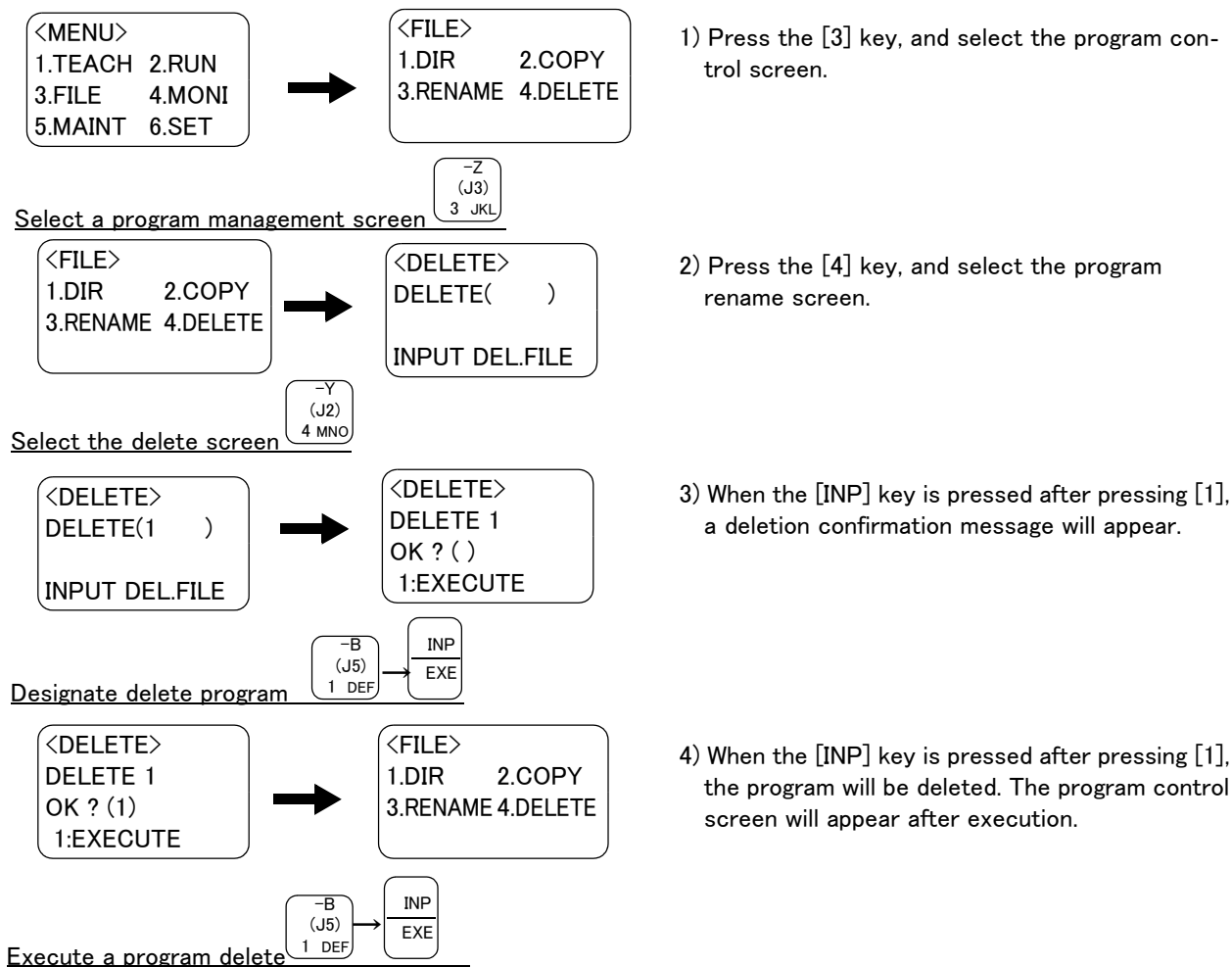
If either the program protection or variable protection is ON, the program cannot be renamed.

In this case, turn the protection OFF before renaming.

(5) Deleting a program

This function deletes a registered program.

The case for deleting program 1 is shown below.



◆◆◆ Deletion not possible when protected ◆◆◆

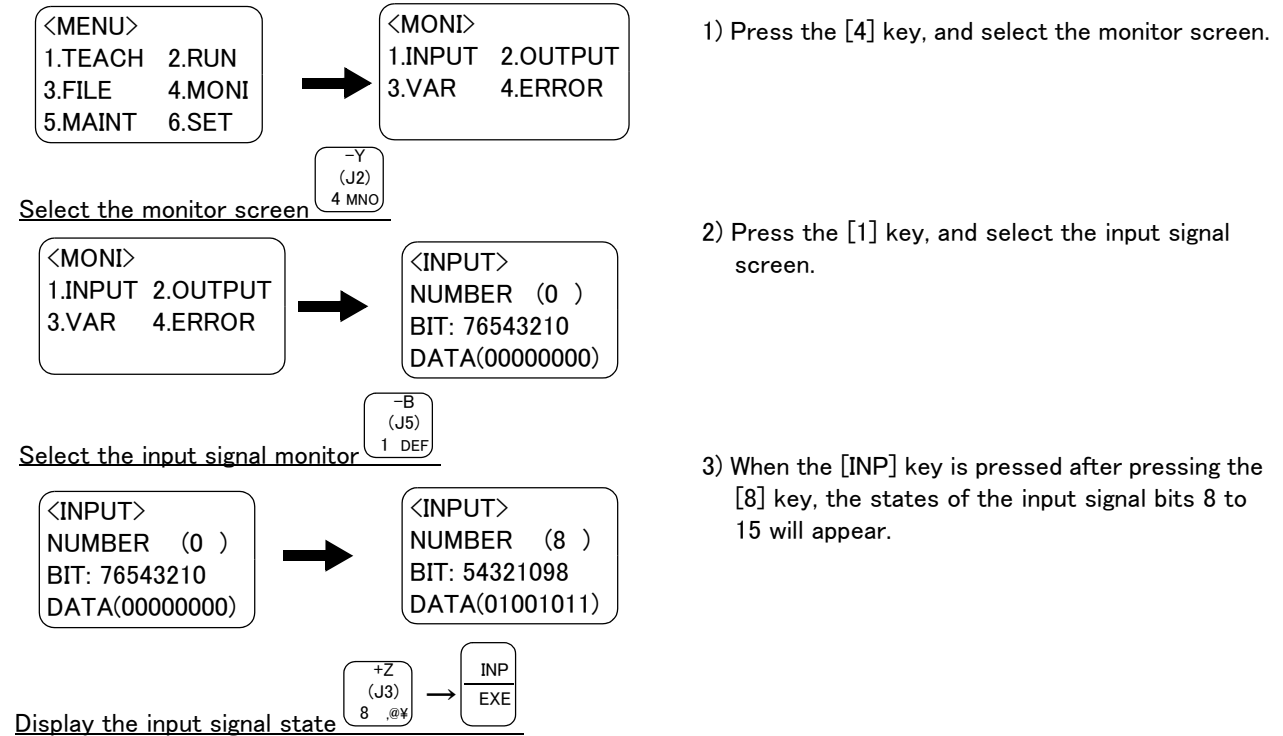
If either the program protection or variable protection is ON, the program cannot be deleted.
In this case, turn the protection OFF before deleted.

3.12 Operating the monitor screen

(1) Input signal monitor

This function allows the state of the input signals from an external source to be confirmed at real-time.

An example for confirming the states of the input signal bits 8 to 15 is shown below.



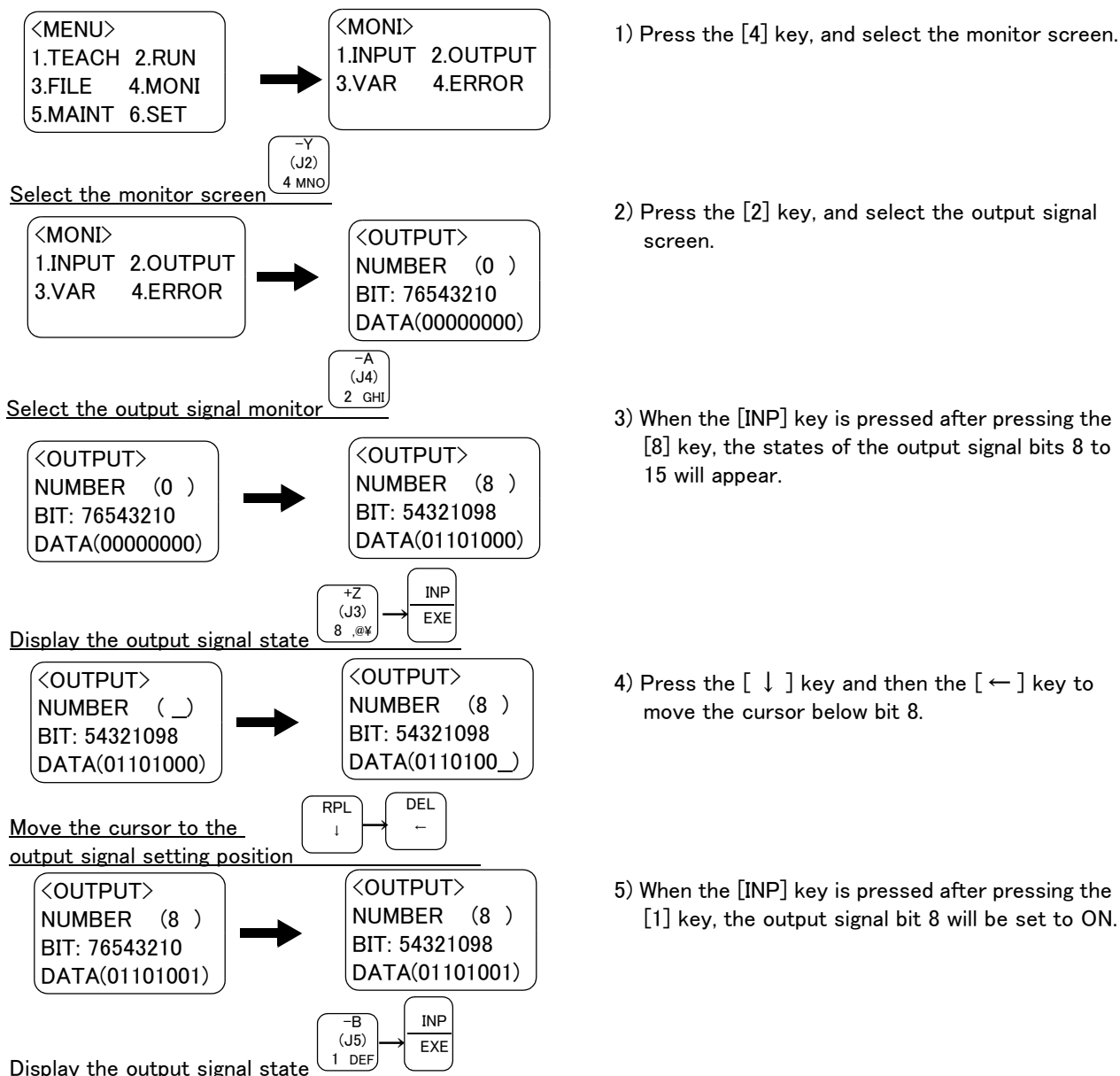
◆◆◆ Operation rights not required ◆◆◆

This operation can be carried out even if the T/B does not have the operation rights.

(2) Output signal monitor

This function allows the state of the signals output to an external source to be confirmed and set.

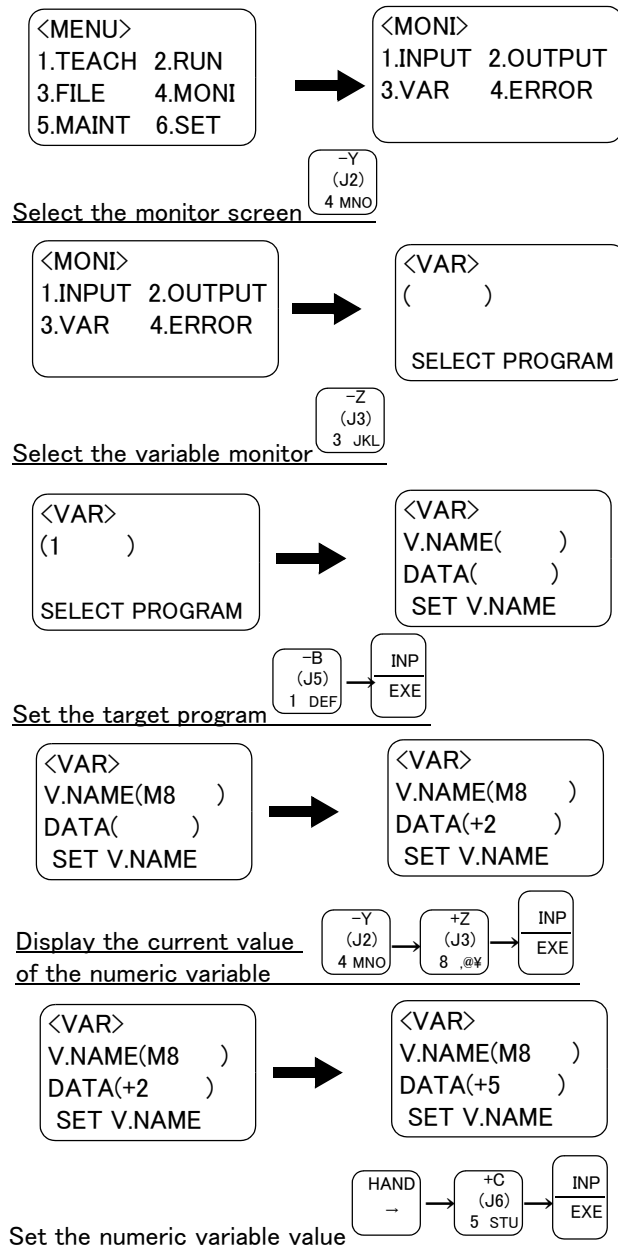
An example for turning the input signal bit 8 ON is shown below.



(3) Variable monitor

This function allows the details of the numeric variables used in the program to be displayed and changed.

An example for changing the program 1 numeric variable M8 value from 2 to 5 is shown below.



1) Press the [4] key, and select the monitor screen.

2) When the [3] key is pressed, the program selection screen for displaying the variables will appear. Input the name of the program for monitoring the variables.

3) When the [INP] key is pressed after pressing [1], the variable display and setting screen will appear.

4) When the [-Y/MNO] key, [8] key and then [INP] key are pressed, the current value +2 of the program 1 numeric variable M8 will appear.

5) When the [→] key, [5] key and then [INP] key are pressed, the current value +2 of the program 1 numeric variable M8 will change to +5.

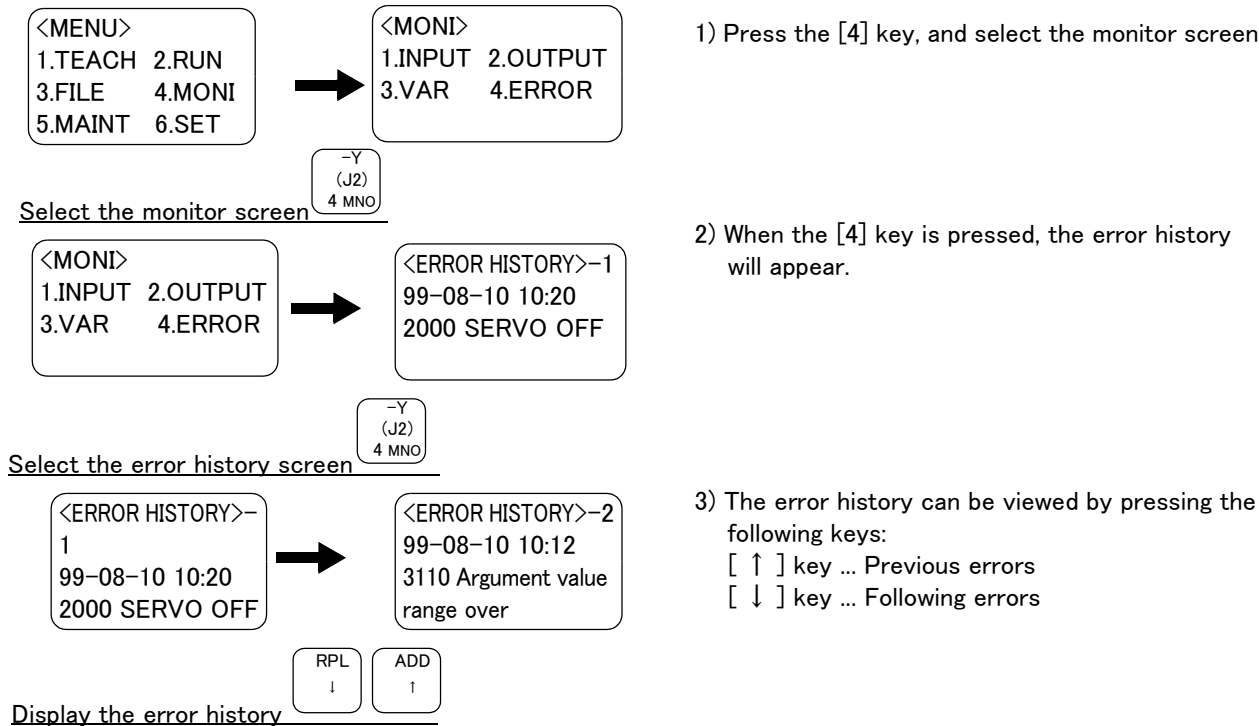
◆◆◆ Operation rights not required for only display ◆◆◆

If this function is used to only display the values, the T/B does not require the operation rights.

The robot status variables cannot be directly monitored. In this case, the variable must be substituted in the program variables once, and then monitored with the program variable.

(4) Error history

This function displays the history of the errors that have occurred in the robot. Use this as reference when trouble occurs.



◆◆◆ Operation rights not required ◆◆◆

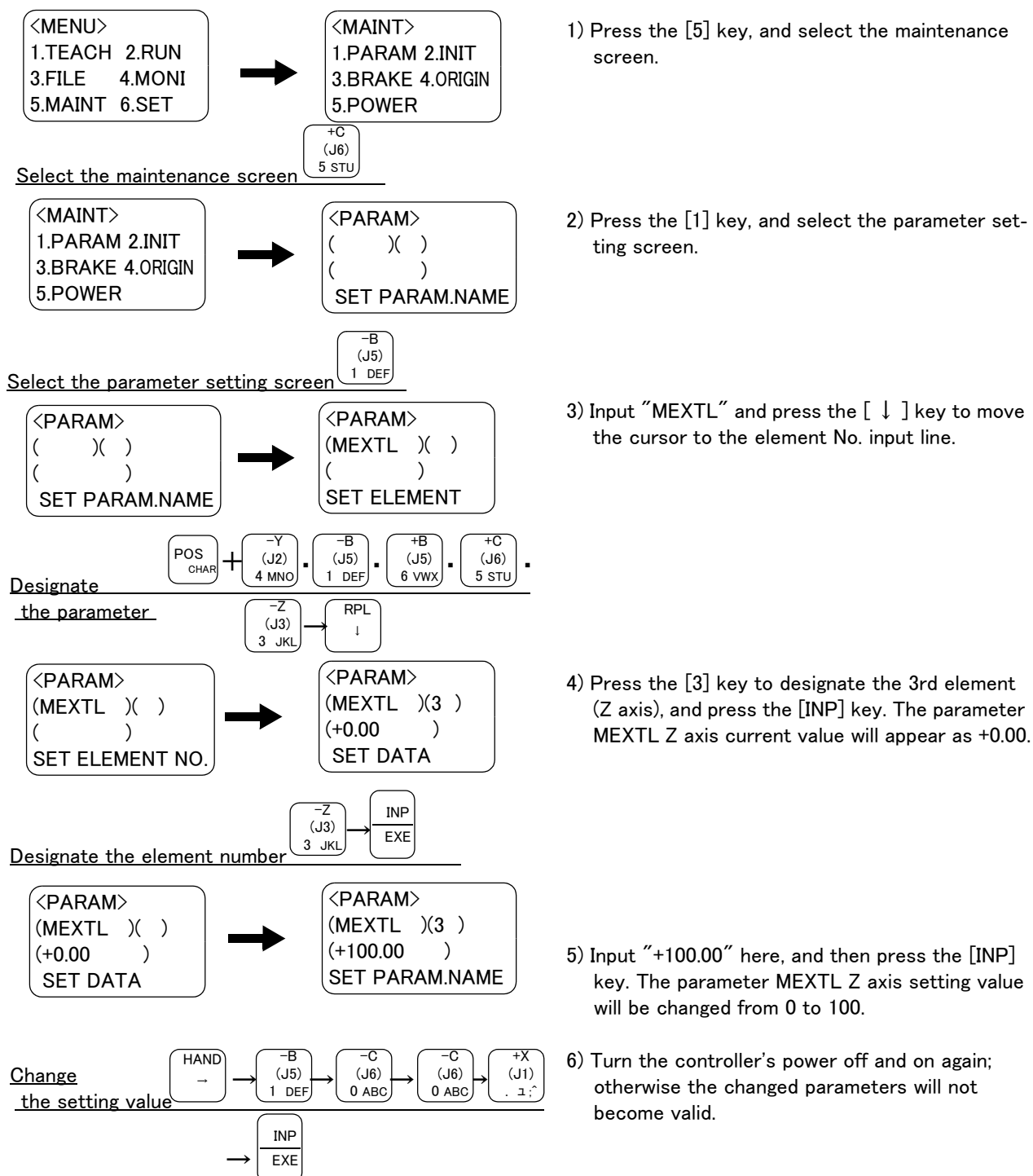
This operation can be carried out even if the T/B does not have the operation rights.

3.13 Operation of maintenance screen

(1) Setting the parameters

The parallel I/O designated input/output settings and settings for the tool length, etc., are registered as parameters. The robot moves based on the values set in each parameter. This function allows each parameter setting value to be displayed and registered.

An example of changing the parameter "MEXTL (tool data)" Z axis (3rd element) setting value from 0 to 100mm is shown below.



◆◆◆ Power must be turned ON again ◆◆◆

The changed parameter will be validated only after the controller power has been turned OFF and ON once.

(2) Initializing the program

This function erases all programs.

```
<MENU>
1.TEACH 2.RUN
3.FILE 4.MONI
5.MAINT 6.SET
```



```
<MAINT>
1.PARAM 2.INIT
3.BRAKE 4.ORIGIN
5.POWER
```

```
+C
(J6)
5 STU
```

Select the maintenance screen

```
<MAINT>
1.PARAM 2.INIT
3.BRAKE 4.ORIGIN
5.POWER
```



```
<INIT>
INIT ( )
1.PROGRAM 2.BATT.
```

```
-A
(J4)
2 GHI
```

Select the initialization screen

```
<INIT>
INIT (1)
1.PROGRAM 2.BATT.
```



```
<INIT>
PROGRAM
OK ? ( )
1.EXECUTE
```

```
-B
(J5)
1 DEF
```

```
INP
EXE
```

Select the program initialization screen

```
<INIT>
PROGRAM
OK ? (1)
1.EXECUTE
```



```
<INIT>
INIT ( )
1.PROGRAM 2.BATT.
```

```
-B
(J5)
1 DEF
```

```
INP
EXE
```

Execute program initialization

1) Press the [5] key, and select the maintenance screen.

2) Press the [2] key, and select the initialization screen.

3) Press [1] and then [INP] to select the program initialization screen.

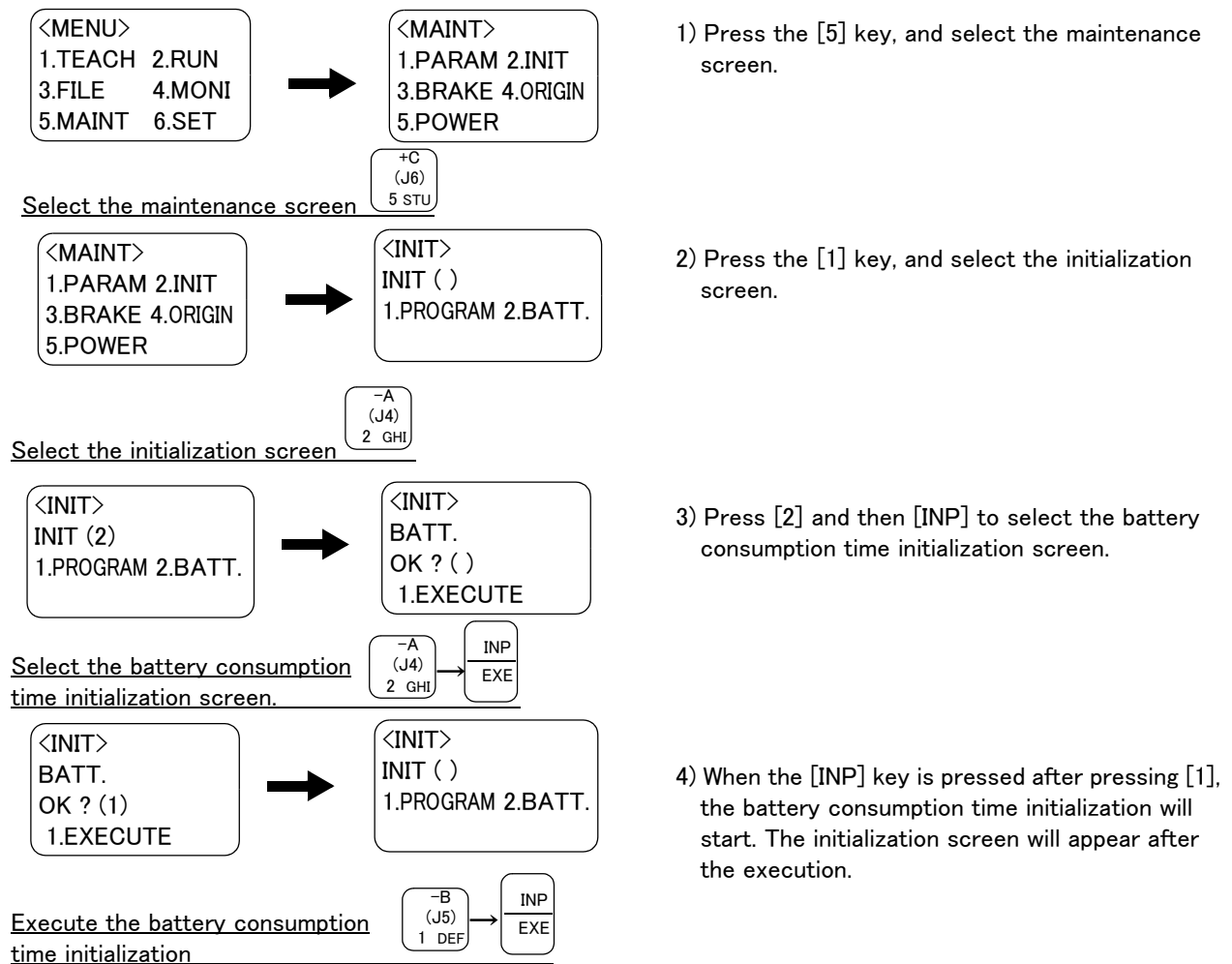
4) When the [INP] key is pressed after pressing [1], the program initialization will start. The initialization screen will appear after the execution.

◆◆◆ Executed even when protected ◆◆◆

The program will be initialized even if the program protection or variable protection is set to ON.

(3) Initializing the battery consumption time

The usage time of the battery built into the controller and robot arm is calculated to indicate battery replacement on the caution message screen when the battery is spent. Thus, always initialize this setting after replacing the battery.



◆◆◆ Always initialize after battery replacement ◆◆◆

The battery usage time is calculated in the controller, and a caution message is displayed when the battery is spent. Always initialize the battery consumption time after replacing the battery to ensure that the caution message is displayed correctly.

If this initialization is carried out when the battery has not been replaced, the display timing of the caution message will deviate. Thus, carry this step out only when the battery has been replaced.

(4) Releasing the brakes

This function releases the servomotor brakes when the servo is OFF. Refer to [Page 39, "3.8 Turning the servo ON/OFF"](#) for details on turning the servo OFF.

This function is used to directly move the robot arm by hand, etc.

**CAUTION**

Due to the robot configuration, when the brakes are released, the robot arm will drop with its own weight depending on the released axis.
Always assign an operator other than the T/B operator to prevent the arm from dropping. This operation must be carried out with the T/B operator giving signals.
Refer to [Table 3-5](#) and accurately designate the axis for which the brakes are to be released.

Note that the minimum axis unit for which the brakes can be released at once will differ according to the model of the robot in use. [Table 3-5](#) shows the minimum axis unit for which the brake release operation can be performed at once for each model.

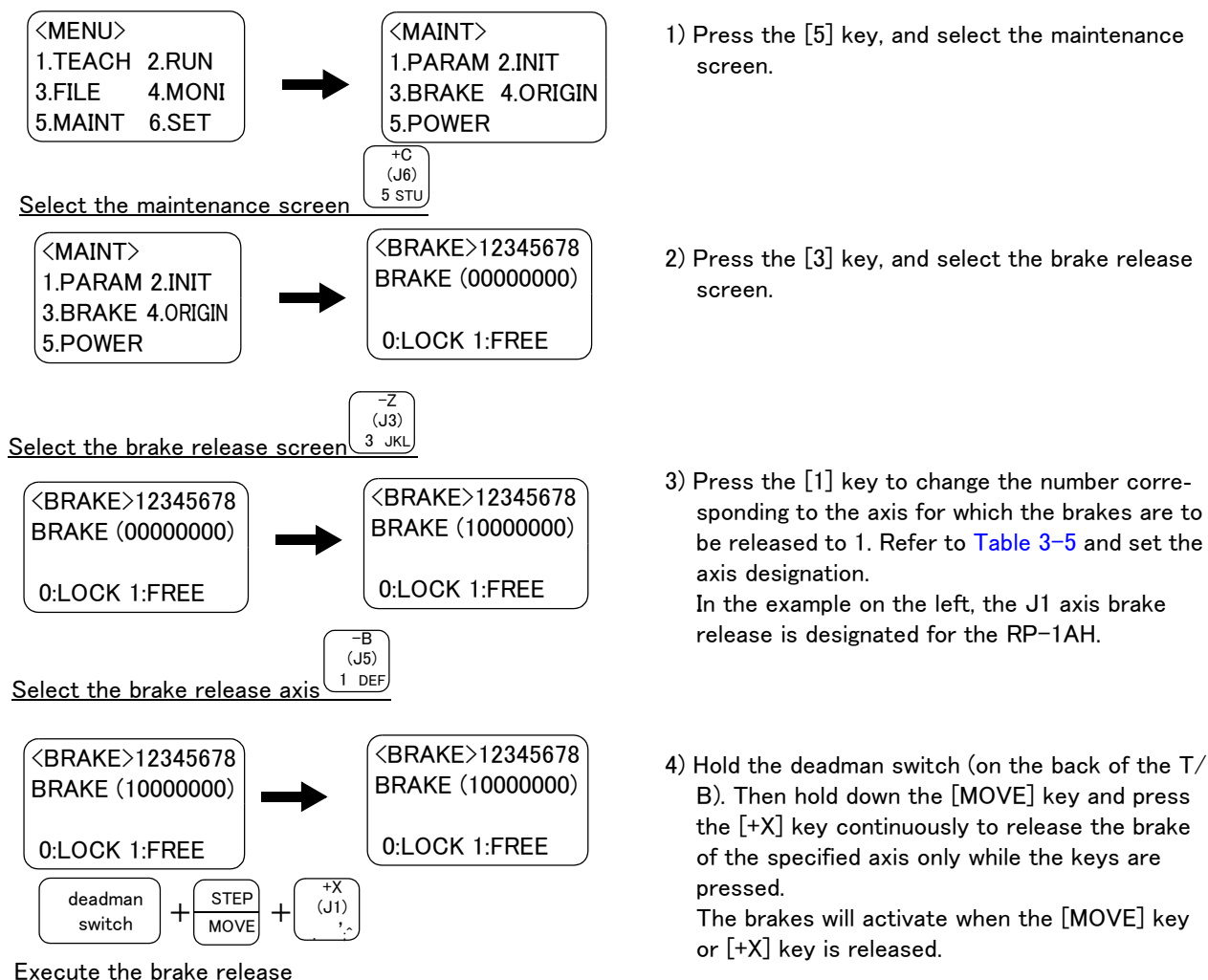
The minimum axis units can be combined to release the brakes form multiple axes at the same time.

Table 3-5 : Brake release axis unit per mode

Model	Axes								Remarks
	1	2	3	4	5	6	7	8	
RP-1AH,3AH,5AH RH-5AH35/45/55 RH-10AH55/70/85 RH-15AH85 RH-15UHC RV-100THL	■	■	■	■					The third axis (the Z-axis) of RH-10AH55/70/85, RH-15AH85, and RH-15UHC is in the intermittent brake mode. When the brake release operation is performed, the brake operates in such a way that release and application are repeated, the arm is thus prevented from dropping suddenly.
RV-1A,4A,3AL	■	■	■	■	■	■			
RV-2AJ,5AJ,4AJL	■	■	■		■	■			
RH-1000GHLC,RC-1000GHWLC RC-1000GHWDC	■ = ■	■	■	■					Axes 1 and 2 are paired. It is not possible to release the brake if both axes are not set to 1 simultaneously.
RH-1000GHLC-RL	■ = ■	■	■	■				■	
RH-1000GJLC,RH-1500GJC	■ = ■	■	■	■ = ■					Axes 1 and 2, and 4 and 5 are paired. It is not possible to release the brake if both axes are not set to 1 simultaneously.
RH-1000GJLC-RL,RH-1500GJC-RL	■ = ■	■	■	■ = ■				■	
RH-1500GC	■ = ■	■	■	■ = ■	■				
RH-1500GC-RL,RH1500GVC-RL	■ = ■	■	■	■ = ■	■ = ■	■ = ■			Axes 1 and 2, and 4 and 5, and 6 and 8 are paired. It is not possible to release the brake if both axes are not set to 1 simultaneously.
RC-1000GHWLC-RL,RC-1000GHWDC-RL	■ = ■	■	■	■ = ■	■ = ■	■ = ■			Axes 1 and 2, and 4 and 8 are paired. It is not possible to release the brake if both axes are not set to 1 simultaneously.
RV-100TH,RV-150TH,RV-150THL RS-30AG,RH-1000GHDC	■ = ■	■	■ = ■						Axes 1 and 2, and 3 and 4 are paired. It is not possible to release the brake if both axes are not set to 1 simultaneously.
RH-1000GHDC-RL	■ = ■	■	■ = ■					■	
RH-1000GJDC	■ = ■	■	■ = ■	■					
RH-1000GJDC-RL	■ = ■	■	■ = ■	■				■	
RV-20A	■ = ■	■	■ = ■	■ = ■					Axes 1 and 2, and 3 and 4, and 5 and 6 are paired. It is not possible to release the brake if both axes are not set to 1 simultaneously.

Note) The symbol [■ = ■] means that it is necessary to set two axes at the same time to release the brake.
The symbol [■] means that it is possible to release the brake for an independent axis.

The operation method is shown below. The following operations are carried out while lightly pressing the deadman switch on the T/B.



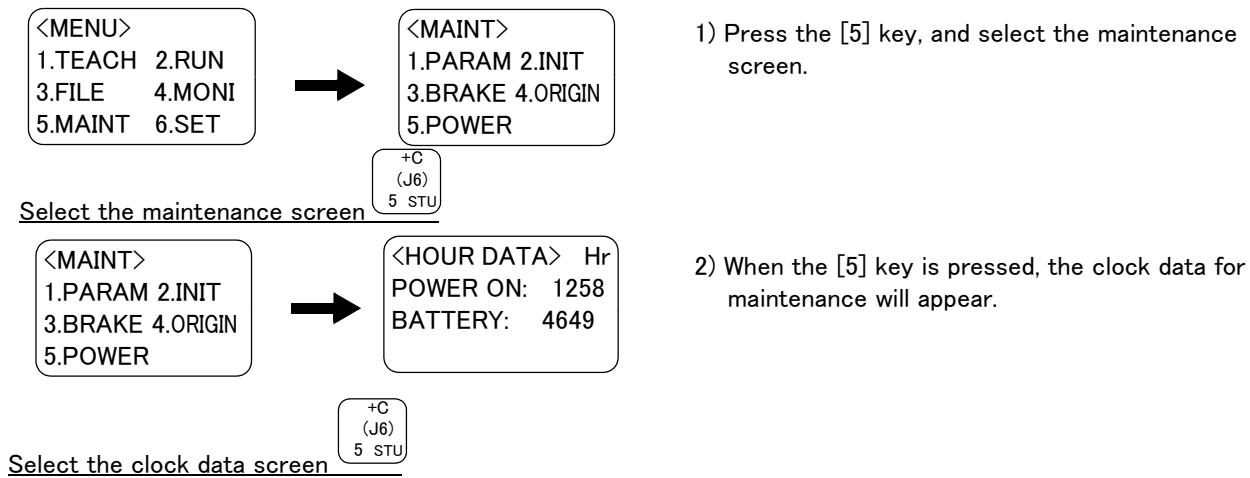
(5) Setting the origin

If the origin position has been lost or deviated when the parameters are lost or due to robot interference, etc., the robot origin must be set again using this function.

Refer to the separate manual: "Robot arm setup & maintenance" for details on the operation.

(6) Displaying the clock data for maintenance

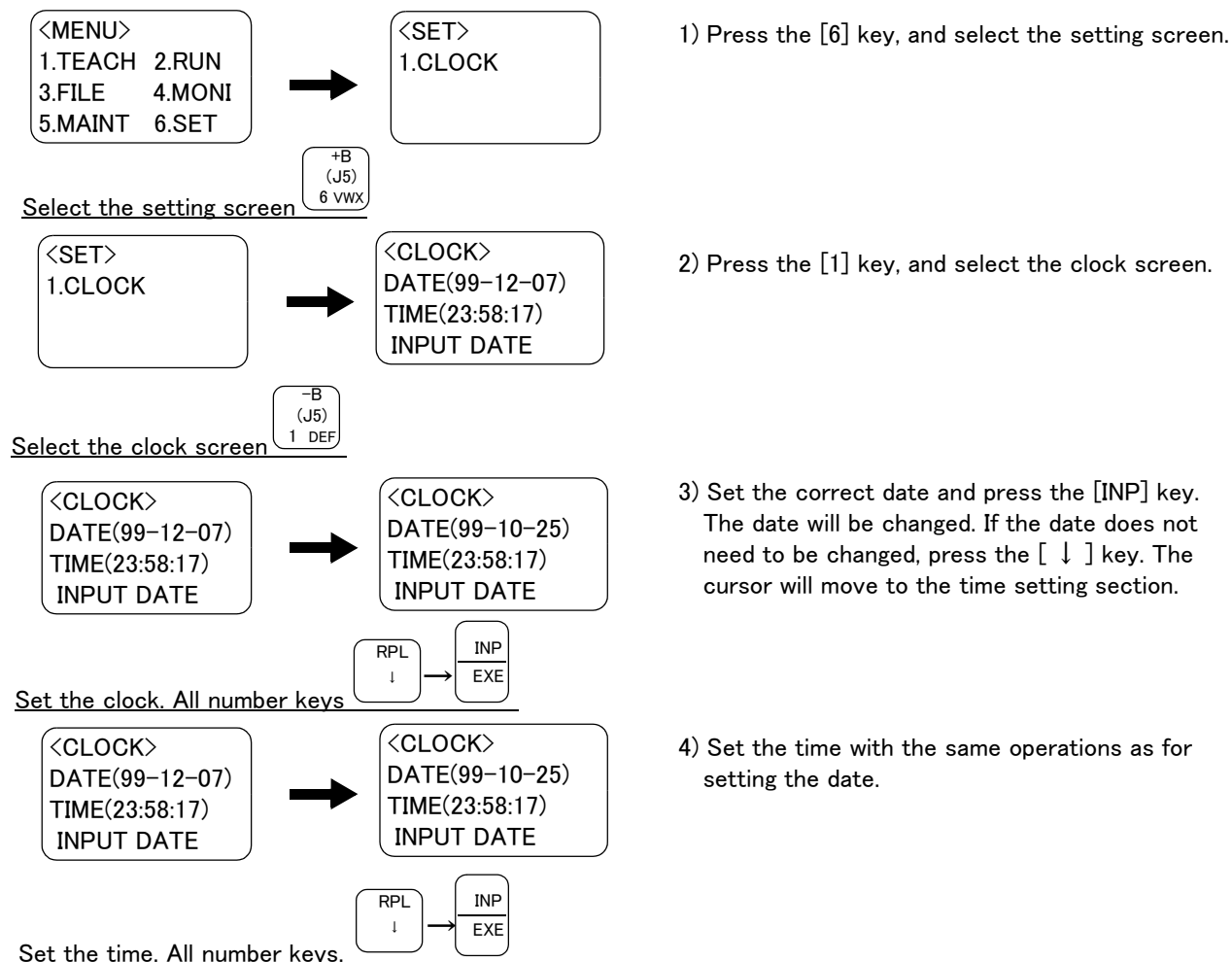
The controller’s cumulative power ON time and remaining battery time are displayed.(Unit:hour)



3.14 Operation of the setting screen

(1) Setting the time

A clock function, used when registering the program, and displaying the change time or error time, etc., is provided in the controller. If the times are deviated from the current date and time, change the date and time to the correct values.



4 MELFA-BASIC IV

In this chapter, the functions and the detailed language specification of the programming language "MELFA-BASIC IV" are explained.

4.1 MELFA-BASIC IV functions

The outline of the programming language "MELFA-BASIC IV" is explained in this section. The basic movement of the robot, signal input/output, and conditional branching methods are described.

Table 4-1 : List of items described

	Item	Details	Related instructions, etc.
1	4.1.1 Robot operation control	(1) Joint interpolation movement	MOV
2		(2) Linear interpolation movement	MVS
3		(3) Circular interpolation movement	MVR, MVR2, MVR3, MVC
4		(4) Continuous movement	CNT
5		(5) Acceleration/deceleration time and speed control	ACCEL, OADL
6		(6) Confirming that the target position is reached	FINE, MOV and DLY
7		(7) High path accuracy control	PREC
8		(8) Hand and tool control	HOPEN, HCLOSE, TOOL
9	4.1.2 Pallet operation	-----	DEF PLT, PLT
10	4.1.3 Program control	(1) Unconditional branching, conditional branching, waiting	GOTO, IF THEN ELSE, WAIT, etc
11		(2) Repetition	FOR NEXT, WHILE WEND
12		(3) Interrupt	DEF ACT, ACT
13		(4) Subroutine	GOSUB, CALLP, ON GOSUB, etc
14		(5) Timer	DLY
15		(6) Stopping	END(Pause for one cycle), HLT
16	4.1.4 Inputting and outputting external signals	(1) Input signals	M_IN, M_INB, M_INW, etc
17		(2) Output signals	M_OUT, M_OUTB, M_OUTW, etc
18	4.1.5 Communication	-----	OPEN, CLOSE, PRINT, INPUT, etc
19	4.1.6 Expressions and operations	(1) List of operator	+, -, *, /, <>, <, >, etc
20		(2) Relative calculation of position data (multiplication)	P1 * P2
21		(3) Relative calculation of position data (Addition)	P1 + P2
22	4.1.7 Appended statement	-----	WTH, WTHIF

For the detailed description of each instruction, please refer to [Page 111, "4.11 Detailed explanation of command words"](#).

4.1.1 Robot operation control

(1) Joint interpolation movement

The robot moves with joint axis unit interpolation to the designated position. (The robot interpolates with a joint axis unit, so the end path is irrelevant.)

■ Command word

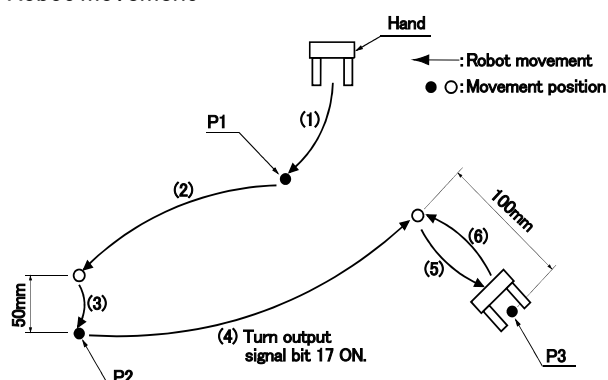
Command word	Explanation
MOV	The robot moves to the designated position with joint interpolation. It is possible to specify the interpolation form using the TYPE instruction. An appended statement WTH or WTHIF can be designated

■ Statement example

Statement example	Explanation
MOV P1	' Moves to P1.
MOV P1 + P2	' Moves to the position obtained by adding the P1 and P2 coordinate elements. Refer to Page 81 .
MOV P1 * P2	' Moves to the position relatively converted from P1 to P2. Refer to Page 81 .
MOV P1, - 50 Note)	' Moves from P1 to a position retracted 50mm in the hand direction.
MOV P1 WTH M_OUT (17) = 1	' Starts movement toward P1, and simultaneously turns output signal bit 17 ON.
MOV P1 WTHIF M_IN (20) = 1, SKIP	' If the input signal bit 20 turns ON during movement to P1, the movement to P1 is stopped, and the program proceeds to the next stop.
MOV P1 TYPE 1, 0	' Specify either roundabout (or shortcut) when the operation angle of each axis exceeds 180 deg.. (Default value: Long way around)

■ Program example

• Robot movement



CAUTION

Specification of forward/backward movement of the hand

The statement examples and program examples are for a vertical 6-axis robot (e.g., RV-20A). The hand advance/retrace direction relies on the Z axis direction (+/- direction) of the tool coordinate set for each model.

Refer to the tool coordinate system shown in "Confirmation of movement" in the separate "From Robot unit setup to maintenance", and designate the correct direction.

• Program example

Program	Explanation
10 MOV P1	' (1) Moves to P1.
20 MOV P2, -50 Note)	' (2) Moves from P2 to a position retracted 50mm in the hand direction.
30 MOV P2	' (3) Moves to P2
40 MOV P3, -100 WTH M_OUT (17) = 1	' (4) Starts movement from P3 to a position retracted 100mm in the hand direction, and turns ON output signal bit 17.
50 MOV P3	' (5) Moves to P3
60 MOV P3, -100 Note)	' (6) Returns from P3 to a position retracted 100mm in the hand direction.
70 END	' Ends the program.

■ Related functions

Function	Explanation page
Designate the movement speed.....	Page 64, "(5) Acceleration/deceleration time and speed control"
Designate the acceleration/deceleration time.	Page 64, "(5) Acceleration/deceleration time and speed control"
Confirm that the target position is reached.	Page 66, "(6) Confirming that the target position is reached"
Continuously move to next position without stopping at target position.	Page 63, "(4) Continuous movement"
Move linearly.	Page 60, "(2) Linear interpolation movement"
Move while drawing a circle or arc.	Page 61, "(3) Circular interpolation movement"
Add a movement command to the process.	Page 194, " WTH"

(2) Linear interpolation movement

The end of the hand is moved with linear interpolation to the designated position.

■ Command word

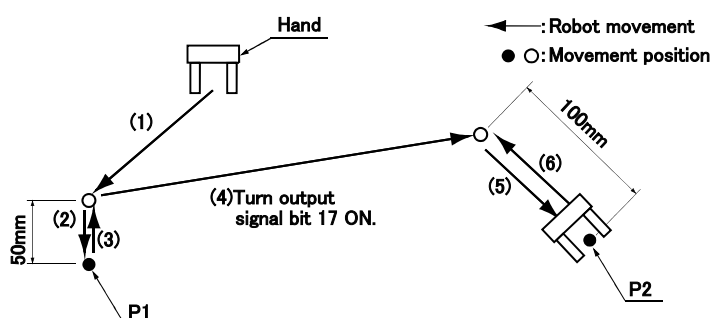
Command word	Explanation
MVS	The robot moves to the designated position with linear interpolation. It is possible to specify the interpolation form using the TYPE instruction. An appended statement WTH or WTHIF can be designated.

■ Statement example

Statement example	Explanation
MVS P1	' Moves to P1
MVS P1 + P2	' Moves to the position obtained by adding the P1 and P2 coordinate elements. Refer to Page 81 .
MVS P1 * P2	' Moves to the position relatively converted from P1 to P2.
MVS P1, - 50 Note)	' Moves from P1 to a position retracted 50mm in the hand direction.
MVS , - 50 Note)	' Moves from the current position to a position retracted 50mm in the hand direction.
MVS P1 WTH M_OUT (17) = 1	' Starts movement toward P1, and simultaneously turns output signal bit 17 ON.
MVS P1 WTHIF M_IN (20) = 1, SKIP	' If the input signal bit 20 turns ON during movement to P1, the movement to P1 is stopped, and the program proceeds to the next stop.
MVSP1 TYPE 0, 0	' Moves to P1 with ABC interpolation.
MVSP1 TYPE 0, 1	' Moves to P1 with 3-axis orthogonal interpolation.

■ Program example

• Robot movement



CAUTION Specification of forward/backward movement of the hand

The statement examples and program examples are for a vertical 6-axis robot (e.g., RV-20A). The hand advance/retrace direction relies on the Z axis direction (+/- direction) of the tool coordinate set for each model.

Refer to the tool coordinate system shown in "Confirmation of movement" in the separate "From Robot unit setup to maintenance", and designate the correct direction.

• Program example

Program	Explanation
10 MVS P1, -50 Note)	' (1) Moves with linear interpolation from P1 to a position retracted 50mm in the hand direction.
20 MVS P1	' (2) Moves to P1 with linear interpolation.
30 MVS , -50 Note)	' (3) Moves with linear interpolation from the current position (P1) to a position retracted 50mm in the hand direction.
40 MVS P2, -100 WTH M_OUT(17) = 1 Note)	' (4) Output signal bit 17 is turned on at the same time as the robot starts moving.
50 MVS P2	' (5) Moves with linear interpolation to P2.
60 MVS , -100 Note)	' (6) Moves with linear interpolation from the current position (P2) to a position retracted 50mm in the hand direction.
70 END	' Ends the program.

■ Related functions

Function	Explanation page
Designate the movement speed.....	Page 64, "(5) Acceleration/deceleration time and speed control"
Designate the acceleration/deceleration time.	Page 64, "(5) Acceleration/deceleration time and speed control"
Confirm that the target position is reached.	Page 66, "(6) Confirming that the target position is reached"
Continuously move to next position without stopping at target position.....	Page 63, "(4) Continuous movement"
Move with joint interpolation.	Page 59, "(1) Joint interpolation movement"
Move while drawing a circle or arc.....	Page 61, "(3) Circular interpolation movement"
Add a movement command to the process.....	Page 194, "WTH"

(3) Circular interpolation movement

The robot moves along an arc designated with three points using three-dimensional circular interpolation.

If the current position is separated from the start point when starting circular movement, the robot will move to the start point with linear operation and then begin circular interpolation.

■ Command word

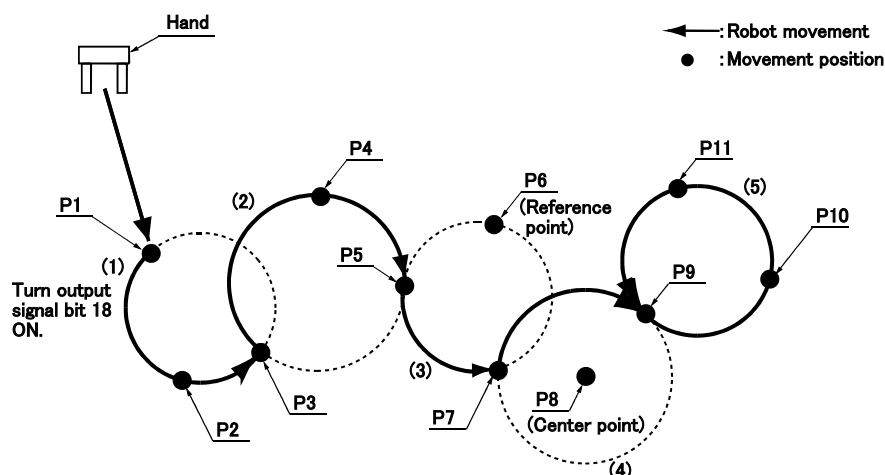
Command word	Explanation
MVR	Designates the start point, transit point and end point, and moves the robot with circular interpolation in order of the start point → transit point → end point. It is possible to specify the interpolation form using the TYPE instruction. An appended statement WTH or WTHIF can be designated.
MVR2	Designates the start point, end point and reference point, and moves the robot with circular interpolation from the start point → end point without passing through the reference point. It is possible to specify the interpolation form using the TYPE instruction. An appended statement WTH or WTHIF can be designated.
MVR3	Designates the start point, end point and center point, and moves the robot with circular interpolation from the start point to the end point. The fan angle from the start point to the end point is 0 deg. < fan angle < 180 deg. It is possible to specify the interpolation form using the TYPE instruction. An appended statement WTH or WTHIF can be designated.
MVC	Designates the start point (end point), transit point 1 and transit point 2, and moves the robot with circular interpolation in order of the start point → transit point 1 → transit point 2 → end point. An appended statement WTH or WTHIF can be designated.

■ Statement example

Statement example	Explanation
MVR P1, P2, P3	' Moves with circular interpolation between P1 → P2 → P3.
MVR P1, P2, P3 WTH M_OUT (17) = 1	' Circular interpolation between P1 → P2 → P3 starts, and the output signal bit 17 turns ON.
MVR P1, P2, P3 WTHIF M_IN (20) = 1, SKIP.....	' If the input signal bit 20 turns ON during circular interpolation between P1 → P2 → P3, circular interpolation to P1 is stopped, and the program proceeds to the next step.
MVR P1, P2, P3 TYPE 0, 1.....	' Moves with circular interpolation between P1 → P2 → P3.
MVR2 P1, P3, P11.....	' Circular interpolation is carried out from P1 to P3 in the direction that P11 is not passed. P11 is the reference point.
MVR3 P1, P3, P10.....	' Moves with circular interpolation from P1 to P3 in the direction with the smallest fan angle. P10 is the center point.
MVC P1, P2, P3.....	' Moves with circular movement from P1 → P2 → P3 → P1.

■ Program example

• Robot movement



• Program example

Program	Explanation
10 MVR P1, P2, P3 WTH M_OUT(18) = 1	' (1) Moves between P1 → P2 → P3 as an arc. The robot current position before movement is separated from the start point, so first the robot will move with linear operation to the start point. (P1) output signal bit 18 turns ON simultaneously with the start of circular movement.
20 MVR P3, P4, P5	' (2) Moves between P3 → P4 → P5 as an arc.
30 MVR2 P5, P7, P6	' (3) Moves as an arc over the circumference on which the start point (P5), reference point (P6) and end point (P7) in the direction that the reference point is not passed between the start point and end point.
40 MVR3 P7, P9, P8	' (4) Moves as an arc from the start point to the end point along the circumference on which the center point (P8), start point (P7) and end point (P9) are designated.
50 MVC P9, P10, P11	' (5) Moves between P9 → P10 → P11 → P9 as an arc. The robot current position before movement is separated from the start point, so first the robot will move with linear operation to the start point.(1 cycle operation)
60 END	' Ends the program.

■ Related functions

Function	Explanation page
Designate the movement speed.....	Page 64, "(5) Acceleration/deceleration time and speed control"
Designate the acceleration/deceleration time.	Page 64, "(5) Acceleration/deceleration time and speed control"
Confirm that the target position is reached.	Page 66, "(6) Confirming that the target position is reached"
Continuously move to next position without stopping at target position.....	Page 63, "(4) Continuous movement"
Move with joint interpolation.	Page 59, "(1) Joint interpolation movement"
Move linearly.....	Page 60, "(2) Linear interpolation movement"
Add a movement command to the process.....	Page 194, " WTH"

(4) Continuous movement

The robot continuously moves to multiple movement positions without stopping at each movement position. The start and end of the continuous movement are designated with the command statement. The speed can be changed even during continuous movement.

■ Command word

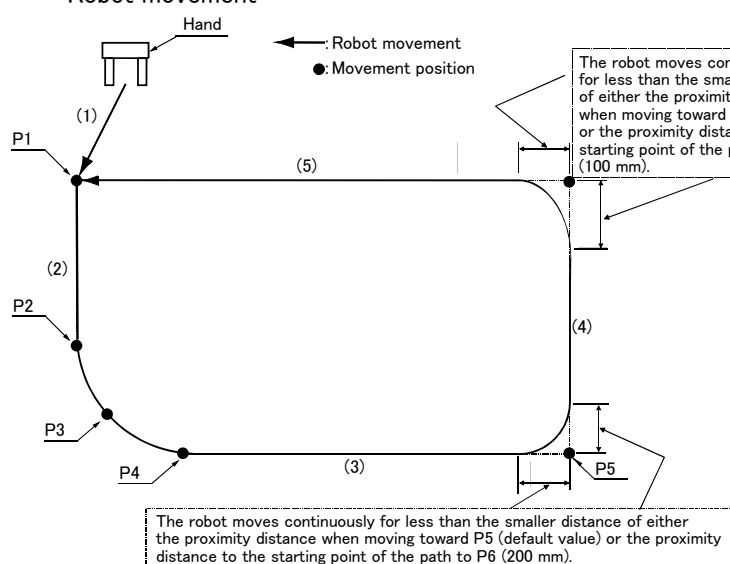
Command word	Explanation
CNT	Designates the start and end of the continuous movement.

■ Statement example

Statement example	Explanation
CNT 1	Designates the start of the continuous movement.
CNT 1, 100, 200.....	Designates the start of the continuous movement, and designates that the start point neighborhood distance is 100mm, and the end point neighborhood distance is 200mm.
CNT 0	Designates the end of the continuous movement.

■ Program example

• Robot movement



⚠ CAUTION

Specification of forward/backward movement of the hand

The statement examples and program examples are for a vertical 6-axis robot (e.g., RV-20A). The hand advance/retrace direction relies on the Z axis direction (+/- direction) of the tool coordinate set for each model. Refer to the tool coordinate system shown in "Confirmation of movement" in the separate "From Robot unit setup to maintenance", and designate the correct direction.

• Program example

Program	Explanation
10 MOV P1	' (1) Moves with joint interpolation to P1.
20 CNT 1	' Validates continuous movement. (Following movement is continuous movement.)
30 MVR P2, P3, P4	' (2) Moves linearly to P2, and continuously moves to P4 with arc movement.
40 MVS P5	' After arc movement, moves linearly to P5.
50 CNT 1, 200, 100	' (3) Sets the continuous movement's start point neighborhood distance to 200mm, and the end point neighborhood distance to 100mm.
60 MVS P6	' (4) After moving to previous P5, moves in succession linearly to P6.
70 MVS P1	' (5) Continuously moves to P1 with linear movement.
80 CNT 0	' Invalidates the continuous movement.
90 END	' Ends the program.

■ Related functions

Function	Explanation page
Designate the movement speed.....	Page 64, "(5) Acceleration/deceleration time and speed control"
Designate the acceleration/deceleration time.	Page 64, "(5) Acceleration/deceleration time and speed control"
Confirm that the target position is reached.	Page 66, "(6) Confirming that the target position is reached"
Move with joint interpolation.....	Page 59, "(1) Joint interpolation movement"
Move linearly.	Page 60, "(2) Linear interpolation movement"
Move while drawing a circle or arc.	Page 61, "(3) Circular interpolation movement"

(5) Acceleration/deceleration time and speed control

The percentage of the acceleration/deceleration in respect to the maximum acceleration/deceleration, and the movement speed can be designated.

■ Command word

Command word	Explanation
ACCEL	Designates the acceleration during movement and the deceleration as a percentage (%) in respect to the maximum acceleration/deceleration speed.
OVRD	Designates the movement speed applied on the entire program as a percentage (%) in respect to the maximum speed.
JOVRD	Designates the joint interpolation speed as a percentage (%) in respect to the maximum speed.
SPD	Designate the linear and circular interpolation speed with the hand end speed (mm/s).
OADL	This instruction specifies whether the optimum acceleration/deceleration function should be enabled or disabled.

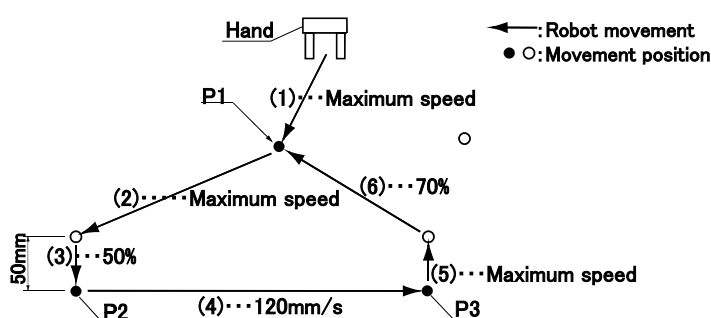
■ Statement example

Statement example	Explanation
ACCEL.....	Sets both the acceleration and deceleration to 100%.
ACCELL 60, 80.....	Sets the acceleration to 60% and the deceleration to 80%. (For maximum acceleration/deceleration is 0.2 sec. acceleration $0.2 \div 0.6=0.33$ sec. deceleration $0.2 \div 0.8=0.25$ sec.)
OVRD 50.....	Sets the joint interpolation, linear interpolation and circular interpolation to 50% of the maximum speed.
JOVRD 70.....	Set the joint interpolation operation to 70% of the maximum speed.
SPD 30.....	Sets the linear interpolation and circular interpolation speed to 30mm/s.
OADL ON.....	This instruction enables the optimum acceleration/deceleration function.

- Movement speed during joint interpolationController (T/B) setting value × OVRD command setting value × JOVRD command setting value.
- Movement speed during linear and circular interpolationController (T/B) setting value × OVRD command setting value × SPD command setting value.

■ Program example

- Robot movement



CAUTION Specification of forward/backward movement of the hand

The statement examples and program examples are for a vertical 6-axis robot (e.g., RV-20A). The hand advance/retrace direction relies on the Z axis direction (+/- direction) of the tool coordinate set for each model.

Refer to the tool coordinate system shown in "Confirmation of movement" in the separate "From Robot unit setup to maintenance", and designate the correct direction.

• Program example

Program	Explanation
10 OVRD 100	' Sets the movement speed applied on the entire program to the maximum speed.
20 MVS P1	' (1) Moves at maximum speed to P1.
30 MVS P2, -50 Note)	' (2) Moves at maximum speed from P2 to position retracted 50mm in hand direction.
40 OVRD 50	' Sets the movement speed applied on the entire program to half of the maximum speed.
50 MVS P2	' (3) Moves linearly to P2 with a speed half of the default speed.
60 SPD 120	' Sets the end speed to 120mm/s. (Since the override is 50%, it actually moves at 60 mm/s.)
70 OVRD 100	' Sets the movement speed percentage to 100% to obtain the actual end speed of 120mm/s.
80 ACCEL 70, 70	' Sets the acceleration and deceleration to 70% of the maximum speed.
90 MVS P3	' (4) Moves linearly to P3 with the end speed 120mm/s.
100 SPD M_NSPD	' Returns the end speed to the default value.
110 JOVRD 70	' Sets the speed for joint interpolation to 70%.
120 ACCEL	' Returns both the acceleration and deceleration to the maximum speed.
130 MVS , -50 Note)	' (5) Moves linearly with the default speed for linear movement from the current position (P3) to a position retracted 50mm in the hand direction.
140 MVS P1	' (6) Moves to P1 at 70% of the maximum speed.
150 END	' Ends the program.

■ Related functions

Function	Explanation page
Move with joint interpolation.....	Page 59, "(1) Joint interpolation movement"
Move linearly.....	Page 60, "(2) Linear interpolation movement"
Move while drawing a circle or arc.....	Page 61, "(3) Circular interpolation movement"
Continuously move to next position without stopping at target position.....	Page 63, "(4) Continuous movement"

(6) Confirming that the target position is reached

The positioning finish conditions can be designated with as No. of pulses. (FINE instruction) This designation is invalid when using continuous movement.

■ Command word

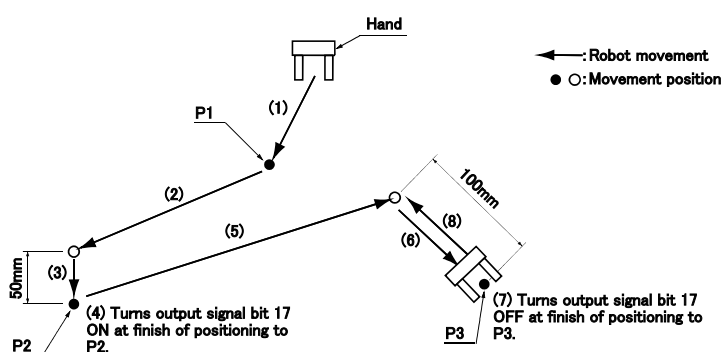
Command word	Explanation
FINE	Designates the positioning finish conditions with a No. of pulses. Specify a small number of pulses to allow more accurate positioning.
MOV and DLY	After the MOV movement command, command the DLY instruction (timer) to complete positioning . (this is effective for belt-driven robots, e.g., RP-1AH, 3AH, and 5AH).

■ Statement example

Statement example	Explanation
FINE 100.....	Sets the positioning finish conditions to 100 pulses.
MOV P1	Moves with joint interpolation to P1. (The movement completes at the command value level.)
DLY 0.1	Positioning after the movement instruction is performed by the timer. (this is effective for belt-driven robots, e.g., RP-1AH, 3AH, and 5AH).

■ Program example

• Robot movement

⚠ CAUTION Specification of forward/backward movement of the hand

The statement examples and program examples are for a vertical 6-axis robot (e.g., RV-20A). The hand advance/retrace direction relies on the Z axis direction (+/- direction) of the tool coordinate set for each model.

Refer to the tool coordinate system shown in "Confirmation of movement" in the separate "From Robot unit setup to maintenance", and designate the correct direction.

• Program example

Program	Explanation
10 CNT 0	* The FINE instruction is valid only when the CNT instruction is OFF.
20 MVS P1	* (1) Moves with joint interpolation to P1.
30 MVS P2, -50 Note)	* (2) Moves with joint interpolation from P2 to position retracted 50mm in hand direction.
40 FINE 50	* Sets positioning finish pulse to 50.
50 MVS P2	* (3) Moves with linear interpolation to P2 (MVS completes if the positioning complete pulse count is 50 or less.)
60 M_OUT(17)=1	* (4) Turns output signal 17 ON when positioning finish pulse reaches 50 pulses.
70 FINE 1000	* Sets positioning finish pulse to 1000.
80 MVS P3, -100 Note)	* (5) Moves linearly from P3 to position retracted 100mm in hand direction.
90 MVS P3	* (6) Moves with linear interpolation to P3.
100 DLY 0.1	* Performs the positioning by the timer.
110 M_OUT(17)=0	* (7) Turns output signal 17 off.
120 MVS , -100 Note)	* (8) Moves linearly from current position (P3) to position retracted 100mm in hand direction.
130 END	* Ends the program.

■ Related functions

Function	Explanation page
Move with joint interpolation.	Page 61, "(3) Circular interpolation movement"
Move linearly.....	Page 60, "(2) Linear interpolation movement"
Continuously move to next position without stopping at target position.....	Page 63, "(4) Continuous movement"

(7) High path accuracy control

It is possible to improve the motion path tracking when moving the robot. This function is limited to certain types of robot. Currently, the PREC instruction is available for vertical multi-joint type 5-axis and 6-axis robots, RV-20A, RV-4A, 5AJ, 3AL, 4AJL, RV-1A, and 2AJ.

■ Command word

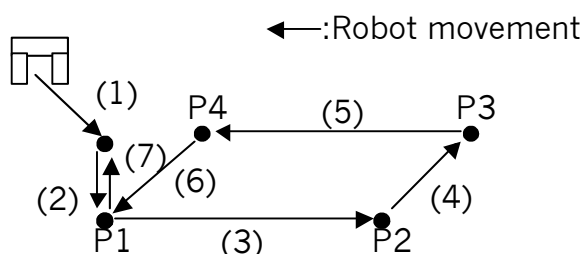
Command word	Explanation
PREC	This instruction specifies whether the high path accuracy mode should be enabled or disabled.

■ Statement example

Statement example	Explanation
PREC ON	Enables the high path accuracy mode.
PREC OFF	Disables the high path accuracy mode.

■ Program example

• Robot movement



⚠ CAUTION Specification of forward/backward movement of the hand

The statement examples and program examples are for a vertical 6-axis robot (e.g., RV-20A). The hand advance/retrace direction relies on the Z axis direction (+/- direction) of the tool coordinate set for each model.

Refer to the tool coordinate system shown in "Confirmation of movement" in the separate "From Robot unit setup to maintenance", and designate the correct direction.

• Program example

Program	Explanation
10 MOV P1, -50 Note)	' (1) Moves with joint interpolation from P1 to position retracted 50mm in hand direction.
20 OVRD 50	' Sets the movement speed to half of the maximum speed.
30 MVS P1	' (2) Moves with linear interpolation to P1.
40 PREC ON	' The high path accuracy mode is enabled.
50 MVS P2	' (3) Moves the robot from P1 to P2 with high path accuracy.
60 MVS P3	' (4) Moves the robot from P2 to P3 with high path accuracy.
70 MVS P4	' (5) Moves the robot from P3 to P4 with high path accuracy.
80 MVS P1	' (6) Moves the robot from P4 to P1 with high path accuracy.
90 PREC OFF	' The high path accuracy mode is disabled.
100 MVS P1, -50	' (7) Returns the robot to the position 50 mm behind P1 in the hand direction using linear interpolation.
110 END	' Ends the program.

⚠ CAUTION

The PREC instruction improves the tracking accuracy of the robot's hand tip, but lowers the acceleration/deceleration of the robot movement, which means that the cycle time may become longer. The tracking accuracy will be further improved if the CNT instruction is not included. However, the hand tip speed cannot be guaranteed in this case.

(8) Hand and tool control

The hand open/close state and tool shape can be designated.

■ Command word

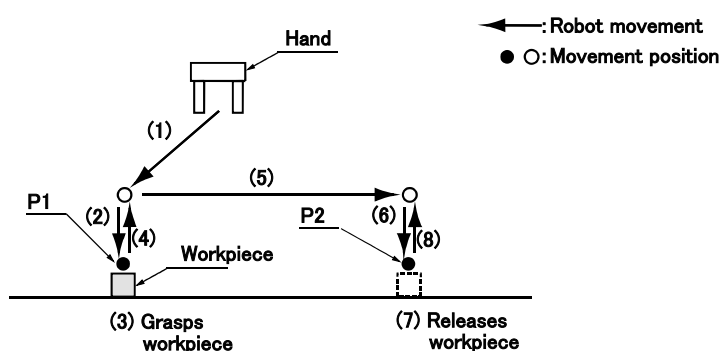
Command word	Explanation
HOPEN	Opens the designated hand.
HCLOSE	Closes the designated hand.
TOOL	Sets the shape of the tool being used, and sets the control point.

■ Statement example

Statement example	Explanation
HOPEN 1	Opens hand 1.
HOPEN 2	Opens hand 2.
HCLOSE 1	Closes hand 1.
HCLOSE 2	Closes hand 2.
TOOL (0, 0, 95, 0, 0, 0)	Sets the robot control point to the position 95 mm from the flange plane in the extension direction.

■ Program example

• Robot movement



CAUTION Specification of forward/backward movement of the hand

The statement examples and program examples are for a vertical 6-axis robot (e.g., RV-20A). The hand advance/retrace direction relies on the Z axis direction (+/- direction) of the tool coordinate set for each model.

Refer to the tool coordinate system shown in "Confirmation of movement" in the separate "From Robot unit setup to maintenance", and designate the correct direction.

• Program example

Program	Explanation
10 TOOL (0, 0, 95, 0, 0, 0)	; Sets the hand length to 95 mm.
20 MVS P1, -50 (注)	; (1) Moves with joint interpolation from P1 to position retracted 50mm in hand direction.
30 OVRD 50	; Sets the movement speed to half of the maximum speed.
40 MVS P1	; (2) Moves with linear interpolation to P1. (Goes to grasp workpiece.)
50 DLY 0.5	; Wait for the 0.5 seconds for the completion of arrival to the target position.
60 HCLOSE 1	; (3) Closes hand 1. (Grasps workpiece.)
70 DLY 0.5	; Waits 0.5 seconds.
80 OVRD 100	; Sets movement speed to maximum speed.
90 MVS , -50 Note)	; (4) Moves linearly from current position (P1) to position retracted 50mm in hand direction. (Lifts up workpiece.)
100 MVS P2, -50 Note)	; (5) Moves with joint interpolation from P2 to position retracted 50mm in hand direction.
110 OVRD 50	; Sets movement speed to half of the maximum speed.
120 MVS P2	; (6) Moves with linear interpolation to P2. (Goes to place workpiece.)
130 DLY 0.5	; Wait for the 0.5 seconds for the completion of arrival to the target position.
140 HOPEN 1	; (7) Opens hand 1. (Releases workpiece.)
150 DLY 0.5	; Waits 0.5 seconds.
160 OVRD 100	; Sets movement speed to maximum speed.
170 MVS , -50 Note)	; (8) Moves linearly from current position (P2) to position retracted 50mm in hand direction. (Separates from workpiece.)
180 END	; Ends the program.

■ Related functions

Function	Explanation page
Appended statement.....	Page 194, " WTH"

4.1.2 Pallet operation

When carrying out operations with the workpieces neatly arranged (palletizing), or when removing workpieces that are neatly arranged (depalletizing), the pallet function can be used to teach only the position of the reference workpiece, and obtain the other positions with operations.

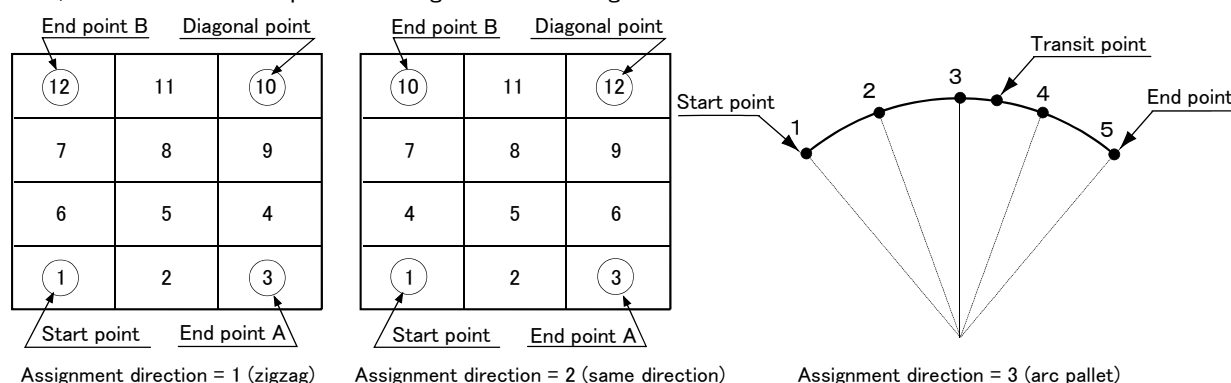
■ Command word

Command word	Explanation
DEF PLT	Defines the pallet to be used.
PLT	Obtains the designated position on the pallet with operations.

■ Statement example

Statement example	Explanation
DEF PLT 1, P1, P2, P3, P4, 4, 3, 1.....	Defines to operate pallet No. 1 with a start point = P1, end point A = P2, end point B = P3 and diagonal point = P4, a total of 12 work positions (quantity A = 4, quantity B = 3), and an assignment direction = 1 (Zigzag).
DEF PLT 2, P1, P2, P3, , 8, 5, 2.....	Defines to operate pallet No. 2 with a start point = P1, end point A = P2, and end point B = P3, a total of 40 work positions (quantity A = 8, quantity B = 5), and an assignment direction = 2 (Same direction).
DEF PLT 3, P1, P2, P3, , 5, 1, 3.....	Define that pallet No. 3 is an arc pallet having give five work positions on an arc designated with start point = P1, transit point = P2, end point = P3 (total three points).
(PLT 1, 5).....	Operate the 5th position on pallet No. 1.
(PLT 1, M1).....	Operate position in pallet No. 1 indicated with the numeric variable M1.

Note 1) The relation of the position designation and assignment direction is shown below



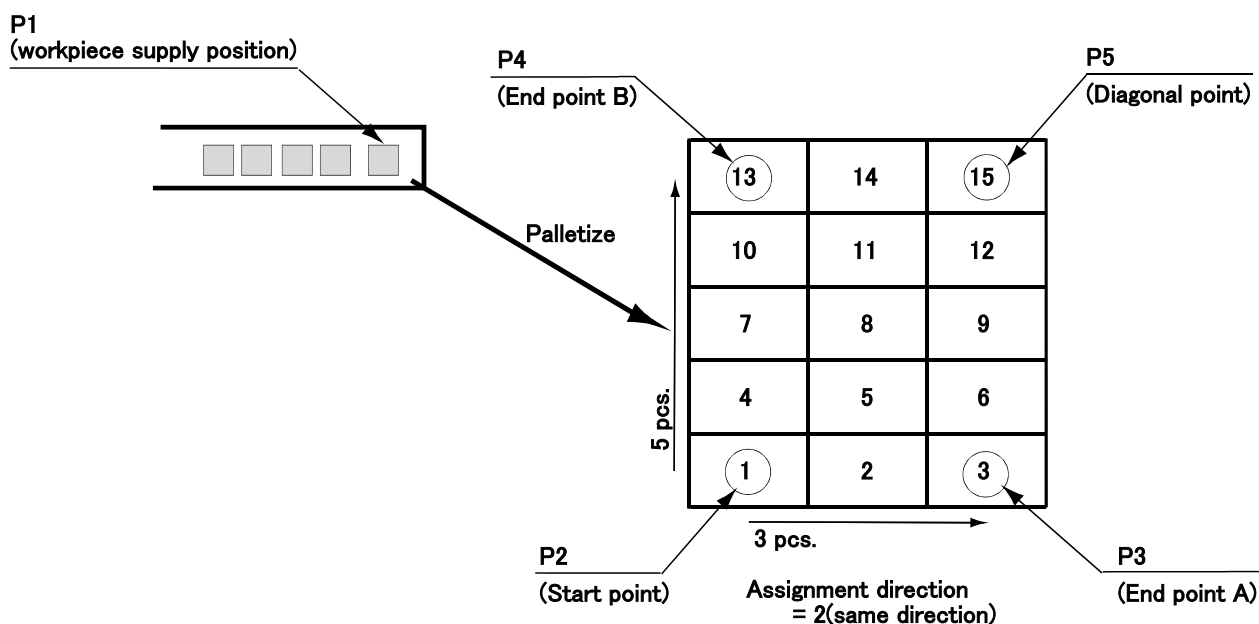
<About the posture of position data defining a pallet>

The signs of the posture data (A, B, and C) at four points, the starting point, endpoint A, endpoint B, and the diagonal point, must match. In the case of a vertical multi-joint robot, if the mechanical interface plane (flange plane) is facing downward, the A, B, and C axis coordinates (especially the A and C axes) may reach 180 degrees. In this case, the sign can be either + or -. Each position of the pallet is calculated from the position data of the starting point and endpoints; if the signs do not match, the hand rotates as a result.

+180 and -180 result in the same position. Use + or - consistently when the signs are different, and perform this correction only when the values are exactly 180 degrees.

■ Program example

- Robot movement



• Program example

Program	Explanation
10 DEF PLT 1, P2, P3, P4, P5, 3, 5, 2	; Defines the pallet. Pallet No. = 1, start point = P2, end point A = P3, end point B = P4, diagonal point = P5, quantity A = 3, quantity B = 5, assignment direction = 2 (Same direction).
20 M1=1	; Substitutes value 1 in numeric variable M1. (M1 is used as a counter.)
30 * LOOP	; Designates label LOOP at the jump destination.
40 MOV P1, -50 (注)	; Moves with joint interpolation from P1 to a position retracted 50mm in hand direction.
50 OVRD 50	; Sets movement speed to half of the maximum speed.
60 MVS P1	; Moves linearly to P1. (Goes to grasp workpiece.)
70 HCLOSE 1	; Closes hand 1. (Grasps workpiece.)
80 DLY 0.5	; Waits 0.5 seconds.
90 OVRD 100	; Sets movement speed to maximum speed.
100 MVS, -50 (注)	; Moves linearly from current position (P1) to a position retracted 50mm in hand direction. (Lifts up workpiece.)
110 P10=(PLT 1, M1)	; Operates the position in pallet No. 1 indicated by the numeric variable M1, and substitutes the results in P10.
120 MOV P10, -50 (注)	; Moves with joint interpolation from P10 to a position retracted 50mm in hand direction.
130 OVRD 50	; Sets movement speed to half of the maximum speed.
140 MVS P10	; Moves linearly to P10. (Goes to place workpiece.)
150 HOPEN 1	; Opens hand 1. (Places workpiece.)
160 DLY 0.5	; Waits 0.5 seconds.
170 OVRD 100	; Sets movement speed to maximum speed.
180 MVS, -50	; Moves linearly from current position (P10) to a position retracted 50mm in hand direction. (Separates from workpiece.)
190 M1=M1+1	; Increments numeric variable M1 by 1. (Advances the pallet counter.)
200 IF M1 <= 15 THEN * LOOP	; If numeric variable M1 value is less than 15, jumps to label LOOP and repeat process. If more than 15, goes to next step.
210 END	; Ends the program.

■ Related functions

Function	Explanation page
Substitute, operation.....	Page 80, "4.1.6 Expressions and operations"
Condition branching.....	Page 71, "(1) Unconditional branching, conditional branching, waiting"

4.1.3 Program control

The program flow can be controlled with branching, interrupting, subroutine call, and stopping, etc.

(1) Unconditional branching, conditional branching, waiting

The flow of the program to a specified line can be set as unconditional or conditional branching.

■ Command word

Command word	Explanation
GOTO	Jumps unconditionally to the designated line.
ON GOTO	Jumps according to the value of the designated variable. The value conditions follow the integer value order.
IF THEN ELSE (Instructions written in one line)	Executes the command corresponding to the designated conditions.. The value conditions can be designated randomly. There is only one type of condition per command statement. If the conditions are met, the instruction after THEN is executed. If the conditions are not met, the instruction after ELSE is executed. They are written in one line.
IF THEN ELSE END IF (Instructions written in several lines)	Several lines can be processed according to the specified variables and specified conditions of the values. It is possible to specify any conditions for values. Only one type of condition is allowed for one instruction. If the conditions are met, the lines following THEN until the ELSE line are executed. If the conditions are not met, the lines after ELSE until END IF are executed. Note) This function is available for controller version G1 or later.
SELECT CASE END SELECT	Jumps according to the designated variable and the designated conditions of that value. The value conditions can be designated randomly. Multiple types of conditions can be designated per command statement.
WAIT	Waits for the variable to reach the designated value.

■ Statement example

Statement example	Explanation
GOTO 200	Jumps unconditionally to line 200.
GOTO * FIN	Jumps unconditionally to the label FIN line.
ON M1 GOTO 100, 200, 300	If the numeric variable M1 value is 1, jumps to line 100, if 2 jumps to line 200, and if 3 jumps to line 300. If the value does not correspond, proceeds to next step.
IF M1 = 1 THEN 100	If the numeric variable M1 value is 1, branches to line 100. If not, proceeds to the next step.
IF M1 = 1 THEN 100 ELSE 200	If the numeric variable M1 value is 1, branches to line 100. If not, branches to line 200.
IF M1 = 1 THEN M2 = 1 M3 = 2 ELSE M2 = -1 M3 = -2 ENDIF	If the numerical variable of M1 is 1, the instructions M2 = 1 and M3 = 2 are executed. If the value of M1 is different from 1, the instructions M2 = -1 and M3 = -2 are executed.
SELECT M1 CASE 10 : BREAK CASE IS 11 : BREAK CASE IS < 5 : BREAK CASE 6 TO 9 : BREAK DEFAULT : BREAK END SELECT	Branches to the CASE statement corresponding to the value of numeric variable M1. If the value is 10, executes only between CASE 10 and the next CASE 11. If the value is 11, executes only between CASE 11 and the next CASE IS <5. If the value is smaller than 5, executes only between CASE IS <5 and next CASE 6 TO 9. If value is between 6 and 9, executes only between CASE 6 TO 9 and next DEFAULT. If value does not correspond to any of the above, executes only between DEFAULT and next END SELECT.
WAIT M_IN (1) = 1	Waits for the input signal bit 1 to turn ON.

■ Related functions

Function	Explanation page
Repetition.....	Page 73, "(2) Repetition"
Interrupt.....	Page 74, "(3) Interrupt"
Subroutine	Page 75, "(4) Subroutine"
External signal input	Page 78, "(1) Input signals"

(2) Repetition

Multiple command statements can be repeatedly executed according to the designated conditions.

■ Command word

Command word	Explanation
FOR NEXT	Repeat between FOR statement and NEXT statement until designated conditions are satisfied.
WHILE WEND	Repeat between WHILE statement and WEND statement while designated conditions are satisfied.

■ Statement example

Statement example	Explanation
FOR M1 = 1 TO 10 : NEXT	Repeat between FOR statement and NEXT statement 10 times. The initial numeric variable M1 value is 1, and is incremented by one with each repetition.
FOR M1 = 0 TO 10 STEP 2 : NEXT	Repeat between FOR statement and NEXT statement 6 times. The initial numeric variable M1 value is 0, and is incremented by two with each repetition.
WHILE (M1 >= 1) AND (M1 <= 10) : WEND	Repeat between WHILE statement and WEND statement while the value of the numeric variable M1 is 1 or more and less than 10.

■ Related functions

Function	Explanation page
Unconditional branching, branching	Page 71, "(1) Unconditional branching, conditional branching, waiting"
Interrupt	Page 74, "(3) Interrupt"
Input signal wait	Page 78, "(1) Input signals"

(3) Interrupt

Once the designated conditions are established, the command statement being executed can be interrupted and a designated line branched to.

■ Command word

Command word	Explanation
DEF ACT	Defines the interrupt conditions and process for generating interrupt.
ACT	Designates the validity of the interrupt.
RETURN	If a subroutine is called for the interrupt process, returns to the interrupt source line.

■ Statement example

Statement example	Explanation
DEF ACT 1, M_IN (10) = 1 GOSUB 100	If input signal bit 10 is turned on for interrupt number 1, the subroutine on line 100 is defined to be called after the robot decelerates and stops. The deceleration time depends on the ACCEL and OVRD instructions.
DEF ACT 2, M_IN (11) = 1 GOSUB 200, L	If input signal bit 11 is turned on for interrupt number 2, the subroutine on line 200 is defined to be called after the statement currently being executed is completed.
DEF ACT 3, M_IN (12) = 1 GOSUB 300, S	If input signal bit 12 is turned on for interrupt number 3, the subroutine on line 300 is defined to be called after the robot decelerates and stops in the shortest time and distance possible.
ACT 1 = 1	Enables the priority No. 1 interrupt.
ACT 2 = 0	Disables the priority No. 1 interrupt.
RETURN 0	Returns to the line where the interrupt occurred.
RETURN 1	Returns to the line following the line where the interrupt occurred.

■ Related functions

Function	Explanation page
Unconditional branching, branching.....	Page 71, "(1) Unconditional branching, conditional branching, waiting"
Subroutine	Page 75, "(4) Subroutine"
Communication	Page 79, "4.1.5 Communication"

(4) Subroutine

Subroutine and subprograms can be used.

By using this function, the program can be shared to reduce the No. of steps, and the program can be created in a hierarchical structure to make it easy to understand.

■ Command word

Command word	Explanation
GOSUB	Calls the subroutine at the designated line or designated label.
ON GOSUB	Calls the subroutine according to the designated variable number. The value conditions follow the integer value order. (1,2,3,4,.....)
RETURN	Returns to the line following the line called with the GOSUB command.
CALLP	Calls the designated program. The next line in the source program is returned to at the END statement in the called program. Data can be transferred to the called program as an argument.
FPRM	An argument is transferred with the program called with the CALLP command.

■ Statement example

Statement example	Explanation
GOSUB 100	Calls the subroutine from line 100.
GOSUB * GET	Calls the subroutine from label GET.
ON M1 GOSUB 100, 200, 300	If the numeric variable M1 value is 1, calls the subroutine at line 100, if 2 calls the subroutine at line 200, and if 3 calls the subroutine at line 300. If the value does not correspond, proceeds to next step.
RETURN	Returns to the line following the line called with the GOSUB command.
CALLP " 10"	Calls the No. 10 program.
CALLP " 20", M1, P1	Transfers the numeric variable M1 and position variable P1 to the No. 20 program, and calls the program.
FPRM M10, P10	Receives the numeric variable transferred with the CALLP in M10 of the subprogram, and the position variable in P10.

■ Related functions

Function	Explanation page
Interrupt	Page 74, "(3) Interrupt"
Communication.....	Page 79, "4.1.5 Communication"
Unconditional branching	Page 71, "(1) Unconditional branching, conditional branching, waiting"

(5) Timer

The program can be delayed by the designated time, and the output signal can be output with pulses at a designated time width.

■ Command word

Command word	Explanation
DLY	Functions as a designated-time timer.

■ Statement example

Statement example	Explanation
DLY 0.05	Waits for only 0.05 seconds.
M_OUT (10) = 1 DLY 0.5	Turns on output signal bit 10 for only 0.5 seconds.

■ Related functions

Function	Explanation page
Pulse signal output.....	Page 78, "(1) Input signals"

(6) Stopping

The program execution can be stopped. The moving robot will decelerate to a stop.

■ Command word

Command word	Explanation
HLT	This instruction stops the robot and pauses the execution of the program. When the program is started, it is executed from the next line.
END	This instruction defines the end of one cycle of a program. In continuous operation, the program is executed again from the start line upon the execution of the END instruction. In cycle operation, the program ends upon the execution of the END instruction when the cycle is stopped.

■ Statement example

Statement example	Explanation
HLT	Interrupt execution of the program.
IF M_IN(20) = 1 THEN HLT	Pauses the program if input signal bit 20 is turned on.
MOV P1 WTHIF M_IN (18) = 1, HLT	Pauses the program if input signal bit 18 is turned on while moving toward P1.
END	Terminates the program even in the middle of the execution.

■ Related functions

Function	Explanation page
Appended statement.....	Page 194, " WTH"

4.1.4 Inputting and outputting external signals

This section explains the general methods for signal control when controlling the robot via an external device (e.g., PLC).

(1) Input signals

Signals can be retrieved from an external device, such as a programmable logic controller.

The input signal is confirmed with a robot status variable (M_IN(), etc.) Refer to [Page 102, "4.3.26 Robot status variables"](#) for details on the robot status variables.

■ Command word

Command word	Explanation
WAIT	Waits for the input signal to reach the designated state.

■ System variables

M __ IN, M __ INB, M __ INW, M __ DIN

■ Statement example

Statement example	Explanation
WAIT M_IN (1) = 1	Waits for the input signal bit 1 to turn ON.
M1 = M_INB (20)	Substitutes the input signal bit 20 to 27, as an 8-bit state, in numeric variable M1.
M1 = M_INW (5)	Substitutes the input signal bit 5 to 20, as an 16-bit state, in numeric variable M1.

■ Related functions

Function	Explanation page
Signal output.....	Page 78, "(2) Output signals"
Branching with input signal.....	Page 71, "(1) Unconditional branching, conditional branching, waiting"
Interrupting with input signal	Page 74, "(3) Interrupt"

(2) Output signals

Signals can be output to an external device, such as a programmable logic controller.

The signal is output with the robot status variable (M_OUT(), etc.). Refer to [Page 102, "4.3.26 Robot status variables"](#) for details on the robot status variables.

■ Command word

Command word	Explanation
CLR	Clears the general-purpose output signal according to the output signal reset pattern in the parameter.

■ System variables

M __ OUT, M __ OUTB, M __ OUTW, M __ DOUT

■ Statement example

Statement example	Explanation
CLR 1	Clears based on the output reset pattern.
M_OUT (1) = 1	Turns the output signal bit 1 ON.
M_OUTB (8) = 0	Turns the 8 bits, from output signal bit 8 to 15, OFF.
M_OUTW (20) = 0	Turns the 16 bits, from output signal bit 20 to 35, OFF.
M_OUT (1) = 1 DLY 0.5	Turns the output signal bit 1 ON for 0.5 seconds. (Pulse output)
M_OUTB (10) = & HOF	Turns the 4 bits, from output signal bit 10 to 13 ON, and turns the four bits from 14 to 17 OFF.

■ Related functions

Function	Explanation page
Signal input.....	Page 78, "(1) Input signals"
Timer.....	Page 76, "(5) Timer"

4.1.5 Communication

Data can be exchanged with an external device, such as a personal computer.

■ Command word

Command word	Explanation
OPEN	Opens the communication line.
CLOSE	Closes the communication line.
PRINT#	Outputs the data in the ASCII format. CR is output as the end code.
INPUT#	Inputs the data in the ASCII format. The end code is CR.
ON COM GOSUB	Defines the subroutine to be called when an interrupt is generated from the communication line. The interrupt is generated when data is input from an external device.
COM ON	Enables the interrupt process from the communication line.
COM OFF	Disables the interrupt process from the communication line. The interrupt will be invalid even if it occurs.
COM STOP	Stops the interrupt process from the communication line. If there is an interrupt, it is saved, and is executed after enabled.

■ Statement example

Statement example	Explanation
OPEN " COM1 : " AS # 1	Opens the communication line COM1 as file No. 1.
CLOSE # 1	Closes file No. 1.
CLOSE	Closes all files that are open.
PRINT # 1, " TEST"	Outputs the character string "TEST" to file No. 1.
PRINT # 2, " M1 = " ; M1	Output the character string "M1=" and then the M1 value to file No. 2. Output data example: "M1 = 1" + CR (When M1 value is 1)
PRINT # 3, P1	Outputs the position variable P1 coordinate value to file No. 3. Output data example: "(123.7, 238.9, 33.1, 19.3, 0, 0)(1, 0)" + CR (When X = 123.7, Y=238.9, Z=33.1, A=19.3, B=0, C=0, FL1=1, FL2=0)
PRINT # 1, M5, P5	Outputs the numeric variable M5 value and position variable coordinate value to file No. 1. M5 and P5 are separated with a comma (hexadecimal, 2C). Output data example: "8, (123.7, 238.9, 33.1, 19.3, 0, 0)(1, 0)" + CR (When M5=8, P5 X=123.7, Y=238.9, Z=33.1, A=19.3, B=0, C=0, FL1=1, FL2=0)
INPUT # 1, M3	Converts the input data into a value, and substitutes it in numeric variable M3. Input data example: "8" + CR (when value 8 is to be substituted)
INPUT # 1, P10	Converts the input data into a value, and substitutes it in position variable P10. Input data example: "8, (123.7, 238.9, 33.1, 19.3, 0, 0)(1, 0)" + CR (P5 will be X= 123.7, Y=238.9, Z=33.1, A=19.3, B=0, C=0, FL1=1, FL2=0)
INPUT # 1, M8, P6	Converts the first data input into a value, and substitutes it in numeric variable M8. Converts the data following the command into a coordinate value, and substitutes it in position variable P6. M8 and P6 are separated with a comma (hexadecimal, 2C) Input data example: "7, (123.7, 238.9, 33.1, 19.3, 0, 0)(1, 0)" + CR (The data will be M8 = 7, P6 X=123.7, Y=238.9, Z=33.1, A=19.3, B=0, C=0, FL1=1, FL2=0)
ON COM(1) GOSUB 300	Defines to call line 300 subroutine when data is input in communication line COM1.
ON COM(2) GOSUB * RECV	Defines to call subroutine at label RECV line when data is input in communication line COM2.
COM(1) ON	Enables the interrupt from communication line COM1.
COM(2) OFF	Disables (prohibits) the interrupt from communication line COM2.
COM(1) STOP	Stops (holds) the interrupt from communication line COM1.

■ Related functions

Function	Explanation page
Subroutine	Page 75, "(4) Subroutine"
Interrupt.....	Page 74, "(3) Interrupt"

4.1.6 Expressions and operations

The following table shows the operators that can be used, their meanings, and statement examples.

(1) List of operator

Class	Operator	Meaning	Statement example	
Arithmetic operation	=	The right side is substituted in the left side.	P1 = P2 P5 = P__CURR P10.Z = 100.0 M1 = 1 STS\$="OK"	; Substitute P2 in position variable P1. ; Substitute the current coordinate value in current position variable P5. ; Set the position variable P10 Z coordinate value to 100.0. ; Substitute value 1 in numeric variable M1. ; Substitute the character string OK in the character string variable STS\$.
	+	Add	P10 = P1 + P2 MOV P8 + P9 M1 = M1 + 1 STS\$ = "ERR" + " 001"	; Substitute the results obtained by adding the P1 and P2 coordinate elements to position variable P10. ; Move to the position obtained by adding the position variable P8 and P9 coordinate elements. ; Add 1 to the numeric variable M1. ; Add the character string 001 to the character string ERR and substitute in character string variable STS\$.
	-	Subtract	P10 = P1 - P2 MOV P8 - P9 M1 = M1 - 1	; Substitute the results obtained by subtracting the P2 coordinate element from P1 in position variable P10. ; Move to the position obtained by subtracting the P9 coordinate element from the position variable P8. ; Subtract 1 from the numeric variable M1.
	*	Multiply	P1 = P10 * P3 M1 = M1 * 5	; Substitute the relative conversion results from P10 to P3 in position variable P1. ; Multiply the numeric variable M1 value by 5.
	/	Divide	P1 = P10 / P3 M1 = M1 / 2	; Substitute the reverse relative conversion results from P10 to P3 in position variable P1. ; Divide the numeric variable M1 value by 2.
	^	Exponential operation	M1 = M1 ^ 2	; Square the numeric variable M1 value.
	¥	Integer division	M1 = M1 ¥ 3	; Divide the numeric variable M1 value by 3 and make an integer (round down).
	MOD	Remainder operation	M1 = M1 MOD 3	; Divide the numeric variable M1 value by 3 and leave redundant.
Comparison operation	-	Sign reversal	P1 = - P1 M1 = - M1	; Reverse the sign for each coordinate element in position variable P1. ; Reverse the sign for the numeric variable M1 value.
	=	Compare whether equal	IF M1 = 1 THEN 200 IF STS\$ = "OK" THEN 100	; Branch to line 200 if numeric variable M1 value is 1. ; Branch to line 100 if character string in character string variable STS\$ is OK.
	<> or ><	Compare whether not equal	IF M1 <> 2 THEN 300 IF STS\$ <> "OK" THEN 900	; Branch to line 300 if numeric variable M1 value is 2. ; Branch to line 900 if character string in character string variable STS\$ is not OK.
	<	Compare whether smaller	IF M1 < 10 THEN 300 IF LEN(STS\$) < 3 THEN 100	; Branch to line 300 if numeric variable M1 value is less than 10. ; Branch to line 100 if No. of characters in character string STS\$ variable is less than 3.
	>	Compare whether larger	IF M1 > 9 THEN 200 IF LEN(STS\$) > 2 THEN 300	; Branch to line 200 if numeric variable M1 value is more than 9. ; Branch to line 300 if No. of characters in character string variable STS\$ is more than 2.
	=< or <=	Compare whether equal to or less than	IF M1 <= 10 THEN 200 IF LEN(STS\$) <= 5 THEN 300	; Branch to line 200 if numeric variable M1 value is equal to or less than 10. ; Branch to line 300 if No. of characters in character string variable STS\$ is equal to or less than 5.
	=> or >=	Compare whether equal to or more than	IF M1 >= 11 THEN 200 IF LEN(STS\$) >= 6 THEN 300	; Branch to line 200 if numeric variable M1 value is equal to or more than 11. ; Branch to line 300 if No. of characters in character string variable STS\$ is equal to or more than 6.
Logical operation	AND	Logical AND operation	M1 = M_INB(1) AND &H0F	; Convert the input signal bit 1 to 4 status and substitute in numeric variable M1. (Input signal bits 5 to 8 remain OFF.)
	OR	Logical OR operation	M_OUTB(20) = M1 OR &H80	; Output the numeric variable M1 value to output signal bit 20 to 27. Output bit signal 27 is always ON at this time.
	NOT	NOT operation	M1 = NOT M_INW(1)	; Reverse the status of input signal bit 1 to 16 to create a value, and substitute in numeric variable M1.
	XOR	Exclusive OR operation	M2 = M1 XOR M_INW(1)	; Obtain the exclusive OR of the states of M1 and the input signal bits 1 to 16, convert into a value and substitute in numeric variable M2.
	<<	Logical left shift operation	M1 = M1 << 2	; Shift numeric variable M1 two bits to the left.
	>>	Logical right shift operation.	M1 = M1 >> 1	; Shift numeric variable M1 bit to the right.

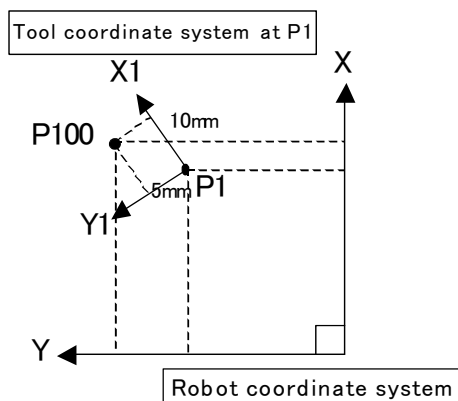
Note1) Please refer to [Page 81, "Relative calculation of position data \(multiplication\)"](#).

Note2) Please refer to [Page 81, "Relative calculation of position data \(Addition\)"](#).

(2) Relative calculation of position data (multiplication)

Numerical variables are calculated by the usual four arithmetic operations. The calculation of position variables involves coordinate conversions, however, not just the four basic arithmetic operations. This is explained using simple examples.

Multiplication between P variables
(relative calculation in the tool coordinate system)



An example of relative calculation (multiplication)

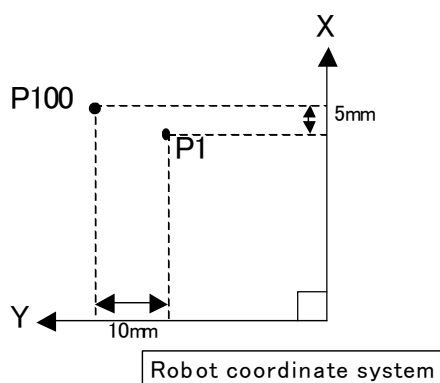
```
10 P2=(10,5,0,0,0,0)(0,0)
20 P100=P1*P2
30 MOV P1
40 MVS P100
P1=(200,150,100,0,0,45)(4,0)
```

In this example, the hand tip is moved relatively within the P1 tool coordinate system at teaching position P1. The values of the X and Y coordinates of P2 become the amount of movement within the tool coordinate system. The relative calculation is given by multiplication of the P variables. Be aware that the result becomes different if the order of multiplication is different. The variable that specifies the amount of relative movement (P2) should be entered lastly.

If the posture axis parts of P2 (A, B, and C) are 0, the posture of P1 is used as is. If there are non-zero values available, the new posture is determined by rotating the hand around the Z, Y, and X axes (in the order of C, B, and A) relative to the posture of P1. Multiplication corresponds to addition within the tool coordinate system, while division corresponds to subtraction within the tool coordinate system.

(3) Relative calculation of position data (Addition)

Addition of P variables
(relative calculation in the robot coordinate system)



An example of relative calculation(Addition)

```
10 P2=(5,10,0,0,0,0)(0,0)
20 P100=P1+P2
30 MOV P1
40 MVS P100
P1=(200,150,100,0,0,45)(4,0)
```

In this example, the hand is moved relatively within the robot coordinate system at teaching position P1. The values of the X and Y coordinates of P2 become the amount of movement within the robot coordinate system. The relative calculation is given by addition of the P variables.

If a value is entered for the C-axis coordinate of P2, it is possible to change the C-axis coordinate of P100. The resulting value will be the sum of the C-axis coordinate of P1 and the C-axis coordinate of P2.

CAUTION)

In the example above, the explanation is made in two dimensions for the sake of simplicity. In actuality, the calculation is made in three dimensions. In addition, the tool coordinate system changes depending on the posture.

4.1.7 Appended statement

A process can be added to a movement command.

■ Appended statement

Appended statement	Explanation
WTH	Unconditionally adds a process to the movement command.
WTHIF	Conditionally adds a process to the movement command.

■ Statement example

Statement example	Explanation
MOV P1 WTH M_OUT (20) = 1	Turns output signal bit 20 ON simultaneously with the start of movement to P1.
MOV P1 WTHIF M_IN (20) = 1, HLT	Stops if the input signal bit 20 turns ON during movement to P1.
MOV P1 WTHIF M_IN (19) = 1, SKIP	Stops movement to P1 if the input signal bit 19 turns ON during movement to P1, and then proceeds to the next step.

■ Related functions

Function	Explanation page
Joint interpolation movement	Page 59, "(1) Joint interpolation movement"
Linear interpolation movement	Page 60, "(2) Linear interpolation movement"
Circular interpolation movement	Page 61, "(3) Circular interpolation movement"
Stopping	Page 77, "(6) Stopping"

4.2 Multitask function

4.2.1 What is multitasking?

The multitask function is explained in this section.

Multitasking is a function that runs several programs as parallel, to shorten the tact time and enable control of peripheral devices with the robot program.

Multitasking is executed by placing the programs, to be run in parallel, in the items called “slots” (There is a total of 32 task slots. The maximum factory default setting is 8.) .

The execution of multitask operation is started by activating it from the operation panel or by a dedicated input signal, or by executing an instruction related to multitask operation.

The execution environment for multitasking is shown in Fig. 4-1.

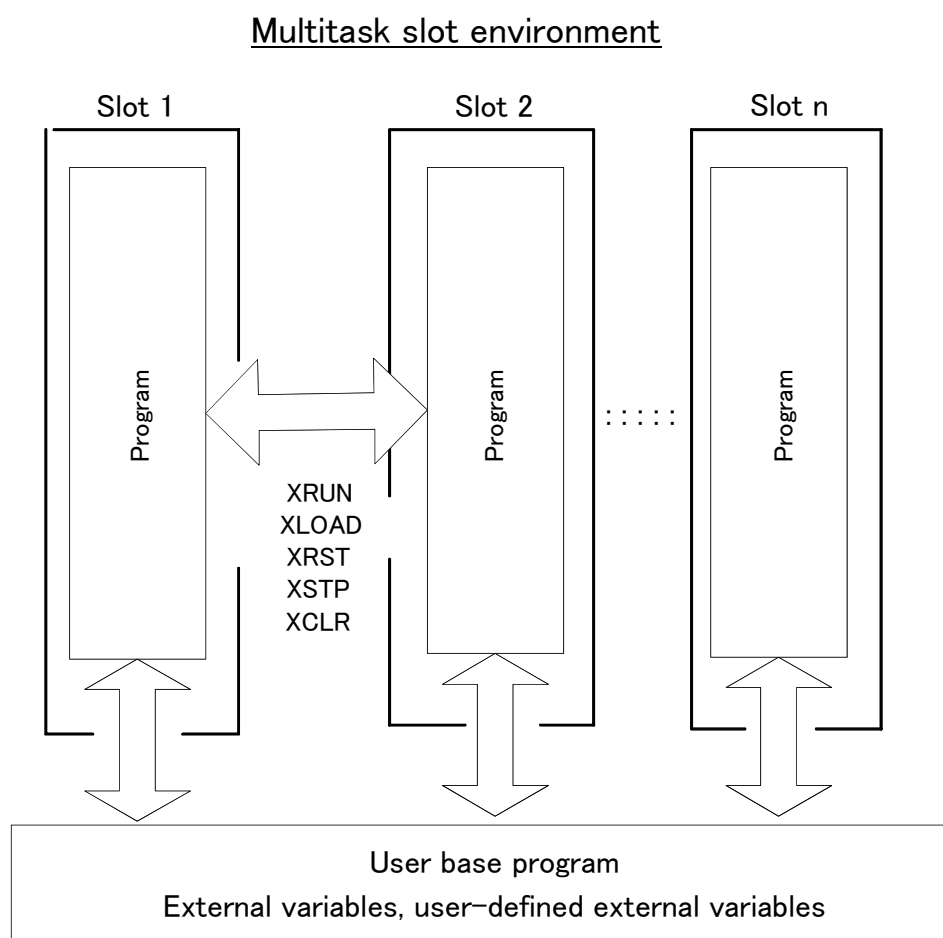


Fig.4-1 : Multitask slot environment

◇◆◇ Execution of a program ◇◆◇

A program is executed by placing it in an item called a “slot” and running it. For example, when running one program (when normally selecting and running the program with the controller’s operation panel), the controller system unconditionally places the program selected with the operation panel in slot 1.

4.2.2 Executing a multitask

The multitask can be executed with the following three methods.

	Types of execution	Explanation
1	Execution from a program	This method starts parallel operation of the programs from a random position in the program using a MELFA-BASIC IV command. The programs to be run in parallel can be designated, and a program running in parallel can be stopped. This method is effective when selecting the programs to be run in parallel according to the program flow. The related commands include the XLOAD, XRUN, XSTP and XRST commands. Refer to Page 111, "4.11 Detailed explanation of command words" in this manual for details.
2	Execution from controller operation panel or external input/output signal	In this execution type, depending on the setting of the information of the "SLT*" parameter, the start operation starts concurrent execution or constant concurrent execution, or starts concurrent execution at error occurrence. It is necessary to set the "SLT*" parameter in advance. This method does not rely on the program flow, and is effective for carrying out simultaneous execution with a preset format, or for sequential execution.
3	Executing automatically when the power is turned on	It is possible to start constant execution immediately after turning the controller's power on. If ALWAYS is specified for the start condition of the SLT* parameter, the program is executed in constant execution mode immediately after the controller's power is turned on. This eliminates the trouble of starting the programs in task slots used for monitoring input/output signals from the PLC side. In addition, it is possible to execute a program from within another program that controls movement continuously. In this case, set the value of the "ALWENA" parameter to 7 in order to execute X** instructions such as XRUN and XLOAD, the SERVO instruction, and the RESET instruction.

4.2.3 Operation state of each slot

The operation state of each slot changes as shown in [Fig. 4-2](#) according to the operations and commands. Each state can be confirmed with the robot status variable or external output signal.

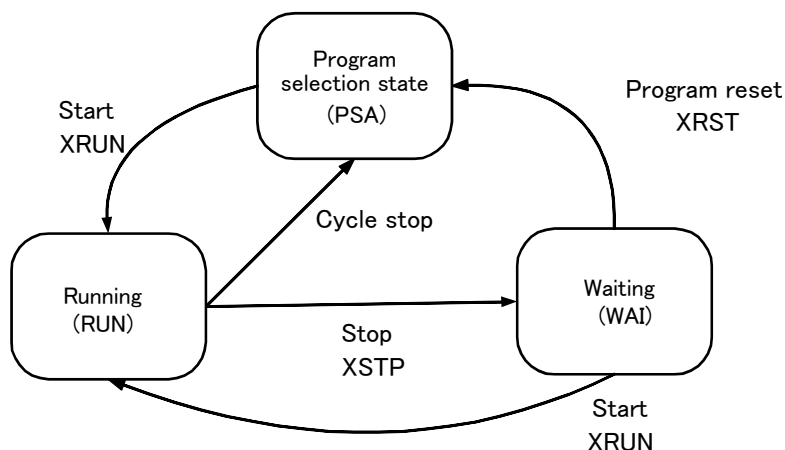


Fig.4-2 : Operation state of each slot

<About parameters related to task slots>

The parameters SLT1 to SLT32 contain information about the name of the program to be executed, operation mode, start condition, and priority for each of the 32 task slots (set to 8 slots at the factory default setting).

Please refer to [Page 266, "5 Functions set with parameters"](#) for details.

■ Designation format

Parameter name = 1. program name, 2. operation format, 3. starting conditions, 4. order of priority

■ Various setting values and meanings

Item of parameter	Default value	Setting value	Explanation
1. Program name	SLT1: Program selected on the operation panel. SLT2 to 32: Name of the program to be specified with a parameter.	Possible to set a registered program	Use the parameter to specify the execution of predetermined programs in multitask operation. If the programs to be executed vary depending on conditions, it is possible to specify the program using the XLOAD and XRUN instructions in another program. The programs selected on the operation panel are set if SLT1 is specified.
2. Operation format	REP	• REP : Continuous operation	If REP is specified, the program is executed again from the top after the program ends when the final line of the program is reached, or by execution of the END instruction.
		• CYC : One cycle operation	If CYC is specified, the program ends after being executed for one cycle and the selected status is canceled. Change the SLOTON setting of the parameter if it is desired to keep the program in the selected status. Please refer to the section for SLOTON in Page 266, "5 Functions set with parameters" for details.
3. Starting conditions	START	• START : Execution of a program using the START button on the operation panel or the I/O START signal	Select START when it is desired to start normally. ^{Note1)}
		• ALWAYS : Execution of a program when the controller's power is turned on	Use ALWAYS when it is desired to execute the program in constant execution mode. Note, however, that it is not possible to execute movement instructions such as MOV during constant execution of a program. Programs in constant execution mode can be stopped via the XSTP instruction. They cannot be stopped via the operation panel and external input signals, or emergency stop. The operation mode (REP/CYC) is ignored.
		• ERROR : Execution of a program when the controller is in error status	Specify ERROR when it is desired to execute a program in case an error occurs. It is not possible to execute instructions for moving the robot, such as the MOV instruction. The operation mode (REP/CYC) is one-cycle operation (CYC) regardless of the setting value.
4. Order of priority (number of lines executed in priority)	1	1 to 31: Number of lines executed at one time at multitask operation	If this number is increased, the number of lines executed at one time for the task slot is increased. For example, if 10 is specified for SLT1, 5 for SLT2, and 1 for SLT3, then after 10 lines of the program specified in SLT1 have been executed, five lines of the program specified in SLT2 are executed, and then one line of the program specified in SLT3 is executed. Afterward this cycle will be repeated.

Note1) The start operation conducted from the operation panel or by sending the dedicated input signal START will start the execution of programs of all the task slots whose start conditions are set to "START" at the same time.

To start the program independently, start in slot units with the dedicated input signal (S1 START to S32START). In this case, the line No. is preassigned to the same dedicated input/output parameter.

Refer to [Page 308, "6.3 Dedicated input/output"](#) in this manual for details on the assignment of the dedicated input/output.

■ Setting example

An example of the parameter settings for designating the following items in slot 2 is shown below.

Designation details) Program name : 5
 Operation format : Continuous operation
 Starting conditions : Always
 Order of priority : 10
 SLT2=5, REP, ALWAYS, 10

4.2.4 Precautions for creating multitask program

(1) Relationship between number of tasks and processing time

During multitask operation, it appears as if several robot programs are being processed concurrently. However, in reality, only one line is executed at any one time, and the processing switches from program to program (it is possible to change the number of lines being executed at a time. See the section for the "SLTn" parameter in "Setting Functions by Parameters" on page 247). This means that if the number of tasks increases, the overall program execution time becomes longer. Therefore, when using multitask operation, the number of tasks should be kept to a minimum. However, programs of other tasks executing movement instructions (the MOV and MVS instructions) are processed at any time.

(2) Specification of the maximum number of programs executed concurrently

The number of programs to be run in parallel is set with parameter TASKMAX. (The default value is 8.) To run more than 8 programs in parallel, change this parameter.

(3) How to pass data between programs via external variables

Data is passed between programs being executed in multitask operation via program external variables such as M_00 and P_00 (refer to [Page 100, "4.3.22 External variables"](#)) and the user-defined external variables (refer to [Page 101, "4.3.24 User-defined external variables"](#)). An example is shown below. In this example, the on/off status of input signal 8 is judged by the program specified in task slot 2. Then this program notifies the program specified in task slot 1 that the signal is turned on by means of the external variable M_00.

```
<Slot 1>
10 M_00=0                ; Substitute 0 in M_00
20 IF M_00=0 THEN 20      ; Wait for M_00 value to change from 0.
30 M_00=0                ; Substitute 0 in M_00
40 MOV P1                ; Proceed with the target work.
50 MOV P2
    :
100 GOTO 20              ; Repeat from line 20.
```

```
<Slot 2> (Program of signals and variables)
10 IF M_IN(8) <> 1 THEN 30 ; Branch to line 30 if input signal 8 is not ON.
20 M_00=1                ; Substitute 1 in M_00
30 MOV P1                ; Proceed with the target work.
    :
```

(4) Confirmation of operating status of programs via robot status variables

The status of the program running with multitask can be referred to from any slot using the robot status variables (M_RUN, M_WAI, M-ERR).

Example) M1 = M_RUN (2) The operation status of slot 2 is obtained.

Refer to [Page 102, "4.3.26 Robot status variables"](#) for details on the robot status variables.

(5) The program that operates the robot is basically executed in slot 1.

The program that describes the robot arm's movement, such as with the MOV commands, is basically set and executed in slot 1. To run the program in a slot other than slot 1, the robot arm acquisition and release command (GETM, RELM) must be used. Refer to [Page 111, "4.11 Detailed explanation of command words"](#) in this manual for details on the commands.

(6) How to perform the initialization processing via constantly executed programs

Programs specified in task slots whose start condition is set to ALWAYS are executed continuously even if the operation mode is set to CYC. Therefore, in order to perform the initialization processing, they should be programmed in such a way that the initialization processing is not executed more than once by jumping within the program using the GOTO instruction, etc.

◆◆◆ Mechanism 1 is assigned to slot 1 ◆◆◆

In the default state, mechanism 1 (robot arm of standard system) is automatically assigned to slot 1.

Because of this, slot 1 can execute the movement command even without acquiring mechanism 1 (without executing GETM command). However, when executing the movement command in a slot other than slot 1, the slot 1 mechanism acquisition state must be released (RELM command executed), and the mechanism must be acquired with the slot that is to execute the movement command (execute the GETM command).

4.2.5 Precautions for using a multitask program

(1) Starting the multitask

When starting from the operation panel or with the dedicated input signal START, the programs in all slots for which the "start request execution" is set in the slot parameter start conditions will start simultaneously.

When starting with the dedicated input signals S1START to S32START, the program can be started in each slot.

In this case, the line No. is preassigned to the same dedicated input/output parameter. Refer to [Page 308](#), "6.3 Dedicated input/output" for details on the assignment of the dedicated input/output.

(2) Display of operation status

The LEDs of the [START] and [STOP] switches on the operation panel and the dedicated input/output signals START and STOP display the operation conditions of programs specified in task slots for which the start conditions are set to "START" in the corresponding "SLT*" parameter. If at least one program is operating, the LED of the [START] switch lights up and the dedicated output signal START turns on. If all the programs stop, the LED of the [STOP] switch is lit and the dedicated output signal STOP turns on.

The dedicated output signals S1START to S32START and S1STOP to S32STOP output the operation status for each of the task slots. If it is necessary to know the individual operation status, signal numbers can be assigned to the dedicated input/output parameters and their status checked with the status of the external signals.

For a detailed description of assignment of dedicated input/output, please refer to "6.3 Dedicated Input/Output" on page 272 of this manual.

The status of programs whose start condition is set to ALWAYS or ERROR does not affect the LEDs of the [START] and [STOP] switches. The operation status of programs in constant execution mode can be checked using the monitoring tool of the PC support software (optional).

4.2.6 Example of using multitask

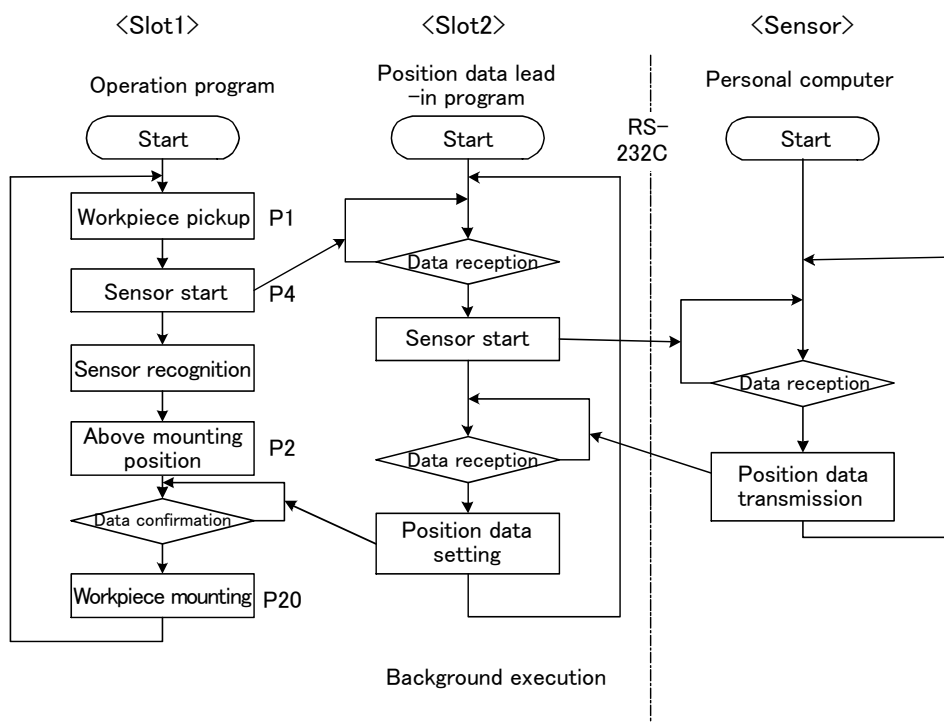
An example of the multitask execution is given in this section.

(1) Robot work details.

The robot programs are the "movement program" and "position data lead-in program".

The "movement program" is executed with slot 1, and the "position data lead-in program" is executed with slot 2. If a start command is output to the sensor while the robot is moving, a request for data will be made to the personal computer via the position data lead-in program. The personal computer sends the position data to the robot based on the data request. The robot side leads in the compensation data via the position data lead-in program.

< Process flow >



P1: Workpiece pickup position (Vacuum timer DLY 0.05)

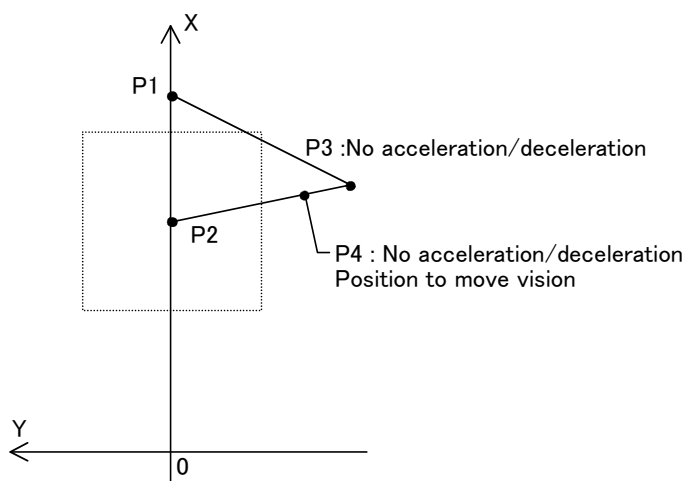
P2: Workpiece placing position (Release timer DLY 0.05)

P3: Vision pre-position (Do not stop at penetration point CNT)

P4: Vision shutter position (Do not stop at penetration point CNT)

P_01: Vision compensation data

P20: Position obtained by adding P2 to vision compensation data (relative operation)



(2) Procedures to multitask execution

■ Procedure 1: Program creation

<1> Movement program (Program name: 1)

```

100 CNT 1           'Validate path connected movement
110 MOV P2,10       'Move to +10mm above P2
120 MOV P1,10       'Move to +10mm above P1
130 MOV P1          'Move to P1 workpiece pickup position
135 M_OUT(10)=0     'Pickup workpiece
140 DLY 0.05        'Timer 0.05 second
150 MOV P1,10       'Move to +10mm above P1
160 MOV P3          'Move to vision pre-position P3
165 SPD 500         'Set linear speed to 500mm/sec.
170 MVS P4          'Start vision lead-in with P4 passage
180 M_02#=0         'Start data lead-in with background process at interlock variable
                      (M_01=1/M_02=0)

190 M_01#=1         'Start data load-in with background process
200 MVS P2,10       'Move to +10mm above P2
210 IF M_02#=0 THEN GOTO 210 'Wait for interlock variable M_02 to reach 1
220 P20=P2*P_01     'Add vision compensation P_01 to P20, and move to +10mm above
230 MOV P20,10      'Move to +10mm above P20
240 MOV P20         'Go to P20 workpiece placing position
245 M_OUT(10)=1     'Place workpiece
250 DLY 0.05        'Timer 0.05 second
260 MOV P20,10      'Move to +10mm above P20
270 CNT 0           'Invalidate path connected movement
280 END             'End one cycle

```

<2> Position data lead-in program (Program name: 2)

```

100 IF M_01#=0 THEN GOTO 100 'Wait for interlock variable M_01 to reach 1
105 OPEN "COM1:" AS #1       'Open RS-232-C line
110 DLY M_03#                'Hypothetical process timer (0.05 second)
115 PRINT #1,"SENS"         'Transmit character string "SENS" to RS-232-C (vision side)
117 INPUT #1,M1,M2,M3        'Wait to lead-in vision compensation value (relative data)
120 P_01.X=M1                'Substitute  $\Delta$  X coordinate
130 P_01.Y=M2                'Substitute  $\Delta$  Y coordinate
140 P_01.Z=0.0               '
150 P_01.A=0.0               '
160 P_01.B=0.0               '
170 P_01.C=RAD(M3)           'Substitute  $\Delta$  C coordinate
175 CLOSE                    'Close RS-232-C line
180 M_01#=0                  'Interlock variable M_01 = 0
190 M_02#=1                  'Interlock variable M_02 = 0
200 END                       'End process

```

■ Procedure 2: Setting the slot parameters

Set the slot parameters as shown below.

Parameters	Program name	Operation mode	Operation format	Number of executed lines
SLT1	1	REP	START	1
SLT2	2	REP	START	1

■ Procedure 3: Reflecting the slot parameters

Turn the power OFF and ON to validate the slot parameters.

■ Procedure 4: Starting

Start the program 1 and program 2 operation by starting from the operation panel.

4.3 Detailed specifications of MELFA-BASIC IV

In this section, detailed explanations of the MELFA-BASIC IV format and syntax such as configuration are given, as well as details on the functions of each command word. The following explains the components that constitute a statement.

(1) Program name

A program name can be specified using up to 12 characters. However, the operation panel display can display only up to four characters; it is therefore recommended to specify the program name using up to four characters.

Moreover, the characters that may be used are as follows.

Class	Usable characters
Alphabetic characters	A B C D E F G H I J K L M N O P Q R S T U V W X Y Z (Use uppercase characters only. If a program name is registered using lowercase characters, the program may not be executed normally.)
Numerals	0 1 2 3 4 5 6 7 8 9

If a program name is specified using more than four characters, the program cannot be selected from the operation panel. In addition, if it is desired to use an external output signal to select a program to be executed, the program name should be specified using the numbers. If a program is executed as a sub-program via the CALLP instruction, more than four alphabetic characters may be used. However, such programs may not be selected from the operation panel.

(2) Command statement

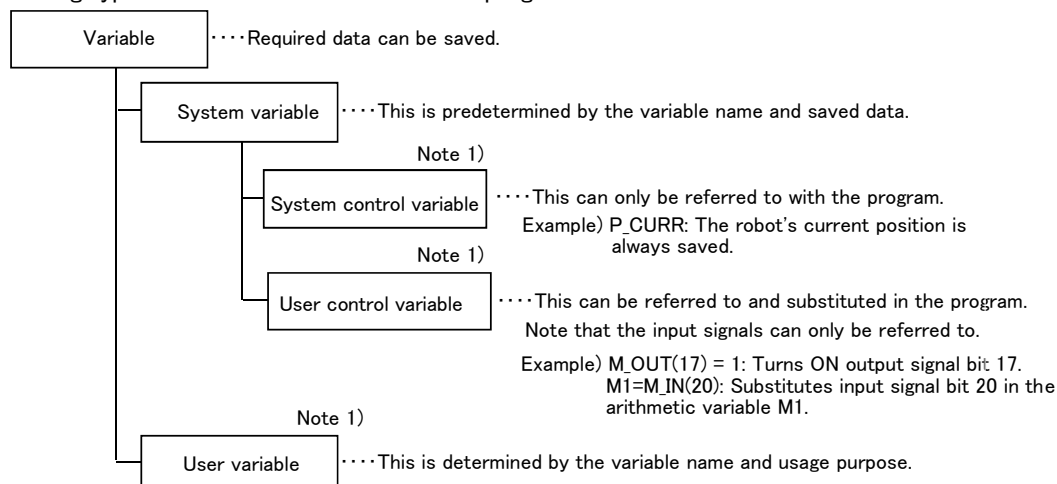
Example of constructing a statement

10 MOV P1 WTH M OUT(17)=1
 1) 2) 3) 4)

- 1) Line No. : Numbers for determining the order of execution within the program. Lines are executed in ascending order.
- 2) Command word : Instructions for specifying the robot's movement and tasks
- 3) Data : Variables and numerical data necessary for each instruction
- 4) Appended statement : Specify these as necessary when adding robot tasks.

(3) Variable

The following types of variables can be used in a program.



Note 1) Each variable is categorized into the following classes.

Position type variable	The robot's orthogonal coordinate value is saved. The variable name starts with "P". Example) MOV P1: The robot moves to the position saved in variable name P1.
Joint type variable	The robot's joint angle is saved. The variable name starts with "J". Example) MOV J1: The robot moves to the position saved in variable name J1.
Arithmetic type variable	A numeric value (integer, real value, etc.) is saved. The variable name starts with "M". Example) M1 = 1: The value 1 is substituted in variable name M1.
Character type variable	A character string is saved. A "\$" is added to the end of the variable name. Example) C1\$ = "ERROR": the character string "ERROR" is substituted in variable name C1\$.

4.3.1 Statement

A statement is the minimum unit that configures a program, and is configured of a command word and data issued to the word.

Example) MOV P1
 Command word Data
 Command statement

4.3.2 Appended statement

Command words can be connected with an appended statement, but this is limited to movement commands.

This allows some commands to be executed in parallel with a movement command.

Example) MOV P1 WTH M OUT (17) = 1
 Command statement Appended statement Command statement

Please refer to [Page 194, "WTH"](#) or [Page 195, "WTHIF"](#), as well as each of the movement instructions (MOV, MVS, MVR, MVR2, MVR3, MVC, MVA) for detailed descriptions.

4.3.3 Line

A line is consisted of a line No. and one command statement. Note that if an appended statement is used, there will be two command statements.

One line can have up to 127 characters. (This does not include the last character of the line.)

◆◆◆ Only one command statement per line ◆◆◆

Multiple command statements cannot be separated with a semicolon and described on one line as done with the general BASIC.

4.3.4 Line No.

Line Nos. should be in ascending order, starting from the first line, in order for the program to run properly. When a program is stored in the memory, it is stored in the order of the line Nos.

Line Nos. can be any integer from 1 to 32767.

◆◆◆ Direct execution if line No. is not assigned ◆◆◆

If an instruction statement is described without a line number on the instruction screen of the T/B, the statement is executed as soon as it is input. This is called direct execution. In this case, the command statement will not be saved in the memory, but the value substitution to the variable will be saved.

4.3.5 Label

A label is a user-defined name used as a marker for branching.

A label can be created by inserting an asterisk (*) followed by uppercase or lowercase alphanumeric characters after the line No. The head of the label must be an alphabetic character, and the entire label must be within eight characters long. If a label starting with the alphabetic character L is described after the asterisk, an underscore (_) can be used immediately after the character.

* Characters that cannot be used in labels:

- Reserved words (DLY, HOPEN, etc.)
- Any name that begins with a symbol or numeral
- Any name that is already used for a variable name or function name

Example) 10 GOTO *LBL
 100 *LBL

4.3.6 Types of characters that can be used in program

The character which can be used within the program is shown in [Table 4-2](#). However, there are restrictions on the characters that can be used in the program name, variable name and label name. The characters that can be used are indicated by ○, those that cannot be used are indicated by ×, and those that can be used with restrictions are indicated by △.

Table 4-2 : List of characters that can be used

Class	Available characters	Program name	Variable name	Label name
Alphabetic characters	A B C D E F G H I J K L M N O P Q R S T U V W X Y Z	○	○	○
	a b c d e f g h i j k l m n o p q r s t u v w x y z	×	△ Note1)	△ Note 1)
Numerals	0 1 2 3 4 5 6 7 8 9	○	△ Note2)	○
Symbols	" ' & () * + - . , / : ; = < > ? @ ` [¥] ^ { } ~	×	×	×
	! # \$ %	×	Available for type specification	×
	_ (Underscore)	×	△ Note3)	△ Note4)
Spaces	Space character	×	×	×

Note1) Alphanumerics in variable names and label names in the program are automatically converted into uppercase characters.

Note2) Only alphabetical characters can be used as the first character of the variable name. Numerals can be used as the second and succeeding characters.

Note3) They can be used as the second and succeeding characters. Any variable having an underscore () as the second character becomes an external variable.

Note4) If an underscore () is used in a label name, start with an asterisk (*) followed by alphabetic character "L."

Refer to [Page 90, "\(1\) Program name"](#) for detail of program names, refer to [Page 97, "4.3.15 Variables"](#) for detail of variable names, and refer to [Page 91, "4.3.5 Label"](#) for detail of label names.

4.3.7 Characters having special meanings

(1) Uppercase and lowercase identification

Lowercase characters will be resigned as lowercase characters when they are used in comments or in character string data. In all other cases, they will be converted to uppercase letters when the program is read.

(2) Underscore (_)

The underscore is used for the second character of an identifier (variable name) to identify the variable as an external variable between programs. Refer to [Page 100, "4.3.22 External variables"](#) for details.

Example) P_CURR, M_01, M_ABC

(3) Apostrophe (')

The apostrophe (') is used at the head of all comments lines. When assigned at the head of a character it is a substitute for the REM statement.

Example) 100 MOV P1 'GET ; GET will be set as the comment.
150 'GET PARTS ; This is the same as 150 REM GET PARTS.

(4) Asterisk (*)

The asterisk is placed in front of label names used as the branch destination.

Example) 200 *CHECK

(5) Comma (,)

The comma is used as a delimiter when there are several parameters or suffixes.

Example) P1=(200, 150,)

(6) Period (.)

The period is used for obtaining certain components out of multiple data such as decimal points, position variables and joint variables.

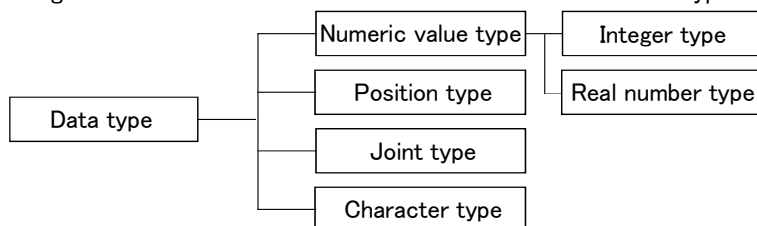
Example) M1 = P2.X ; Substitute the position variable P2.X coordinate element in numeric variable M1.

(7) Space

The space character, when used as part of a character string constant or within a comments line, is interpreted as a character. The space character is required as a delimiter immediately after a line No. or a command word, and between data items. In the [Format] given in section [Page 111, "4.11 Detailed explanation of command words"](#), the space is indicated with a " □ " where required.

4.3.8 Data type

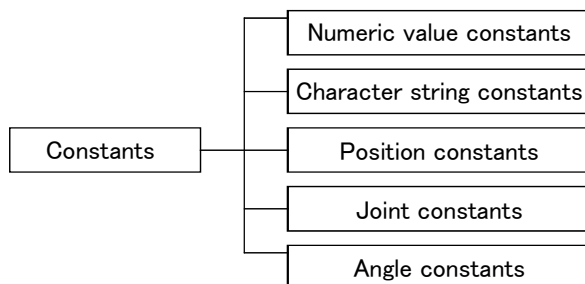
In MELFA BASIC IV it is possible to use four data types: numerical values, positions, joints, and character strings. Each of these is called a "data type." The numerical value data type is further classified into real numbers and integers. There can be variables and constants of each data type.



Example) Numeric value type	M1 [Numeric value variables], 1 [Numeric value constants] (Integer), 1.5 [Numeric value constants](Real number)
Position type	P1 [Position variables], (0,0,0,0,0,0) (0,0) [Position constants]
Joint type	J1 [Joint variables], (0,0,0,0,0,0) [Joint constants]
Character type	C1\$ [Character string variables], "ABC" [Character string constants]

4.3.9 Constants

The constant types include the numeric value constant, character string constant, position constant, joint constant and angle constant.



4.3.10 Numeric value constants

The syntax for numeric value constants is as follows. Numerical constants have the following characteristics.

(1) Decimal number

Example) 1, 1.7, -10.5, +1.2E+5 (Exponential notation)

Valid range -1.7976931348623157e+308 to 1.7976931348623157e+308

(2) Hexadecimal number

Example) &H0001, &HFFFF

Valid range &H0000 to &HFFFF

(3) Binary number

Example) &B0010, &B1111

Valid range &B0000000000000000 to &B1111111111111111

(4) Types of constant

The types of constants are specified by putting symbols after constant characters.

Example) 10% (Integer), 1.0005! (Single-precision real number), 10.000000003# (Double-precision real number)

4.3.11 Character string constants

String constants are strings of characters enclosed by double quotation marks (").

Example) "ABCDEFGH I JKLMN" "123"

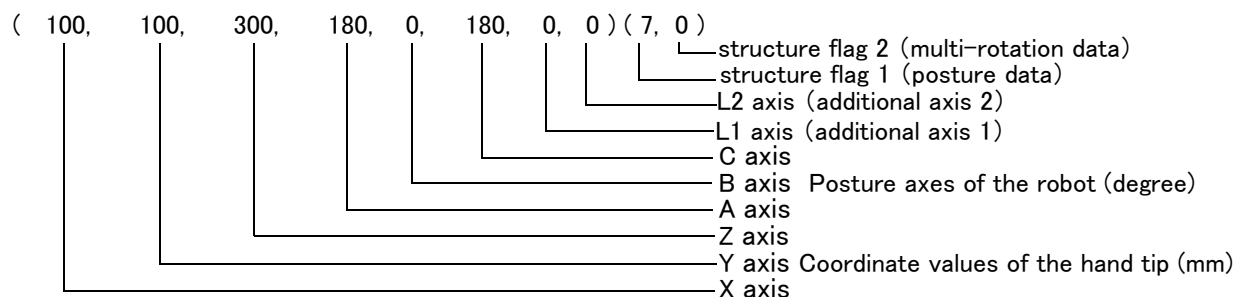
Up to 127 characters for character string

The character string can have up to 127 characters, including the line No. and double quotations.

Enter two double quotation marks successively in order to include the double quotation mark itself in a character string. For the character string AB"CD, input "AB""CD".

4.3.12 Position constants

The syntax for position constants is as shown below. Variables cannot be described within position constants.



Example)

P1=(300, 100, 400, 180, 0, 180, 0, 0) (7, 0)

P2=(0, 0, -5, 0, 0, 0) (0, 0) [A case where there is no traveling axis data]

P3=(100, 200, 300, 0, 0, 90) (4, 0) [A case of a 4-axis horizontal multi-joint robot]

(1) Coordinate, posture and additional axis data types and meanings

[Format] X, Y, Z, A, B, C, L1, L2

[Meaning] X, Y, Z: coordinate data. The position of the tip of the robot's hand in the XYZ coordinates.
(The unit is mm.)

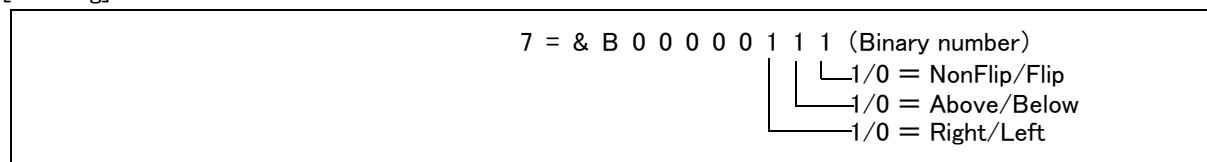
A, B, C: posture data. This is the angle of the posture. (The unit is deg.)

L1, L2: additional axis data. These are the coordinates for additional axis 1 and additional axis 2, respectively. (The unit is mm or deg.)

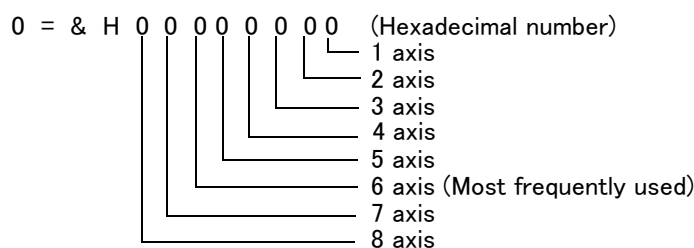
(2) Meaning of structure flag data type and meanings

[Format] FL1, FL2

[Meaning] FL1: Posture data



FL2: Multi-rotation data information - Default value = 0 (The range is 0 to +4294967295 ... Information for eight axes is held with a 1-axis 4-bit configuration.) Two types of screens are available for the PC: screens that display the number of rotations for each axis (-8 to 7) in decimal and those that display the number of rotations for each axis in hexadecimal..



Value of multiple rotation data

	-900	-540	-180	0	180	540	900
Angle of each axis							
Value of multiple rotation data	...	-2 (E)	-1 (F)	0	1	2	...

◇◆◇ Designation of axis No. ◇◆◇

1. There is no need to describe the coordinate and posture data for all eight axes. However, if omitted, the following axis data will be processed as undefined.

For a 4-axis robot (X,Y,Z,C axis configuration), describe as (X, Y, Z, , , C) or (X,Y,Z,0,0,C).

2. To omit all axes, insert at least one ", "(comma), such as (,).

◇◆◇ Use of variables in position element data ◇◆◇

The coordinate, position, additional axis data and structure flag data are called the position element data.

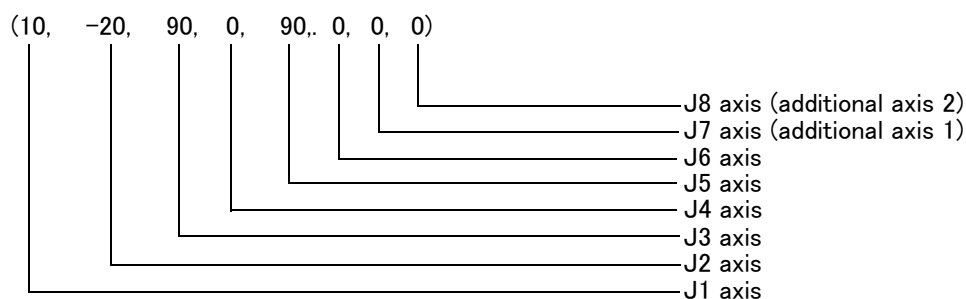
A variable cannot be contained in the position element data that configures the position constant.

◇◆◇ Omitting the structure flag data ◇◆◇

If the structure flag data is omitted, the default value will be applied. ((7,0) Varies depending on the machine model.)

4.3.13 Joint constants

The syntax for the joint constants is as shown below



Example) 6 axis robot	J1 = (0, 10, 80, 10, 90, 0)
6 axis + Additional axis	J1 = (0, 10, 80, 10, 90, 0, 10, 10)
5 axis robot	J1 = (0, 10, 80, 0, 90, 0)
5 axis + Additional axis	J1 = (0, 10, 80, 0, 90, 0, 10, 10)
4 axis robot	J1 = (10, 20, 90, 0)
4 axis + Additional axis	J1 = (10, 20, 90, 0, , , 10, 10)

(1) Axis data format and meanings

[Format] J1,J2,J3,J4,J5,J6,J7,J8

[Meaning] J1 to J6: Robot axis data (Unit is mm or deg.)

J7, J8: Additional axis data, and may be omitted (optional).

(Unit is mm or deg. Depending on the parameter setting.)

The unit is mm, not degrees, if the J3 axis of a horizontal multi-joint type robot is a direct-driven axis.

◇◆◇ Use of variables in joint element data ◇◆◇

The axis data is called the joint element data.

A variable cannot be contained in the joint constant data that configures the joint constant.

4.3.14 Angle value

The angle value is used to express the angle in "degrees" and not in "radian".

If written as 100DEG, this value becomes an angle and can be used as an argument of trigonometric functions.

Example) SIN(90DEG)-----A 90° sine is indicated.

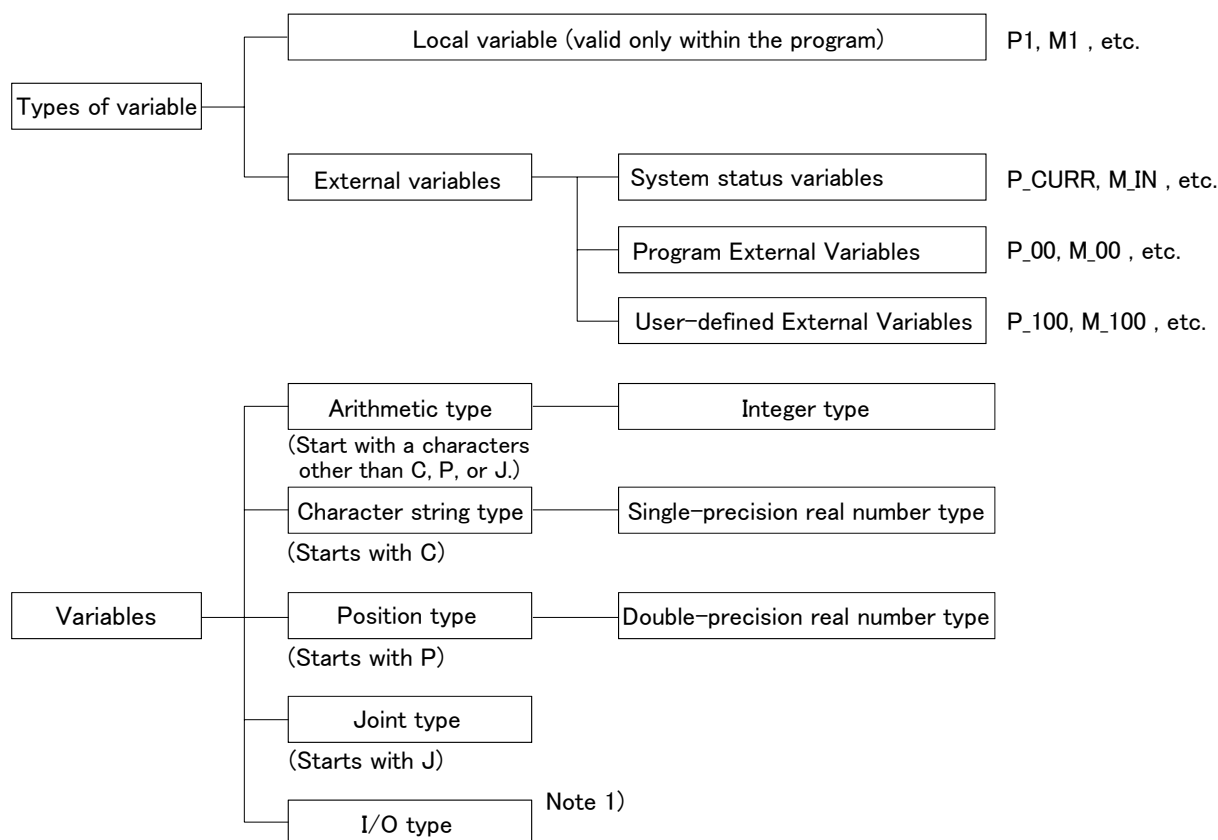
4.3.15 Variables

A variable name should be specified using up to eight characters.

The variable types include the arithmetic type, character string type, position type, joint type and I/O type. Each is called a "variable type". The variable type is determined by the head character of the identifier (variable name). The arithmetic type can be further classified as integer type, single-precision real number type, or double-precision real number type.

The following two types of data valid ranges are used.

1. Local variable valid only in one program
2. Robot status variable, program external variable and user-defined external variable valid over programs. (The user-defined external variable has a _ for the second character of the variable name. Refer to [Page 100, "4.3.22 External variables"](#) for details.)



Note 1) The identifiers include those determined by the robot status variable (M_IN, M_OUT, etc.), and those declared in the program with the DEFIO command.

◆◆◆ Variables are not initialized ◆◆◆

The variables will not be cleared to zero when generated, when the program is loaded, or when reset.

4.3.16 Arithmetic variables

Variables whose names begin with a character other than P, J, or C are considered arithmetic variables. In MELFA BASIC IV, it is often specified that a variable is an arithmetic variable by placing an M at the head. M is the initial letter of mathematics.

Example) M1 = 100

M2! = -1.73E+10

M3# = 0.123

ABC = 1

- 1) It is possible to define the type of variable by attaching an arithmetic type indicator at the end of the variable name. If it is omitted, the variable type is assumed to be of the single-precision real number type.

Arithmetic type suffix	Meaning
%	Integer
!	Single-precision real number type
#	Double-precision real number type

- 2) Once the type of a variable is registered, it can only be converted from integer to single-precision real number. For example, it is not possible to convert the type of a variable from integer to double-precision real number, or from single-precision real number to double-precision real number.
- 3) It is not possible to add an arithmetic type indicator to an already registered variable. Include the type indicator at the end of the variable name at the declaration when creating a new program.
- 4) If the value is exceeded during a single precision = double precision execution, an error will occur.

Table 4-3 : Range of arithmetic variable data

Type	Range	
Integer type	-32768 to 32767	
Single-precision real number type	-3. 40282347e+38 to 3. 40282347e+38	Note) E expresses a power of 10.
Double-precision real number type	-1. 7976931348623157e+308 to 1. 7976931348623157e+308	

4.3.17 Character string variables

A character string variable should start with C and end with "\$." If it is defined by the DEF CHAR instruction, it is possible to specify a name beginning with a character other than C.

Example) C1\$ = "ABC"

CS\$ = C1\$

DEF CHAR MOJI

MOJI = "MOJIMOJI"

4.3.18 Position variables

Variables whose names begin with character P are considered position variables. If it is defined by the DEF POS instruction, it is possible to specify a name beginning with a character other than P. It is possible to reference individual coordinate data of position variables. In this case, add "." and the name of a coordinate axis, e.g. "X," after the variable name.

P1.X, P1.Y, P1.Z, P1.A, P1.B P1.C, P1.L1, P1.L2

The unit of the angular coordinate axes A, B, and C is radians. Use the DEG function to convert it to degrees.

Example) P1 = PORG

DIM P3(10)

M1 = P1. X (Unit : mm)

M2 = DEG(P1. A) (Unit : degree)

DEG POS L10

MOV L10

4.3.19 Joint variables

A character string variable should start with J. If it is defined by the DEF JNT instruction, it is possible to specify a name beginning with a character other than J.

It is possible to reference individual coordinate data of joint variables.

In this case, add "." and the name of a coordinate axis, e.g. "J1," after the variable name.

JDATA.J1, JDATA.J2, JDATA.J3, JDATA.J4, JDATA.J5, JDATA.J6, JDATA.J7, JDATA.J8

The unit of the angular coordinate axes A, B, and C is radians. Use the DEG function to convert it to degrees.

Example) JSTARAT = (0, 0, 90, 0, 90, 0, 0, 0)

JDATA = JSTART

DIM J3 (10)

M1 = J1.J1 (Unit : radian)

M2 = DEG (J1.J2)

DEF JNT K10

MOV K 10

4.3.20 Input/output variables

The following types of input/output variables are available. They are provided beforehand by the robot status variables.

Input/output variables name	Explanation
M_IN	For referencing input signal bits
M_INB	For referencing input signal bytes (8-bit signals)
M_INW	For referencing input signal words (16-bit signals)
M_OUT	For referencing/assigning output signal bits
M_OUTB	For referencing/assigning output signal bytes (8-bit signals)
M_OUTW	For referencing/assigning output signal words (16-bit signals)
M_DIN	For referencing input registers for CC-Link
M_DOUT	For referencing output registers for CC-Link

Please refer to the robot status variables [Page 216](#), "M_IN/M_INB/M_INW", [Page 221](#), "M_OUT/M_OUTB/M_OUTW", and [Page 213](#), "M_DIN/M_DOUT".

4.3.21 Array variables

Arithmetic variables, character string variables, position variables, and joint variables can all be used in arrays.

Designate the array elements at the subscript section of the variables. Array variables should be declared with the DIM instruction. It is possible to use arrays of up to three dimensions.

Example) Example of definition of an array variable

DIM M1 (10) Single-precision real number type

DIM M2% (10) Integer type

DIM M3 ! (10) Single-precision real number type

DIM M4# (10) Double-precision real number type

DIM P1 (20)

DIM J1 (5)

DIM ABC (10, 10, 10)

The subscript of an array starts from 1.

However, among the robot status variables, the subscript starts from 0 for individual input/output signal variables (M_IN, M_OUT, etc.) only.

Whether it is possible to secure sufficient memory for the variable is determined by the free memory size.

4.3.22 External variables

External variables have a “_” (underscore) for the second character of the identifier (variable name). (It is necessary to register user-defined external variables in the user base program.) The value is valid over multiple programs. Thus, these can be used effectively to transfer data between programs.

There are four types of external variables, numeric value, position, joint and character, in the same manner as the [Page 94, “4.3.8 Data type”](#). The following three types of external variables are available.

Table 4-4 : Types of external variables

External variables	Explanation	Example
Program external variables	Types of external variables	P_01,M_01,P_100(1), etc.
User-defined external variables	The user can determine the name freely. Declare the variables using the DEF POS, DEF JNT, DEF CHAR, or DEF INTE/FLOAT/DOUBLE instructions in the user base program.	P_GENTEN,M_MACHI
Robot status variables (System status variables)	The robot status variables are controlled by the system, and their usage is determined in advance.	M_IN,M_OUT,P_CURR,M_PI, etc.

4.3.23 Program external variables

[Table 4-5](#) lists the program external variables that have been prepared for the controller in advance. As shown in the table, the variable name is determined, but the application can be determined by the user.

Table 4-5 : Program external variables

Data type	Variable name	Qty.	Remarks
Position	P_00 to P_19	20	
Position array (No. of elements 10)	P_100() to P_104()	5	Use the array element in the first dimensions.
Joint	J_00 to J_19	20	
Joint array (No. of elements 10)	J_100() to J_104()	5	Use the array element in the first dimensions.
Numeric value	M_00 to M_19	20	The data type of the variables is double-precision real numbers.
Numeric value array (No. of elements 10)	M_100() to M_104()	5	Use the array element in the first dimensions. The data type of the variables is double-precision real numbers.
Character string	C_00 to C_19	20	
Character string array (No. of elements 10)	C_100() to C_104()	5	Use the array element in the first dimensions.

4.3.24 User-defined external variables

If the number of program external variables listed above is insufficient or it is desired to define variables with unique names, the user can define program external variables using a user base program.

Procedure before using user-defined external variables
1) First, write a user base program. Use “_” for the second character of the variables. 2) Register the program name in the “PRGUSR” parameter and turn the power off and on again. 3) Write a normal program using the user-defined external variables.

- (1) By defining a variable having an underscore () for the second character of the identifier with the DEF statement in the base program (Note), that variable will be handled as an external variable.
- (2) It is not necessary to execute the user base program.
- (3) Write only the lines necessary for declaring variables in the user base program.
- (4) If it is desired to define array variables in a user base program and use them as external variables, it is necessary to declare them using the DIM instruction again in the program in which they will be used. It is not necessary to declare single variables again.

Example) Example of using user-defined external variables

On the main program (program name 1) side

```

10 DIM P_100(10)      ' Re-declaration of external variables
20 DIM M_200(10)      ' Re-declaration of external variables
30 MOV P_100(1)
40 IF M_200(1)=1 THEN HLT

```

On the user base program (program name UBP) side

```

10 DEF POS P_900, P_901, P_902, P_903
20 DIM P_100(10)      ' It is necessary to declare this variable again in the program in which they will
                        ' be used.
30 DEF INTE M_100
40 DIM M_200(10)      ' It is necessary to declare this variable again in the program in which they will
                        ' be used.

```

Parameter name	Value
PRGUSR	UBP

4.3.25 Creating User Base Programs

◆◆◆ Note) What is a user base program? ◆◆◆

A user base program is written when user-defined external variables are to be used, but it is not necessary to execute the program. Simply create a program containing the necessary declaration lines and register it in the “PRGUSR” parameter. After changing the parameter, turn the power off and on again.

◆◆◆ How to register a new user base program using the Personal Computer Support Software ◆◆◆

Using the Personal Computer Support Software, write only instructions to the robot controller first, and write only position data next.

User base programs can be created by using either the teaching box or Personal Computer Support Software, in the same way as the normal programs. To create user base programs using the Personal Computer Support Software, please follow the procedure below:

- 1) Store a program created as a user base program on your personal computer.
- 2) Start Program Manager from Program Editor of the Personal Computer Support Software.
- 3) Specify the program created in step 1) above as the transfer source and the robot as the transfer destination in Program Manager, and perform a “copy” operation. At this point, uncheck the “Position Variables” check box so that only the “Instructions” check box is checked.
- 4) When the copy operation is complete, perform the operation in step 3) above again. Uncheck the “Instructions” check box and check the “Position Variables” check box this time, and then execute.

4.3.26 Robot status variables

The available robot status variables are shown in [Table 4-6](#). As shown in the table, the variable name and application are predetermined.

The robot status can be checked and changed by using these variables.

Table 4-6 : Robot status variables

No	Variable name	Array designation Note1)	Details	Attribute Note2)	Data type, Unit	Page
1	P_CURR	Mechanism No.(1 to 3)	Current position (XYZ)	R	Position type	228
2	J_CURR	Mechanism No.(1 to 3)	Current position (joint)	R	Joint type	207
3	J_ECURRE	Mechanism No.(1 to 3)	Current encoder pulse position	R	Joint type	207
4	J_FBC	Mechanism No.(1 to 3)	Joint position generated based on the feedback value from the servo	R	Joint type	208
5	P_FBC	Mechanism No.(1 to 3)	XYZ position generated based on the feedback value from the servo	R	Position type	229
6	M_CMPDST	Mechanism No.(1 to 3)	Amount of difference between a command value and the actual position when the compliance function is being performed	R	Single-precision real number type, mm	210
7	M_CMPLMT	Mechanism No.(1 to 3)	This is used to recover from the error status by using interrupt processing when an error has occurred while the command value in the compliance mode attempted to exceed the limit.	R	Integer type	211
8	P_TOOL	Mechanism No.(1 to 3)	Currently designated tool conversion data	R	Position type	230
9	P_BASE	Mechanism No.(1 to 3)	Currently designated base conversion data	R	Position type	227
10	P_NTOOL	Mechanism No.(1 to 3)	System default value (tool conversion data)	R	Position type	230
11	P_NBASE	Mechanism No.(1 to 3)	System default value (base conversion data)	R	Position type	227
12	M_OPOVRD	None	Speed override on the operation panel (0 to 100%)	R	Integer type, %	217
13	M_OVRD	Slot No.(1to 32)	Override in currently designated program (0 to 100%)	R	Integer type, %	217
14	M_JOVRD	Slot No.(1to 32)	Currently designated joint override (0 to 100%)	R	Integer type, %	217
15	M_NOVRD	Slot No.(1to 32)	System default value (default value of M_OVRD) (%)	R	Single-precision real number type, %	217
16	M_NJOVRD	Slot No.(1to 32)	System default value (default value of M_JOVRD) (%)	R	Single-precision real number type, %	217
17	M_SKIPCQ	Slot No.(1to 32)	A value of 1 is input if execution of an instruction is skipped as a result of executing the line that includes the last executed SKIP command, otherwise a value of 0 is input.	R	Integer type	224
18	M_RATIO	Slot No.(1to 32)	Fraction of the current movement left before reaching the target position (%)	R	Integer type, %	222
19	M_RDST	Slot No.(1to 32)	Remaining distance left of the current movement (only the three dimensions of X, Y, and Z are taken into consideration: mm)	R	Single-precision real number type, mm	223
20	M_SPD	Slot No.(1to 32)	Current specified speed (valid only for linear/circular interpolation)	R	Single-precision real number type, mm/s	225
21	M_NSPD	Slot No.(1to 32)	System default value (default value of M_SPD) (mm/s)	R	Single-precision real number type, mm/s	225
22	M_RSPD	Slot No.(1to 32)	Current directive speed (mm/s)	R	Single-precision real number type, mm/s	225
23	M_ACL	Slot No.(1to 32)	Current specified acceleration rate (%)	R	Single-precision real number type, %	209
24	M_DACL	Slot No.(1to 32)	Current specified deceleration rate (%)	R	Single-precision real number type, %	209
25	M_NACL	Slot No.(1to 32)	System default value (default value of M_ACL) (%)	R	Single-precision real number type, %	209
26	M_NDACL	Slot No.(1to 32)	System default value (default value of M_DACL) (%)	R	Single-precision real number type, %	209

No	Variable name	Array designation Note1)	Details	Attribute Note2)	Data type, Unit	Page
27	M_ACLSTS	Slot No.(1 to 32)	Current acceleration/deceleration status 0 = Stopped, 1 = Accelerating, 2 = Constant speed, 3=Decelerating	R	Integer type	209
28	M_RUN	Slot No.(1 to 32)	Operation status (1: Operating, 0: Not operating)	R	Integer type	223
29	M_WAI	Slot No.(1 to 32)	Pause status (1: Pausing, 0: Not pausing)	R	Integer type	227
30	M_PSA	Slot No.(1 to 32)	Specifies whether or not the program selection is possible in the specified task slot. (1: Selection possible, 0: Selection not possible, in pause status)	R	Integer type	222
31	M_CYS	Slot No.(1 to 32)	Cycle operation status (1: Cycle operation, 0: Non-cycle operation)	R	Integer type	212
32	M_CSTP	None	Cycle stop operation status (1: Cycle stop, 0: Not cycle stop)	R	Integer type	212
33	C_PRG	Slot No.(1 to 32)	Execution program name	R	Character string type	205
34	M_LINE	Slot No.(1 to 32)	Currently executed line No.	R	Integer type	218
35	M_ERR	None	Error occurring (1: An error has occurred, 0: No errors have occurred)	R	Integer type	214
36	M_ERRLVL	None	Reads an error level. caution/low/high1/high2=1/2/3/4	R	Integer type	214
37	M_ERRNO	None	Reads an error number.	R	Integer type	214
38	M_SVO	Mechanism No.(1 to 3)	Servo motor power on (1: Servo power on, 0: Servo power off)	R	Integer type	225
39	M_UAR	Mechanism No.(1 to 3)	Bit data. (1: Within user specified area, 0: Outside user specified area) (Bit 0: area 1 to Bit 7: area 8)	R	Integer type	226
40	M_IN	Input No.(0 to 32767)	Use this variable when inputting external input signals (bit units). General-purpose bit device: bit signal input 0=off 1=on The signal numbers will be 6000s for CC-Link	R	Integer type	216
41	M_INB	Input No.(0 to 32767)	Use this variable when inputting external input signals (8-bit units) General-purpose bit device: byte signal input The signal numbers will be 6000s for CC-Link	R	Integer type	216
42	M_INW	Input No.(0 to 32767)	Use this variable when inputting external input signals (16- bit units) General-purpose bit device: word signal input The signal numbers will be 6000s for CC-Link	R	Integer type	216
43	M_OUT	Output No.(0 to 32767)	Use this variable when outputting external output signals (bit units). General-purpose bit device: bit signal input 0=off 1=on The signal numbers will be 6000s for CC-Link	RW	Integer type	221
44	M_OUTB	Output No.(0 to 32767)	Use this variable when outputting external output signals (8-bit units) General-purpose bit device: byte signal input The signal numbers will be 6000s for CC-Link	RW	Integer type	221
45	M_OUTW	Output No.(0 to 32767)	Use this variable when outputting external output signals (16-bit units) General-purpose bit device: word signal input The signal numbers will be 6000s for CC-Link	RW	Integer type	221
46	M_DIN	Input No.(from 6000)	CC-Link's remote register: Input register	R	Integer type	213
47	M_DOUT	Output No.(from 6000)	CC-Link's remote register: output register	RW	Integer type	213
48	M_HNDCQ	Input No.(1 to 8)	Returns a hand check input signal.	R	Integer type	215
49	P_SAFE	Mechanism No.(1 to 3)	Returns an evasion point position.	R	Position type	229
50	J_ORIGIN	Mechanism No.(1 to 3)	Returns the joint coordinate data when setting the origin.	R	Joint type	208
51	M_OPEN	File No.(1 to 8)	Returns the open status of the specified file or the communication line.	R	Integer type	220

No	Variable name	Array designation Note1)	Details	Attribute Note2)	Data type, Unit	Page
52	C_MECHA	Slot No.(1 to 32)	Returns the type name of the robot.	R	Character string type	205
53	C_MAKER	None	Shows manufacturer information (a string of up to 64 characters).	R	Character string type	204
54	C_USER	None	Returns the content of the parameter "USERMSG." (a string of up to 64 characters).	R	Character string type	206
55	C_DATE	None	Current date expressed as "year/month/date".	R	Character string type	204
56	C_TIME	None	Current time expressed as "time/minute/second".	R	Character string type	206
57	M_BTIME	None	Returns the remaining battery capacity time (hours).	R	Integer type, Time	210
58	M_TIMER	Timer No. (1 to 8)	Constantly counting. Value can be set. [ms] It is possible to measure the precise execution time by using this variable in a program.	RW	Single-precision real number type	226
59	P_ZERO	None	A variable whose position coordinate values (X, Y, Z, A, B, C, FL1, FL2) are all 0	R	Position type	230
60	M_PI	None	Circumference rate (3.1415...)	R	Double-precision real number type	221
61	M_EXP	None	Base of natural logarithm (2.71828...)	R	Double-precision real number type	214
62	M_G	None	Specific gravity constant (9.80665)	R	Double-precision real number type	215
63	M_ON	None	1 is always set	R	Integer type	219
64	M_OFF	None	0 is always set	R	Integer type	219
65	M_MODE	None	Contains the status of the key switch of the operation panel TEACH/AUTO(OP)/AUTO(Ext.)(1/2/3)	R	Integer type	218

Note1) Mechanism No.1 to 3, Specifies a mechanism number corresponding to the multitask processing function.

Slot No.1 to 32, Specifies a slot number corresponding to the multitask function.

Input No.0 to 32767: (theoretical values). Specifies a bit number of an input signal.

Output No.0 to 32767: (theoretical values). Specifies a bit number of an output signal.

Note2) R.....Only reading is possible.

RW.....Both reading and writing are possible.

4.4 Logic numbers

Logic numbers indicate the results of such things as comparison and input/output.

If not 0 when evaluated with an Integer, then it is true, and if 0, it is false. When substituted, if true, 1 is assigned.

The processes that can use logic numbers are shown in [Table 4-7](#).

Table 4-7 : Values corresponding to true or false logic number

Items expressed with logic number "1"	Items expressed with logic number "0"
• Result of comparison operation (if true) • Result of logic operation (if true) • Switch ON • Input/output signal ON • Hand open (supply current to the hand) • Settings for enable/valid such as for interrupts	• Result of comparison operation (if false) • Result of logic operation (if false) • Switch OFF • Input/output signal OFF • Hand close (do not supply current to the hand) • Settings for disable/invalid such as for interrupts

4.5 Functions

A function carries out a specific operation for an assigned argument, and returns the result as a numeric value type or character string type. There are built-in functions, that are preassembled, and user-defined functions, defined by the user.

(1) User-defined functions

The function is defined with the DEF FN statement.

Example) DEF FNMAADD (MA, MB) = MA + MB

• • • • • The function to obtain the total of two values is defined with FNMADD.

The function name starts with FN, and the data type identification character (C: character string, M: numeric value, P: position, J: joint) is described at the third character. The function is designated with up to eight characters.

(2) Built-in functions

A list of assembled functions is given in [Table 4-8](#).

Table 4-8 : List of built-in functions

Class	Function name (format)	Functions	Page	Result
Numeric functions	ABS (<Numeric expression>)	Produces the absolute value	232	Numeric value
	CINT (<Numeric expression>)	Rounds off the decimal value and converts into an integer.	238	
	DEG (<Numeric expression:radian>)	Converts the angle unit from radian (rad) to degree (deg).	241	
	EXP (<Numeric expression>)	Calculates the value of the expression's exponential function	242	
	FIX (<Numeric expression>)	Produces an integer section	243	
	INT (<Numeric expression>)	Produces the largest integer that does not exceed the value in the expression.	245	
	LEN(<Character string expression>)	Produces the length of the character string.	247	
	LN (<Numeric expression>)	Produces the logarithm.	248	
	LOG (<Numeric expression>)	Produces the common logarithm.	248	
	MAX (<Numeric expression>...)	Obtains the max. value from a random number of arguments.	249	
	MIN (<Numeric expression>...)	Obtains the min. value from a random number of arguments.	250	
	RAD (<Numeric expression: deg.>)	Converts the angle unit from radian (rad) to degree (deg).	254	
	SGN (<Numeric expression>)	Checks the sign of the number in the expression	260	
	SQR (<Numeric expression>)	Calculates the square root	261	
	STRPOS(<Character string expression>, <Character string expression>)	Obtains the 2nd argument character string position in the 1st argument character string.	261	
	RND (<Numeric expression>)	Produces the random numbers.	257	
	ASC(<Character string expression>)	Provides a character code for the first character of the character string in the expression.	234	
	CVI(<Character string expression>)	Converts a 2-byte character string into integers.	240	
	CVS(<Character string expression>)	Converts a 4-byte character string into a single-precision real number.	240	
	CVD(<Character string expression>)	Converts an 8-byte character string into a double-precision real number.	241	
	VAL(<Character string expression>)	Converts a character string into a numeric value.	263	
Trigonometric functions	ATN(<Numeric expression>)	Calculates the arc tangent. Unit: radian Definition range: Numeric value, Value range: $-\pi/2$ to $+\pi/2$	234	Numeric value
	ATN2(<Numeric expression>, <Numeric expression>)	Calculates the arc tangent. Unit: radian $\Theta = \text{ATN2}(\angle y, \angle x)$ Definition range: Numeric value of $\angle y$ or $\angle x$ that is not 0 Value range: $-\pi$ to $+\pi$	234	
	COS(<Numeric expression>)	Calculates the cosine Unit: radian Definition range: Numeric value range, Value range: -1 to +1	239	
	SIN(<Numeric expression>)	Calculates the sine Unit: radian Definition range: Numeric value range, Value range: -1 to +1	260	
	TAN(<Numeric expression>)	Calculates the tangent. Unit: radian Definition range: Numeric value range, Value range: Range of numeric value	263	

Class	Function name (format)	Functions	Page	Result
Character string functions	BIN\$(<Numeric expression>)	Converts numeric expression value into binary character string.	235	Character string
	CHR\$(<Numeric expression>)	Provides character having numeric expression value character code.	238	
	HEX\$(<Numeric expression>)	Converts numeric expression value into hexadecimal character string.	245	
	LEFT\$(<Character string expression> , <Numeric expression>)	Obtains character string having length designated with 2nd argument from left side of 1st argument character string.	247	
	MID\$(<Character string expression> , <Numeric expression> , <Numeric expression>)	Obtains character string having length designated with 3rd argument from the position designated with the 2nd argument in the 1st argument character string.	249	
	MIRROR\$(<Character string expression>)	Mirror reversal of the character string binary bit is carried out.	250	
	MKI\$(<Numeric expression>)	Converts numeric expression value into 2-byte character string.	251	
	MKS\$(<Numeric expression>)	Converts numeric expression value into 4-byte character string.	251	
	MKD\$(<Numeric expression>)	Converts numeric expression value into 8-byte character string.	252	
	RIGHT\$(<Character string expression> , <Numeric expression>)	Obtains character string having length designated with 2nd argument from right side of 1st argument character string.	257	
	STR\$(<Numeric expression>)	Converts the numeric expression value into a decimal character string.	262	
	CKSUM(<Character string expression> , <Numeric expression> , <Numeric expression>)	Creates the checksum of a character string. Returns the value of the lower byte obtained by adding the character value of the second argument position to that of the third argument position, in the first argument character string.	239	Numeric value
Position variables	DIST(<Position> , <Position>)	Obtains the distance between two points.	242	Position
	FRAM (<Position 1> , <Position 2> , <Position 3>)	Calculates the coordinate system designated with three points. Position 1 is the plane origin, position 2 is the point on the +X axis, and position 3 is the point on the +Y axis direction plane. The plane origin point and posture are obtained from the XYZ coordinates of the three position, and is returned with a return value (position). This is operated with 6-axis three dimensions regardless of the mechanism structure. This function cannot be used in 5-axis robots, because the A, B, and C posture data has different meaning.	244	
	RDFL1(<Position> , <Numeric value>)	Returns the structure flag of the designated position as character data. Argument <numeric value>) 0 = R/L, 1 = A/B, 2 = F/N is returned.	255	Character
	SETFL1(<Position> , <Character>)	Changes the structure flag of the designated position. The data to be changed is designated with characters.(R/L/A/B/F/N)	258	Numeric value
	RDFL2(<Position> , <Numeric value>)	Returns the multi-rotation data of the designated position as a numeric value (-2 to 1). The argument <numeric expression> returns the axis No. (1 to 8).	256	
	SETFL2 (<Position> , <Numeric value> , <Numeric value>)	Changes the multi-rotation data of the designated position as a numeric value (-2 to 1). The left side of the expression is the axis No. to be changed; the right side is the value to be set.	259	
	ALIGN(<Position>)	Returns the value of the XYZ position (0, ± 90 , ± 180) closest to the position 1 posture axis (A, B, C). This function cannot be used in 5-axis robots, because the A, B, and C posture data has different meaning.	233	
	INV(<Position>)	Obtains the reverse matrix.	246	Position
	PTOJ(<Position>)	Converts the position data into joint data.	254	Joint
	JTOP(<Position>)	Converts the joint data into position data.	246	Position
	ZONE (<Position 1> , <Position 2> , <Position 3>)	Checks whether position 1 is within the space (Cube) created by the position 2 and position 3 points. Outside the range=0, Within the range=1 For position coordinates that are not checked or non-existent, the following values should be assigned to the corresponding position coordinates: If the unit is degrees, assign -360 to position 2 and 360 to position 3 If the unit is mm, assign -10000 to position 2 and 10000 to position 3	264	Numeric value
	ZONE2 (<Position 1> , <Position 2> , <Position 3> , <Numeric value1> , <Numeric value2> , <Numeric value3> , <Position 4>)	Checks whether position 1 is within the space (cylinder) created by the position 2 and position 3 points. Outside the range=0, Within the range=1 Only the X, Y, and Z coordinate values are considered; the A, B, and C posture data is ignored.	265	Numeric value
	POSCQ(<Position>)	Checks whether <position> is within the movement range.	253	Numeric value
	POSMID (<Position1> , <Position2> , <Numeric value1> , <Numeric value2>)	Calculates the middle position between <position 1> and <position 2> .	253	Position
	CALARC (<Position 1> , <Position 2> , <Position 3> , <Numeric value1> , <Numeric value2> , <Numeric value3> , <Position 4>)	Returns information of an arc created from <position 1> , <position 2> , and <position 3> .	236	

4.6 List of Instructions

A list of pages with description of each instruction is shown below. They are listed in the order of presumed usage frequency.

(1) Instructions related to movement control

Command	Explanation	Page
Page 162, "MOV"	Joint interpolation	162
Page 168, "MVS"	Linear interpolation	168
Page 165, "MVR"	Circular interpolation	165
Page 166, "MVR2"	Circular interpolation 2	166
Page 167, "MVR3"	Circular interpolation 3	167
Page 164, "MVC"	Circular interpolation	164
Page 163, "MVA"	Arch motion interpolation	163
Page 175, "OVRD"	Overall speed specification	175
Page 187, "SPD"	Speed specification during linear or circular interpolation movement	187
Page 158, "JOVRD"	Speed specification during joint interpolation movement	158
Page 128, "CNT"	Continuous path mode specification	128
Page 112, "ACCEL"	Acceleration/deceleration rate specification	112
Page 120, "CMP JNT"	Specification of compliance in the JOINT coordinate system	120
Page 122, "CMP POS"	Specification of compliance in the XYZ coordinate system	122
Page 124, "CMP TOOL"	Specification of compliance in the TOOL coordinate system	124
Page 126, "CMP OFF"	Compliance setting invalid	126
Page 127, "CMPG"	Compliance gain specification	127
Page 170, "OADL"	Optimum acceleration/deceleration rate specification	170
Page 161, "LOADSET"	Hand's optional condition specification	161
Page 177, "PREC (Precision)"	High accuracy mode specification	177
Page 191, "TORQ"	Torque specification of each axis	191
Page 159, "JRC"	Enables multiple rotation of the tip axis	159
Page 147, "FINE"	Robot's positioning range specification	147
Page 185, "SERVO"	Servo motor power ON/OFF	185
Page 194, "WTH"	Addition instruction of movement instruction	194
Page 195, "WTHIF"	Additional conditional instruction of movement instruction	195

(2) Instructions related to program control

Command	Explanation	Page
Page 180, "RELM"	Comment(')	180
Page 155, "IF...THEN...ELSE...ENDIF"	Conditional branching	155
Page 184, "SELECT CASE"	Enables multiple branching	184
Page 152, "GOTO"	Jump	152
Page 151, "GOSUB"	Subroutine jump	151
Page 182, "RESET ERR"	Resets an error (use of default is not allowed)	182
Page 115, "CALLP"	Program call	115
Page 149, "FPRM"	Program call argument definition	149
Page 144, "DLY"	Timer	144
Page 153, "HLT"	Suspends a program	153
Page 146, "END"	End a program	146
Page 172, "ON...GOSUB"	Subroutine jump according to the value	172
Page 173, "ON...GOTO"	Jump according to the value	173
Page 148, "FOR - NEXT"	Repeat	148
Page 193, "WHILE-WEND"	Conditional repeat	193

Command	Explanation	Page
Page 174, "OPEN"	Opens a file or communication line	174
Page 178, "PRINT"	Outputs data	178
Page 157, "INPUT"	Inputs data	157
Page 118, "CLOSE"	Closes a file or communication line	118
Page 131, "COM ON/COM OFF/COM STOP"	Enables, disables or pauses communication interrupt	131
Page 171, "ON COM GOSUB"	Communication interrupt subroutine jump	171
Page 154, "HOPEN / HCLOSE"	Hand's open/close	154
Page 145, "ERROR"	User error	145
Page 186, "SKIP"	Skip while moving	186
Page 192, "WAIT"	Waiting for conditions	192
Page 119, "CLR"	Signal clear	119

(3) Definition instructions

Command	Explanation	Page
Page 143, "DIM"	Array variable declaration	143
Page 136, "DEF PLT"	Pallet declaration	136
Page 176, "PLT"	Pallet position calculation	176
Page 132, "DEF ACT"	Interrupt definition	132
Page 113, "ACT"	Starts or ends interrupt monitoring	113
Page 140, "DEF JNT"	Joint type position variable definition	140
Page 141, "DEF POS"	XYZ type position variable definition	141
Page 137, "DEF INTE/DEF FLOAT/DEF DOUBLE"	Integer or real number variable definition	137
Page 142, "DEF CHAR"	Character variable definition	142
Page 138, "DEF IO"	Signal variable definition	138
Page 135, "DEF FN"	User function definition	135
Page 190, "TOOL"	Hand length setting	190
Page 114, "BASE"	Robot base position setting	114

(4) Multi-task related

Command	Explanation	Page
Page 197, "XLOAD"	Loads a program to another task slot	197
Page 199, "XRUN"	Execute the program in another task slot	199
Page 200, "XSTP"	Stop the program in another task slot	200
Page 198, "XRST"	Resets the program in another task slot being suspended	198
Page 196, "XCLR"	Cancels the loading of the program from the specified task slot	196
Page 199, "XRUN"	Obtains mechanical control right	199
Page 199, "XRUN"	Releases mechanical control right	199
Page 179, "PRIORITY"	Changes the task slot priority	179
Page 180, "RELM"	Resets an error (use of default is not allowed)	180

(5) Multi-task related

Command	Explanation	Page
Page 117, "CHRSRCH"	Searches the character string out of the character array.	117

4.7 Operators

The value's real number or integer type do not need to be declared. Instead, the type may be forcibly converted according to the operation type. (Refer to Table 4-9.) The operation result data type is as follows according to the combination of the left argument and right argument data types.

Example)	Left argument	Operation	Right argument	Operation results
	• <u>15</u> (Numeric value type)	<u>AND</u>	<u>256</u> (Numeric value type)	<u>15</u> (Numeric value type)
	• <u>P1</u> (Position type)	<u>*</u>	<u>M1</u> (Numeric value type)	<u>P2</u> (Position type)
	• <u>M1</u> (Numeric value type)	<u>*</u>	<u>P1</u> (Position type)	Description error

Table 4-9 : Table of data conversions according to operations

Left argument type	Operation	Left argument type				
		Character string	Numeric value		Position	Joint
			Integer	Real number		
Character string	Substitution =	Character string	—	—	—	—
	Addition +	Character string	—	—	—	—
	Comparison (Comparison operators)	Integer	—	—	—	—
Integer	Addition +	—	Integer	Real number	—	—
	Subtract —	—	Integer	Real number	—	—
	Multiplication *	—	Integer	Real number	—	—
	Division /	—	Integer	Real number	—	—
	Integer division ¥	—	Integer	Integer	—	—
	Remainder MOD	—	Integer	Integer	—	—
	Exponent ^	—	Integer	Integer	—	—
	Substitution =	—	Integer	Integer	—	—
	Comparison (Comparison operators)	—	Integer	Integer	—	—
	Logic (Logic operators)	—	Integer	Integer	—	—
Real number	Addition +	—	Real number	Real number	—	—
	Subtract —	—	Real number	Real number	—	—
	Multiplication *	—	Real number	Real number	—	—
	Division /	—	Real number	Real number	—	—
	Integer division ¥	—	Integer	Integer	—	—
	Remainder MOD	—	Integer	Integer	—	—
	Exponent ^	—	Integer	Real number	—	—
	Substitution =	—	Integer	Real number	—	—
	Comparison (Comparison operators)	—	Integer	Integer	—	—
	Logic (Logic operators)	—	Integer	Integer	—	—
Position	Addition +	—	—	—	Position	—
	Subtract —	—	—	—	Position	—
	Multiplication *	—	Position	Position	Position	—
	Division /	—	Position	Position	Position	—
	Integer division ¥	—	—	—	—	—
	Remainder MOD	—	—	—	—	—
	Exponent ^	—	—	—	—	—
	Substitution =	—	—	—	Position	—
	Comparison (Comparison operators)	—	—	—	—	—
	Logic (Logic operators)	—	—	—	—	—
Joint	Addition +	—	—	—	—	Joint
	Subtract —	—	—	—	—	Joint
	Multiplication *	—	Joint	Joint	—	—
	Division /	—	Joint	Joint	—	Joint
	Integer division ¥	—	—	—	—	—
	Remainder MOD	—	—	—	—	—
	Exponent ^	—	—	—	—	—
	Substitution =	—	—	—	—	Joint
	Comparison (Comparison operators)	—	—	—	—	—
	Logic (Logic operators)	—	—	—	—	—
Right argument only (Single argument)	Reversal — Negate NOT	— —	Integer Integer	Integer Integer	Position —	Joint —

Reversal: Sign reversal Negate: Logical negate Substitute: substitute operation Remainder: Remainder operation
Comparison: Comparison operation Logic: Logical Operation (excluding logical negate).

[Caution]

- The operation of the section described with a “-” is not defined.
- The results of the integer and the interger multiplication/division is an integer type for multiplication, and a real number type for division.
- If the right argument is a 0 divisor (divide by 0), an operation will not be possible.
- During exponential operation, remainder operation or logical operation (including negate), all real numbers will be forcibly converted into integers (rounded off), and operated.

4.8 Priority level of operations

In the event there are many operators within an expression being calculated, the order of operations is as shown in [Table 4-10](#).

Table 4-10 : Priority level of operations

Operations, (operators)	Type of operation	Priority level
1) Operations inside parentheses ()		High
2) Functions	Functions	:
3) Exponents	Arithmetic operation	:
4) Single argument operator (+, -)	Arithmetic operation	:
5) * /	Arithmetic operation	:
6) %	Arithmetic operation	:
7) MOD	Arithmetic operation	:
8) + -	Arithmetic operation	:
9) << >>	Logic operation	:
10) Comparison operator (=, <, >, <=, >=, <>, =, >=, <=, <>)	Comparison operation	:
11) NOT	Logic operation	:
12) AND	Logic operation	:
13) OR	Logic operation	:
14) XOR	Logic operation	Low

4.9 Depth of program's control structure

When creating a program, the depth of the control structure must be considered.

When using the commands in the table below, the program's level of control structure becomes one level deeper. Each command has a limit to the depth of the control structure. Exceeding these limits will cause an error.

Table 4-11 : Limit to control structure depth

	No. of levels	Applicable commands
User stack in program	16 levels	• Repeated controls (FOR-NEXT, WHILE-WEND)
	8 levels	• Function calling (CALLP)
	800 levels max.	• Subroutine calling (GOSUB) The number decreases by usage frequency of FOR-NEXT, WHILE-WEND, and CALLP instructions.

4.10 Reserved words

Reserved words are those that are already used for the system.

A name that is the same as one of the reserved words cannot be used in the program.

Instructions, functions, and system status variables, etc. are considered reserved words.

4.11 Detailed explanation of command words

4.11.1 How to read the described items

[Function]	: Indicates the command word functions.
[Format]	: Indicates how to input the command word argument. The argument is shown in <>. [] indicates that the argument can be omitted. □ indicates that a space is required.
[Terminology]	: Indicates the meaning and range, etc. of the argument.
[Reference Program]	: Indicates a program example.
[Explanation]	: Indicates detailed functions and cautions, etc.
[The available robot type]	: Indicates the available robot type.
[Related parameter]	: Indicates the related parameter.
[Related system variables]	: Indicates the related system variables.
[Related instructions]	: Indicates the related instructions.

4.11.2 Explanation of each command word

Each instruction is explained below in alphabetical order.

ACCEL (Accelerate)

[Function]

Designate the robot's acceleration and deceleration speeds as a percentage (%).

[Format]

ACCEL □ [<Acceleration rate>] [, <Deceleration rate>]

Controller software version G2 or later

ACCEL □ [<Acceleration rate>] [, <Deceleration rate>]
 , [<Acceleration rate when moving upward>], [<Deceleration rate when moving upward>]
 [<Acceleration rate when moving downward>], [<Deceleration rate when moving downward>]

[Terminology]

<Acceleration/Deceleration>

1 to 100. Designate the acceleration/deceleration to reach the maximum speed from speed 0 as a percentage. This can be described as a constant or variable. A default value of 100 is set if the argument is omitted. A value of 100 corresponds to the maximum rate of acceleration/deceleration. Unit: [%]

<Acceleration/Deceleration rate when moving upward>

Specify the acceleration/deceleration rate when moving upward in an arch motion due to the MVA instruction.

A default value of 100 is set if the argument is omitted. It is possible to specify the argument either by a constant or variable.

<Acceleration/Deceleration rate when moving downward>

Specify the acceleration/deceleration rate when moving downward in an arch motion due to the MVA instruction.

A default value of 100 is set if the argument is omitted. It is possible to specify the argument either by a constant or variable.

[Reference Program]

10 ACCEL 50,100	' Heavy load designation (when acceleration/deceleration is 0.2 seconds, the acceleration will be 0.4, and the deceleration will be 0.2 seconds).
20 MOV P1	
30 ACCEL 100,100	' Standard load designation.
40 MOV P2	
50 DEF ARCH 1,10,10,25,25,1,0,0	
60 ACCEL 100,100,20,20,20,20	' Specify the override value to 20 when moving upward or downward due to the MVA instruction.
70 MVA P3,1	

[Explanation]

- (1) The maximum acceleration/deceleration is determined according to the robot being used. Set the corresponding percentage. The system default value is 100,100.
- (2) The acceleration percentage changed with this command is reset to the system default value when the program is reset or the END statement executed.
- (3) If a value greater than 100 is set, the system will forcefully make the robot move at 100%. If a value less than 1 is set, an error occurs at registration.
- (4) The smooth operation when CNT is valid will have a different locus according to the acceleration speed or operation speed. To move smoothly at a constant speed, set the acceleration and deceleration to the same value. CNT is invalid in the default state.
- (5) If the optimum acceleration/deceleration control is active (OADL ON), the setting of the ACCEL instruction is ignored.

[Related instructions]

[OADL](#), [LOADSET](#)

[Related system variables]

[M_ACL/M_DACL/M_NACL/M_NDAACL/M_ACLSTS](#)

ACT (Act)

[Function]

This instruction specifies whether to allow or prohibit interrupt processing caused by signals, etc. during operation.

[Format]

ACT □ <Priority No.> = <1/0>

[Terminology]

<Priority No.>	0: Either enables or disables the entire interrupt. 1 to 8: Designate the priority No. for the interrupt defined in the DEF ACT statement. When entering the priority No., always leave a space (character) after the ACT command. If described as ACT1, it will be a variable name declaration statement.
<1/0>	1: Allows interrupts, 0: Prohibits interrupts.

[Reference Program]

When the input signal 1 turns on (set to 1) while moving from P1 to P2, it loops until that signal is set to 0.

```

10 DEF ACT 1,M_IN(1)=1 GOSUB *INTR      ' Assign input signal 1 to the interrupt 1 condition
20 MOV P1
30 ACT 1=1                              ' Enable interrupt 1.
40 MOV P2
50 ACT 1=0                              ' Disable interrupt 1.
:
100 *INTR                                '
110 IF M_IN(1)=1 GOTO 110                ' Loops until the M_IN(1) signal becomes 0.
120 RETURN 0                            '

```

[Explanation]

- (1) When the program starts, the status of <Priority No.> 0 is "enabled." When <Priority No.> 0 is "disabled," even if <Priority No.> 1 to 8 are set to "enabled," no interrupt will be enabled.
- (2) The statuses of <Priority No.> 1 to 8 are all "disabled" when the program starts.
- (3) An interrupt will occur only when all of the following conditions have been satisfied:
 - <Priority No.> 0 is set to "enabled."
 - The status of the DEF ACT statement has been defined.
 - When the <Priority No.> designated by DEF ACT is made valid by an ACT statement.
- (4) The return from an interrupt process should be done by describing either RETURN 0 or RETURN 1.
- (5) Even if the robot is in the middle of interpolation, an interrupt defined by a DEF ACT statement will be executed.
- (6) During an interrupt process, that <Priority No.> will be executed with the status as "disable."
- (7) A communications interrupt (COM) has a higher priority than an interrupt defined by a DEF ACT statement.
- (8) The relationship of priority rankings is as shown below:

COM > ACT > WTHIF (WTH) > Pulse substitution

[Related instructions]

DEF ACT, RETURN

BASE (Base)

[Function]

With this instruction, it is possible to move or rotate the robot coordinate system. Specify the base conversion data for this instruction. Pay extra attention when making changes in a program, as it may be mistaken in jog operations, etc.

[Format]

BASE □ < Base conversion data >

[Terminology]

<Base conversion data> Specify with position constants or position variables.

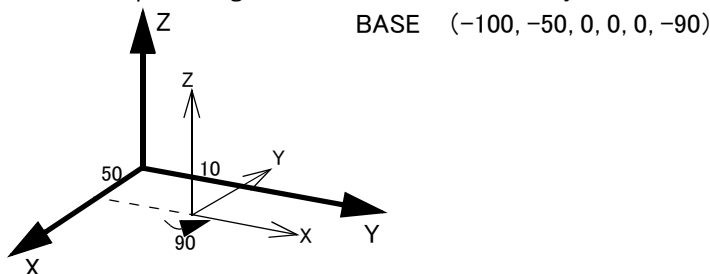
[Reference Program]

10 BASE (50,100,0,0,0,90)	' Input the conversion data as a constant.
20 MVS P1	
30 BASE P2	' Input the conversion data as a variable.
40 MVS P1	
50 BASE P_NBASE	' Reset the conversion data to the default values.

[Explanation]

- (1) The X, Y, and Z components of the position data represent the amount of parallel movement from the origin of the world coordinate system to the origin of the base coordinate system. The base conversion data can be changed only with the BASE command. The components A, B, and C of the position data represent the amount that the base coordinate system is tilted in relation to the world coordinate system.
 - X · · · Distance to move parallel to X axis
 - Y · · · Distance to move parallel to Y axis
 - Z · · · Distance to move parallel to Z axis
 - A · · · Angle to turn toward the X axis
 - B · · · Angle to turn toward the Y axis
 - C · · · Angle to turn toward the Z axis
 - L1 · · Movement amount of additional axis 1
 - L2 · · Movement amount of additional axis 2
- (2) For A, B and C, the clockwise direction looking from the front of the origin of the coordinate, used as the center of rotation, is the forward rotation direction.
- (3) The contents of the structural flag have no meaning.
- (4) The system's default value for this data is P_NBASE=(0,0,0,0,0) (0,0). This is calculated with the 6-axis three dimensional regardless of the mechanism structure.
- (5) The valid axis element of base conversion data is different depending on the type of robot. Set up the appropriate data referring to the [Page 278, "Table 5-3 : Valid axis elements of the base conversion data depending on the robot model"](#).

Fig.4-3 : Conceptual diagram of the base coordinate system



[Related parameter]

After it has been changed by the MEXBS BASE instruction, the base coordinate system is stored in the MEXBS parameter and maintained even after the controller's power is turned off. Refer to [Page 278, "About Standard Base Coordinates"](#).

[Related system variables]

[P_BASE/P_NBASE](#)

CALLP (Call P)

[Function]

This instruction executes the specified program (by calling the program in a manner similar to using GOSUB to call a subroutine). The execution returns to the main program when the END instruction or the final line in the sub program is reached.

[Format]

CALLP □ "<Program name>" [, <Argument> [, <Argument>]

[Terminology]

- <Program name> Designate the program name with a character string constant or character string variable.
 For the standards for program names, please refer to [Page 90, "\(1\) Program name"](#).
 <Argument> Designate the variable to be transferred to the program when the program is called.
 Up to 16 variables can be transferred.

[Reference Program]

(1) When passing the argument to the program to call.

Main program

```

10 M1=0
20 CALLP "10",M1,P1,P2
30 M1=1
40 CALLP "10",M1,P1,P2
:
100 CALLP "10", M2,P3,P4
:
150 END

```

"10" sub program side

```

10 FPRM M01, P01,P02
20 IF M01<>0 THEN GOTO *LBL1
30 MOV P01
40 *LBL1
50 MVS P02
60 END

```

'Return to the main program at this point.

* When lines 20 and 40 of the main program are executed, M1, P1 and P2 are set in M01, P01 and P02 of the sub program, respectively. When line 100 of the main program is executed, M2, P3 and P4 are set in M01, P01 and P02 of the sub program, respectively.

(2) When not passing the argument to the program to call.

Main program

```

10 MOV P1
20 CALLP "20"
30 MOV P2
40 CALLP "20"
50 END

```

"20" sub program side

```

10 MOV P1
20 MVS P002
30 M_OUT(17)=1
40 END

```

'P1 of the sub program differs from P1 of the main program.
 'Return to the main program at this point.

[Explanation]

- (1) A program (sub program) called by the CALLP instruction will return to the parent program (main program) when the END instruction (equivalent to the RETURN instruction of GOSUB) is reached. If there is no END instruction, the execution is returned to the main program when the final line of the sub program is reached.
- (2) If arguments need to be passed to the sub program, they should be defined using the FPRM instruction at the beginning of the sub program.
- (3) If the type or the number of arguments passed to the sub program is different from those defined (by the FPRM instruction) in the sub program, an error occurs at execution.
- (4) If a program is reset, the control returns to the beginning of the top main program.
- (5) Definition statements (DEF ACT, DEF FN, DEF PLT, and DIM instructions) executed in the main program are invalid in a program called by the CALLP instruction. They become valid when the control is returned to the main program from the program called by the CALLP instruction again.
- (6) Speed and tool data are all valid in a sub program. Values of ACCEL and SPD are invalid. The mode of OADL is valid.
- (7) Another sub program can be executed by calling CALLP in a sub program. However, a main program or a program that is currently being executed in another task slot cannot be called. In addition, own program cannot be called, either.
- (8) Eight levels (in a hierarchy) of sub programs can be executed by calling CALLP in the first main program.
- (9) Variable values may be passed from a main program to a sub program using arguments, however, it is not possible to pass the processing result of a sub program to a main program by assigning it in an argument. To use the processing result of a sub program in a main program, pass the values using external variables.

[Related instructions]

FPRM

CHRSRCH (Character search)

[Function]

Searches the character string out of the character array.

[Format]

CHRSRCH □ <Character string array variable>,<Character string>,<Search result storage destination>

[Terminology]

<Character string array variable> Specify the character string array to be searched.
 <Character string> Specify the character string to be searched.
 <Search result storage destination> The number of the element for which the character string to be searched is found is set.

[Reference Program]

```

10 DIM C1$(10)
20 C1$(1)="ABCDEFG"
30 C1$(2)="MELFA"
40 C1$(3)="BCDF"
50 C1$(4)="ABD"
60 C1$(5)="XYZ"
70 C1$(6)="MELFA"
80 C1$(7)="CDF"
90 C1$(8)="ROBOT"
100 C1$(9)="FFF"
110 C1$(10)="BCD"
120 CHRSRCH C1$(1), "ROBOT", M1      ' 8 is set in M1.
130 CHRSRCH C1$(1), "MELFA", M2    ' 2 is set in M2.
```

[Explanation]

- (1) The specified character string is searched from the character string array variables, and the element number of the completely matched character string array is set in <search result storage destination>. Partially matched character strings are not searched.
 Even if CHRSRCH C1\$(1), "ROBO", M1 are described in the above statement example, the matched character string is not searched.
- (2) If the character string to be searched is not found, 0 is set in <search result storage destination>.
- (3) Character string search is performed sequentially beginning with element number 1, and the element number found first is set.
 Even if CHRSRCH C1\$(3), "MELFA", M2 are described in the above statement example, 2 is set in M2. (The same character string is set in C1\$(2) and C1\$(6).)
- (4) The <character string array variable> that can be searched is the one-dimensional array only. If a two-dimensional or higher array is specified as a variable, an error will occur at the time of execution.

CLOSE (Close)**[Function]**

Closes the designated file.(including communication lines)

[Format]

CLOSE □ [[#] <File No.> [, [[#] <File No.> ...]

[Terminology]

<File No.>

Designate the No. of the file to be closed. Only a numerical constant is allowed.
If this argument is omitted, all open files are closed.

[Reference Program]

```

10 OPEN "COM1:" AS #1      ' "Open "COM1:" as file No. 1.
20 PRINT #1,M1
   :
100 INPUT #1,M2
110 CLOSE #1              ' Close file No. 1, "COM1:".
   :
200 CLOSE                 ' Close all open files.
```

[Explanation]

- (1) This instruction closes files (including communication lines) opened by the OPEN instruction. Data remaining in the buffer is flushed.

The data left in the buffer will be processed as follows when the file is closed:

Table 4-12 : Processing of each buffer when the file is closed

Buffer types	Processing when the file is closed
Communication line reception buffer	The contents of the buffer are destroyed
Communication line transmission buffer	(No data remains in the transmission buffer since the data in the transmission buffer is sent immediately by executing the PRINT instruction.)
File load buffer	The contents of the buffer are destroyed.
File unload buffer	The contents of the buffer are written into the file, and then the file is closed.

- (2) Executing an END statement will also close a file.
(3) If the file number is omitted, all files will be closed.

[Related instructions]

OPEN, PRINT, INPUT

CLR (Clear)

[Function]

This instruction clears general-purpose output signals, local numerical variables in a program, and numerical external variables.

[Format]

CLR □ <Type>

[Terminology]

<Type>

It is possible to specify either a constant or a variable.

0 : All steps 1 to 3 below are executed.

1 : The general-purpose output signal is cleared based on the output reset pattern.

The output reset pattern is designated with parameters ORST0 to ORST224.

Refer to [Page 286, "5.9 About the output signal reset pattern"](#).

(0: OFF, 1: ON, *: Hold)

2 : All local numeric variables and numeric array variables used in the program are cleared to zero

3: Clears all external numerical variables (External system variables and user-defined external variables) and external numerical array variables, setting them to 0. External position variables are not cleared.

[Reference Program]

(1) The general-purpose output signal is output based on the output reset pattern.

10 CLR 1

(2) The local numeric variables and numeric array variables in the program are cleared to 0.

10 DIM MA(10)

20 DEF INTE IVAL

30 CLR 2 ' Clears MA(1) through MA(10), IVAL and local numeric variables in the program to 0.

(3) All external numeric array variables and external numeric array variables are cleared to 0

10 CLR 3

(4) (1) though (3) above are performed simultaneously.

10 CLR 0

[Related parameter]

ORST0 to ORST224

[Related system variables]

[M_IN/M_INB/M_INW](#), [M_OUT/M_OUTB/M_OUTW](#)

CMP JNT (Comp Joint)

[Function]

Start the soft control mode (compliance mode) of the specified axis in the JOINT coordinates system.

Note) The available robot type is limited such as RH-nAH. Refer to "[Available robot type]".

[Format]

CMP □ JNT, <Axis designation>

[Terminology]

<Axis designation>

Specify the axis to be controlled in a pliable manner with the bit pattern.

1 : Enable, 0 : Disable &B00000000

This corresponds to axis 87654321.

[Reference Program]

10 MOV P1

20 CMPG 0.0,0.0,1.0,1.0, , , , ' Set softness.

30 CMP JNT,&B11 ' The J1 and J2 axes are put in the state where they are controlled in a pliable manner.

40 MOV P2

50 HOPEN 1

60 MOV P1

70 CMP OFF ' Return to normal state.

[Explanation]

- (1) It is possible to control each of the robot's axes in the joint coordinate system in a pliable manner. For example, if using a horizontal multi-joint robot to insert pins in a workpiece by moving the robot's hand up and down, it is possible to insert the pins more smoothly by employing pliable control of the J1 and J2 axes (see the statement example above).
- (2) The degree of compliance can be specified by the CMPG instruction, which sets the spring constant. If the robot is of the RH- □ AH type, specify 0.0 for the horizontal axes J1 and J2 to make the robot behave equivalently to a servo free system (the spring constant is zero). (Note that the vertical axes cannot be made to behave equivalently to a servo free system even if 0.0 is set for them. Also, be careful not to let these axes reach a position beyond the movement limit or where the amount of diversion becomes too large.) Note that 4) and 5) below do not function if this servo-free equivalent behavior is in use.
- (3) The soft state is maintained even after the robot program execution is stopped. To cancel the soft status, execute the "CMP OFF" command or turn OFF the power.
- (4) When pressing in the soft state, the robot cannot move to positions that exceed the operation limit of each joint axis.
- (5) If the amount of difference between the original target position and the actual robot position becomes greater than 200 mm by pushing the hand, etc., the robot will not move any further and the operation shifts to the next line of the program.
- (6) It is not possible to use CMP JNT, POS, and TOOL at the same time. In other words, an error occurs if the CMP POS or CMP TOOL instruction is executed while the CMP JNT instruction is being performed. Cancel the CMP JNT instruction once using the CMP OFF instruction to execute these instructions.
- (7) Be aware that the position of the robot may change if the servo status is switched on while this instruction is active.
- (8) It is possible to perform jog operations while the robot is in compliance mode. However, the setting of the compliance mode cannot be canceled by the T/B; in order to do so, execute this instruction in a program or execute it directly via the program edit screen of the T/B.
- (9) To change the axis specification, cancel the compliance mode with the CMP OFF instruction first, and then execute the CMP JNT instruction again.



CAUTION

The compliance mode is in effect continuously until the CMP OFF instruction is executed, or the power is turned off.

 CAUTION

To execute a jog operation after setting the compliance mode with the CMP JNT instruction, use the JOINT jog mode.

If any other jog mode is used, the robot may operate in a direction different from the expected moving direction because the directions of the coordinate systems controlled by the jog operation and the compliance mode differ.

 CAUTION

When performing the teaching of a position while in the compliance mode, perform servo OFF first.

Be careful that if teaching operation is performed with Servo ON, the original command position is taught, instead of the actual robot position. As a result, the robot may move to a location different from what has been taught.

[Available robot type]

RH-5AH/10AH/15AH series

[Related system variables]

[M_CMPDST](#)

[Related instructions]

[CMP OFF](#), [CMPG](#), [CMP POS](#), [CMP TOOL](#)

CMP POS (Composition Posture)

[Function]

Start the soft control mode (compliance mode) of the specified axis in the XYZ coordinates system.

Note) The available robot type is limited such as RV-4A. Refer to "[Available robot type]".

[Format]

CMP □ POS, <Axis designation>

[Terminology]

<Axis designation> Designate axis to move softly with a bit pattern.
1 : Enable, 0 : Disable &B00000000
This corresponds to axis L2L1CBAZYX

[Reference Program]

10 MOV P1	' Move in front of the part insertion position.
20 CMPG 0.5, 0.5, 1.0, 0.5, 0.5, , ,	' Set softness
30 CMP POS, &B011011	' The X, Y, A, and B axes are put in the state where they are controlled in a pliable manner.
40 MVS P2	' Moves to the part insertion position.
50 M_OUT(10)=1	' Instructs to close the chuck for positioning.
60 DLY 1.0	' Waits for the completion of chuck closing.(1 sec.)
70 HOPEN 1	' Open the hand.
80 MVS, -100	' Retreats 100 mm in the Z direction of the TOOL coordinate system.
90 CMP OFF	' Return to normal state.

[Explanation]

- (1) The robot can be moved softly with the XYZ coordinate system.
For example, when inserting a pin in the vertical direction, if the X, Y, A and B axes are set to soft operation, the pin can be inserted smoothly.
- (2) The degree of softness can be designated with the CMPG command.
- (3) The soft state is maintained even after the robot program execution is stopped. To cancel the soft status, execute the "CMP OFF" command or turn OFF the power.
- (4) When pressing in the soft state, the robot cannot move to positions that exceed the operation limit of each joint axis.
- (5) The deviation of the command position and actual position can be read with M_CMPDST. The success/failure of pin insertion can be checked using this variable.
- (6) If the amount of difference between the original target position and the actual robot position becomes greater than 200 mm by pushing the hand, etc., the robot will not move any further and the operation shifts to the next line of the program.
- (7) It is not possible to use CMP JNT, POS, and TOOL at the same time. In other words, an error occurs if the CMP POS or CMP TOOL instruction is executed while the CMP JNT instruction is being performed. Cancel the CMP JNT instruction once using the CMP OFF instruction to execute these instructions.
- (8) If the servo turns from OFF to ON while this command is functioning, the robot position could change.
- (9) It is possible to perform jog operations while the robot is in compliance mode. However, the setting of the compliance mode cannot be canceled by the T/B; in order to do so, execute this instruction in a program or execute it directly via the program edit screen of the T/B.
- (10) To change the axis specification, cancel the compliance mode with the CMP OFF instruction first, and then execute the CMP POS instruction again.
- (11) If the robot is operated near a singular point, an alarm may be generated or control may be disabled. Do not operate the robot near a singular point.

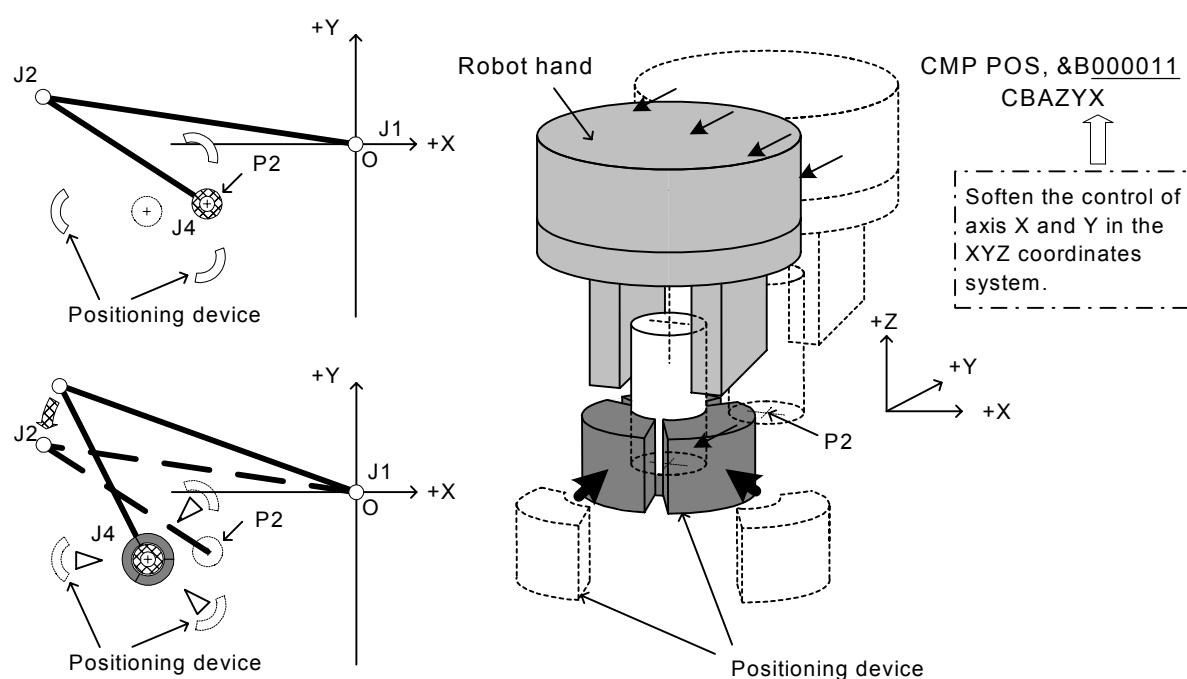


Fig.4-4 : The example of compliance mode use

⚠ CAUTION

The compliance mode is in effect continuously until the CMP OFF instruction is executed, or the power is turned off. Exercise caution when changing the executable program number or operating the jog.

⚠ CAUTION

To execute a jog operation after setting the compliance mode with the CMP POS instruction, use the XYZ jog mode.

If any other jog mode is used, the robot may operate in a direction different from the expected moving direction because the directions of the coordinate systems controlled by the jog operation and the compliance mode differ.

⚠ CAUTION

When performing the teaching of a position while in the compliance mode, perform servo OFF first.

Be careful that if teaching operation is performed with Servo ON, the original command position is taught, instead of the actual robot position. As a result, the robot may move to a location different from what has been taught.

[Available robot type]

RV-20A
RV-4A/5AJ series
RV-1A/2AJ
RV-2A/3AJ series
RH-5AH/10AH/15AH series

[Related system variables]

[M_CMPDST](#)

[Related instructions]

[CMP OFF](#), [CMPG](#), [CMP TOOL](#), [CMP JNT](#)

CMP TOOL (Composition Tool)

[Function]

Start the soft control mode (compliance mode) of the specified axis in the TOOL coordinates system.

Note) The available robot type is limited such as RV-4A. Refer to "[Available robot type]".

[Format]

CMP □ TOOL, <Axis designation>

[Terminology]

<Axis designation> Designate axis to move softly with a bit pattern.

1 : Enable, 0 : Disable &B00000000

This corresponds to axis L2L1CBAZYX

[Reference Program]

10 MOV P1	' Moves to in front of the part insertion position.
20 CMPG 0.5, 0.5, 1.0, 0.5, 0.5, , ,	' Set softness.
30 CMP TOOL, &B011011	' The X, Y, A, and B axes are put in the state where they are controlled in a pliable manner.
40 MVS P2	' Moves to the part insertion position.
50 M_OUT(10)=1	' Instructs to close the chuck for positioning.
60 DLY 1.0	' Waits for the completion of chuck closing.(1 sec.)
70 HOPEN 1	' Open the hand.
80 MVS, -100	' Retreats 100 mm in the Z direction of the TOOL coordinate system.
90 CMP OFF	' Return to normal state.

[Explanation]

- (1) The robot can be moved softly with the tool coordinate system. For the tool coordinate system, please refer to [Page 275, "5.1 Standard Tool Coordinates"](#).
- (2) For example, when inserting a pin in the tool coordinate Z axis direction, if the X, Y, A and B axes are set to soft operation, the pin can be inserted smoothly.
- (3) The degree of softness can be designated with the CMPG command.
- (4) The soft state is maintained even after the robot program execution is stopped. To cancel the soft status, execute the "CMP OFF" command or turn OFF the power.
- (5) When pressing in the soft state, the robot cannot move to positions that exceed the operation limit of each joint axis.
- (6) The deviation of the command position and actual position can be read with M_CMPDST. The success/failure of pin insertion can be checked using this variable.
- (7) If the amount of difference between the original target position and the actual robot position becomes greater than 200 mm by pushing the hand, etc., the robot will not move any further and the operation shifts to the next line of the program.
- (8) It is not possible to use CMP JNT, POS, and TOOL at the same time. In other words, an error occurs if the CMP POS or CMP TOOL instruction is executed while the CMP JNT instruction is being performed. Cancel the CMP JNT instruction once using the CMP OFF instruction to execute these instructions.
- (9) If the servo turns from OFF to ON while this command is functioning, the robot position could change.
- (10) It is possible to perform jog operations while the robot is in compliance mode. However, the setting of the compliance mode cannot be canceled by the T/B; in order to do so, execute this instruction in a program or execute it directly via the program edit screen of the T/B.
- (11) To change the axis specification, cancel the compliance mode with the CMP OFF instruction first, and then execute the CMP TOOL instruction again.
- (12) For vertical 5-axis robots (such as the RV-5AJ), only the X and Z axes can be used for axis specification.
- (13) If the robot is operated near a singular point, an alarm may be generated or control may be disabled. Do not operate the robot near a singular point.

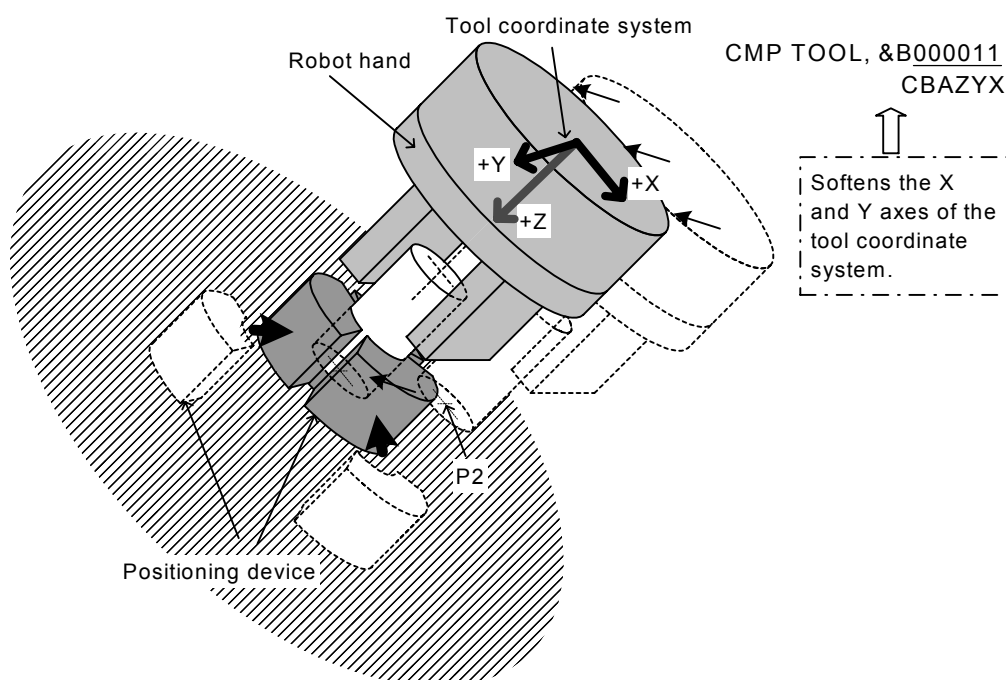


Fig.4-5 : The example of using the compliance mode

⚠ CAUTION

The compliance mode is in effect continuously until the CMP OFF instruction is executed, or the power is turned off. Exercise caution when changing the executable program number or operating the jog.

⚠ CAUTION

To execute a jog operation after setting the compliance mode with the CMP TOOL instruction, use the TOOL jog mode.

If any other jog mode is used, the robot may operate in a direction different from the expected moving direction because the directions of the coordinate systems controlled by the jog operation and the compliance mode differ.

⚠ CAUTION

When performing the teaching of a position while in the compliance mode, perform servo OFF first.

Be careful that if teaching operation is performed with Servo ON, the original command position is taught, instead of the actual robot position. As a result, the robot may move to a location different from what has been taught.

[Available robot type]

RV-20A
RV-4A/5AJ series
RV-1A/2AJ
RV-2A/3AJ series
RH-5AH/10AH/15AH series

[Related system variables]

[M_CMPDST](#)

[Related instructions]

[CMP OFF](#), [CMPG](#), [CMP POS](#), [CMP JNT](#)

CMP OFF (Composition OFF)

[Function]

Release the soft control mode (compliance mode).

Note) The available robot type is limited such as RV-4A. Refer to "[Available robot type]".

[Format]

CMP ☐ OFF

[Reference Program]

10 MOV P1	' Moves to in front of the part insertion position.
20 CMPG 0.5, 0.5, 1.0, 0.5, 0.5, , ,	' Set softness.
30 CMP TOOL, &B011011	' The X, Y, A, and B axes are put in the state where they are controlled in a pliable manner.
40 MVS P2	' Moves to the part insertion position.
50 M_OUT(10)=1	' Instructs to close the chuck for positioning.
60 DLY 1.0	' Waits for the completion of chuck closing.(1 sec.)
70 HOPEN 1	' Open the hand.
80 MVS, -100	' Retreats 100 mm in the Z direction of the TOOL coordinate system.
90 CMP OFF	' Return to normal state.

[Explanation]

- (1) This instruction cancels the compliance mode started by the CMP TOOL, CMP POS, or CMP JNT instruction.
- (2) In order to cancel jog operations in the compliance mode, either execute this instruction in a program or execute it directly via the program edit screen of the T/B.

[Available robot type]

RV-20A
RV-4A/5AJ series
RV-1A/2AJ
RV-2A/3AJ series
RH-5AH/10AH/15AH series

[Related instructions]

[CMPG](#), [CMP TOOL](#), [CMP POS](#), [CMP JNT](#)

CMPG (Composition Gain)

[Function]

Specify the softness of robot control.

Note) The available robot type is limited such as RV-4A. Refer to "[Available robot type]".

[Format]

CMP POS, CMP TOOL

CMPG □ [<X axis gain>], [<Y axis gain>], [<Z axis gain>], [<A axis gain>], [<B axis gain>], [<C axis gain>], [<L1 axis gain>], [<L2axis gain>],
--

CMP JNT

CMPG □ [<J1 axis gain>], [<J2 axis gain>], [<J3 axis gain>], [<J4 axis gain>], [<J5 axis gain>], [<J6 axis gain>], [<J7 axis gain>], [<J8 axis gain>],

[Terminology]

<X to L2 axis gain>

<J1 to J8 axis gain>

Specify this argument using a constant.

The softness can be set for each axis.

Value 1 .0 indicates the normal status, and the 0.2 is the softest.

If the value is omitted, the current setting value will be applied.

[Reference Program]

10 CMPG , ,0.5, , , , ,

' This statement selects only the Z-axis. For axes that are omitted, keep the corresponding entries blank and just enter commas.

[Explanation]

- (1) The softness can be designated in each axis units.
- (2) The soft state will not be entered unless validated with the CMP POS or CMP TOOL commands.
- (3) A spring-like force will be generated in proportion to the deviation of the command position and actual position. CMPG designates that spring constant.
- (4) The deviation of the command position and actual position can be read with M_CMPDST. The success/failure of pin insertion can be checked using this variable.
- (5) If a small gain is set, and the soft state is entered with the CMP POS, CMP TOOL, and CMP JNT commands, the robot position could drop. Set the softness state gradually while checking it.
- (6) The softness can be changed halfway when this command executed under the soft control status.
- (7) The gain value of less than 0.2 is invalid. The robot is controlled using the value 0.2.
(However, up to 0.1 can be set for the RV-1A/2AJ, and up to 0.0 for the RH- □ AH.)
Also, two or more decimal positions can be set for gain values.

[Available robot type]

RV-20A
RV-4A/5AJ series
RV-1A/2AJ
RV-2A/3AJ series
RH-5AH/10AH/15AH series

CNT (Continuous)

[Function]

Designates continuous movement control for interpolation. Shortening of the operating time can be performed by carrying out continuous movement.

[Format]

CNT ☐ <Continuous movement mode/acceleration/deceleration movement mode>
 [, <Numeric value 1>] [, <Numeric value 2>]

[Terminology]

- <1/0> Designate the continuous operation or acceleration/deceleration operation mode.
 1 : Continuous movement.
 0 : Acceleration/deceleration movement.(default value.)
- <Numeric value 1> Specify the maximum proximity distance in mm for starting the next interpolation when changing to a new path segment.
 The default value is the position where the acceleration/deceleration is started.
- <Numeric value 2> Specify the maximum proximity distance in mm for ending the previous interpolation when changing to a new path segment.
 The default value is the position where the acceleration/deceleration is started.

[Reference Program]

When the maximum neighborhood distance is specified when changing a locus.

10 CNT 0	' Invalidate CNT (Continuous movement).
20 MVS P1	' Operate with acceleration/deceleration
30 CNT 1	' Validate CNT (Continuous movement).
	(Operate with continuous movement after this line.)
40 MVS P2	' The connection with the next interpolation is continuous movement.
50 CNT 1,100,200	' Continuous operation specification at 100 mm on the starting side and at 200 mm on the end side.
60 MVS P3	' Continuous operation at a specified distance before and after an interpolation.
70 CNT 1,300	' Continuous operation specification at 300 mm on the starting side and at 300 mm on the end side.
80 MOV P4	' Continuous operation specification at 300 mm on the starting side.
90 CNT 0	' Invalidate CNT (Continuous movement).
100 MOV P5	' Operate with acceleration/deceleration

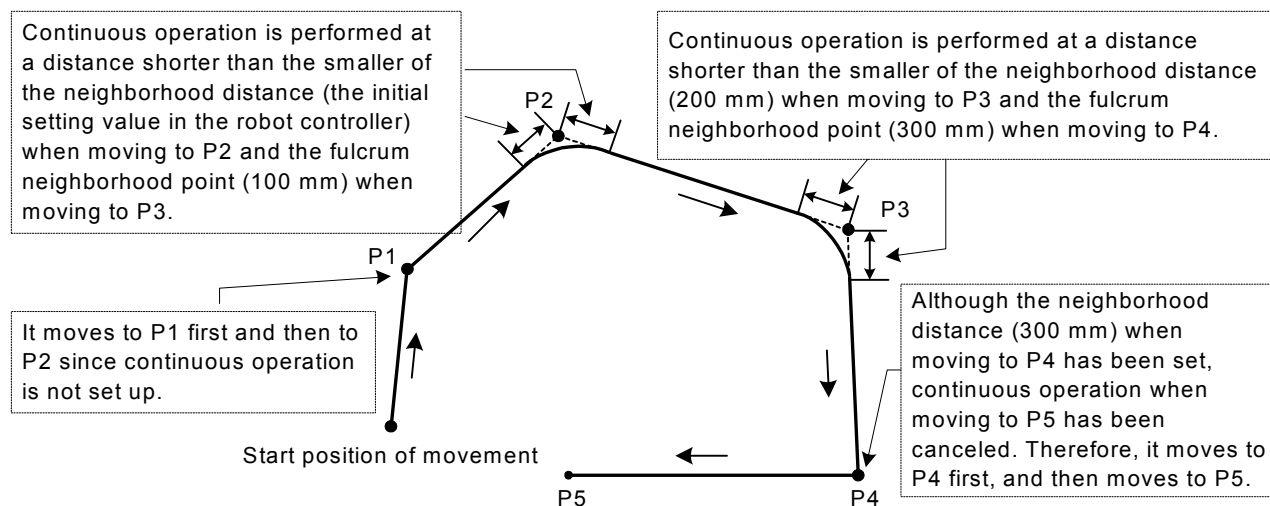


Fig.4-6 : Example of continuous path operation

[Explanation]

- (1) The interpolation (40 line to 80 line of the example) surrounded by CNT 1 – CNT 0 is set as the target of continuous action.
- (2) The system default value is CNT 0 (Acceleration/deceleration movement).
- (3) If values 1 and 2 are omitted, the connection with the next path segment is started from the time the deceleration is started.
- (4) As shown in Fig. 4-7, in the acceleration and deceleration operating mode, the speed is reduced in front of the target position. After moving to the target position, the speed for moving to the next target position starts to be accelerated. On the other hand, in the continuous operating mode, the speed is reduced in front of the target position, but it does not stop completely. The speed for moving to the next target position starts to be accelerated at that point. Therefore, it does not pass through each target position, but it passes through the neighborhood position.

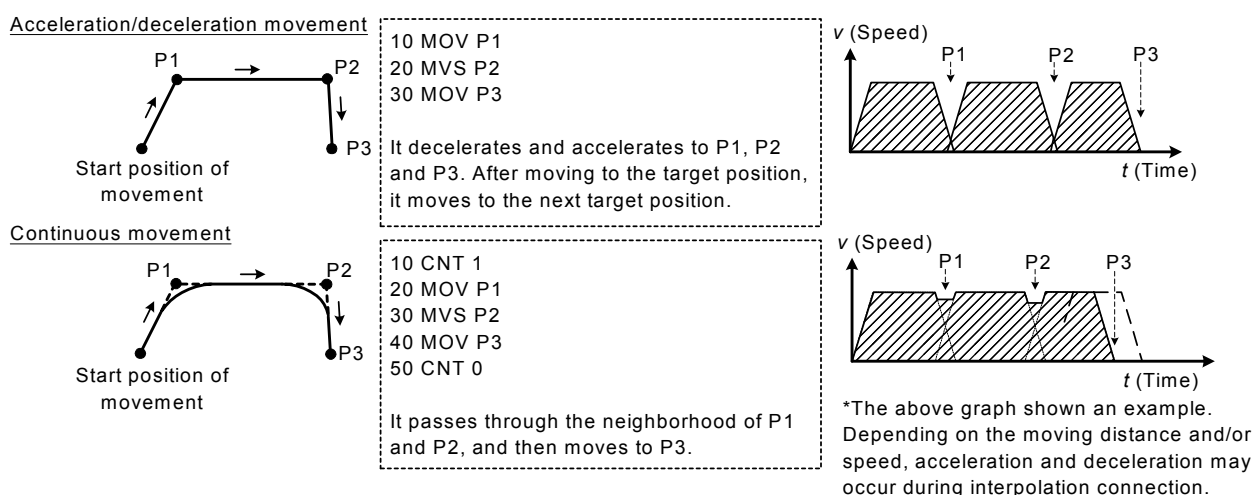


Fig.4-7 : Acceleration/deceleration movement and continuous movement

- (5) The neighborhood distance denotes the changing distance to the interpolation operation at the next target position. If this neighborhood distance (numerical value 1, numerical value 2) is omitted, the accelerate and deceleration starting position will be the changing position to the next interpolation. In this case, it passes through a location away from the target position, but the operating time will be the shortest. To pass through a location closer to the target position, set this neighborhood distance (numerical value 1, numerical value 2).

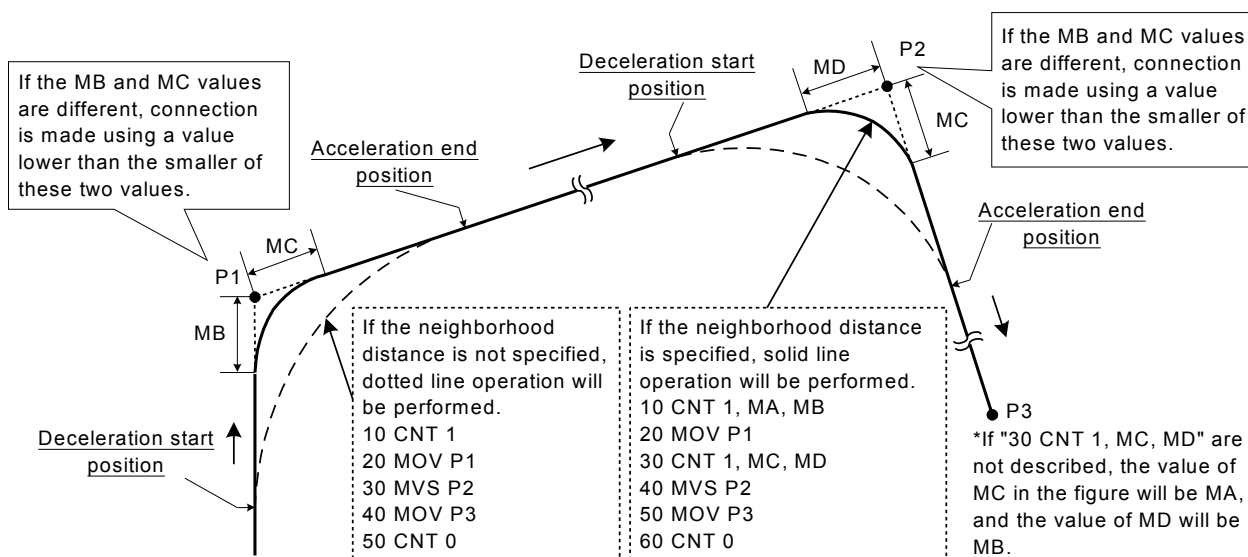


Fig.4-8 : Setting Up the Neighborhood Distance

- (6) If the specifications of numerical value 1 and numerical value 2 are different, continuous operation will be performed at the position (distance) that is the smaller of these two.
- (7) If numeric value 2 is omitted, the same value as numeric value 1 will be applied.
- (8) When continuous operation is specified, the positioning completion specification by the FINE instruction will be invalid.
- (9) If the proximity distance (value 1, value 2) is set small, the movement time may become longer than in the status where CNT 0.

COM ON/COM OFF/COM STOP (Communication ON/OFF/STOP)

[Function]

- COM ON : Allows interrupts from a communication line.
 COM OFF : Prohibits interrupts from a communication line.
 COM STOP : Prevents interrupts from a communication line temporarily (data is received).
 Jump immediately to the interrupt routine the next time the COM ON instruction is executed.

[Format]

COM ☐ [<Communication Line No.>] ☐ ON

COM ☐ [<Communication Line No.>] ☐ OFF

COM ☐ [<Communication Line No.>] ☐ STOP

[Terminology]

<Communication Line No.>

Describes numbers 1 to 3 assigned to the communication line.
 (If the argument is omitted, 1 is set as the default value.)

[Reference Program]

Refer to [Page 171](#), "ON COM GOSUB".

[Explanation]

- (1) When COMMON OFF is executed, even if communications are attempted, the interrupt will not be generated.
- (2) For information on communication line Nos., refer to the [Page 174](#), "OPEN".
- (3) After COM STOP is executed, even if communication is attempted, the interrupt will not be generated. Note that the receiving data and the fact of the interrupt will be recorded, and be executed the next time the line is reopened.

DEF ACT (Define act)

[Function]

This instruction defines the interrupt conditions for monitoring signals concurrently and performing interrupt processing during program execution, as well as the processing that will take place when an interrupt occurs.

[Format]

DEF □ ACT □ <Priority No.>, <Expression> □ <Process> [, <Type>]

[Terminology]

<Priority No.>	This is the priority No. of the interrupt. It can be set with constant Nos. 1 to 8.
<Expression>	For the interrupt status, use the formats described below: (Refer to the syntax diagram) <Numeric type data> <Comparison operator> <Numeric type data> or <Numeric type data> <Logical operator> <Numeric type data> <Numeric type data> refers to the following: <Numeric type constant> <Numeric variable> <Numeric array variable> <Component data>
<Process>	Refer to a GOTO statement or a GOSUB statement used to process an interrupt when it occurs.
<Type>	When omitted: Stop type 1 The robot stops at the stop position, assuming 100% execution of the external override. If the external override is small, the time required for the robot to stop becomes longer, but it will always stop at the same position. S : Stop type 2 (only for software version E3 or later) The robot decelerates and stops in the shortest time and distance possible, independently of the external override. L : Execution complete stop The interrupt processing is performed after the robot has moved to the target position (the line being executed is completed).

[Reference Program]

10 DEF ACT 1, M_IN(17)=1 GOSUB 100	' Defines the subroutine at line 100 to be the one to be called up when the status for the general purpose input signal No. 17 is ON.
20 DEF ACT 2, MFG1 AND MFG2 GOTO 200	' Defines the line 200 as the one to jump to when the logic operation of AND applied to MFG1 or MFG2 results in "true."
30 DEF ACT 3, M_TIMER(1)>10.5 GOSUB *LBL	' When 10.5 seconds pass, the program jumps to the line 300 subroutine.
:	
100 M_TIMER(1)=0	' Sets the timer to zero.
110 ACT 3=1	' Enables ACT 3.
120 RETURN 0	
200 MOV P_SAFE	
210 END	
300 *LBL	
310 M_TIMER(1)=0.0	' Resets the timer to zero.
320 ACT 3=0	' Disables ACT 3.
320 RETURN 0	

[Explanation]

- (1) The priority level for the interrupts is decided by the <Priority No.>, and the priority level, from the highest ranges from 1 to 8.
- (2) There can be up to 8 settings for the interrupts. Use the <Priority No.> to differentiate them.
- (3) An <expression> should be either a simple logical operation or a comparison operation (one operator). Parentheses cannot be used either.
- (4) If two DEF ACT instructions with the same priority number are included in a program, the latter one defined becomes valid.

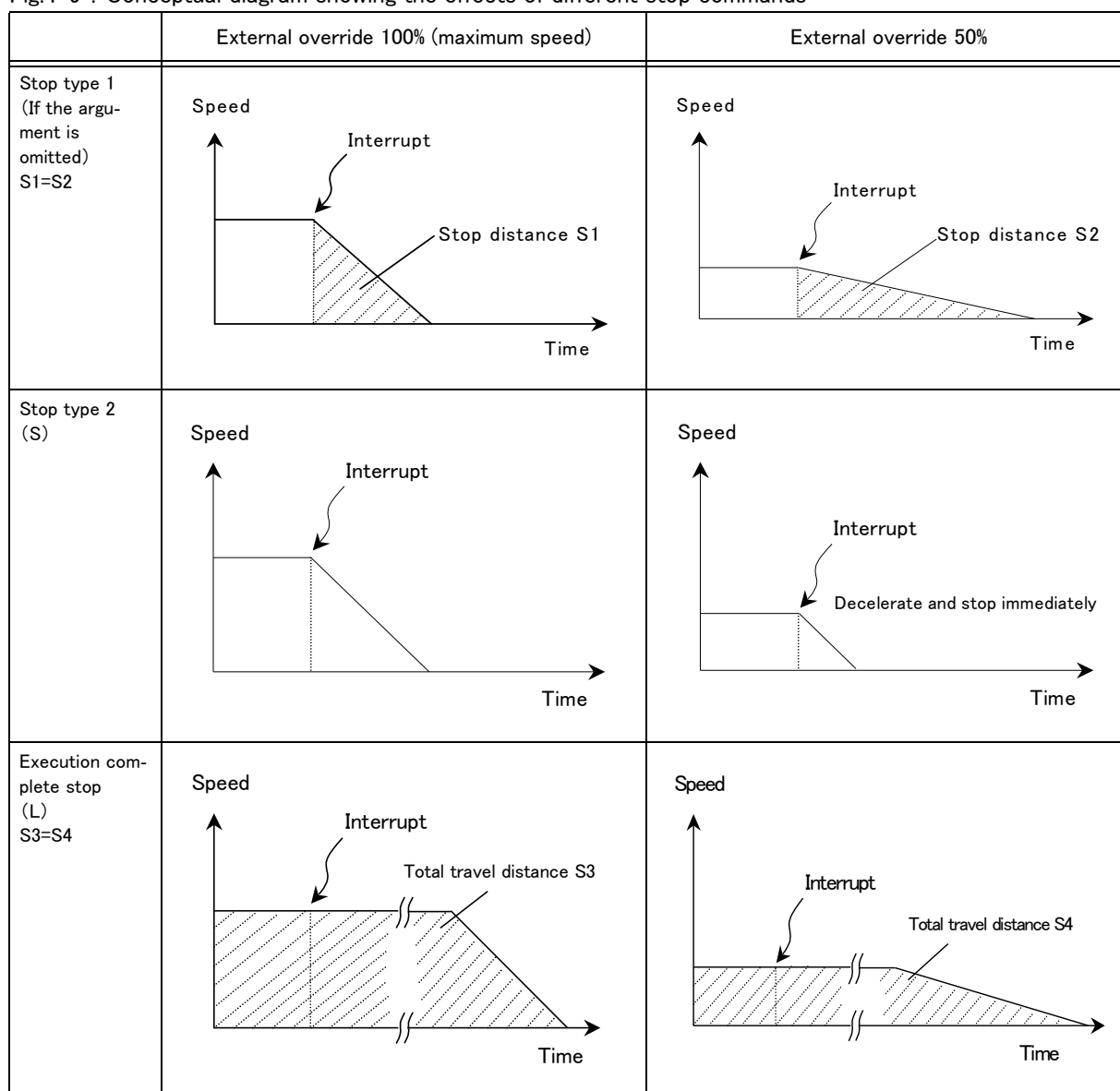
- (5) Since DEF ACT defines only the interrupt, always use the ACT command to designate the enable/disable status of the interrupt.
- (6) The communications interrupt (COM) has a higher priority level than any of the interrupts defined by DEF ACT.
- (7) DEF ACT definitions are valid only in the programs where they are defined. These are invalid when called up in a program by CALLP. If necessary, the data in a sub program may need to be redefined.
- (8) If an interrupt is generated when a GOTO command is designated by <Process> for a DEF ACT command, during execution of the remaining program, the interrupt in progress will remain, and only interrupts of a higher level will be accepted. The interrupt in progress for a GOTO statement can be canceled with the execution of an END statement.
- (9) Expressions containing conditional expressions combined with logical operations, such as (M1 AND &H001) = 1, are not allowed.

⚠ CAUTION

Specify the proper interrupt stop type according to the purpose. Specify "S" for the stop type if it is desired to stop the robot in the shortest time and distance possible by an interrupt while the robot is executing a movement instruction.

The following conceptual diagrams illustrate the effects of the three types of execution program stop commands when the interrupt conditions are met while the robot is moving according to a movement instruction.

Fig.4-9 : Conceptual diagram showing the effects of different stop commands



[Related instructions]

ACT

DEF ARCH (Define arch)

[Function]

This instruction defines an arch shape for the arch motion movement corresponding to the MVA instruction.

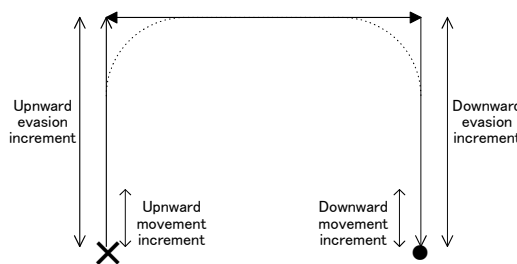
[Format]

This function is available for controller software version G2 or later.

```
DEF ARCH □ <Arch number>, [<upward movement increment>][<downward movement increment >]
                [<Upward evasion increment>], [<downward evasion increment>],
                [<interpolation type>], [<interpolation type 1>, <interpolation type 2> ]
```

[Terminology]

- <Arch number> Arch motion movement pattern number. Specify a number from 1 to 4 using a constant or a variable.
- <Upward movement increment>
<Downward movement increment >
Refertofigureatright.Itispos-
sible to specify either a con-
stant or a variable.
- <Upward evasion increment>
<Downward evasion increment>
- <Interpolation type> Interpolation type for upward
and downward movements.
Linear/joint = 1/0
- <Interpolation type 1> Detour/short cut = 1/0,
- <Interpolation type 2> 3-axis XYZ/Equivalent rotation = 1/0



If any of the arguments besides the arch number is omitted, the default value is employed.

The default values are set by the following parameters. Check the corresponding parameters to see the values; it is also possible to modify the values.

Parameter name	Arch number	Upward movement increment (mm)	Downward movement increment (mm)	Upward evasion increment (mm)	Downward evasion increment (mm)
ARCH1S	1	0.0	0.0	30.0	30.0
ARCH2S	2	10.0	10.0	30.0	30.0
ARCH3S	3	20.0	20.0	30.0	30.0
ARCH4S	4	30.0	30.0	30.0	30.0

Vertical multi-joint robot(RV-1A/2AJ, RV-4A/5AJ, etc.) Horizontal multi-joint robot(RH-5AH**, etc.)

Parameter name	Arch number	Interpolation type	Interpolation type 1	Interpolation type 2	Parameter name	Arch number	Interpolation type	Interpolation type 1	Interpolation type 2
ARCH1T	1	1	0	0	ARCH1T	1	0	0	0
ARCH2T	2	1	0	0	ARCH2T	2	0	0	0
ARCH3T	3	1	0	0	ARCH3T	3	0	0	0
ARCH4T	4	1	0	0	ARCH4T	4	0	0	0

[Reference Program]

```
10 DEF ARCH 1,5,5,20,20
20 MVA P1,1
30 MVA P2,2
```

'Performs the arch motion movement defined in the shape definition in line 10.

'The robot moves according to the default values specified by the parameters.

[Explanation]

- (1) If the MVA instruction is executed without the DEF ARCH instruction, the robot moves according to the arch shape specified by the parameters.
- (2) Used to change the increments in a program, etc.

[Related instructions]

MVA, ACCEL, OVRD, MVS(Used as a reference for interpolation types 1 and 2)

DEF FN (Define function)

[Function]

Defines a function and gives it name.

[Format]

```
DEF □ FN <Name> [( <Dummy Argument> [, <Dummy Argument> ] ... )]
                = <Function Definition Expression>
```

[Terminology]

<Name>	Describe an identification character + character string.
<Dummy argument>	When a function has been called up, it is transferred to the function. It is possible to describe all the variables, and up to 16 variables can be used.
<Function Definition Expression>	Describe the expression for what operation to use as a function.

[Reference Program]

```
10 DEF FNMAVE(MA,MB)=(MA+MB)/2      ' Define FNMAVE to obtain the average of two numeric values.
20 MDATA1=20
30 MDATA2=30
40 MAVE=FNMAVE(MDATA1,MDATA2)      ' Substitute average value 25 of 20 and 30 in numeric variable
                                     MAVE.
```

[Explanation]

- (1) FN + <Name> becomes the name of the function. The function name can be up to 8 characters long.
Example) Numeric value type ... FNMMAX Identification character: M
Character string type ... FNCAME\$ Identification character: C (Describe \$ at the end of the name)
- (2) A function defined with DEF FN is called a user-defined function. A function as long as one line can be described.
- (3) Built-in functions and user-defined functions that have already been defined can be used in the function definition expression. In this case, up to 16 levels of user-defined functions can be written.
- (4) If the variables used in <Function Definition Expression> are not located in <Dummy Argument>, then the value that the variable has at that time will be used. Also, an error will occur if during execution, the number or argument type (numeric value or character string) of arguments differs from the number or type declared.
- (5) A user-defined function is valid only in the program where it is defined. It cannot be used by a CALLP designation program.

DEF PLT (Define pallet)**[Function]**

Defines the pallet. (3-point pallet, 4-point pallet)

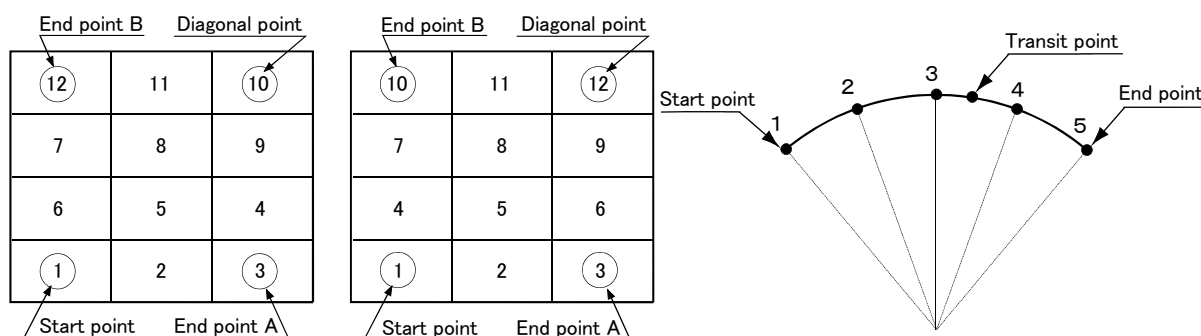
[Format]

DEF □ PLT □ <Pallet No.>, <Start Point>, <End Point A>, <End Point B>, [<Diagonal Point>],
<Quantity A>, <Quantity B>, <Assignment Direction>

[Terminology]

<Pallet No.>	This is the selection No. of the set pallet. (Constants from 1 to 8 only).
<Start Point>	Refers to the pallet's start point.
<End Point A>	One of the ending points for the pallet. Transit point of arc for arc pallet.
<End Point B>	Another ending point for the pallet. Ending point of arc for arc pallet.
<Diagonal Point>	The diagonal point from the pallet's start point. Insignificant for arc pallet.
<Quantity A>	The No. of workpieces from the pallet's start point to the end point A.
	The No. of workpieces between the pallet start point and arc end point when using an arc pallet.
<Quantity B>	The No. of workpieces from the pallet's start point to the end point B.
	Insignificant for an arc pallet. (1, etc., must be designated.)
<Assignment Direction>	Describes the direction of the number assignment when numbering divided grid points.

1 : Zigzag 2 : Same direction 3 : Arc pallet

**[Reference Program]**

```
10 DEF PLT 1,P1,P2,P3, ,4,3,1      ' Define a 3-point pallet.
20 DEF PLT 1,P1,P2,P3,P4,4,3,1     ' Define a 4-point pallet.
```

[Explanation]

- (1) The accuracy of the position calculation will be higher for a 3-point pallet than for a 4-point pallet.
- (2) The command is valid only within the program being executed. The command is invalid in the program that calls up the command from another program. If necessary, redefine.
- (3) Quantity A and B should be a non-zero positive number, while if 0 or a negative number is assigned, an error will occur.
- (4) If Quantity A × Quantity B exceeds 32,767, an error will occur when operation starts.
- (5) The value of quantity B is insignificant for the arc pallet, but it must not be omitted. The diagonal point will be insignificant even when designated. Set a dummy value.
- (6) If the hand is facing downward, the signs of the A, B, and C axis coordinates at the starting point, endpoint A, and endpoint B must match. If the hand is facing downward, A = 180 (or -180), B = 0, and C = 180 (or -180). If the signs of the A and C axis coordinates at the three positions do not match, the hand may rotate in the middle position. In this case, modify the signs so that they match in the position edit screen of the T/B. +180 and -180 result in the same posture; modifying signs poses no problem.

Please refer to the illustrations in [4.1.2Pallet operation](#), which explain this concept.

[Related instructions]

PLT

DEF INTE/DEF FLOAT/DEF DOUBLE (Define Integer/Float/Double)**[Function]**

Use this instruction to declare numerical values. INTE stands for integer, FLOAT stands for single-precision real number, and DOUBLE stands for double-precision real number.

[Format]

```
DEF □ INTE □ <Arithmetic variable name> [, <Arithmetic variable name>]...
DEF □ FLOAT □ <Arithmetic variable name> [, <Arithmetic variable name>]...
DEF □ DOUBLE □ <Arithmetic variable name> [, <Arithmetic variable name>]...
```

[Terminology]

<Arithmetic variable name> Designate the variable name.

[Reference Program]

(1) The definition of the integer type variable.

```
10 DEF INTE WORK1, WORK2    ' Declare WORK 1 and WORK 2 as an arithmetic variable name.
20 WORK1 = 100              ' Substitute the value 100 in WORK 1.
30 WORK2 = 10.562          ' Numerical "11" is set to WORK2.
40 WORK2 = 10.12            ' Numerical "10" is set to WORK2.
```

(2) The definition of the single precision type real number variable.

```
10 DEF FLOAT WORK3
20 WORK3 = 123.468          ' Numerical "123.468000" is set to WORK3.
```

(3) The definition of the double precision type real number variable.

```
10 DEF DOUBLE WORK
20 WORK4 = 100/3            ' Numerical "33.333332061767599" is set to WORK4.
```

[Explanation]

- (1) The variable name can have up to eight characters. Refer to the [Page 92, "4.3.6 Types of characters that can be used in program"](#) for the characters that can be used.
- (2) When designating multiple variable names, the maximum value (123 characters including command) can be set on one line.
- (3) The variable declared with INTE will be an integer type. (−32768 to +32767)
- (4) The variable declared with FLOAT will be a single-precision type. (± 1.70141E+38)
- (5) The variable declared with DOUBLE will be a double-precision type. (± 1.701411834604692E+308)

DEF IO (Define IO)

[Function]

Declares an input/output variable. Use this instruction to specify bit widths. M_IN and M_OUT variables are used for normal single-bit signals, M_INB and M_OUTB are used in the case of 8-bit bytes, and M_INW and M_OUTW are used in the case of 16-bit words.

Be aware that it is not allowed to reference output signals with variables declared using this instruction.

[Format]

DEF □ IO □ <Input/output variable name> = <Type designation>, <Input/output bit No.>
[, <Mask information>]

[Terminology]

<Input/output variable name>	Designate the variable name.
<Type designation>	Designate BIT(1bit), BYTE(8bit), WORD(16bit) or INTEGER.
<Input/output bit No.>	Designate the input(When referencing) or output(When assigning) bit No.
<Mask information>	Designate when only a specific signal is to be validated.

[Reference Program]

(1) Assign the input variable named PORT1 to input/output signal number 6 in bit type.

```
10 DEF IO PORT1 = BIT,6
:
100 PORT1 = 1      ' Output signal number 6 turns on.
:
200 PORT1 = 2      ' Output signal number 6 turns off.(Because the lowest bit of the numerical value 2 is 0.)
210 M1 = PORT1     ' Substitute the state of the input signal number 6 for M11.
```

(2) Assign the input variable named PORT2 to input/output signal number 5 in byte type, and specify the mask information as 0F in hexadecimal.

```
10 DEF IO PORT2 = BYTE, 5, &H0F
:
100 PORT2 = &HFF      ' Output signal number 5 to 8 turns on.
:
200 M2 = PORT2        ' Substitute the value of the input signals 5 to 8 for the variable M2.
```

(3) Assign the input variable named PORT3 to input/output signal number 8 in word type, and specify the mask information as 0FFE in hexadecimal.

```
10 DEF IO PORT3 = WORD, 8, &H0FFF
:
100 PORT3 = 9         ' Output signal number 8 and 11 turns on.
:
200 M3 = PORT3        ' Substitute the value of the input signals 8 to 19 for the variable M3.
```

[Explanation]

- (1) An input signal is read when referencing this variable.
- (2) An output signal is written when assigning a value to this variable.
- (3) It is not allowed to reference an output signal by this variable. Use the M_OUT variable in order to reference an output signal.
- (4) The variable name can have up to eight characters. Refer to the [Page 92, "4.3.6 Types of characters that can be used in program"](#) for the characters that can be used.
- (5) When mask information is designated, only the specified signal will be validated.

Example) In the above example on the 20th line, the input/output data with a bit width of eight is masked by 0F in hexadecimal. Thus, if PORT 2 is used thereafter,

· When used as an input signal (M1 = PORT 2):

Numbers 5 to 8 are used for input, and numbers 9 to 12 are always treated as 0.

No. 12 No.5 (Input/output bit No.)

0000 1111

Invalid Valid

· When used as an output signal (PORT 2 = M1):

Data to be output this time is output to numbers 5 to 8, and the status currently being output is retained at numbers 9 to 12.

No. 12 No.5 (Input/output bit No.)

**** 1111

↑

↑

Retains the current output status Output data of this time

DEF JNT(Define Joint)

[Function]

This instruction declares joint type position variables. It is used when using a variable with a name that begins with a character other than "J." It is not necessary to declare variables whose names begin with the character "J" using the DEF JNT instruction.

[Format]

DEF □ JNT □ <Joint variable name> [, <Joint variable name>]...
--

[Terminology]

<Joint variable name> Designate a variable name.

[Reference Program]

10 DEF JNT SAFE	' Declare "SAFE" as a joint variable.
20 MOV J1	' For joint type position variables starting with J, the definition of "DEF JNT" is not required.
30 SAFE = (-50,120,30,300,0,0,0,0)	
40 MOV SAFE	' Move to SAFE.

[Explanation]

- (1) Use this instruction to define a joint position variable by a name beginning with a character other than J.
- (2) The variable name can have up to eight characters. Refer to the [Page 92, "4.3.6 Types of characters that can be used in program"](#) for the characters that can be used. When designating multiple variable names, the maximum value (127 characters including command) can be set on one line.
- (3) A variable becomes a global variable that is shared among programs by placing "_" after J in the variable name and writing it in a base program.
Refer to [Page 101, "4.3.24 User-defined external variables"](#) for details.

DEF POS (Define Position)

[Function]

This instruction declares XYZ type position variables. It is used when using a variable with a name that begins with a character other than "P." It is not necessary to declare variables whose names begin with the character "P" using the DEF POS instruction.

[Format]

```
DEF □ POS □ <Position variable name> [, <Position variable name>]...
```

[Terminology]

<Position variable name> Designate a variable name.

[Reference Program]

10 DEF POS WORKSET	' Declare "WORKSET" as the XYZ type position variable.
20 MOV P1	' For XYZ type position variables starting with P, the definition of "DEF POS" is not required.
30 WORKSET=(250,460,100,0,0,-90,0,0)(0,0)	
40 MOV WORKSET	' Move to WORKSET.

[Explanation]

- (1) Use this instruction to define a XYZ type position variable by a name beginning with a character other than P.
- (2) The variable name can have up to eight characters. Refer to the [Page 92, "4.3.6 Types of characters that can be used in program"](#) for the characters that can be used.
- (3) When designating multiple variable names, the maximum value (127 characters including command) can be set on one line.
- (4) A variable becomes a global variable that is shared among programs by placing "_" after P in the variable name and writing it in a base program.
Refer to [Page 101, "4.3.24 User-defined external variables"](#) for details.

DEF CHAR (Define Character)

[Function]

Declares a character string variable. It is used when using a variable with a name that begins with a character other than "C." It is not necessary to declare variables whose names begin with the character "C" using the DEF CHAR instruction.

[Format]

```
DEF □ CHAR □ <Character string variable name>  
[, <Character string variable name>...
```

[Terminology]

<Character string variable name> Designate a variable name.

[Reference Program]

10 DEF CHAR MESSAGE	' Declare "MESSAGE" as a character string variable.
20 MESSAGE = "WORKSET"	' Substitute "WORKSET" in the MESSAGE variable.
30 CMSG = "ABC"	' Substitute "ABC" for variable CMSG. For variables starting with C, the definition of "DEF CHAR" is not required.

[Explanation]

- (1) The variable name can have up to eight characters. Refer to the [Page 92, "4.3.6 Types of characters that can be used in program"](#) for the characters that can be used.
- (2) When designating multiple variable names, the maximum value (127 characters including command) can be set on one line.
- (3) A variable becomes a global variable that is shared among programs by placing "_" after C in the variable name and writing it in a base program.
Refer to [Page 101, "4.3.24 User-defined external variables"](#) for details.

DIM (Dim)

[Function]

Declares the quantity of elements in the array variable. (Arrays up to the third dimension are possible.)

[Format]

```
DIM □ <Variable name> (<Element Value> [, <Element Value> [, <Element Value>]])
      [, <Variable name> (<Element Value> [, <Element Value> [, <Element Value>]])]...
```

[Terminology]

<Variable name>	Describe the name of the array variable.
<Element Value>	Describe in terms of constants, the number of elements in an array variable.

[Reference Program]

10 DIM PDATA(10)	' Define the position array variable PDATA having ten elements.
20 DIM MDATA#(5)	' Define double-precision type array variable MDATA# having the five elements.
30 DIM M1%(6)	' Define integer-type array variable M1% having the six elements.
40 DIM M2!(4)	' Define single-precision real number type array variable M2! having the four elements.
50 DIM CMOJI(7)	' Define the character-string type variable CMOJI having the seven elements.
60 DIM MD6(2,3), PD1(5,5)	' Define the 2-dimensional single precision real number type array variable MDATA having the element of 2x3.
	' Define the 2-dimensional position array variable PD 1 having the element of 5x5.

[Explanation]

- (1) A one-dimensional, two-dimensional or three-dimensional array can be used.
- (2) In the case of numeric variables, it is possible to use integer, single-precision real and double-precision real variables differently by adding a symbol that indicates the type of each variable to the variable name. If the variable type is omitted, a single-precision real variable will be assumed.
DIM MABC(10) ' Define the single-precision real number type array variable MABC having ten elements.
- (3) Element number start from 1 when actually referencing array variables. For PDATA on line 10 of the statement example, the element number will be 1 to 10.
- (4) <Element Value> can be described with numeric constants from 1 to 999. It is not allowed to use a numerical value operation expression.
If the number of elements is specified using a real number, an integer with rounded decimal part will be assumed. Depending on the system memory's free space, arrays may not be allocated for the number of specified elements. In this case, an error will occur when lines are registered.
- (5) If an element number larger than the number of defined elements is specified, an error will occur at the time of execution.
- (6) At the point when array variables are defined, variable values are indeterminate.
- (7) To use array variables, it is necessary to define them using the DIM instruction.
- (8) The arrays defined by the DIM instruction are valid only in the program where they are defined. To use these arrays by a sub program called by the CALLP instruction, it is necessary to define them again.
- (9) Array variables can be used similar to normal variables. However, note that variables of which variable names and/or the number of characters for specifying element numbers exceed eight characters cannot be used on the monitor variable screen and position edit screen of the teaching pendant.
- (10) If a variable name whose second character is underlined "_" is registered in a user program, a user defined external variable (a variable common among programs) will be assumed.
Refer to [Page 101](#), "4.3.24 User-defined external variables" for details.

DLY (Delay)

[Function]

- 1) When used as a single command:
At a designated time, it causes a wait. It is used for positioning the robot and timing input/output signals.
- 2) When used as an additional pulse output:
Designates an output time for a pulse.

[Format]

- 1) When used as a single command

DLY □ <Time>

- 2) When used as an additional pulse output

Example) M_OUT (1) = 1 DLY □ <Time>

[Terminology]

<Time> Describes the waiting time or the output time for the pulse output, in terms of a numeric operation expression. Unit: [Seconds]
The minimum value that can be set is 0.01 seconds. It is allowed to specify 0.00 as well.

[Reference Program]

(1) Waiting for time

10 DLY 30 ' Wait for 30 seconds

(2) Pulse output of the signal

20 M_OUT(17)=1 DLY 0.5 ' Send the signal output to the general-purpose output signal 17 for 0.5 seconds.

30 M_OUTB(18)=1 DLY 0.5 ' Among general-purpose output signals 18 to 25, only signal 18 is output (on) for the first 0.5 seconds, and signals 19 to 25 are output (on) after 0.5 seconds have passed.

(3) Wait for the completion of positioning.

10 MOV P1 ' Moves to P1.

20 DLY 0.1 ' Positions to 1.

(4) Wait for completion of hand opening. (closing)

10 HOPEN 1 ' Open the hand 1.

20 DLY 0.5 ' Wait for hand 1 to open securely.

[Explanation]

- (1) This instruction sets the wait time in a program. It is used for timing input/output signals, positioning movement instructions, and for specifying pulse output times when used in a signal output statement (such as line 20 in [statement example] above).
- (2) The pulse output will be executed simultaneously as the next command in the lines that follow.
- (3) Up to 50 pulse outputs can be issued of all programs simultaneously. Exceeding this, an error will occur when the program tries to execute it.
- (4) A pulse output reverses each of its bits after the specified time. This means that if M_OUTB (8-bit signal) or M_OUTW (16-bit signal) is used, the corresponding number of bits are reversed.
- (5) As for pulse output, the execution of a program ends without waiting the elapse of the specified duration if the END instruction or the last line of the program is executed during the specified duration. However, output turns off after the specified duration.
- (6) The relation of the priority levels for other interrupts is as shown below:
COM>ACT>WTHIF (WTH) >Pulse output (Time setting ON)
- (7) Even if stop is input during the execution of a pulse output, the pulse output operation will not stop.

Note1) If stop is input at line 20 in the following program, the output signal state will be held, and the execution is stopped.

```
10 M_OUT(17)=1
20 DLY 10
30 M_OUT(17)=0
```

Note2) If a pulse output by the M_OUTB (8-bit signal) or the M_OUTW (16-bit signal) is used, each bits in the corresponding bit width are reversed after the designated time.

```
M_OUTB(1)=1 DLY 1.0
```

In this case the bit pattern 00000001 is output for one second, and the bit pattern 11111110 is output thereafter.

ERROR(error)

[Function]

This instruction makes a program generate an error (9000s number).

[Format]

ERROR □ <Error No.>

[Terminology]

<Error No.> Either a constant or numeric operation expression can be set. Designate the No. within the range of 9000 to 9299.

[Reference Program]

(1) Generate the error 9000.

```
100 ERROR 9000
```

(2) Change the error number to generate corresponding to the value of M1.

```

40 IF M1 <> 0 THEN *LERR      ' When M1 is not 0, branches to "*LERR".
:
140 *LERR
150 MERR=9000+M1*10          ' Calculate the error number according to the value of M1.

160 ERROR MERR              ' The calculated error number is generated.
170 END

```

[Explanation]

- (1) It is possible to generate any error in the 9000's number range by executing this instruction.
- (2) If a LOW level or HIGH level error is generated, the program is paused.
Lines after the ERROR instruction are not executed. A CAUTION error does not pause a program; the next line and onward are executed. The action of system by error number is shown in the [Table 4-13](#).
- (3) It is possible to create up to 20 error messages using parameters UER1 to UER20.
- (4) A system error occurs if a value outside the error number range shown in [Table 4-13](#) is specified.

Table 4-13 : Action of system by error number

No.	System behavior
9000 to 9099 (H level error)	The program execution is stopped, and the servo power is shut off. The error state is reset when error reset is input.
9100 to 9199 (L level error)	The program execution is stopped. The error state is reset when error reset is input.
9200 to 9299 (CAUTION)	The program execution is continued. The error state is reset when error reset is input.

[Related parameter]

UER1 to 20

END (End)

[Function]

This instruction defines the final line of a program.

It is also used to indicate the end of a program explicitly, by entering the END instruction at the end of the main processing, in case a sub program is attached after the main program. In the case of a sub program called up by the CALLP instruction, the control is returned to the main program when the END instruction is executed.

[Format]

END

[Reference Program]

```
10  MOV P1
20  GOSUB *ABC
30  END          ' End the program.
:
100 *ABC
110  M1=1
120  RETURN
```

[Explanation]

- (1) This instruction defines the final line of a program. Use the HLT instruction to stop a program in the middle and put it in the pause status.
- (2) If executed from the operation panel, a program is executed in the continuous operation mode; it will be executed again from the top even if it contains an END instruction. If it is desired to end a program at the END instruction, press the END key on the operation panel to stop the cycle.
- (3) It is allowed to have several END statements within one program.
- (4) The END statement does not need to be described at the end of the program.
- (5) If the END command is executed by the sub program called by CALLP, control will return to the main program. The operation will be similar to the RETURN command of GOSUB.
- (6) The file and communication line which are opened are all closed by execution of the END command.
- (7) At program END, the SPD, ACCEL, OADL, JOVRD, OVRD, FINE and CNT settings will be initialized.

[Related instructions]

HLT, CALLP

FINE (Fine)

[Function]

This instruction specifies completion conditions of the robot's positioning. It is invalid during the smooth movement control (CNT 1).

Depending on the type of robot (RP series), positioning using the DLY instruction may be more effective than using the FINE instruction.

[Format]

FINE □ <No. of pulses> [, <Axis No.>]

[Terminology]

<No. of pulses>	Specify the positioning pulses number. This will be invalid to when set to 0. The default value is 0.
<Axis No.>	Designate the axis No. to which the positioning pulses are to be designated. The positioning pulses will be applied on all axes when omitted.

[Reference Program]

10 FINE 300	' Designate 300 for the positioning pulses.
20 MOV P1	
30 FINE 100,2	' Change the 2nd axis positioning pulses to 100.
40 MOV P2	
50 FINE 0	' Invalidate the positioning pulse designation.
60 MOV P3	
70 FINE 100	' Designate 100 for the positioning pulses.
80 MOV P4	

[Explanation]

- (1) The FINE instruction does not complete movement instructions such as MOV by giving commands to the servo; rather, it completes positioning by determining whether or not the feedback pulse value from the servo is within the specified range. It is thus possible to confirm positioning more accurately.
- (2) There are cases when the DLY instruction (timer) is used for positioning instead of the FINE instruction. This instruction is easier to specify.

10 MOV P1
20 DLY 0.1
- (3) FINE is invalid in the program until the FINE command is executed. Once FINE is validated, it remains valid until invalidated.
- (4) FINE is invalidated at the end of the program (Execution of the END instruction, program reset after pausing).
- (5) When the continuous movement control valid state (CNT 1) is entered, the FINE command will be ignored even if it is valid (i.e., it will be treated as invalid, but the status will be kept).
- (6) To the addition axis (general-purpose servo axis), although the valid/invalid change of FINE is possible, specification of the pulse number cannot be performed. The value registered in the "INP" parameter on the servo amplifier side is used. Thus, when the integers other than zero are specified, the FINE becomes effective by the parameter set value of servo amplifier, and the FINE becomes invalid when 0 is specified.

FOR – NEXT(For-next)

[Function]

Repeatedly executes the program between the FOR statement and NEXT statement until the end conditions are satisfied.

[Format]

```
FOR □ <Counter> = <Default value> TO <End Value> [STEP <Increment>]
:
NEXT □ [<Counter 1>]
```

[Terminology]

<Counter>	Describe the numerical variable that represents the counter for the number of repetitions. Same for <Counter 1> and <Counter 2>.
<Default Value>	Set default value of the counter for the number of repetitions as a numeric operation expression.
<End Value>	Set the end value of the counter for the number of repeats as a numeric operation expression.
<Increment>	Set the value of the increments for the counter for the number of repetitions as a numeric operation expression. It is allowed to omit this argument, including STEP.

[Reference Program]

(1) A program that adds the numbers 1 to 10

```
10 MSUM=0           ' Initialize the total MSUM.
20 FOR M1=1 TO 10    ' Increase the counter by 1 from 1 to 10 for the numeric variable M1.
30  MSUM=MSUM+M1     ' Add M1 value to numeric variable MSUM.
40 NEXT M1           ' Return to line 20.
```

(2) A program that puts the result of a product of two numbers into a 2-dimensional array variable

```
10 DIM MBOX(10,10)   ' Reserve space for a 10 × 10 array.
20 FOR M1=1 TO 10 STEP 1 ' Increase the counter by 1 from 1 to 10 for the numeric variable M1.
30  FOR M2=1 TO 10 STEP 1 ' Increase the counter by 1 from 1 to 10 for the numeric variable M2.
40    MBOX(M1,M2)=M1*M2 ' Substitute the value of M1*M2 for the array variable MBOX (M1, M2).
50  NEXT M2           ' Return to line 30.
60 NEXT M1           ' Return to line 20.
```

[Explanation]

- (1) It is possible to describe FOR-NEXT statements between other FOR-NEXT statements. Jumps in the program caused by the FOR-NEXT instruction will add one more level to the control structure in a program. It is possible to make the control structure of a program up to 16 levels deep. An error occurs at execution if 16 levels are exceeded.
- (2) If a GOTO instruction forces the program to jump out from between a FOR statement and a NEXT statement, the free memory available for control structure (stack memory) decreases. Thus, if a program is executed continuously, an error will eventually occur. Write a program in such a way that the loop exits when the condition of the FOR statement is met.
- (3) A run-time error occurs under the following conditions.
 - The counter's <Default Value> is greater than <End Value> and <Increment> is a positive number.
 - The counter's <Default Value> is smaller than <End Value>, and <Increment> is a negative number.
- (4) A run-time error occurs if a FOR statement and a NEXT statement are not paired.
- (5) When the NEXT statement corresponds to the closest FOR statement, the variable name in the NEXT statement can be omitted. In the example, "M2" in line 50 and "M1" in line 60 can be omitted. The processing speed will be slightly faster to omit the counter variable.

FPRM (FPRM)

[Function]

Defines the order of the arguments, the type, and number for the main program that uses arguments in a sub program (i.e., when the host program uses another program with CALL P).

[Format]

FPRM □ <Dummy Argument> [, <Dummy Argument>] ...
--

[Terminology]

<Dummy Argument>	The variable in the sub program that is transferred to the main statement when executed. All variables can be used. Up to 16 variables may be used.
------------------	---

[Reference Program]

<Main program>

10 M1=1

20 P2=P_CURR

30 P3=P100

40 CALLP "100",M1,P2,P3 ' It can be described like "CALLP "100", 1, P_CURR, P100" also.

<Sub program "100">

10 FPRM M1,P1,P2

20 IF M1=1 THEN GOTO 40

30 MOV P1

40 MVS P2

50 END ' Return to the main program.

[Explanation]

- (1) FPRM is unnecessary if there are no arguments in the sub program that is called up.
- (2) An error occur when the type or number is different between the argument of CALLP and the dummy argument that defined by FPRM.
- (3) It is not possible to pass the processing result of a sub program to a main program by assigning it in an argument.
To use the processing result of a sub program in a main program, pass the values using external variables.

[Related instructions]

[CALLP](#)

GETM (Get Mechanism)

[Function]

This instruction is used to control the robot by a program other than the slot 1 program when a multi-task is used, or to control a multi-mechanism by setting an additional axis as a user-defined mechanism.

Control right is acquired by specifying the mechanism number of the robot to be controlled. To release control right, use the RELM instruction.

[Format]

GETM □ <Mechanism No.>

[Terminology]

<Mechanism No.> 1 to 3, Specify this argument using a numerical or a variable.
The standard system's robot arm is assigned to mechanism 1.

[Reference Program]

(1) Start the task slot 2 from the task slot 1, and control the mechanism 1 in the task slot 2.

Task slot 1.

10 RELM	' Releases the mechanism in order to control mechanism 1 using slot 2.
20 XRUN 2,"10"	' Start the program 10 in slot 2.
30 WAIT M_RUN(2)=1	' Wait for the starting confirmation of the slot 2.
:	

Task slot 2. (Program "10")

10 GETM 1	' Get the control of mechanism 1.
20 SERVO ON	' Turn on the servo of mechanism 1.
30 MOV P1	
40 MVS P2	
50 P3=P_CURR	' Substitute P3 in mechanism 1 current position.
60 SERVO OFF	' Turn mechanism 1 servo OFF.
70 RELM	' Releases the control right of mechanism 1.
80 END	

[Explanation]

- (1) Normally (in single task operation), mechanism 1 is obtained in the initial status; it is not necessary to use the GETM instruction.
- (2) Because the control right of the same mechanism cannot be acquired simultaneously by multiple tasks, the following procedure is required in order to operate the robot by other than slot 1:
First, release control right using the RELM instruction by the slot 1 program. Next, acquire control right using the GETM instruction by the slot program that operates the robot. An error will be generated if the GETM instruction is executed again using a slot that has already acquired control right.
- (3) The instructions requiring control right include the motor power ON/OFF instruction, the interpolation instruction, the speed acceleration deceleration specification instruction, and the TOOL/BASE instruction.
- (4) If the argument is omitted from the system status variable requiring the mechanism designation, the currently acquired mechanism will be designated.
- (5) If the program is stopped, RELM will be executed automatically by the system. When the program is restarted, GETM will be executed automatically.
- (6) This instruction cannot be used in a constantly executed program.

[Related instructions]

REL

*GOSUB (RETURN)(Go Subroutine)***[Function]**

Calls up the subroutine at the designated line No. or line label. Be sure to return from the jump destination using the RETURN instruction.

[Format]

GOSUB □ <Call Destination>

[Terminology]

<Call Destination> Describe the line No. or label name.

[Reference Program]

<For a line number>

```
100 GOSUB 1000
110 END
```

```
1000 MOV P1
1010 RETURN
```

' Be sure to use the RETURN instruction to return.

<For a label>

```
100 GOSUB *LBL
110 END
```

```
1000 *LBL
1010 MOV P1
1020 RETURN
```

' Be sure to use the RETURN instruction to return.

[Explanation]

- (1) Make sure to return from the subroutine by using the RETURN command. If return by GOTO command, the memory for control structure (stack memory) will decrease, and it will cause the error at continuous executing.
- (2) The call of other subroutines is possible again by the GOSUB command out of the subroutine. This approach can be employed approximately up to 800 times.
- (3) The line number or label can be specified as a jump destination.
When the line or label of the call place does not exist, it becomes the execution-time error.

[Related instructions]

[RETURN](#)

GOTO (Go To)

[Function]

This instruction makes a program branch to the specified line number or label line unconditionally.

[Format]

GOTO □ <Branch Destination>

[Terminology]

<Branch Destination> Describe the line No. or label name.

[Reference Program]

(1) Specify the line number.

```
10 MOV P1
:
100 GOTO 10      ' Returns to the line number 10.
110 END
```

(2) Specify the label.

```
100 GOTO *LBL    ' Branches to the label *LBL.
:
1000 *LBL
1010 MOV P1
```

[Explanation]

- (1) A line number or label can be specified as a branch destination.
- (2) If a branch destination or label does not exist, an error will occur during execution.

[Function]

[Format]

HLT

(1) Stop the robot without condition during program execution.

(2) Stop the robot on some conditions.

200 MOV P1 WTHIF M_IN(17)=1, HLT	' When the input signal 17 turns on during moving to P1, the program execution is stopped.
----------------------------------	--

- (1) Interrupts the execution of a program and decelerates the robot to a stop. The system will enter the waiting state.
- (2) If the HLT instruction is used in multitask operation, only the task slot that executed the HLT instruction is paused.
- (3) To restart, start the O/P or issue the start signal from an external source. The program will be restarted at the next line after the HLT statement. Note that if the HLT statement is an appended statement, the operation will restart from the same line of the program where it was interrupted.

END

HOPEN / HCLOSE (Hand Open/Hand Close)**[Function]**

Commands the hand to open or close.

[Format]

HOPEN □ <Hand No.> [, <Starting grasp force>, <Holding grasp force>, <Starting grasp force holding time>]
 HCLOSE □ <Hand No.>

[Terminology]

<Hand No.>	Select a numeric value between 1 and 8. Specify this argument using a constant or a variable.
<Starting grasp force>	This parameter is valid for the motorized hand, and invalid for any other type of hand. Set the required grasping force for starting the hand open/close. Set the grasping force as a step between 0 and 63 (63 = 3.5kg). The default value is 63. When omitted, the previous setting value will be applied.
<Holding grasp force>	This parameter is valid for the motorized hand, and invalid for any other type of hand. Set the required grasping force for holding the hand open/close. Set the grasping force as a step between 0 and 63 (63 = 3.5kg). The default value is 63. When omitted, the previous setting value will be applied.
<Starting grasp force holding timer>	This parameter is valid for the motorized hand. Set the time to hold the starting grasping force as a value from 0.00(sec.). The default value is 0.3 sec.

[Reference Program]

```

10 HOPEN 1      ' Open hand 1.
20 DLY 0.2      ' Set the timer to 0.2 sec. (Wait for the hand to open securely.)
30 HCLOSE 1     ' Close hand 1.
40 DLY 0.2      ' Set the timer to 0.2 sec. (Wait for the hand to close securely.)
50 MOV PUP      '

```

[Explanation]

- (1) The operation (single/double) of each hand is set with parameter HANDTYPE.
- (2) If the hand type is set to double solenoid, hands 1 to 4 can be supported. If the hand type is set to single solenoid, hands 1 to 8 can be supported.
- (3) The status of the hand output signal when the power is turned ON is set with parameter HANDINIT.
- (4) The hand input signal can be confirmed with the robot status variable M_HNDCQ ("Hand input number"). The signal can also be confirmed with the input signals No. 900 to 907 (when there is one mechanism).


```

10 HCLOSE 1
20 IF M_HNDCQ(1)<>1 THEN GOTO 20
30 MOV P1

```
- (5) There are related parameters. Refer to [Page 281, "5.5 Automatic return setting after jog feed at pause"](#) and, [Page 285, "5.8 About default hand status"](#) of this manual.

[Related system variables]

M_IN/M_INB/M_INW (900s number), M_OUT/M_OUTB/M_OUTW (900s number), M_HNDCQ

[Related instructions]

LOADSET, OADL

[Related parameter]

HANDTYPE, HANDINIT

Refer to [Page 281, "5.5 Automatic return setting after jog feed at pause"](#) and, [Page 285, "5.8 About default hand status"](#).

IF...THEN...ELSE...ENDIF (If Then Else)**[Function]**

A process is selected and executed according to the results of an expression.

[Format]

```
IF □ <Expression> □ THEN □ <Process> □ [ELSE <Process>]
```

This function is available for controller software version G1 or later.

```
IF □ <Expression> □ THEN
  <Process>
  <Process>
  .
[ELSE]
  <Process>
  <Process>
  .
ENDIF
```

[Terminology]

<Expression>	Describe the expression targeted for comparison as a comparison operation expression or logic operation expression.
<Process>	Describe the process following THEN for when the comparison results are true, and the process following ELSE for when the comparison results are false.

[Reference Program](1) The software version earlier than G1 edition.

<pre>100 IF M1>10 THEN 1000 : 200 M1=10 210 MOV P1 220 GOTO 400 300 M1=-10 310 MOV P2 320 GOTO 400</pre>	<pre>' When M1 is larger than 10, jump to the line number 1000. ' If M1 is larger than 10, it jumps to line number 200; if smaller than 10, it jumps to line number 300. The "GOTO" after "THEN" or "ELSE" can be omitted.</pre>
---	---

(2) The software version is G1 edition or later.

```
100 IF M1>10 THEN
110   M1=10
120  MOV P1
130 ELSE
140   M1=-10
150  MOV P2
160 ENDIF
```

* The description method of earlier than G1 edition is also possible.

```
250 IF M2=0 THEN GOSUB *SUB1 ELSE GOSUB *SUB2
```

(3) When a IF statement is described inside THEN or ELSE (allowed in revision G1 and later)

```
300 IF M1>10 THEN
310   IF M2 > 20 THEN
320     M1 = 10
330     M2 = 10
340   ELSE
350     M1 = 0
360     M2 = 0
370   ENDIF
380 ELSE
390   M1 = -10
400   M2 = -10
410 ENDIF
```

[Explanation]

- (1) The IF .. THEN .. ELSE .. statements should be contained in one line.
- (2) It is allowed to split an IF .. THEN .. ELSE .. ENDIF block over several lines.
- (3) ELSE can be omitted.
- (4) Make sure to include the ENDIF statement in the IF .. THEN .. ELSE .. ENDIF block.
- (5) If the GOTO instruction is used to jump out from inside an IF .. THEN .. ELSE .. ENDIF block, an error will occur when the memory for control structure (stack memory) becomes insufficient.
- (6) For IF .. THEN .. ELSE .. ENDIF, it is possible to describe IF .. THEN .. ELSE .. ENDIF inside THEN or ELSE. (UP to eight levels of nesting is allowed.)
- (7) GOTO following THEN or ELSE may be omitted.

Example) IF M1 > 10 THEN 200 ELSE 300

Also, only when THEN is followed by GOTO, either one of THEN or GOTO may be omitted. ELSE cannot be omitted.

Example) IF M1 > 10 THEN GOTO 200 (The program at left can be rewritten as shown below.)

⇒ IF M1 > 10 THEN 200

⇒ IF M1 > 10 GOTO 200

INPUT (Input)

[Function]

Inputs data into a file (including communication lines). Only ASCII character data can be received.

Please refer to [Page 288, "5.10 About the communication setting"](#), which lists related parameters.

[Format]

```
INPUT □ # <File No.>, <Input data name> [, <Input data name>] ...
```

[Terminology]

<File No.> Describe a number between 1 and 8.

This corresponds to the file No. assigned with the OPEN command.

<Input data name> Describe the variable name for saving the input data. All variables can be described.

[Reference Program]

```

10 OPEN "COM1:" AS #1 ' Assign RS-232-C to file No. 1.
20 INPUT #1, M1       ' The value will be set to the numerical variable M1 if data are inputted from the
                       ' keyboard.
30 INPUT #1, CABC$    '
:
100 CLOSE #1

```

[Explanation]

- (1) Data is input from file having the file No. opened with the OPEN statement, and is substituted in the variable. If the OPEN statement has not been executed, an error will occur.
- (2) The type of data input and the type of variable that is substituting it must be the same.
- (3) When describing multiple variable names, use a comma (,) between variable names as delimiters.
- (4) When the INPUT statement is executed, the status will be "standby for input." The input data will be substituted for the variables at the same time as the carriage return (CR and LF) are input.
- (5) If the protocol (in the case of the standard port: the "CRPC232" parameter is 0) of the specified port is for PC support (non procedure), it is necessary to attach "PRN" at the head of any data sent from a PC. Normally, the standard port is connected to a PC and used for transferring and debugging robot programs. Therefore, it is recommended to use the optional expansion serial interface if a data link is used.
- (6) If the number of elements input is greater than the number of arguments in the INPUT statement, they will be read and discarded.

When the END or CLOSE statement is executed, the data saved in the buffer will be erased.

Example) To input both a character string, numeric value and position.

```
10 INPUT #1,C1$,M1,P1
```

Data sent from the PC side

(when received by the standard port of the robot: the "CRPS232" parameter is 0)

```
PRNMELFA,125.75,(130.5,-117.2,55.1,16.2,0,0)(1,0) CR
```

MELFA is substituted in C1\$, 125.75 in M1, and (130.5, -117.2,55.1,16.2,0,0)(1,0) in P1.

[Related instructions]

[OPEN](#), [CLOSE](#), [PRINT](#)

JOVRD (J Override)

[Function]

Designates the override that is valid only during the robot's joint movements.

[Format]

JOVRD □ <Designated override>

[Terminology]

<Designated override> Describe the override as a real number.
A numeric operation expression can also be described.
Unit: [%] (Recommended range: 1 to 100.0)

[Reference Program]

```
10 JOVRD 50
20 MOV P1
30 JOVRD M_NJOVRD      ' Set the default value.
```

[Explanation]

- (1) The JOVRD command is valid only during joint interpolation.
- (2) The actual override is = (Operation panel (T/B) override setting value) x (Program override (OVRD command)) x (Joint override (JOVRD command)). The JOVRD command changes only the override for the joint interpolation movement.
- (3) The 100% <Designate override> is the maximum capacity of the robot. Normally, the system default value (M_NOVRD) is set to 100%. The value is reset to the default value when the END statement is executed or the program is reset.

[Related instructions]

[OVRD](#), [SPD](#)

[Related system variables]

[M_JOVRD](#)/[M_NJOVRD](#)/[M_OPOVRD](#)/[M_OVRD](#)/[M_NOVRD](#)
(M_NJOVRD (System default value) , M_JOVRD(Currently specified joint override))

JRC (Joint Roll Change)

[Function]

- The current position is rewritten to a position obtained by adding ± 360 degrees to the current joint position of the J6 axis.
- User-defined axis (additional axis, user-defined mechanism)
The current position is rewritten to a position obtained by adding or subtracting a value designated in the parameters to the current joint position of the designated axis.
Either a joint axis or linear axis can be used.
The origin can also be reset at the current position.

[Format]

JRC < [+] 1 / -1 / 0 > [, < Axis No >]

[Terminology]

<+1>	The current joint angle of the designated axis is incremented by the amount designated in parameter JRCQTT. (The sign can be omitted.) For the robot arm's J6 axis, the value is fixed to 360 degrees.
<-1>	The current joint angle of the designated axis is decremented by the amount designated in parameter JRCQTT. For the robot arm's J6 axis, the value is fixed to 360 degrees.
<0>	The origin for the designated axis is reset at the value designated in parameter JRCORG. This can be used only for the user-defined axis.
<Axis No>	The target axis is specified with the number. The default axis is J6 of robot arm.

[Reference Program]

10 MOV P1	' Moves to P1.
20 JRC 1	' Increases J6 axis by 360° .
30 MOV P1	' Moves to P1.

[Explanation]

- (1) The current joint angle of the designated axis is incremented/decremented by the designated amount with the JRC 1/-1 command.
The origin for the designated axis is reset with the JRC 0 command.
The joint angle will change, but the robot will not move.
- (2) When using this command, change the movement range of the target axis beforehand so that it does not leave the movement range when the command is executed. The range can be changed by changing the - side and + side value of the corresponding axis in the joint movement range parameter "MEJAR". Set the movement range for the rotating axis in the range of -2340 deg. to 2340 deg.
- (3) If the designated axis is omitted, the priority axis will be the target. The priority axis is the rotating axis (J6 axis) at the end of the robot.
- (4) If the designated axis is omitted when a priority axis does not exist (robot incapable of JRC), or if the designated axis is not a target for JRC, an error will occur when the command is executed.
- (5) If the origin is not set, an error will occur when the command is executed.
- (6) The robot is stopped while the JRC command is executed. Even if CNT is validated, the interpolation connection will not be continuous when this command is executed.
- (7) The following parameter must be set before using the JRC command.
Set JRCEXE to 1. (JRC execution enabled)
Change the movement range of the target axis with MEJAR.
Set the position change amount during the JRC 1/-1 execution with JRCQTT.
(Only for the additional axis or user-defined mechanism.)
Set the origin position for executing JRC 0 with JRCORG.
(Only for the additional axis or user-defined mechanism.)
- (8) When parameter JRCEXE is set to 0, no process will take place even if JRC command is executed.
- (9) If the movement amount designated with parameter JRCQTT is not within the pulse data 0 to MAX., an error will occur during the initialization. Here, MAX. is $2^{\wedge}(\text{Number of encoder bits} + 15) - 1$. For example,
with a 13-bit encoder (8192 pulses), this will be MAX. = $2^{\wedge}(13+15) - 1 = 0x0fffff$,
and for a 14-bit encoder (16384 pulses), this will be MAX. $2^{\wedge}(14+15) - 1 = 0x1fffff$.

The movement amount to pulse data conversion is as follows:

For rotating axis

Pulse data = movement amount (deg.)/360 * gear ratio denominator/gear ratio numerator * Number of encoder pulses

For linear axis

Pulse data = movement amount (mm) * gear ratio denominator/gear ratio numerator * Number of encoder pulses

(10) The origin data will change when JRC is executed, so the default origin data will be unusable.

If the controller needs to be initialized due to a version upgrade, etc., the parameters must be backed up beforehand in the original state.

(11) Step return operation is not possible with the JRC command.

(12) This instruction cannot be used in a constantly executed program.

[Related parameter]

JRC EXE Set whether to enable/disable the JRC execution.

Execution disabled = 0 (default value)/execution enabled = 1

JRC QTT Designate the amount to move (1 deg./1mm unit) when incrementing or decrementing with the JRC command in additional axis or user-defined mechanism.

The value is fixed to 360 degrees for the JRC target axis on the robot arm.

JRC ORG Designate the origin for executing JRC 0. in additional axis or user-defined mechanism.

Refer to [Page 266, "5 Functions set with parameters"](#) for detail.

[Target mechanism and target axis]

- RV-1A/2AJ, RV-4A/5AJ and related models, RV-20A J6 axis
- User-defined additional axis for all mechanisms
- All user-defined mechanism axes

LOADSET (Load Set)

[Function]

This instruction specifies the condition of the hand/workpiece at execution of the OADL instruction.

[Format]

LOADSET □ <Hand condition No.>, <Workpiece condition No.>

[Terminology]

- | | |
|---------------------------|--|
| <Hand condition No.> | Designate the hand condition (HNDDAT 1 to 8) No. for which the weight and size are designated. |
| <Workpiece condition No.> | Designate the hand condition (WRKDAT 1 to 8) No. for which the weight and size are designated. |

[Reference Program]

```

10 OADL ON
20 LOADSET 1,1      ' Hand 1(HNDDAT1) and workpiece 1(WRKDAT1) conditions.
30 MOV P1
40 MOV P2
50 LOADSET 1,2      ' Hand 1(HNDDAT1) and workpiece 2(WRKDAT1) conditions.
60 MOV P1
70 MOV P2
80 OADL OFF

```

[Explanation]

- (1) Set the hand conditions and workpiece conditions used for optimum acceleration/deceleration. This is used when setting the optimum acceleration/deceleration for workpiece types having different weights.
- (2) The maximum load is set for the hand when the program execution starts.
- (3) Set the weight, size (X, Y, Z) and center of gravity position (X, Y, Z) as the hand conditions in parameter (HND-DAT 1 to 8).
- (4) Set the weight, size (X, Y, Z) and center of gravity position (X, Y, Z) as the workpiece conditions in parameter (WRKDAT 1 to 8).
- (5) The hand conditions and workpiece conditions changed when this command is executed are reset to the system default value when the program is reset and when the END statement is executed.
As the system default values, the hand conditions are set to the rated load, and the workpiece conditions are set to none (0kg).
- (6) Refer to [Page 291, "5.11 About the optimum acceleration/deceleration setting"](#) for details on the optimum acceleration/deceleration.

[Related instructions]

[OADL](#), [HOPEN](#) / [HCLOSE](#)

[Related parameter]

HNDDAT1 to 8, WRKDAT1 to 8, HNDHOLD1 to 8

Refer to [Page 291, "5.11 About the optimum acceleration/deceleration setting"](#).

Refer to [Page 266, "Table 5-1 : List parameter"](#) for the ACCMODE.

MOV (Move)

[Function]

Using joint interpolation operation, moves from the current position to the destination position.

[Format]

MOV ☐ <Target Position> [, <Close Distance>] [☐ TYPE ☐ <Constants 1>, <Constants 2>] ☐
 [<Appended conditions>]

[Terminology]

<Movement Target Position>	This is the final position for interpolation operation. This position may be specified using a position type variable and constant, or a joint variable.
<Close Distance>	If this value is designated, the actual movement target position will be a position separated by the designated distance in the tool coordinate system Z axis direction (+/- direction).
<Constants 1>	1/0 : Detour/short cut. The default value is 1 (detour).
<Constants 2>	Invalid (Specify 0).
<Appended conditions>	The WTH and WTHIF statements can be used.

[Reference Program]

```

10 MOV P1 TYPE 1,0
20 MOV J1
30 MOV (PLT 1,10),100.0 WTH M_OUT(17)=1
40 MOV P4+P5,50.0 TYPE 0,0 WTHIF M_IN(18)=1,M_OUT(20)=1

```

[Explanation]

- (1) The joint angle differences of each axis are evenly interpolated at the starting point and endpoint positions.
This means that the path of the tip cannot be guaranteed.
- (2) By using the WTH and WTHIF statement, the signal output timing and motion can be synchronized.
- (3) The numeric constant 1 for the TYPE designates the posture interpolation amount.
- (4) Detour refers to the operating exactly according to the teaching posture. Short cut operation may take place depending on the teaching posture.
- (5) Short cut operation refers to posture interpolation between the start point and end point in the direction with less motion.
- (6) The detour/short cut designation is significant when the posture axis has a motion range of (180 deg. or more).
- (7) Even if short cut is designated, if the target position is outside the motion range, the axis may move with the detour in the reverse direction.
- (8) The TYPE numeric constant 2 setting is insignificant for joint interpolation.
- (9) This instruction cannot be used in a constantly executed program.
- (10) If paused during execution of a MOV instruction and restarted after jog feed, the robot returns to the interrupted position and restarts the MOV instruction. The interpolation method (JOINT interpolation / XYZ interpolation) which returns to the interrupted position can be changed by the "RETPATH" parameter.
Moreover, it is also possible by changing the value of this RETPATH parameter to move to the direct target position, without returning to the interrupted position. (Refer to [Page 281](#), "5.5 Automatic return setting after jog feed at pause")

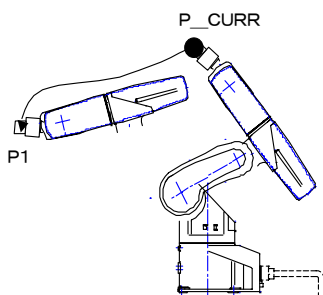


Fig.4-10 : Example of joint interpolation motion path

MVA (Move Arch)

[Function]

This instruction moves the robot from the current position to the target position with an arch movement (arch interpolation).

[Format]

This function is available for controller software version G2 or later.

MVA □ <Target Position> [, <Arch number>]

[Terminology]

<Target Position>	Final position of interpolation movement. This position may be specified using a position type variable and constant, or a joint variable.
<Arch number>	A number defined by the DEF ARCH instruction (1 to 4). If the argument is omitted, 1 is set as the default value.

[Reference Program]

10 DEF ARCH 1,5,5,20,20	' Defines the arch shape configuration.
20 OVRD 100,20,20	' Specifies override.
30 ACCEL 100,100,50,50,50,50	' Specifies acceleration/deceleration rate.
20 MVA P1,1	' Performs the arch motion movement according to the shape configuration defined in line 10.
30 MVA P2,2	' Moves the robot according to the default values registered in the parameters.

[Explanation]

- (1) The robot moves upward along the Z-axis direction from the current position, then moves to a position above the target position, and finally moves downward, reaching the target position. This so-called arch motion movement is performed with one instruction.
- (2) If the MVA instruction is executed without the DEF ARCH instruction, the robot moves with the arch shape configuration set in the parameters. Refer to [Page 134, "DEF ARCH"](#) for a detailed description about the parameters.
- (3) The interpolation form, type and other items are also defined by the DEF ARCH instruction; refer to [Page 134, "DEF ARCH"](#).
- (4) This instruction cannot be used in a constantly executed program.
- (5) If paused during execution of a MVA instruction and restarted after jog feed, the robot returns to the interrupted position and restarts the MVA instruction. (this can be changed by the "RETPATH" parameter). The interpolation method (JOINT interpolation / XYZ interpolation) which returns to the interrupted position can be changed by the "RETPATH" parameter. (Refer to [Page 281, "5.5 Automatic return setting after jog feed at pause"](#))

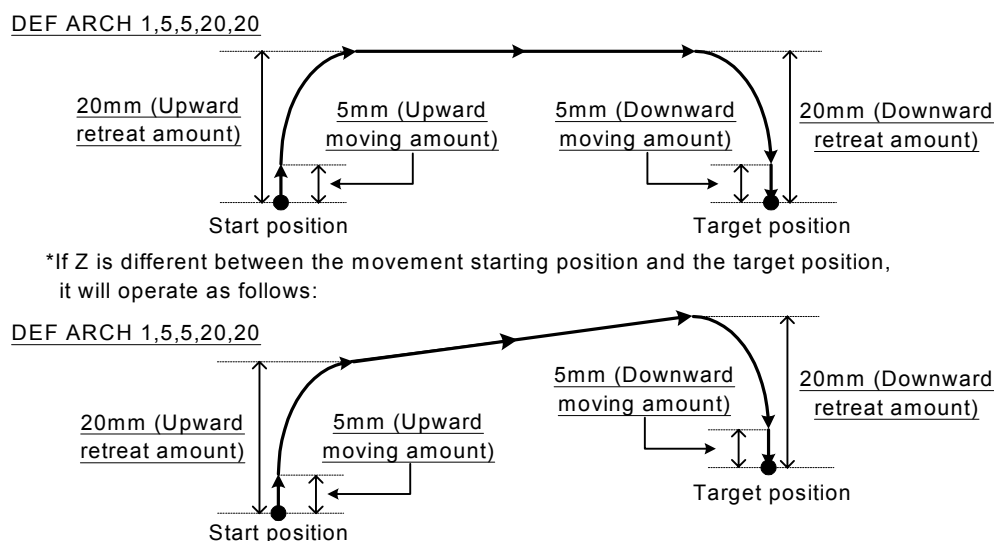


Fig.4-11 : Example of arch interpolation motion path (seen from the side)

[Related instructions]

[DEF ARCH](#), [ACCEL](#), [OVRD](#)

MVC (Move C)

[Function]

Carries out 3D circular interpolation in the order of start point, transit point 1, transit point 2 and start point.

[Format]

MVC □ <Start point>,<Transit point 1>,<Transit point 2> □ [<Additional condition>]

[Terminology]

<Start point>	The start point and end point for a circle. Describe a position operation expression or joint operation expression.
<Transit point 1>	Transit point 1 for a circular arc. Describe a position operation expression or joint operation expression.
<Transit point 2>	Transit point 2 for a circular arc. Describe a position operation expression or joint operation expression.
<Additional condition>	Describe a WTH conjunction or a WTHIF conjunction

[Reference Program]

```

10 MVC P1,P2,P3
20 MVC P1,J2,P3
30 MVC P1,P2,P3 WTH M_OUT(17)=1
40 MVC P3,(PLT 1,5),P4 WTHIF M_IN(20)=1,M_OUT(21)=1

```

[Explanation]

- (1) In circular interpolation motion, a circle is formed with the 3 given points, and the circumference is moved. (360 degrees)
- (2) The posture at the starting point is maintained during circle interpolation. The postures while passing points 1 and 2 are not considered.
- (3) If the current position and the starting position do not match, the robot automatically moves to the starting point based on the linear interpolation (3-axis XYZ interpolation), and then performs the circle interpolation.
- (4) If paused during execution of a MVC instruction and restarted after jog feed, the robot returns to the interrupted position by JOINT interpolation and restarts the remaining circle interpolation.
The interpolation method (JOINT interpolation / XYZ interpolation) which returns to the interrupted position can be changed by the "RETPATH" parameter. (Refer to [Page 281](#), "5.5 Automatic return setting after jog feed at pause")
- (5) This instruction cannot be used in a constantly executed program.

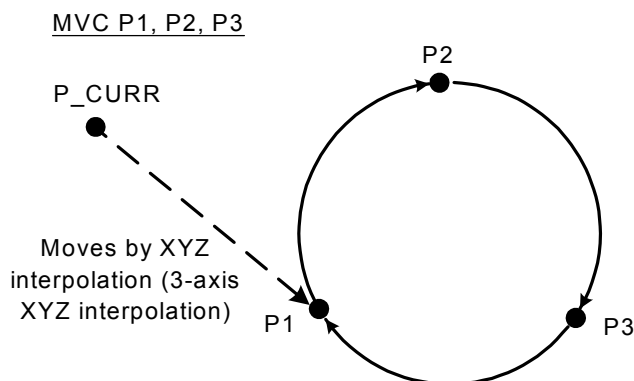


Fig.4-12 : Example of circle interpolation motion path

MVR (Move R)

[Function]

Carries out 3-dimensional circular interpolation movement from the start point to the end point via transit points.

[Format]

MVR □ <Start Point>, <Transit Point>, <End Point>
 [□ TYPE □ <Constants 1>, <Constants 2>] □ [<Appended Condition>]

[Terminology]

<Start Point>	Start point for the arc. Describe a position operation expression or joint operation expression.
<Transit Point>	Transit point for the arc. Describe a position operation expression or joint operation expression.
<End Point>	End point for the arc. Describe a position operation expression or joint operation expression.
<Constants 1>	Detour/short cut = 1/0, The default value is 0.
<Constants 2>	3-axis XYZ/Equivalent rotation = 1/0, The default value is 0.
<Appended conditions>	The WTH and WTHIF statements can be used.

[Reference Program]

```
10 MVR P1,P2,P3
20 MVR P1,J2,P3
30 MVR P1,P2,P3 WTH M_OUT(17)=1
40 MVR P3,(PLT 1,5),P4 WTHIF M_IN(20)=1,M_OUT(21)=1
```

[Explanation]

- (1) In circular interpolation motion, a circle is formed with three given points, and robot moves along the circumference.
- (2) The posture is interpolation from the start point to the end point; the transit point posture has no effect.
- (3) If the current position and start point do not match, the robot will automatically move with linear interpolation (3-axis XYZ interpolation) to the start point.
- (4) If paused during execution of a MVR instruction and restarted after jog feed, the robot returns to the interrupted position by JOINT interpolation and restarts the remaining circle interpolation.
 The interpolation method (JOINT interpolation / XYZ interpolation) which returns to the interrupted position can be changed by the "RETPATH" parameter. (Refer to [Page 281](#), "5.5 Automatic return setting after jog feed at pause")
- (5) If the start point and end point structure flags differ for an interpolation method other than 3-axis XYZ interpolation, an error will occur at the execution.
- (6) Of the three designated points, if any points coincide with the other, or if three points are on a straight line, linear interpolation will take place from the start point to the end point. An error will not occur.
- (7) If 3-axis XYZ is designated for the numeric constant 2, the numeric constant 1 will be invalidated, and the robot will move with the taught posture.
- (8) Numeric constant 2 designates the posture interpolation type. 3-axis XYZ is used when carrying out interpolation on the (X, Y, Z, J4, J5, J6) coordinate system, and the robot is to move near a particular point.
- (9) This instruction cannot be used in a constantly executed program.

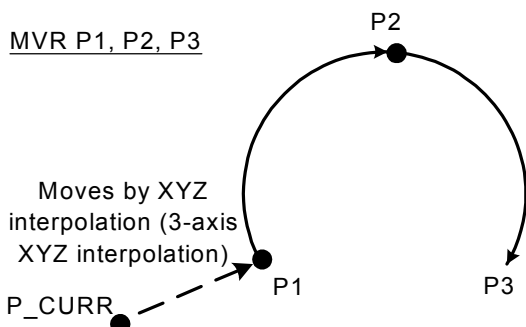


Fig.4-13 : Example of circular interpolation motion path 1

MVR2 (Move R2)

[Function]

Carries out 3-dimensional circular interpolation motion from the start point to the end point on the arc composed of the start point, end point, and reference points.

The direction of movement is in a direction that does not pass through the reference points.

[Format]

MVR2 □ <Start Point>, <End Point>, <Reference point>
 [□ TYPE □ <Constants 1>, <Constants 2>] □ [<Appended Condition>]

[Terminology]

<Start Point>	Start point for the arc. This position may be specified using a position type variable and constant, or a joint variable.
<End Point>	Endpoint for the arc. This position may be specified using a position type variable and constant, or a joint variable.
<Reference point>	Reference point for a circular arc. This position may be specified using a position type variable and constant, or a joint variable.
<Constants 1>	Detour/short cut = 1/0, The default value is 0.
<Constants 2>	3-axis XYZ/Equivalent rotation = 1/0, The default value is 0.
<Appended conditions>	The WTH and WTHIF statements can be used.

[Reference Program]

```
10 MVR2 P1,P2,P3
20 MVR2 P1,J2,P3
30 MVR2 P1,P2,P3 WTH M_OUT(17)=1
40 MVR2 P3,(PLT 1,5),P4 WTHIF M_IN(20)=1,M_OUT(21)=1
```

[Explanation]

- (1) In circular interpolation motion, a circle is formed with three given points, and robot moves along the circumference.
- (2) The posture is interpolation from the start point to the end point; the reference point posture has no effect.
- (3) If the current position and start point do not match, the robot will automatically move with linear interpolation (3-axis XYZ interpolation) to the start point.
- (4) If paused during execution of a MVR instruction and restarted after jog feed, the robot returns to the interrupted position by JOINT interpolation and restarts the remaining circle interpolation.
The interpolation method (JOINT interpolation / XYZ interpolation) which returns to the interrupted position can be changed by the "RETPATH" parameter. (Refer to [Page 281](#), "5.5 Automatic return setting after jog feed at pause")
- (5) The direction of movement is in a direction that does not pass through the reference points.
- (6) If the start point and end point structure flags differ for an interpolation method other than 3-axis XYZ interpolation, an error will occur at the execution.
- (7) Of the three designated points, if any points coincide with the other, or if three points are on a straight line, linear interpolation will take place from the start point to the end point. An error will not occur.
- (8) If 3-axis XYZ is designated for the numeric constant 2, the numeric constant 1 will be invalidated, and the robot will move with the taught posture.
- (9) Numeric constant 2 designates the posture interpolation type. 3-axis XYZ is used when carrying out interpolation on the (X, Y, Z, J4, J5, J6) coordinate system, and the robot is to move near a particular point.
- (10) This instruction cannot be used in a constantly executed program.

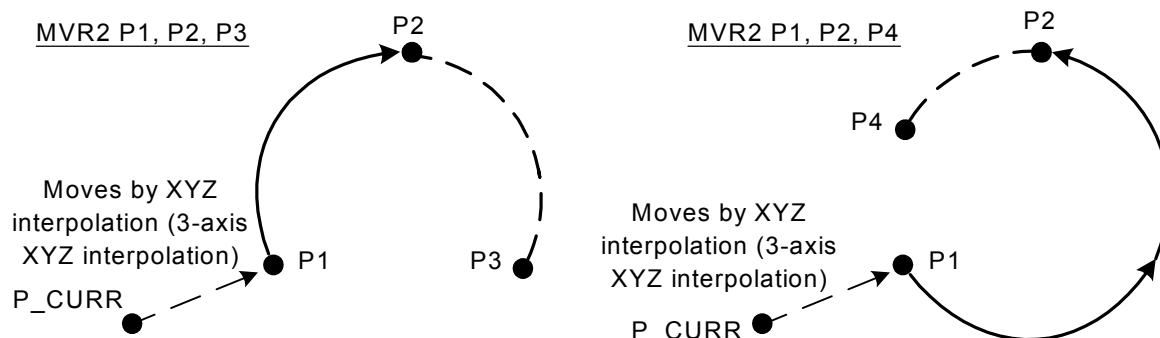


Fig.4-14 : Example of circular interpolation motion path 2

MVR3 (Move R 3)

[Function]

Carries out 3-dimensional circular interpolation movement from the start point to the end point on the arc composed of the center point, start point and end point.

[Format]

MVR3 ☐ <Start Point>, <End Point>, <Center Point>
☐ TYPE ☐ <Constants 1>, <Constants 2>] ☐ [<Appended Condition>]

[Terminology]

<Start Point>	Start point for the arc. This position may be specified using a position type variable and constant, or a joint variable.
<End Point>	Endpoint for the arc. This position may be specified using a position type variable and constant, or a joint variable.
<Center Point>	Center point for the arc. This position may be specified using a position type variable and constant, or a joint variable.
<Constants 1>	Detour/short cut = 1/0, The default value is 0.
<Constants 2>	3-axis XYZ/Equivalent rotation = 1/0, The default value is 0.
<Appended conditions>	The WTH and WTHIF statements can be used.

[Reference Program]

```
10 MVR3 P1,P2,P3
20 MVR3 P1,J2,P3
30 MVR3 P1,P2,P3 WTH M_OUT(17)=1
40 MVR3 P3,(PLT 1,5),P4 WTHIF M_IN(20)=1,M_OUT(21)=1
```

[Explanation]

- (1) In circular interpolation motion, a circle is formed with three given points, and robot moves along the circumference.
- (2) The posture is interpolation from the start point to the end point; the center point posture has no effect.
- (3) If the current position and start point do not match, the robot will automatically move with linear interpolation (3-axis XYZ interpolation) to the start point.
- (4) If paused during execution of a MVR3 instruction and restarted after jog feed, the robot returns to the interrupted position by JOINT interpolation and restarts the remaining circle interpolation.
The interpolation method (JOINT interpolation / XYZ interpolation) which returns to the interrupted position can be changed by the "RETPATH" parameter. (Refer to [Page 281](#), "5.5 Automatic return setting after jog feed at pause")
- (5) If the start point and end point structure flags differ for an interpolation method other than 3-axis XYZ interpolation, an error will occur at the execution.
- (6) If 3-axis XYZ is designated for the numeric constant 2, the numeric constant 1 will be invalidated, and the robot will move with the taught posture.
- (7) Numeric constant 2 designates the posture interpolation type. 3-axis XYZ is used when carrying out interpolation on the (X, Y, Z, J4, J5, J6) coordinate system, and the robot is to move near a particular point.
- (8) The fan angle from the start point to the end point is $0 < \text{fan angle} < 180 \text{ deg}$.
- (9) Designate the positions so that the difference from the center point to the end point and the center point to the distance is within 0.01mm.
- (10) If the three points are on the same line, or if the start point and center point, or end point and center point are the same, an error will occur.
- (11) If the start point and end point are the same or if three points are the same, an error will not occur, and the next command will be executed. Note that if the posture changes at this time, only the posture will be interpolated.
- (12) This instruction cannot be used in a constantly executed program.

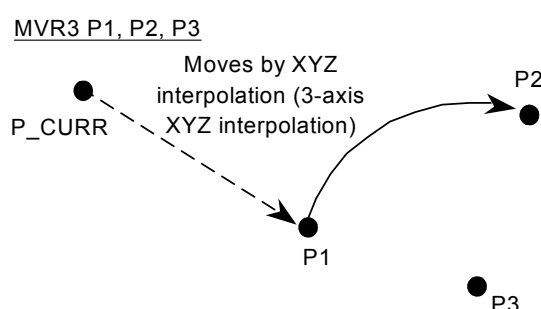


Fig.4-15 : Example of circular interpolation motion path 3

MVS (Move S)

[Function]

Carries out linear interpolation movement from the current position to the movement target position.

[Format 1]

MVS □ <Movement Target Position> [, <Close Distance>]
[□ TYPE <Constants 1>, <Constants 2>] □ [<Appended Condition>]

[Format 2]

MVS □ , <Separation Distance> □ [<Interpolation Type>]

[Terminology]

<Movement Target Position>	The final position for the linear interpolation. This position may be specified using a position type variable and constant, or a joint variable.
<Close Distance>	If this value is designated, the actual movement target position will be a position separated by the designated distance in the tool coordinate system Z axis direction (+/- direction).
<Constants 1>	Detour/short cut = 1/0, The default value is 0(detour).
<Constants 2>	3-axis XYZ/Equivalent rotation = 1/0, The default value is 0(equivalent rotation).
<Appended conditions>	The WTH and WTHIF statements can be used.
<Separation Distance>	When this value is designated, the axis will move the designated distance from the current position to the Z axis direction (+/- direction) of the tool coordinate system.

[Reference Program]

(1) Move to the target position P1 by XYZ interpolation.

10 MVS P1

(2) Turns on the output signal 17 at the same time if it moves to the target position P1 by linear interpolation.

10 MVS P1,100.0 WTH M_OUT(17)=1

(3) Turns on output signal 20 if the input signal 18 is turned on while moving 50 mm in the Z direction of the tool coordinate system of the target position P4+P5 (relative operation position obtained by addition) by linear interpolation.

20 MVS P4+P5, 50.0 WTHIF M_IN(18)=1, M_OUT(20)=1

(4) Moves 50 mm in the Z direction of the tool coordinate system from the current position by linear interpolation.

30 MVS ,50

[Explanation]

- (1) Linear interpolation motion is a type of movement where the robot moves from its current position to the movement target position so that the locus of the control points is in a straight line.
- (2) The posture is interpolation from the start point to the end point.
- (3) In the case of the tool coordinate system specified by using <proximity distance> or <separation distance>, the + and - directions of the Z axis vary depending on the robot model. Refer to [Page 275, "5.1 Standard Tool Coordinates"](#) for detail. The "Fig.4-16: Example of movement at linear interpolation" is the example of RV-1A movement.

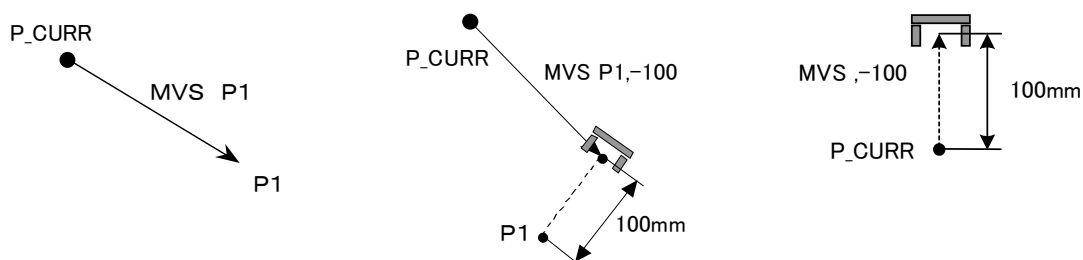


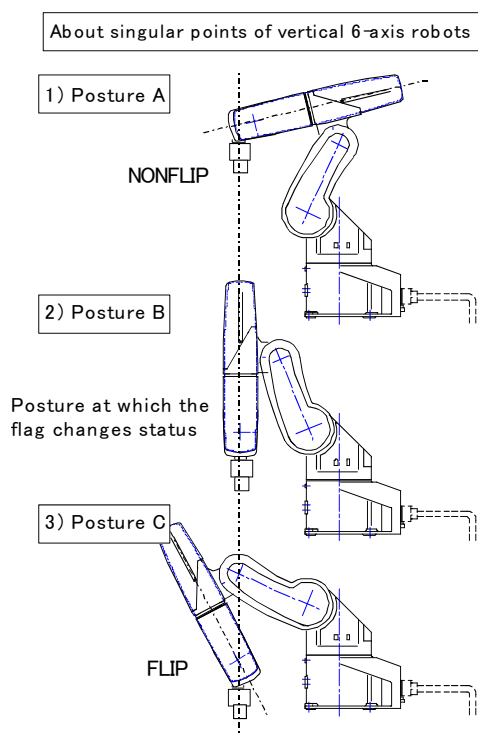
Fig.4-16 : Example of movement at linear interpolation

- (4) If paused during execution of a MVS instruction and restarted after jog feed, the robot returns to the interrupted position and restarts the MVS instruction. This can be changed by the "RETPATH" parameter, and also the interpolation method (JOINT interpolation / XYZ interpolation) which returns to the interrupted position can be changed by same parameter. Some robots for liquid crystal transportation have different default values of this parameter. Refer to [Page 281, "5.5 Automatic return setting after jog feed at pause"](#).
- (5) This instruction cannot be used in a constantly executed program.
- (6) If the start point and end point structure flags differ for an interpolation method other than 3-axis XYZ interpolation, an error will occur at the execution.
- (7) If 3-axis XYZ is designated for the numeric constant 2, the numeric constant 1 will be invalidated, and the robot will move with the taught posture.
- (8) Numeric constant 2 specifies the type of posture interpolation. Three-axis XYZ operation is used to pass through near a singular point in order to perform interpolation in the coordinate system of (X, Y, Z, J4, J5, J6).
- (9) Description of singular points.

<In the case of a vertical 6-axis robot (RV-1A, RV-4A, and RV-20A)>

Movement from posture A, through posture B, to posture C cannot be performed using the normal linear interpolation (MVS).

Fig.4-17 : Singular point 1



This limitation applies only when J4 axis is at zero degrees at all the postures A, B, and C. This is because the structure flag of axis J5 (wrist axis) is FLIP for posture A and NONFLIP for posture C. Moreover, in posture B, the wrist is fully extended and axes J4 and J6 are located on the same line. In this case, the robot cannot perform a linear interpolation position calculation.

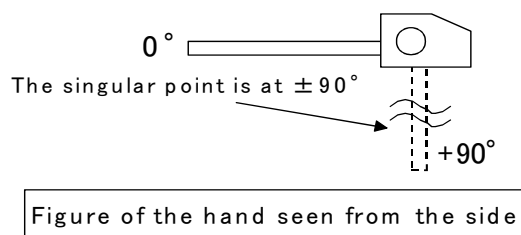
The 3-axis XYZ (TYPE 0, 1) method in the command option of MVS should be used if it is desired to perform linear interpolation based on such posture coordinates. Note that, strictly speaking, this 3-axis XYZ method does not maintain the postures as it evenly interpolates the joint angle of axes J4, J5, and J6 at posture A and C. Therefore, it is expected that the robot hand's posture may move front and back while moving from posture A to posture C.

In this case, add one point in the middle to decrease the amount of change in the hand's posture.

Another singular point is when the center of axis J5 is on the origin and the wrist is facing upward. In this case, J1 and J6 are located on the same axis and it is not possible to calculate the robot position.

<In the case of a 6-axis robot for liquid crystal transportation (Robot RH-1500GVC)>

Fig.4-18 : Singular point 2



The singular points are when the wrist axis J5 is at +90 degrees, and when the center of axis J6 is located at the origin and the wrist is facing upward. In these cases, axes J1 and J5 are located on the same axis and it is not possible to calculate the robot position.

OADL (Optimal Acceleration)

[Function]

Automatically sets the optimum acceleration/deceleration according to the robot hand's load state (Optimum acceleration/deceleration control).

By employing this function, it becomes possible to shorten the robot's motion time (tact).

[Format]

OADL ☐ <ON / OFF>
--

[Terminology]

<ON / OFF>

ON : Start optimum acceleration/deceleration.

OFF : End optimum acceleration/deceleration.

[Reference Program]

10 OADL ON	
20 MOV P1	' Move with maximum load.
30 LOADSET 1, 1	' Set hand 1 and workpiece 1.
40 MOV P2	' Move with hand 1 + workpiece 1 load.
50 HOPEN 1	'
60 MOV P3	' Move with hand 1 load.
70 HCLOSE 1	'
80 MOV P4	' Move with hand 1 + workpiece 1 load.
90 OADL OFF	

*When parameter HNDHOLD1 is set to 0, 1

[Explanation]

- (1) The robot moves with the optimum acceleration/deceleration according to the hand conditions and workpiece conditions designated with the LOADSET command.
- (2) The workpiece grasp/not grasp for when the hand is opened or closed is set with parameter HNDHOLD 1 to 8.
- (3) Initial setting of OADL can be changed by the ACCMODE parameter. (Refer to [Page 266](#), "Table 5-1 : List parameter")
- (4) Once OADL is ON, it is valid until OADL OFF is executed or until the program END is executed.
- (5) Depending on the conditions of the hand and/or workpiece, the motion time may become longer than usual.
- (6) It is possible to perform the optimum acceleration/deceleration operation by using the LOADSET and OADL instructions, and by setting the HNDDAT1 through 8 and WRKDAT1 through 8 parameters to appropriate values. (Refer to [Page 291](#), "5.11 About the optimum acceleration/deceleration setting")

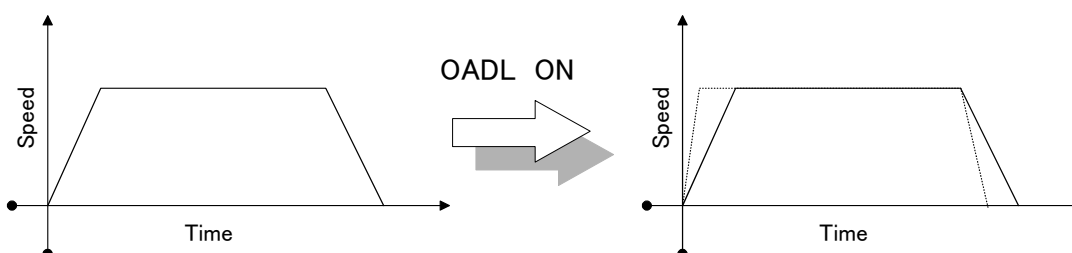


Fig.4-19 : Acceleration/deceleration pattern at light load

[Related instructions]

[ACCEL](#), [LOADSET](#), [HOPEN](#) / [HCLOSE](#)

[Related parameter]

HNDDAT 1 to 8, WRKDAT 1 to 8, HNDHOLD 1 to 8

ON COM GOSUB (ON Communication Go Subroutine)**[Function]**

Defines the starting line of a branching subroutine when an interrupt is generated from a designated communication line.

[Format]

ON ☐ COM ☐ [(<File No.>)] ☐ GOSUB ☐ <Call Destination>
--

[Terminology]

<File No.> Describe a number between 1 and 3 assigned to the communication line.
 <Call Destination> Describe the line No. and label name.

[Reference Program]

If an interrupt is generated from the file No. 1 communication line (COM1:), carry out the label RECV process.

```

10 OPEN "COM1:" AS #1      ' Communication line opening.
20 ON COM(1) GOSUB *RECV   ' The definition of interruption.
30 COM(1) ON               ' Enable interrupt from file No. 1 communication line.
40                          '
100                        ' <<If the communicative interrupt occurs here, it will branch to label *RECV.>>
110 '
120 MOV P1
130 COM(1) STOP            ' Suspend the interrupt during movement only from P1 to P2.
140 MOV P2
150 COM(1) ON              ' If there are some communications during movement from P1 to P2, the
                          ' interrupt occurs here.
160                        '
170                        ' <<If the communicative interrupt occurs here, it will branch to label *RECV.>>
260                        '
270 COM(1) OFF             ' Disable interrupt from file No. 1 communication line.
280 CLOSE #1
290 END
:
:
3000 *RECV                 ' Communication interruption processing.
3010 INPUT #1, M0001        ' Set the received information as M0001 and P0001.
3020 INPUT #1, P0001
:
3100 RETURN 1              ' Returns control to the next line of interrupted line.
```

[Explanation]

- (1) If the file No. is omitted, 1 will be used as the file No.
- (2) The file Nos. with the smallest No. have the order of priority for the interrupt.
- (3) If a communication interrupt is generated while the robot is moving, the robot will stop.
It is possible to use COM STOP to stop the interrupt, and prevent the robot from stopping.
- (4) Interrupts are prohibited in the initial state. To enable interrupts, execute the COM ON instruction after this instruction.
- (5) Make sure to return from a subroutine using the RETURN instruction. An error occurs if the GOTO instruction is used to return, because the free memory available for control structure (stack memory) decreases and eventually becomes insufficient.

[Related instructions]

COM ON/COM OFF/COM STOP, RETURN, OPEN, INPUT, PRINT, CLOSE

ON... GOSUB (ON Go Subroutine)

[Function]

Calls up the subroutine at the line No. or label corresponding to the value.

[Format]

ON □ <Terminology> □ GOSUB □ [<Expression>] [, [<Call Destination>]] ...
--

[Terminology]

<Terminology> Designate the line No. or label on the line to branch to with a numeric operation expression.

<Call Destination> Describe the line No. or the label No. The maximum number is 32.

[Reference Program]

Sets the value equivalent to three bits of input signal 16 in M1, and branches according to the value of M1 (1 through 7).

(Calls line 1000 if M1 is 1, label LSUB if M1 is 2, line 2000 if M1 is 3, 4 or 5, and label L67 if M1 is 6 or 7.)

```
10 M1 = M_INB(16) AND &H7
```

```
20 ON M1 GOSUB 1000,*LSUB,2000,2000,2000,*L67,*L67
```

```
1000 ' Describes processing when M1=1.
```

```
1010 '
```

```
1200 RETURN                                ' Be sure to return by using RETURN.
```

```
1210 *LSUB
```

```
1220                                        ' Describes processing when M1=2.
```

```
1300 RETURN                                ' Be sure to return by using RETURN.
```

```
1700 *L67
```

```
1710 ' Describes processing when M1=6 or M1=7.
```

```
1720 RETURN                                ' Be sure to return by using RETURN.
```

```
2000 ' Describes processing when M1=3, M1=4, or M1=5.
```

```
2010 '
```

```
2020 RETURN                                ' Be sure to return by using RETURN.
```

[Explanation]

- (1) The value of <Expression> determines which line No. or label subroutine to call.
For example, if the value of <Expression> is 2, the line No. or label described for the second value is called.
- (2) If the value of <expression> is larger than the number of <destinations called up>, the program control jumps to the next line. For example, the program control jumps to the next line if the value of <expression> is 5 and there are only three <destinations called up>.
- (3) When a line No. or label that is called up does not exist, or when there are two definitions, an error will occur.
- (4) Make sure to return from a subroutine using the RETURN instruction. An error occurs if the GOTO instruction is used to return, because the free memory available for control structure (stack memory) decreases and eventually becomes insufficient.

Value of <Expression>	Process <Control>
Real number	Value is converted to an integer by rounding it off, and then branching is executed.
When 0, or when the value exceeds the number of line Nos. or labels	Control proceeds to the next line
Negative number or 32767 is exceeded	Execution error

ON...GOTO (On Go To)**[Function]**

Branches to the line with the line No. or label that corresponds to the designated value.

[Format]

ON □ <Expression> □ GOTO □ [<Branch Destination>] [, [<Branch Destination>]] ...

[Terminology]

<Expression> Designate the line No. or label on the line to branch to with a numeric operation expression.

<Call Destination> Describe the line No. or the label No. The maximum number is 32.

[Reference Program]

Branches based on the value (1-7) of the numerical variable M1.

(Branches to line 1000 if M1 is 1, to label LJMP if M1 is 2, to line 2000 if M1 is 3, 4 or 5, and to label L67 if M1 is 6 or 7.)

```
100 ON M1 GOTO 1000,*LJMP,2000,2000,2000,*L67,*L67
```

```
110 ' Control is passed to this line when M1 is other than 1 through 7 (i.e., 0, or 8 or larger).
```

```
1000 ' Describes processing when M1=1.
```

```
1010 ' :
```

```
1110 *LJMP          ' When M1=2.
```

```
1120 ' Describes processing when M1=2.
```

```
1130 ' :
```

```
1700 *L67
```

```
1710 ' Describes processing when M1=6 or M1=7.
```

```
1720 ' :
```

```
2000 ' Describes processing when M1=3, M1=4, or M1=5.
```

```
2010 ' :
```

[Explanation]

(1) This is the GOTO version of ON GOSUB.

(2) If the value of <expression> is larger than the number of <destinations called up>, the program control jumps to the next line. For example, the program control jumps to the next line if the value of <expression> is 5 and there are only three <destinations called up>.

(3) When a line No. or label that is called up does not exist, or when there are two definitions, an error will occur.

Value of <Expression>	Process <Control>
Real number	Value is converted to an integer by rounding it off, and then branching is executed.
When 0, or when the value exceeds the number of line Nos. or labels	Control proceeds to the next line
Negative number or 32767 is exceeded	Execution error

OPEN (Open)

[Function]

Open the file or communication lines.

[Format]

OPEN □ "<File Descriptor>" □ [FOR <Mode>] □
 AS □ [#] <File No.>

[Terminology]

- <File Descriptor> Describe a file name (including communication lines).
- To use a communication line, set "<Communication Line File Name>:"
 - When not using a communications line, set "<File Name>"

File type	File name	Access method
File	Describe with 16 characters or less.	INPUT,PRINT,APPEND
Communi- cation line	COM1: Standard RS-232C(default value) COM2:The setting in the "COMDEV" parameter. . COM8:The setting in the "COMDEV" parameter.	Omitted = random mode only

- <Mode> Designate the method to access a file.
- Omitted = random mode. This can be omitted when using a communication line.
 - INPUT = input mode. Inputs from an existing file.
 - OUTPUT = output mode (new file). Creates a new file and outputs it there.
 - APPEND=Outputmode(existingfile). Appends output to the end of an existing file.

- <File No.> Specify a constant from 1 to 8.
 To interrupt from communication line: 1 to 3.

[Reference Program]

(1) Communication line.

```

10 OPEN "COM1:" AS #1      ' Open standard RS-232C line as file No. 1.20 MOV P_01
20 MOV P_01
30 PRINT #1,P_CURR        ' Output current position to external source.
                           "(100.00,200.00,300.00,400.00)(7.0)" format
40 INPUT #1,M1,M2,M3      ' Receive from external source with "101.00,202.00,303.00" ASCII format.
50 P_01.X=M1
60 P_01.Y=M2
70 P_01.C=RAD(M3)        ' Copy to global data.
80 CLOSE                  ' Close all opened files.
90 END

```

(2) File operation. (Create the file "temp.txt" to the controller and write "abc")

```

10 OPEN "temp.txt" FOR APPEND AS #1
20 PRINT #1, "abc"
30 CLOSE #1

```

[Explanation]

- (1) Opens the file specified in <File name> using the file number.
 Use this file No. when reading from or writing to the file.
- (2) A communication line is handled as a file.

[Related instructions]

CLOSE, PRINT, INPUT

[Related parameter]

COMDEV

OVRD (Override)

[Function]

This instruction specifies the speed of the robot movement as a value in the range from 1 to 100%. This is the override applied to the entire program.

[Format]

OVRD □ <Override>

This function is available for controller software version G2 or later.

OVRD □ <Override> [, <Override when moving upward>] [, <Override when moving downward>]

[Terminology]

- <Override> Designate the override with a real number. The default value is 100.
 Unit: [%] (Recommended range: 0.1 to 100.0)
 A numeric operation expression can also be described. If 0 or a value over 100 is set, an error will occur.
- <Override when moving upward/downward>
 Sets the override value when moving upward/downward by the arch motion instruction (MVA).

[Reference Program]

```

10 OVRD 50
20 MOV P1
30 MVS P2
40 OVRD M_NOVRD      ' Set default value.
50 MOV P1
60 OVRD 30,10,10      ' Sets the override when moving upward/downward by the arch motion instruction
                        to 10.
70 MVA P3,3

```

[Explanation]

- (1) The OVRD command is valid regardless of the interpolation type.
- (2) The actual override is as follows:
 - During joint interpolation: Operation panel (T/B) override setting value) × (Program override (OVRD command)) × (Joint override (JOVRD command)).
 - During linear interpolation: Operation panel (T/B) override setting value) × (Program override (OVRD command)) × (Linear designated speed (SPD command)).
- (3) The OVRD command changes only the program override. 100% is the maximum capacity of the robot. Normally, the system default value (M_NOVRD) is set to 100%. The designated override is the system default value until the OVRD command is executed in the program.
- (4) Once the OVRD command has been executed, the designated override is applied until the next OVRD command is executed, the program END is executed or until the program is reset. The value will return to the default value when the END statement is executed or the program is reset.

[Related instructions]

[JOVRD](#) (For joint interpolation), [SPD](#) (For linear/circular interpolation)

[Related system variables]

[M_JOVRD](#)/[M_NJOVRD](#)/[M_OPOVRD](#)/[M_OVRD](#)/[M_NOVRD](#)
 (M_NOVRD (System default value), M_OVRD (Current designated speed))

PLT (Pallet)

[Function]

Calculates the position of grid in the pallet.

[Format]

PLT □ <Pallet No.> , <Grid No.>

[Terminology]

<Pallet No.>	Select a pallet No. between 1 and 8 that has already been defined with a DEF PLT command. Specify this argument using a constant or a variable.
<Grid No.>	The position number to calculate in the palette. Specify this argument using a constant or a variable.

[Reference Program]

```
100 DEF PLT 1,P1,P2,P3,P4,4,3,1 ' The definition of the four-point pallet. (P1,P2,P3,P4)
110                               '
120 M1=1                         ' Initialize the counter M1.
130 *LOOP
140  MOV PICK, 50                 ' Moves 50 mm above the work unload position.
150  OVRD 50
160  MVS PICK
170  HCLOSE 1                     ' Close the hand.
180  DLY 0.5                      ' Wait for the hand to close securely (0.5 sec.)
190  OVRD 100
200  MVS,50                       ' Moves 50 mm above the current position.
210  PLACE = PLT 1, M1            ' Calculates the M1th position
220  MOV PLACE, 50                ' Moves 50 mm above the pallet top mount position.
230  OVRD 50
240  MVS PLACE
250  HOPEN 1                      ' Open the hand.
260  DLY 0.5
270  OVRD 100
280  MVS,50                       ' Moves 50 mm above the current position.
290  M1=M1+1                     ' Add the counter.
300  IF M1 <=12 THEN *LOOP        ' If the counter is within the limits, repeats from *LOOP.
310  MOV PICK,50
320 END
```

[Explanation]

- (1) The position of grid of a pallet defined by the DEF PLT statement is operated.
- (2) The pallet Nos. are from 1 to 8, and up to 8 can be defined at once.
- (3) Note that the position of the grid may vary because of the designated direction in the pallet definition.
- (4) If a grid No. is designated that exceeds the largest grid No. defined in the pallet definition statement, an error will occur during execution.
- (5) When using the pallet grid point as the target position of the movement command, an error will occur if the point is not enclosed in parentheses as shown above. Refer to [Page 69, "4.1.2 Pallet operation"](#) for detail.

[Related instructions]

[DEF PLT](#)

PREC (Precision)

[Function]

This instruction is used to improve the motion path tracking. It switches between enabling and disabling the high accuracy mode. This is only available for certain types of robots (RV-1A/2AJ, RV-4A/5AJ/3AL/4AJL, and RV-20A).

[Format]

This function is available for controller software version D1 or later.

```
PREC ☐ <ON / OFF>
```

[Terminology]

<ON / OFF>

ON : When enabling the high accuracy mode.

OFF : When disabling the high accuracy mode.

[Reference Program]

10 PREC ON ' Enables the high accuracy mode.

20 MVS P1

30 MVS P2

40 PREC OFF ' Disables the high accuracy mode.

50 MOV P1

[Explanation]

- (1) The high accuracy mode is enabled using the PREC ON instruction if it is desired to perform interpolation movement with increased path accuracy.
- (2) When this instruction is used, the path accuracy is improved but the program execution time (tact time) may become longer because the acceleration/deceleration times are changed internally.
- (3) The enabling/disabling of the high accuracy mode is activated from the first interpolation instruction after the execution of this instruction.
- (4) The high accuracy mode is disabled if the PREC OFF or END instruction is executed, or a program reset operation is performed.
- (5) The high accuracy mode is disabled immediately after turning the power on.
- (6) The high accuracy mode is always disabled in jog movement.

PRINT (Print)

[Function]

Outputs data into a file (including communication lines). All data uses the ASCII format.

[Format]

PRINT □ # <File No.> □ [, [<Expression> ;] ... [<Expression> [;]]

[Terminology]

<File No.>	Described with numbers 1 to 8.
	Corresponds to the control No. assigned by the OPEN command.
<Expression>	Describes numeric operation expressions, position operation expressions and character string expressions.

[Reference Program]

10 OPEN "COM1" AS #1	' Open standard RS-232-C line as file No. 1.20 MOV P_01.
20 MDATA=150	' Substitute 150 for the numeric variable MDATA.
30 PRINT #1,"***PRINT TEST***"	' Outputs the character string "***PRINT TEST***."
40 PRINT #1	' Issue a carriage return
50 PRINT #1,"MDATA=",MDATA	' Output the character string "MDATA" followed by the value of MDATA, (150).
60 PRINT #1	' Issue a carriage return.
40 PRINT #1,"*****"	' Outputs the character string "*****."
50 END	' End the program.

The output result is shown below.

PRINT TEST

MDATA=150

[Explanation]

- (1) If <Expression> is not described, then a carriage return will be output.
- (2) Output format of data (reference)

The output space for the value for <Expression> and for the character string is in units of 14 characters. When outputting multiple values, use a comma between each <Expression> as a delimiter.

If a semicolon (;) is used at the head of each space unit, it will output after the item that was last displayed.

The carriage return code will always be returned after the output data.
- (3) The error occurs when OPEN command is not executed.

Example)

「10 M1=123.5
 20 P1=(130.5,-117.2,55.1,16.2,0.0,0.0)(1,0)」

1) 「30 PRINT# 1,"OUTPUT TEST",M1,P1」 is described,
 OUTPUT TEST 123.5 (130.5,-117.2,55.1,16.2,0.0,0.0)(1,0) is output.

2) 「30 PRINT# 1,"OUTPUT TEST";M1;P1」 is described,
 OUTPUT TEST 123.5(130.5,-117.2,55.1,16.2,0.0,0.0)(1,0) is output.

If a comma or semicolon is inserted after a <Expression>, the carriage return will not be issued, and instead, printing will continue on the same line.

3) 「30 PRINT# 1,"OUTPUT TEST",
 40 PRINT# 1,M1;
 50 PRINT# 1,P1」 is described,
 OUTPUT TEST 123.5(130.5,-117.2,55.1,16.2,0.0,0.0)(1,0) is output.

[Related instructions]

OPEN, CLOSE, INPUT

PRIORITY (Priority)

[Function]

In multitask program operation, multiple program lines are executed in sequence (one by one line according to the default setting). This instruction specifies the priority (number of lines executed in priority) when programs are executed in multitask operation.

[Format]

This function is available for controller software version C2 or later.

PRIORITY □ <Number of executed lines> [, <Slot number>]

[Terminology]

<Number of executed lines>	Specify the number of lines executed at once . Use a numerical value from 1 to 31.
<Slot number>	1 to 32. If this argument is omitted, the current slot number is set.

[Reference Program]

Slot 1

10 PRIORITY 3 ' Sets the number of executed lines for the current slot to 3.

Slot 2

10 PRIORITY 4 ' Sets the number of executed lines for this slot to 4.

[Explanation]

- (1) Programs of other slots are not executed until the specified number of lines is executed. For example, as in the statement example above, if PRIORITY 3 is set for slot 1's program and PRIORITY 4 is set for slot 2's program, three lines of the slot 1 program are executed first, then four lines of the slot 2 program are executed. Afterward, this cycle is repeated.
- (2) The default value is 1 for all the slots. In other words, the execution moves to the next slot every time one line has been executed.
- (3) An error occurs if there is no program corresponding to the specified task slot.
- (4) It is possible to change the priority even while the program of the specified task slot is being executed.

RELM (Release Mechanism)

[Function]

This instruction is used in connection with control of a mechanism via task slots during multitask operation.
It is used to release the mechanism obtained by the GETM instruction.

[Format]

RELM

[Reference Program]

(1) Start the task slot 2 from the task slot 1, and control the mechanism 1 in the task slot 2.

Task slot 1

10 RELM	' Releases the mechanism in order to control mechanism 1 using slot 2.
20 XRUN 2,"10"	' Start the program 10 in slot 2.
30 WAIT M_RUN(2)=1	' Wait for the starting confirmation of the slot 2.
:	

Task slot 2. (Program "10")

10 GETM 1	' Get the control of mechanism 1.
20 SERVO ON	' Turn on the servo of mechanism 1.
30 MOV P1	
40 MVS P2	
50 SERVO OFF	' Turn off the servo of mechanism 1.
60 RELM	' Releases the control right of mechanism 1.
70 END	

[Explanation]

- (1) Releases the currently acquired mechanism resource.
- (2) If an interrupt is applied while the mechanism is acquired and the program execution is stopped, the acquired mechanism resource will be automatically released.
- (3) This instruction cannot be used in a constantly executed program.

[Related instructions]

[GETM](#)

REM (Remarks)

[Function]

Uses the following character strings as comments.

[Format]

`REM □ [<Comment>]`

[Terminology]

<Comment>	Describe a user-selected character string. Descriptions can be made in the range of position lines.
-----------	--

[Reference Program]

```
10 REM ***MAIN PROGRAM***  
20 ' ***MAIN PROGRAM***  
30 MOV P1                ' Move to P1.
```

[Explanation]

- (1) REM can be abbreviated to be a single quotation mark (').
- (2) It can be described after the instruction like an 30 line in reference program.

RESET ERR (Reset Error)

[Function]

This instruction resets an error generated in the robot controller. It is not allowed to use this instruction in the initial status. If an error other than warnings occurs, normal programs other than constantly executed programs cannot be operated. This instruction is effective if used in constantly executed programs.

[Format]

This function is available for controller software version B1 or later.

RESET ERR

[Reference Program]

Example of execution in a constantly executed program

10 IF M_ERR=1 THEN RESET ERR 'Resets an error when an error occurs in the controller.

[Explanation]

- (1) This instruction is used in a program whose start condition is set to constant execution (ALWAYS) by the "SLT*" parameter when it is desired to reset system errors of the robot.
- (2) It becomes enabled when the controller's power is turned on again after changing the value of the "ALWENA" parameter from 0 to 7.

[Related parameter]

ALWENA

[Related system variables]

[M_ERR/M_ERRLVL/M_ERRNO](#)

RETURN (Return)

[Function]

- (1) When returning from a normal subroutine returns to the next line after the GOSUB.
- (2) When returning from an interrupt processing subroutine, returns either to the line where the interrupt was generated, or to the next line.

[Format]

- (1) When returning from a normal subroutine:

```
RETURN
```

- (2) When returning from an interrupt processing subroutine:

```
RETURN <Return Designation No.>
```

[Terminology]

- <Return Designation No.> Designate the line number where control will return to after an interrupt has been generated and processed.
- 0 ... Return control to the line where the interrupt was generated.
- 1 ... Return control to the next line after the line where the interrupt was issued.
- If the argument is omitted.

[Reference Program]

(1) The example of RETURN from the usual subroutine.

```
10 ' ***MAIN PROGRAM***
20 GOSUB *SUB_INIT           ' Subroutine jumps to label SUB_INIT.
30 MOV P1
:
1000 ' ***SUB INIT***        ' Subroutine
1010 *SUB_INIT
1020  PSTART=P1
1030  M100=123
1040 RETURN                  ' Returns to the line immediately following the line where the subrou-
                              ' tine was called from.
```

(2) The example of RETURN from the subroutine for interruption processing.

Calls the subroutine on line 100 when the input signal of general-purpose input signal number 17 is turned on.

```
10 DEF ACT 1,M_IN(17)=1 GOSUB 100 ' Definition of interrupt of ACT 1.
20 ACT 1=1                         ' Enable the ACT 1.
:
100                                ' The subroutine for interrupt of ACT 1.
110  ACT 1=0                       ' Disable the interrupt.
120  M_TIMER(1)=0                  ' Set the timer to zero.
130  MOV P2                        ' Move to P2.
140  WAIT M_IN(17)=0               ' Wait until the input signal 17 turns off.
150  ACT 1=1                       ' Set up interrupt again.
160 RETURN 0                      ' Returns control to the interrupted line.
```

[Explanation]

- (1) Writes the RETURN instruction at the end of the jump destination processing called up by the GOSUB instruction.
- (2) An error occurs if the RETURN instruction is executed without being called by the GOSUB instruction.
- (3) Always use the RETURN instruction to return from a subroutine when called by the GOSUB instruction. An error occurs if the GOTO instruction is used to return, because the free memory available for control structure (stack memory) decreases and eventually becomes insufficient.
- (4) When there is a RETURN command in a normal subroutine with a return-to designation number, and when there is a RETURN command in an interrupt-processing subroutine with no return-to destination number, an error will occur.
- (5) Please refer to [Page 132, "DEF ACT"](#) for the interrupt processing.

[Related instructions]

GOSUB, ON... GOSUB, ON COM GOSUB, DEF ACT

SELECT CASE (Select Case)

[Function]

Executes one of multiple statement blocks according to the condition expression value.

[Format]

```

SELECT □ <Condition>
  CASE □ <Expression>
    [<Process>]
    BREAK
  CASE □ <Expression>
    [<Process>]
    BREAK
    :
  DEFAULT
    [<Process>]
    BREAK
END □ SELECT

```

[Terminology]

<Condition>	Describe a numeric operation expression.
<Expression>	Describe a numeric operation expression. The type must be the same as the condition expression.
<Process>	Writes any instruction (other than the GOTO instruction) provided by MELFA_BASIC IV.

[Reference Program]

```

10 SELECT MCNT
20  M1=10           ' This line is not executed
30 CASE IS <= 10    ' MCNT  < = 10
40  MOV P1
50  BREAK
60 CASE 11          'MCNT = 11
70  MOV P2
80  BREAK
90 CASE 13 TO 18    '13  < =  MCNT  < = 18
100 MOV P4
110 BREAK
120 DEFAULT         ' Other than the above.
130 M_OUT(10)=1
140 BREAK
150 END SELECT

```

[Explanation]

- (1) If the condition matches one of the CASE items, the process will be executed until the next CASE, DEFAULT or ENDSELECT. If the case does not match with any of the CASE items but DEFAULT is described, that block will be executed.
- (2) If there is no DEFAULT, the program will jump to the line after ENDSELECT without processing.
- (3) The SELECT CASE and END SELECT statements must always correspond. If a GOTO instruction forces the program to jump out from a CASE block of the SELECT CASE statement, the free memory available for control structure (stack memory) decreases. Thus, if a program is executed continuously, an error will eventually occur.
- (4) If an END SELECT statement that does not correspond to SELECT CASE is executed, an execution error will occur.
- (5) In the case of controller software version G1 or later, it is possible to write a SELECT CASE block within another SELECT CASE block (up to eight nesting levels are allowed).
- (6) It is possible to write WHILE-WEND and FOR-NEXT within a CASE block.
- (7) Use "CASE IS", when using the comparison operators (<, =, >, etc.) for the "<Expression>".

SERVO (Servo)

[Function]

Controls the ON and OFF of the servo motor power.

[Format]

(1) The usual program

```
SERVO □ <ON / OFF>
```

(2) The program of always (ALWAYS) execution.

```
SERVO □ <ON / OFF> [, <Mechanism No.>]
```

[Terminology]

<ON / OFF>

ON : When turning the servo motor power on.

OFF : When turning the servo motor power off.

<Mechanism No.>

This is valid only within the program of always execution.

The range of the value is 1 to 3, and describe by constant or variable.

[Reference Program]

10 SERVO ON	' Servo ON.
20 IF M_SVO<>1 GOTO 20	' Wait for servo ON.
30 SPD M_NSPD	
40 MOV P1	
50 SERVO OFF	

[Explanation]

(1) The robot arm controls the servo power for all axes.

(2) If additional axes are attached, the servo power supply for the additional axes is also affected.

(3) If used in a program that is executed constantly, this instruction is enabled by changing the value of the "ALWENA" parameter from 0 to 7 and then turning the controller's power on again.

[Related system variables]

M_SVO (1 : ON, 0 : OFF)

[Related parameter]

ALWENA

SKIP (Skip)

[Function]

Transfers control of the program to the next line.

[Format]

SKIP

[Reference Program]

10 MOV P1 WTHIF M_IN(17)=1,SKIP

' If the input signal (M_IN(1 7)) turns ON while moving with joint interpolation to the position indicated with position variable P1, stop the robot interpolation motion, and stop execution of this command, and execute the next line.

20 IF M_SKIPCQ=1 THEN HLT

' Pauses the program if the execution is skipped.

[Explanation]

(1) This command is described with the WHT or WTHIF statements. In this case, the execution of that line is interrupted, and control is automatically transferred to the next line. Execution of skip can be seen with the M_SKIPCQ information.

[Related system variables]

M_SKIPCQ (1: Skipped, 0: Not skipped)

SPD (Speed)

[Function]

Designates the speed for the robot's linear and circular movements. This instruction also specifies the optimum speed control mode.

[Format]

SPD □ <Designated Speed>

SPD □ M_NSPD (Optimum speed control mode)

[Terminology]

<Designated Speed> Designate the speed as a real number. Unit: [mm/s]

[Reference Program]

```

10 SPD 100
20 MVS P1
30 SPD M_NSPD      ' Set the default value.
```

[Explanation]

- (1) The SPD command is valid only for the robot's linear and circular movements.
- (2) The actual designated override is (Operation panel (T/B) override setting value) × (Program override (OVRD command)) × (Linear designated speed (SPD command)).
- (3) The SPD command changes only the linear/circular designated speed.
- (4) When M_NSPD (default value) is designated for the designated speed, the robot will always move at the maximum possible speed, so the line speed will not be constant(optimum speed control). Note that an error may occur depending on the robot posture.
- (5) The system default value is applied for the designated speed until the SPD command is executed in the program. Once the SPD command is executed, that designated speed is held until the next SPD command.
- (6) The designated speed will return to the system default value when the program END statement is executed.

[Related system variables]

[M_SPD/M_NSPD/M_RSPD](#)

SPDOPT (Speed Optimize)

[Function]

Adjusts the speed so that the speed does not exceed during the linear interpolation operation in the horizontal direction which passes through near the OP ($X=Y=0$: one of the robot's singular points).

Note) This command is limited to the corresponding models such as the RH-1000G series. Refer to "[available robot type]".

[Format]

This function is available for controller software version H7 or later

SPDOPT ☐ <ON/OFF>

[Terminology]

<ON/OFF>

ON: Enable the speed-optimization function.

OFF: Disable the speed-optimization function.

[Reference Program]

```
10 MOV P1
20 SPDOPT ON           ' Enable the speed-optimization function.
30 MVS P2
40 MVS P3
50 SPDOPT OFF         ' Disable the speed-optimization function.
60 MVS P6
```

[Explanation]

- (1) When performing a XYZ interpolation operation while maintaining the speed of the control point, the J1 axis must rotate at a faster speed when passing through a point near the origin point O (one of the robot's singular points) as shown in Fig. 4-20, causing an excessive speed error depending on the specified speed. If SPDOPT ON is executed, the speed is adjusted automatically in order to prevent an excessive speed error from occurring. For example, while in operation at the command speed V, it approaches the origin point O, and the speed will be exceeded if it continues to operate at the current speed, the speed is decreased automatically as shown by A in Fig. 4-21 in order to prevent the speed to be exceeded. Then, when it has passed near the origin point O and it becomes possible to increase the speed, it starts accelerating to reach command speed V as shown by B in Fig. 4-21.

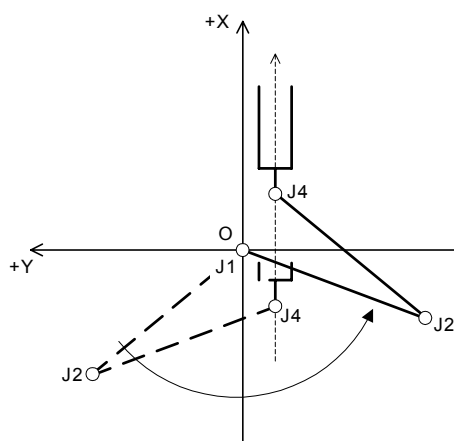


Fig.4-20 : When passing through near the origin point by linear interpolation

- (2) This instruction functions only for XYZ interpolation. It does not function for JOINT interpolation and CIRCULAR interpolation. Also, it does not function for linear interpolation by which the J4 axis does not pass through the speed adjustment area and the singular point area as shown in Page 189 "Fig. 4-22 Speed-optimization area and singular point area."

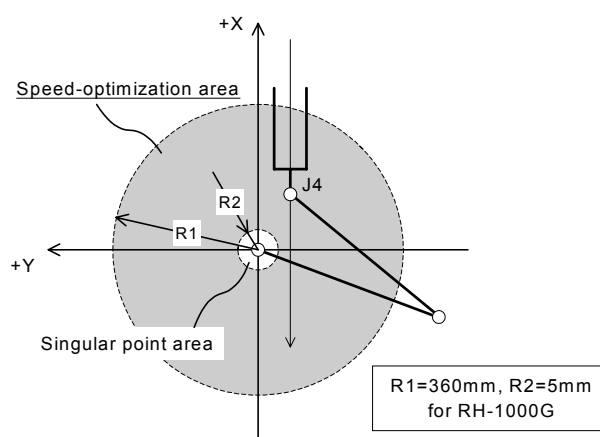
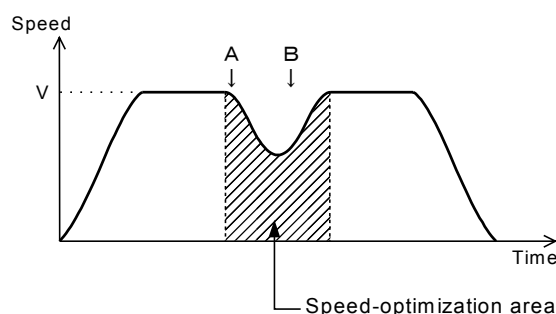


Fig.4-21 : The situation of the speed at speed-optimization. Fig.4-22 : Speed-optimization area and singular point area.

- (3) The initial state of the speed adjustment function immediately after the power is turned on can be changed by the SPDOPT parameter. This parameter also limits the applicable models including the RH-1000G series. Please check that it is listed in the "Command List" in the Additional Handbook/Standard Specifications before using it.
The initial value on an applicable model is SPDOPT=1 (speed adjustment enabled).
- (4) If the END instruction or a program reset operation is executed, the status of the speed adjustment function returns to the initial state immediately after the power is turned on.
- (5) When the speed adjustment function is enabled, error 2804 will be generated if the XYZ interpolation by which the J4 axis passes through a singular point area shown in Fig. 4-22 is executed, and the operation is then suspended.
- (6) Even if this instruction is described in a program, it is ignored on models other than the applicable models.
- (7) Even if the speed adjustment function is enabled, an exceeded speed error may be generated if a path is connected by enabling the CNT instruction near the origin point, or a XYZ interpolation operation that drastically changes the posture is executed. In such a case, move the position where a path is connected away from the origin point, or adjust the speed by using the OVRD instruction.
- (8) In the case of a XYZ interpolation that operates slightly in the horizontal direction but operates significantly in the vertical direction, the operation speed may degrade drastically when the speed adjustment function is enabled vs. when it is disabled. In such a case, disable the speed adjustment function, or operate by using a JOINT interpolation (MOV instruction).

[The available robot type]

RH-1000G series

[Related parameter]

SPDOPT

TOOL (Tool)

[Function]

Designates the tool conversion data. This instruction specifies the length, position of the control point from the mechanical interface, and posture of the tools (hands).

[Format]

TOOL □ <Tool Conversion Data>

[Terminology]

<Tool Conversion Data> Specifies the tool conversion data using the position operation expression.

[Reference Program]

(1) Set up the direct numerical value.

10 TOOL (100,0,100,0,0,0)	' Changes the control position to an X-axis coordinate value of 100 mm and a Z-axis coordinate value of 100 mm in the tool coordinate system.
---------------------------	---

20 MVS P1

30 TOOL P_NTOOL	' Returns the control position to the initial value. (mechanical interface position, flange plane.)
-----------------	---

(2) Set up the variable data in the XYZ coordinates system.

(If (100,0,100,0,0,0,0) are set in PTL01, it will have the same meaning as (1).)

10 TOOL PTL01

20 MVS P1

30 TOOL P_NTOOL

[Explanation]

- (1) The TOOL instruction is used to specify the control points at the tip of each hand in a system using double hands. If both hands are of the same type, the control point should be set by the "MEXTL" parameter instead of by the TOOL instruction.
- (2) The system default value (P_NTOOL) is applied until the TOOL command is executed.
Once the TOOL command is executed, the designated tool conversion data is applied until the next TOOL command is executed. This is operated with 6-axis three-dimension regardless of the mechanism structure.
- (3) The tool conversion data changed with the TOOL command is saved in parameter MEXTL, and is saved even after the controller power is turned OFF.
- (4) If different tool conversion data are used at teaching and automatic operation, the robot may move to an unexpected position. Make sure that the settings at operation and teaching match.
The valid axis element of tool conversion data is different depending on the type of robot.
Set up the appropriate data referring to the [Page 277, "Table 5-2: Valid axis elements of the tool conversion data depending on the robot model"](#).

[Related parameter]

MEXTL Refer to [Page 275, "5.1 Standard Tool Coordinates"](#) for detail.

[Related system variables]

[P_TOOL/P_NTOOL](#)

TORQ (Torque)**[Function]**

Designates the torque limit for each axis.

[Format]

TORQ □ <Axis No.>, <Torque Limitation Rate>

[Terminology]

<Axis No.> Designate the axis No. with a numeric constant. (1 to 6)

<Torque Limitation Rate> Designate the limit of the force generated from the axis as a percentage. (1 to 100)

[Reference Program]

10 TORQ 4, 8

20 MVS P1

[Explanation]

- (1) The torque limit value of the designated axis is limited. The movement is limited so that a torque exceeding the designation is not applied. Designate a percentage in respect to the standard torque limit.
- (2) This instruction is valid only for standard robot axes. It cannot be used for general-purpose servo axes (additional axes and user-defined mechanisms). Change the parameters on the general-purpose servo side to obtain similar movement.
- (3) The available rate of torque limitation is changed by robot type.

WAIT (Wait)

[Function]

Waits for the variable to reach the designated value.

[Format]

WAIT □ <Numeric variable> = <Numeric constant>
--

[Terminology]

<Numeric variable>	Designate a numeric variable. Use the input/output signal variable (in such cases as M_IN, M_OUT) well.
<Numeric constant>	Designate a numeric constant.

[Reference Program]

(1) Signal state

10 WAIT M_IN(1)=1 ' The same meaning as "10 IF M_IN(1)=0 THEN GOTO 10".
20 WAIT M_IN(3)=0

(2) Task slot state

30 WAIT M_RUN(2)=1

(3) Variable state

40 WAIT M_01=100

[Explanation]

- (1) This command is used as the interlock during signal input wait and during multitask execution.
- (2) The WAIT instruction allows the program control to continue to the next line once the specified condition is met.
- (3) In case the WAIT instruction is executed in several tasks at one time in the multitask execution status, the processing time (tact time) may become longer and affect the system. In such cases, use the IF-THEN instruction instead of the WAIT instruction.

Example) 50 WAIT M_ABC=0 → 50 IF M_ABC<>0 THEN GOTO 50

WHILE-WEND (While End)

[Function]

The program between the WHILE statement and WEND statement is repeated until the loop conditions are satisfied.

[Format]

```
WHILE □ <Loop Condition>
:
WEND
```

[Terminology]

<Loop Condition> Describe a numeric operation expression. (Refer to the syntax diagram)

[Reference Program]

Repeat the process while the numeric variable M1 value is between -5 and +5, and transfer control to line after WEND statement if range is exceeded.

10 WHILE (M1>=-5) AND (M1<=5)	' Repeat the process while the value of numeric variable M1 is between -5 and +5.
20 M1=-(M1+1)	' Add 1 to M1, and reverse the sign.
30 PRINT# 1, M1	' Output the M1 value.
40 WEND	' Return to the WHILE statement (line 10)
50 END	' End the program.

[Explanation]

- (1) The program between the WHILE statement and WEND statement is repeated.
- (2) If the result of <Expression> is true (not 0), the control moves to the line following the WHILE statement and the process is repeated.
- (3) If the result of <Expression> is false (is 0), then the control moves to the line following the WEND statement.
- (4) If a GOTO instruction forces the program to jump out from between a WHILE statement and a WEND statement, the free memory available for control structure (stack memory) decreases. Thus, if a program is executed continuously, an error will eventually occur. Write a program in such a way that the loop exits when the condition of the WHILE statement is met.

WTH (With)

[Function]

A process is added to the interpolation motion.

[Format]

例) MOV P1 WTH □ <Process>

[Terminology]

<Process>

Describe the process to be added. The commands that can be described are as follow.

1. <Numeric type data B> <Substitution operator><Numeric type data A> [Substitute, signal modifier command (refer to syntax diagram)]

[Reference Program]

10 MOV P1 WTH M_OUT(17)=1 DLY M1+2 ' Simultaneously with the start of movement to P1, the output signal No. 17 will turn ON for the value indicated with the numeric variable M1 + two seconds.

[Explanation]

- (1) This command can only be used to describe the additional condition for the movement command.
- (2) An error will occur if the WTH command is used alone.
- (3) The process will be executed simultaneously with the start of movement.
- (4) The relationship between the interrupts regarding the priority order is shown below.

COM > ACT > WTHIF (WTH) > Pulse substitution

WTHIF (With If)

[Function]

A process is conditionally added to the interpolation motion command.

[Format]

WTHIF □ <Additional Condition>, <Process>

[Terminology]

<Additional Condition>

Describe the condition for adding the process. (Same as ACT condition expression)

<Process>

Describe the process to be added when the additional conditions are established.
(Same as WTH)

The commands that can be described as a process are as follow. (Refer to syntax diagram.)

1. <Numeric type data B> <Substitution operator><Numeric type data A>
Example) M_OUT (1) = 1, P1 = P2
2. HLT statement
3. SKIP statement

[Reference Program]

(1) If the input signal 17 turns on, the robot will stop.

10 MOV P1 WTHIF M_IN(17)=1, HLT

(2) If the current command speed exceeds 200 mm/s, turn on the output signal 17 for the M1+2 seconds.

20 MVS P2 WTHIF M_RSPD>200, M_OUT(17)=1 DLY M1+2

(3) If the rate of arrival exceeds 15% during movement to P3, turn on the output signal 1.

30 MVS P3 WTHIF M_MRATIO>15, M_OUT(1)=1

[Explanation]

- (1) This command can only be used to describe the additional conditions to the movement command.
- (2) Monitoring of the condition will start simultaneously with the start of movement.
- (3) It is not allowed to write the DLY instruction at the processing part.

XCLR (X Clear)

[Function]

This instruction cancels the program selection status of the specified task slot from within a program. It is used during multitask operation.

[Format]

XCLR □ <Slot No.>

[Terminology]

<Slot No.> Designate the slot number.

[Reference Program]

10 XRUN 2,"1"	' Executes the first program in task slot 2.
·	
100 XSTP 2	' Pauses the program of task slot 2.
110 WAIT M_WAI(2)=1	' Waits until the program of task slot 2 pauses.
150 XRST 2	' Cancels the pause status of the program of task slot 2.
·	
200 XCLR 2	' Cancels the program selection status of task slot 2.
210 END	

[Explanation]

- (1) An error occurs at execution if the specified slot does not select the program.
- (2) If the designated program is being operating, an error will occur at execution.
- (3) If the designated program is being pausing, an error will occur at execution.
- (4) If this instruction is used within a constantly executed program, it becomes enabled by changing the value of the "ALWENA" parameter from 0 to 7 and turning the controller's power off and on again.

[Related instructions]

[XLOAD](#), [XRST](#), [XRUN](#), [XSTP](#)

[Related parameter]

ALWENA

XLOAD (X Load)

[Function]

This instruction commands the specified program to be loaded into the specified task slot from within a program. It is used during multitask operation.

[Format]

XLOAD □ <Slot No.> <Program Name>

[Terminology]

<Slot No.>	Designate the slot number.
<Program Name>	Designate the program name.

[Reference Program]

10 IF M_PSA(2)=0 THEN 60	' Checks whether slot 2 is in the program selectable state.
20 XLOAD 2,"10"	' Select program 10 for slot 2.
30 DLY 0.5	' Waits for a while until the program is loaded.
40 XRUN 2	' Start slot 2.
50 WAIT M_RUN(2)=1	' Wait to confirm starting of slot 2.
60	'
70	' When the slot 2 is already operating, execute from here.

[Explanation]

- (1) An error occurs at execution if the specified program does not exist.
- (2) If the designated program is already selected for another slot, an error will occur at execution.
- (3) If the designated program is being edited, an error will occur at execution.
- (4) If the designated program is being executed, an error will occur at execution.
- (5) Designate the program name in double quotations.
- (6) If used in a program that is executed constantly, this instruction is enabled by changing the value of the "ALWENA" parameter from 0 to 7 and then turning the controller's power on again.

[Related instructions]

XCLR, XRST, XRUN, XSTP

[Related parameter]

ALWENA

XRST (X Reset)

[Function]

This instruction returns the program control to the first line if the program of the specified task slot is paused by a command within the program (program reset). It is used during multitask operation.

[Format]

XRST □ <Slot No.>

[Terminology]

<Slot No.> Designate the slot number.

[Reference Program]

10 XRUN 2	' Start.
20 WAIT M_RUN(2)=1	' Wait to confirm starting.
:	
100 XSTP 2	' Stop.
110 WAIT M_WAI(2)=1	' Wait for stop to complete.
:	
150 XRST 2	' Set program execution start line to head line.
160 WAIT M_PSA(2)=1	' Wait for program reset to complete.
:	
200 XRUN 2	' Restart.
210 WAIT M_RUN(2)=1	' Wait for restart to complete.

[Explanation]

- (1) This is valid only when the slot is in the stopped state.
- (2) If used in a program that is executed constantly, this instruction is enabled by changing the value of the "ALWENA" parameter from 0 to 7 and then turning the controller's power on again.

[Related instructions]

[XCLR](#), [XLOAD](#), [XRUN](#), [XSTP](#)

[Related parameter]

ALWENA

[Related system variables]

[M_PSA](#) (Slot number) (1: Program selection is possible, 0: Program selection is impossible)
[M_RUN](#) (Slot number) (1: Executing, 0: Not executing)
[M_WAI](#) (Slot number) (1: Stopping, 0: Not stopping)

XRUN (X Run)

[Function]

This instruction executes concurrently the specified programs from within a program. It is used during multitask operation.

[Format]

XRUN □ <Slot No.> [, <Program Name> [, <Operation Mode>]]
--

[Terminology]

<Slot No.> If the argument is omitted, the current operation mode is used.
 <Program Name> Designate the program name.
 <Operation Mode> 0 = Continuous operation,
 1 = Cycle stop operation. If the operation mode is omitted, the current operation mode will be used.
 Specify this argument using a constant or a variable.

[Reference Program]

(1) When the program of execution is specified by XRUN command (continuous executing).

```
10 XRUN 2,"1"      ' Start the program 1 with slot 2.
20 WAIT M_RUN(2)=1 ' Wait to have started.
```

(2) When the program of execution is specified by XRUN command (cycle operation)

```
10 XRUN 3,"2",1    ' Start the program 2 with slot 3 in the cycle operation mode
20 WAIT M_RUN(3)=1 ' Wait to have started.
```

(3) When the program of execution is specified by XLOAD command (continuous executing).

```
10 XLOAD 2,"1"      ' Select the program 1 as the slot 2.
20 DLY 0.5          ' Wait for the 0.5 seconds till the completion of the program load.
30 XRUN 2           ' Start the slot 2.
40 WAIT M_RUN(2)=1  ' Wait to have started.
```

(4) When the program of execution is specified by XLOAD command (cycle operation)

```
10 XLOAD 3,"2"      ' Select the program 2 as the slot 3.
20 DLY 0.5          ' Wait for the 0.5 seconds till the completion of the program load.
30 XRUN 3,,1        ' Start the program 1 with cycle operation.
40 WAIT M_RUN(3)=1  ' Wait to have started.
```

[Explanation]

- (1) An error occurs at execution if the specified program does not exist.
- (2) If the designated slot No. is already in use, an error will occur at execution.
- (3) If a program has not been loaded into a task slot, this instruction will load it. It is thus possible to operate the program without executing the XLOAD instruction.
- (4) If XRUN is executed in the "Pausing" state with the program stopped midway, continuous execution will start.
- (5) Designate the program name in double quotations.
- (6) If the operation mode is omitted, the current operation mode will be used.
- (7) If used in a program that is executed constantly, this instruction is enabled by changing the value of the "ALWENA" parameter from 0 to 7 and then turning the controller's power on again.

[Related instructions]

XCLR, XLOAD, XRST, XSTP

[Related parameter]

ALWENA

[Related system variables]

M_RUN (Slot number) (1: Executing, 0: Not executing)

XSTP (X Stop)

[Function]

This instruction pauses the execution of the program in the specified task slot from within a program. If the robot is being operated by the program in the specified task slot, the robot stops. It is used in multitask operation.

[Format]

XSTP □ <Slot No.>

[Terminology]

<Slot No.> Designate the slot No.

[Reference Program]

10	XRUN 2	' Execute.
:		
100	XSTP 2	' Stop.
110	WAIT M_WAI(2)=1	' Wait for stop to complete.
:		
200	XRUN 2	' Restart.

[Explanation]

- (1) If the program is already stopped, an error will not occur.
- (2) XSTP can also stop the constant execution attribute program.
- (3) If used in a program that is executed constantly, this instruction is enabled by changing the value of the "ALWENA" parameter from 0 to 7 and then turning the controller's power on again.

[Related instructions]

[XCLR](#), [XLOAD](#), [XRST](#), [XRUN](#)

[Related parameter]

ALWENA

[Related system variables]

[M_WAI](#) (Slot number) (1: Stopping, 0: Not stopping)

Substitute

[Function]

The results of an operation are substituted in a variable or array variable.

[Format]

$\langle \text{Variable Name} \rangle = \langle \text{Expression 1} \rangle$

For pulse substitution

$\langle \text{Variable Name} \rangle = \langle \text{Expression 1} \rangle \text{ DLY } \langle \text{Expression 2} \rangle$

[Terminology]

- | | |
|--|---|
| $\langle \text{Variable Name} \rangle$ | Designate the variable name of the value is to be substituted.
(Refer to the syntax diagram for the types of variables.) |
| $\langle \text{Expression 1} \rangle$ | Substitution value. Describe an arithmetic operation expression. |
| $\langle \text{Expression 2} \rangle$ | Pulse timer. Describe an arithmetic operation expression. |

[Reference Program]

(1) Substitution of the variable operation result.

10 P100=P1+P2*2

(2) Output of the signal.

20 M_OUT(10)=1 ' Turn on the output signal 10.

(3) Pulse output of the signal.

30 M_OUT(17)=1 DLY 2.0 ' Turn on the output signal 17 for 2 seconds.

[Explanation]

- (1) When using this additionally for the pulse output, the pulse will be executed in parallel with the execution of the commands on the following lines.
- (2) Be aware that if a pulse is output by M_OUTB or M_OUTW, the bits are reversed in 8-bit units or 16-bit units, respectively. It is not possible to reverse at any bit widths.
- (3) If the END command or program's last line is executed during the designated time, or if the program execution is stopped due to an emergency stop, etc., the output state will be held. But, the output reversed after the designated time.

(Label)

[Function]

This indicates the jump site.

[Format]

* <Label Name>

[Terminology]

<Label Name> Describe a character string that starts with an alphabetic character.
Up to 8 characters can be used. (Up to 9 characters including *.)

[Reference Program]

100 *SUB1

200 IF M1=1 THEN GOTO *SUB1

[Explanation]

- (1) An error will not occur even if this is not referred to during the program.
- (2) If the same label is defined several times in the same program, an error will occur at the execution.
- (3) The reserved words can't be used for the label.
- (4) If the underscore is used for the label name, the 1st character is "L." only. If the characters except "L" are used (ex. *A_LABEL), an error occurs.

Ex.) The correct example of the label with using the underscore. (The 1st character is "L")

*L_ABC, *L12_345, *LABEL_1

The mistake example of the label with using the underscore.

*H_ABC, *ABC_123, *NG_, *_LABEL

4.12 Detailed explanation of Robot Status Variable

4.12.1 How to Read Described items

[Function]	: This indicates a function of a variable.
[Format]	: This indicates how to enter arguments of an instruction. [] means that arguments may be omitted. System status variables can be used in conditional expressions, as well as in reference and assignment statements. In the format example, only reference and assignment statements are given to make the description simple.
[Reference Program]	: An example program using variables is shown.
[Terminology]	: This indicates the meaning and range of an argument.
[Explanation]	: This indicates detailed functions and precautions.
[Reference]	: This indicates related items.

4.12.2 Explanation of Each Robot Status Variable

Each variable is explained below in alphabetical order.

C DATE

[Function]

This variable returns the current date in the format of year/month/date.

[Format]

Example) <Character String Variable >=C_DATE
--

[Reference Program]

10 C1\$=C_DATE ' "2000/12/01" is assigned to C1\$.

[Explanation]

- (1) The current date is assigned.
- (2) This variable only reads the data. Use the T/B to set the date.

[Reference]

[C_TIME](#)

C MAKER

[Function]

This variable returns information on the manufacturer of the robot controller.

[Format]

Example) <Character String Variable >=C_MAKER

[Reference Program]

10 C1\$=C_MAKER ' "COPYRIGHT1999....." is assigned to C1\$.

[Explanation]

- (1) This variable returns information on the manufacturer of the robot controller.
- (2) This variable only reads the data.

[Reference]

[C_MECHA](#)

C MECHA

[Function]

This variable returns the name of the mechanism to be used.

[Format]

Example) <Character String Variable >=C_MECHA[(<Mechanism Number>)]

[Terminology]

<Character String Variable >	Specify a character string variable to be assigned.
<Mechanism Number>	Enter the mechanism number. 1 to 3, If the argument is omitted, 1 is set as the default value.

[Reference Program]

10 C1\$=C_MECHA(1) ' "RV-4A" is assigned to C1\$. (If the robot type name is RV-4A)

[Explanation]

- (1) This variable returns the name of the mechanism to be used.
- (2) This variable only reads the data.

C PRG

[Function]

This variable returns the selected program number (name).

[Format]

Example) <Character String Variable >=C_PRG [(<Numeric>)]

[Terminology]

<Character String Variable >	Specify a character string variable to be assigned.
<Numeric><Numeric>	1 to 32, Enter the task slot number. If the argument is omitted, 1 is set as the default value.

[Reference Program]

10 C1\$=C_PRG(1) ' "10" is assigned to C1\$. (if the program number is 10.)

[Explanation]

- (1) The program number (name) set (loaded) into the specified task slot is assigned.
- (2) If this variable is used in single task operation, the task slot number becomes 1.
- (3) If it is set in the operation panel, that number is set.
- (4) This variable only reads the data.
- (5) If a task slot for which a program is not loaded is specified, an error occurs at execution.

C TIME

[Function]

This variable returns the current time in the format of time:minute:second (24 hours notation).

[Format]

Example) <Character String Variable >=C_TME

[Reference Program]

```
10 C1$=C_TIME      ' "01:05:20" is assigned to C1$.
```

[Explanation]

- (1) The current clock is assigned.
- (2) This variable only reads the data.
- (3) Use the T/B to set the time.

[Reference]

[C_DATE](#)

C USER

[Function]

This variable returns the data registered in the "USERMSG" parameter.

[Format]

Example) <Character String Variable >=C_USER
--

[Reference Program]

```
10 C1$=C_USER      ' The characters registered in "USERMSG" are assigned to C1$.
```

[Explanation]

- (1) This variable returns the data registered in the "USERMSG" parameter.
- (2) This variable only reads the data.
- (3) Use the PC support software or the T/B to change the parameter setting.

J_CURR

[Function]

Returns the joint type data at the current position.

[Format]

Example) <Joint Type Variable>=J_CURR [(<Mechanism Number>)]

[Terminology]

<Joint Type Variable>	Specify a joint type variable to be assigned.
<Mechanism Number>	Enter the mechanism number. 1 to 3, If the argument is omitted, 1 is set as the default value.

[Reference Program]

10 J1=J_CURR	' J1 will contain the current joint position.
--------------	---

[Explanation]

- (1) The joint type variable for the current position of the robot specified by the mechanism number will be obtained.
- (2) This variable only reads the data.

[Reference]

[P_CURR](#)

J_ECURRE

[Function]

Returns the current encoder pulse value.

[Format]

Example) <Joint Type Variable>=J_ECURRE [(<Mechanism Number>)]

[Terminology]

<Joint Type Variable>	Specify a joint type variable to be assigned.(Joint type variable will be used even if this is a pulse value.)
<Mechanism Number>	Enter the mechanism number. 1 to 3, If the argument is omitted, 1 is set as the default value.

[Reference Program]

10 J1=J_ECURRE(1)	' J1 will contain the encoder pulse value for mechanism 1.
-------------------	--

[Explanation]

- (1) Although the return values are pulse values, use joint type values when entering data.
- (2) This variable only reads the data.

J FBC

[Function]

Returns the current position of the joint type that has been generated by encoder feedback.

[Format]

Example) <Joint Type Variable>=J_FBC [(<Mechanism Number>)]

[Terminology]

<Joint Type Variable>

Specify a joint type variable to be assigned.

<Mechanism Number>

Enter the mechanism number. 1 to 3, If the argument is omitted, 1 is set as the default value.

[Reference Program]

10 J1=J_FBC

' J1 will contain the current position of the joint that has been generated by servo feedback.

[Explanation]

- (1) Returns the current position of the joint type that has been generated by encoder feedback.
- (2) Delay between the command value to the servo and the actual servo activity can be checked.
- (3) It can be checked when there is a difference by the CMP JNT instruction.
- (4) This variable only reads the data.

[Reference]

[P_FBC](#)

J ORIGIN

[Function]

Returns the joint data when the origin has been set.

[Format]

Example) <Joint Type Variable>=J_ORIGIN [(<Mechanism Number>)]

[Terminology]

<Joint Type Variable>

Specify a joint type variable to be assigned.

<Mechanism Number>

Enter the mechanism number. 1 to 3, If the argument is omitted, 1 is set as the default value.

[Reference Program]

10 J1=J_ORIGIN(1)

' J1 will contain the origin setting position of mechanism 1.

[Explanation]

- (1) Returns the joint data when the origin has been set.
- (2) This can be used to check the origin, for instance, when the position of the robot shifted.
- (3) This variable only reads the data.

M_ACL/M_DACL/M_NACL/M_NDACL/M_ACLSTS**[Function]**

Returns information related to acceleration/deceleration time.

M_ACL : Returns the ratio of current acceleration time. (%)

M_DACL : Returns the ratio of current deceleration time. (%)

M_NACL : Returns the initial acceleration time value. (100%)

M_NDACL : Returns the initial deceleration time value. (100%)

M_ACLSTS : Returns the current acceleration/deceleration status.

(Current status: 0 = Stopped, 1 = Accelerating, 2 = Constant speed, 3 = Decelerating)

[Format]

Example) <Numeric Variable>=M_ACL [(<Numerical Expression>)]

Example) <Numeric Variable>=M_DACL [(<Numerical Expression>)]

Example) <Numeric Variable>=M_NACL [(<Numerical Expression>)]

Example) <Numeric Variable>=M_NDACL [(<Numerical Expression>)]

Example) <Numeric Variable>=M_ACLSTS [(<Numerical Expression>)]

[Terminology]

<Numeric Variable> Specifies the numerical variable to assign.

<Equation> 1 to 32, Enter the task slot number. If this argument is omitted, the current slot will be used as the default.

[Reference Program]

10 M1=M_ACL	' M1 will contain the ratio of acceleration time set for task slot 1.
20 M1=M_DACL(2)	' M1 will contain the ratio of deceleration time set for task slot 2.
30 M1=M_NACL	' M1 will contain the ratio of initial acceleration time value set for task slot 1.
40 M1=M_NDACL(2)	' M1 will contain the ratio of initial deceleration time value set for task slot 2.
50 M1=M_ACLSTS(3)	' M1 will contain the current acceleration/deceleration status for task slot 3.

[Explanation]

- (1) The ratio of acceleration/deceleration time is the ration against each robot's maximum acceleration/deceleration time (initial value). If this value is 50%, the amount of time needed to accelerate/decelerate is doubled, resulting in slower acceleration/deceleration.
- (2) M_NACL and M_NDACL always return 100 (%).
- (3) This variable only reads the data.

M_BTME

[Function]

Returns the remaining hour of battery left. (Unit: hour)

[Format]

Example) <Numeric Variable>=M_BTME

[Terminology]

<Numeric Variable> Specifies the numerical variable to assign.

[Reference Program]

10 M1=M_BTME ' The remaining battery capacity time is assigned to M1.

[Explanation]

- (1) Returns the remaining hours the battery can last from now.
- (2) As for the battery life, 14,600 hours are stored as the initial value.
- (3) After summing the total amount of time the power of robot controller has been off, this value will be subtracted from 14,600 and the result is returned.
- (4) This variable only reads the data.

M_CMPDST

[Function]

Returns the amount of difference (in mm) between the command value and the actual value from the robot when executing the compliance function.

[Format]

Example) <Numeric Variable>=M_CMPDST [(<Mechanism Number>)]

[Terminology]

<Numeric Variable> Specifies the numerical variable to assign.

<Mechanism Number> Enter the mechanism number. 1 to 3, If the argument is omitted, 1 is set as the default value.

[Reference Program]

```
10 MOV P1
20 CMPG 0.5,0.5,1.0,0.5,0.5, , ,      ' Set softness.
30 CMP POS, &B00011011                ' Enter soft state.
40 MVS P2
50 M_OUT(10)=1
60 MVS P1
70 M1=M_CMPDST(1)                     ' M1 will contain the difference between the position specified by the operation command and the actual current position.
80 CMP OFF                             ' Return to normal state.
```

[Explanation]

- (1) This is used to check the positional discrepancy while executing the compliance function.
- (2) This variable only reads the data.

M_CMPLMT

[Function]

Returns whether or not the command value when the compliance function is being executed is about to exceed various limits.

1: The command value is about to exceed a limit.

0: The command value is not about to exceed a limit.

[Format]

Example) DEF ACT 1, M_CMPLMT [(⟨Mechanism Number⟩)]=1 GOTO *LMT

[Terminology]

⟨Mechanism Number⟩ Specify the mechanism number 1 to 3. The default value is 1.

[Reference Program]

```

10 DEF ACT 1, M_CMPLMT(1)=1 GOTO *LMT   ' Define the conditions of interrupt 1.
20 '
30 '
100 MOV P1
110 CMPG 1,1,0,1,1,1,1,1
120 CMP POS, &B100                       ' Enable compliance mode.
130 ACT 1=1                             ' Enable interrupt 1.
140 MVS P2                               '
150                                     '
160                                     '
1000 *LMT
1010 MVS P1                             ' Movement to P2 is interrupted and returns to P1.
1020 RESET ERR                          ' Reset the error.
1030 HLT                                ' Execution is stopped.
```

[Explanation]

- (1) This is used to recover from the error status by using interrupt processing if an error has occurred while the command value in the compliance mode attempted to exceed a limit.
- (2) For various limits, the joint operation range and operation speed of the command value in the compliance mode, and the dislocation between the commanded position and the actual position are checked.
- (3) 0 is set if the servo power is off, or the compliance mode is disabled.
- (4) This is a read only variable.

M_CSTP

[Function]

Returns the status of whether the program is under one cycle operation by cycle stop request. (Either the END key on the operation panel was pressed or a cycle stop signal has been entered.)

- 1: In one cycle operation
- 0: Other than above.

[Format]

Example) <Numeric Variable>=M_CSTP

[Terminology]

<Numeric Variable> Specifies the numerical variable to assign.

[Reference Program]

10 M1=M_CSTP ' 1 is assigned to M1. (When under a cycle stop)

[Explanation]

- (1) When the END key on the operation panel is pressed while the program is under continuous execution, the system enters a cycle operation state.
- (2) This variable only reads the data.

M_CYS

[Function]

Returns whether or not a program is in one cycle operation.

- 1: In one cycle operation (when the operation mode is set as CYC in the parameter slot table, and when cycle operation is specified while the XRUN instruction is being executed)
- 0: Other than above.

[Format]

Example) <Numerical variable> = M_CYS

[Terminology]

<Numerical variable> Specify the numerical variable to substitute.

[Reference Program]

10 M1=M_CYS ' The numerical value 1 is substituted for M1. (If under the cycle operation)

[Explanation]

- (1) When starting a program, the cycle mode – either continuous operation or cycle operation – can be specified using a parameter, etc. Returns this operation mode.
- (2) Even if CYC is specified in the slot table, the value will be 0 when continuous operation is specified during XRUN.
- (3) This is a read only variable.

M DIN/M DOUT

[Function]

This is used to write or reference the remote register of CC-Link (optional).

M_DIN : References the input register.

M_DOUT : Writes or reference the output register.

[Format]

Example) <Numeric Variable>=M_DIN [(<Numerical Expression 1>)]

Example) <Numeric Variable>=M_DOUT [(<Numerical Expression 2>)]

[Terminology]

<Numeric Variable>	Specifies the numerical variable that assigns the CC-Link register value.
<Numerical Expression 1>	Specifies the CC-Link register number (6000 or above).
<Numerical Expression 2>	Specifies the CC-Link register number (6000 or above).

[Reference Program]

10 M1=M_DIN(6000)	' M1 will contain the CC-Link input register value. ' (If CC-Link station number is 1.)
20 M1=M_DOUT(6000)	' M1 will contain the CC-Link output register value.
30 M_DOUT(6000)=100	' Writes 100 to the CC-Link output register.

[Explanation]

- (1) For details, refer to the "CC-Link Interface Instruction Manual."
- (2) Signal numbers in 6,000's will be used for CC-Link.
- (3) M_DIN is read-only.

M_ERR/M_ERRLVL/M_ERRNO

[Function]

Returns information regarding the error generated from the robot.

M_ERR : Returns whether an error has been generated. (1: Error has been generated, 0: No error)

M_ERRLVL : Returns the level of the generated error. (Caution/Low/High1/High2 = 1/2/3/4)

M_ERRNO : Returns the error number of the generated error.

[Format]

Example) <Numeric Variable>=M_ERR

Example) <Numeric Variable>=M_ERRLVL

Example) <Numeric Variable>=M_ERRNO

[Terminology]

<Numeric Variable> Specifies the numerical variable to assign.

[Reference Program]

```
10 IF M_ERR=0 THEN 10      ' Waits until an error is generated.
20 M2=M_ERRLVL             ' M2 will contain the error level
30 M3=M_ERRNO              ' M3 will contain the error number.
```

[Explanation]

- (1) Normal programs will pause when an error (other than cautions) is generated. The error status of the controller may be monitored using this variable for programs whose startup condition is set to ALWAYS by the SLT* parameter. The program set to ALWAYS will not stop even when an error is generated from other programs.
- (2) Level 1 errors are warnings, level 2 errors pause programs. Level 3 errors pause programs and turn the servo power OFF, but error reset can be performed. Level 4 errors pause programs, turn the servo power OFF, and error reset cannot be performed. Thus, when a level 4 error occurs, it is necessary to turn the controller power OFF.
- (3) This variable only reads the data.

[Related instructions]

[RESET ERR](#)

M_EXP

[Function]

Returns the base of natural logarithm (2.7182182845905).

[Format]

Example) <Numeric Variable>=M_EXP

[Terminology]

<Numeric Variable> Specifies the numerical variable to assign.

[Reference Program]

```
10 M1=M_EXP                ' Base of natural logarithm (2.7182182845905) is assigned to M1.
```

[Explanation]

- (1) This is used when processing exponential and logarithmic functions.
- (2) This variable only reads the data.

M_G**[Function]**

Returns gravitational constant (9.80665).

[Format]

Example) <Numeric Variable>=M_G

[Terminology]

<Numeric Variable> Specifies the numerical variable to assign.

[Reference Program]

10 M1=M_G ' Gravitational constant (9.80665) is assigned to M1.

[Explanation]

- (1) This is used to perform calculation related to gravity.
- (2) This variable only reads the data.

M_HNDCQ**[Function]**

Returns the hand check input signal value.

[Format]

Example) <Numeric Variable>=M_HNDCQ [(<Equation>)]

[Terminology]

<Numeric Variable> Specifies the numerical variable to assign.
 <Equation> Enter the hand input signal number.
 1 to 8, (Corresponds to input signals 900 to 907.)

[Reference Program]

10 M1=M_HNDCQ(1) ' M1 will contain the status of hand 1.

[Explanation]

- (1) Returns one bit of the hand check input signal status (such as a sensor).
- (2) M_HNDCQ(1) corresponds to input signal number 900. Same result will be obtained using M_IN (900).
- (3) This variable only reads the data.

M_IN/M_INB/M_INW**[Function]**

Returns the value of the input signal.

M_IN : Returns a bit.

M_INB : Returns a byte (8 bits).

M_INW : Returns a word (16 bits).

[Format]

Example) <Numeric Variable>=M_IN (<Numerical Expression>)

Example) <Numeric Variable>=M_INB (<Numerical Expression>)

Example) <Numeric Variable>=M_INW (<Numerical Expression>)

[Terminology]

<Numeric Variable>	Specifies the numerical variable to assign.
<Numerical Expression>	Enter the input signal number. 0 to 32767 (Theoretical value) 0 to 255 : Standard remote inputs (Normally 32 points. 0 to 31) 900 to 907 : Hand input. 6000 or more : Remote input for CC-Link.

[Reference Program]

10 M1=M_IN(0)	' M1 will contain the value of the input signal 0 (1 or 0).
20 M2=M_INB(0)	' M2 will contain the 8-bit information starting from input signal 0.
30 M3=M_INB(3) AND &H7	' M3 will contain the 3-bit information starting from input signal 3.
40 M4=M_INW(5)	' M4 will contain the 16-bit information starting from input signal 5.

[Explanation]

- (1) Returns the status of the input signal.
- (2) M_INB and M_INW will return 8- or 16-bit information starting from the specified number.
- (3) Although the signal number can be as large as 32767, only the signal numbers with corresponding hardware will return a valid value. Value for a signal number without corresponding hardware is set as undefined.
- (4) This variable only reads the data.

M JOVRD/M NJOVRD/M OPOVRD/M OVRD/M NOVRD**[Function]**

Returns override value.

M_JOVRD : Value specified by the override JOVRD instruction for joint interpolation.

M_NJOVRD : Initial override value (100%) for joint interpolation.

M_OPOVRD : Override value of the operation panel.

M_OVRD : Current override value, value specified by the OVRD instruction.

M_NOVRD : Initial override value (100%).

[Format]

Example) <Numeric Variable>=M_JOVRD [(<Numerical Expression>)]

Example) <Numeric Variable>=M_NJOVRD [(<Numerical Expression>)]

Example) <Numeric Variable>=M_OPOVRD

Example) <Numeric Variable>=M_OVRD [(<Numerical Expression>)]

Example) <Numeric Variable>=M_NOVRD [(<Numerical Expression>)]

[Terminology]

<Numeric Variable>

Specifies the numerical variable to assign.

<Numerical Expression>

1 to 32, Enter the task slot number. If this parameter is omitted, the current slot will be used as the default.

[Reference Program]

10 M1=M_OVRD

' M1 will contain the current override value.

20 M2=M_NOVRD

' M2 will contain the initial override value (100%).

30 M3=M_JOVRD

' M3 will contain the current joint override value.

40 M4=M_NJOVRD

' M4 will contain the initial joint override value.

50 M5=M_OPOVRD

' M5 will contain the current OP (operation panel) override value.

60 M6=M_OVRD(2)

' M6 will contain the current override value for slot 2.

[Explanation]

(1) If the argument is omitted, the current slot status will be returned.

(2) This variable only reads the data.

M LINE

[Function]

Returns the line number that is being executed.

[Format]

Example) <Numeric Variable>=M_LINE [(<Numerical Expression>)]

[Terminology]

<Numeric Variable> Specifies the numerical variable to assign.

<Equation> 1 to 32, Specifies the task slot number. If this parameter is omitted, the current slot will be used as the default.

[Reference Program]

10 M1=M_LINE(2) ' M1 will contain the line number being executed by slot 2.

[Explanation]

(1) This can be used to monitor the line being executed by other tasks during multitask operation.

(2) This variable only reads the data.

M MODE

[Function]

Returns the key switch mode of the operation panel.

1 : TEACH

2 : AUTO(OP)

3 : AUTO(Ext.)

[Format]

Example) <Numeric Variable>=M_MODE

[Terminology]

<Numeric Variable> Specifies the numerical variable to assign.

[Reference Program]

10 M1=M_MODE ' M1 will contain the key switch status.

[Explanation]

(1) This can be used in programs set to ALWAYS (constantly executed) during multitask operation.

(2) This variable only reads the data.

M_ON/M_OFF

[Function]

Always returns 1 (M_ON) or 0 (M_OFF).

[Format]

Example) <Numeric Variable>=M_ON

Example) <Numeric Variable>=M_OFF

[Terminology]

<Numeric Variable> Specifies the numerical variable to assign.

[Reference Program]

10 M1=M_ON ' 1 is assigned to M1.

20 M2=M_OFF ' 0 is assigned to M2.

[Explanation]

- (1) Always returns 1 or 0.
- (2) This variable only reads the data.

M_OPEN

[Function]

RetReturns the status indicating whether or not a file is opened.
Returns the status of other end of the RS-232C cable.

[Format]

This function is available for controller software version H7 or later

Example)<Numerical variable>=M_OPEN [<File number>]

[Terminology]

<Numerical variable>	Specify the numerical variable to substitute.
<File number>	Specify the file number 1–8 by constant value of communication line opened by OPEN command. The default value is 1. If 9 or more are specified, the error will occur when executing.

[Reference Program]

100 OPEN "COM2:" AS #1	' Open the communication line COM2 as the file number 1.
110 IF M_OPEN(1)<>1 THEN GOTO 110	' Wait until the file number 1 opens.

[Explanation]

- (1) This is a read only variable.
(2) The return value differ corresponding to the file type specified by OPEN command as follows.

Kind of files	Meaning	Value
File	Returns the status indicating whether or not a file is opened. Returns 1 until the CLOSE instruction, the END instruction or END in a program is executed after executing the OPEN instruction.	1: Already opened -1: Undefined file number (not opened)
Communication line RS-232C	*Returns the status of other end of the RS-232C cable. Returns the status of the CTS signal input as is. (This can be used only when the RTS signal of other end is enabled using the Mitsubishi genuine cable specification.)	1: Already connected (CTS signal is ON) 0: Unconnected (CTS signal is OFF) -1: Undefined file number (not opened)

*Refer to separate manual "Ethernet Interface INSTRUCTION MANUAL" when using the ethernet.

[Related instructions]

OPEN

[Related parameter]

COMDEV

M OUT/M OUTB/M OUTW

[Function]

Writes or references external output signal.

M_OUT : Output signal bit.

M_OUTB : Output signal byte (8 bits).

M_OUTW : Output signal word (16 bits).

[Format]

Example)M_OUT (<Numerical Expression>) =<Numeric Variable>

Example)M_OUTB (<Numerical Expression>) =<Numeric Variable>

Example)M_OUTW (<Numerical Expression>) =<Numeric Variable>

[Terminology]

<Numeric Variable> Specifies the numerical variable to assign.

<Equation> Specify the output signal number.

0 to 255 : Standard remote outputs.

900 to 907 : Hand output.

6000 or more :Remote output for CC-Link.

[Reference Program]

10 M_OUT(2)=1 ' Turn ON output signal 2 (1 bit).

20 M_OUTB(2)=&HFF ' Turns ON 8-bits starting from the output signal 2.

30 M_OUTW(2)=&HFFFF ' Turns ON 16-bits starting from the output signal 2.

40 M4=M_OUTB(2) AND &H0F ' M4 will contain the 4-bit information starting from output signal 2.

[Explanation]

(1) This is used when writing or referencing external output signals.

(2) Numbers in 900's will be used as I/O signals for the hand.

(3) Numbers 6000 and beyond will be referenced/assigned to the CC-Link (optional).

M PI

[Function]

Returns pi (3.14159265358979).

[Format]

Example)<Numeric Variable>=M_PI

[Terminology]

<Numeric Variable> Specifies the numerical variable to assign.

[Reference Program]

10 M1=M_PI ' 3.14159265358979 is assigned to M1.

[Explanation]

(1) A variable to be assigned will be a real value.

(2) This variable only reads the data.

M_PSA

[Function]

Returns whether the program is selectable by the specified task slot.

1 : Program is selectable.

0 : Program not selectable (when the program is paused).

[Format]

Example) <Numeric Variable>=M_PSA [(<Numerical Expression>)]
--

[Terminology]

<Numeric Variable> Specifies the numerical variable to assign.

<Numerical Expression> 1 to 32, Specifies the task slot number. If this parameter is omitted, the current slot will be used as the default.

[Reference Program]

10 M1=M_PSA(2) ' M1 will contain the program selectable status of task slot 2.

[Explanation]

(1) Returns whether the program is selectable by the specified task slot.

(2) This variable only reads the data.

M_RATIO

[Function]

Returns how much the robot has approached the target position (0 to 100%) while the robot is moving.

[Format]

Example) <Numeric Variable>=M_RATIO [(<Numerical Expression>)]
--

[Terminology]

<Numeric Variable> Specifies the numerical variable to assign.

<Numerical Expression> 1 to 32, Specifies the task slot number. If this parameter is omitted, the current slot will be used as the default.

[Reference Program]

10 MOV P1 WTHIF M_RATIO>80 M_OUT(1)=1 ' The output signal 1 will turn ON when the robot has moved 80% of the distance until the target position is reached while moving toward P1.

[Explanation]

(1) This is used, for instance, when performing a procedure at a specific position while the robot is moving.

(2) This variable only reads the data.

M_RDST

[Function]

Returns the remaining distance to the target position (in mm) while the robot is moving.

[Format]

Example) <Numeric Variable>=M_RDST [(<Numerical Expression>)]

[Terminology]

<Numeric Variable>	Specifies the numerical variable to assign.
<Numerical Expression>	1 to 32, Specifies the task slot number. If this parameter is omitted, the current slot will be used as the default.

[Reference Program]

10 MOV P1 WTHIF M_RDST<10 M_OUT(10)=1 ' The output signal 1 will turn ON when the remaining distance until the target position is reached becomes 10 mm or less while moving toward P1.

[Explanation]

- (1) This is used, for instance, when performing a procedure at a specific position while the robot is moving.
- (2) This variable only reads the data.

M_RUN

[Function]

Returns whether the program for the specified task slot is being executed.

1 : Executing.

0 : Not executing (paused or stopped).

[Format]

Example) <Numeric Variable>=M_RUN [(<Numerical Expression>)]
--

[Terminology]

<Numeric Variable>	Specifies the numerical variable to assign.
<Numerical Expression>	1 to 32, Specifies the task slot number. If this parameter is omitted, the current slot will be used as the default.

[Reference Program]

10 M1=M_RUN(2) ' M1 will contain the execution status of slot 2.

[Explanation]

- (1) This will contain 1 if the specified slot is running, or 0 if the slot is stopped (or paused).
- (2) Combine M_RUN and M_WAI to determine if the program has stopped (in case the currently executed line is the top line).
- (3) This variable only reads the data.

M SKIPCQ

[Function]

Returns the result of executing the line containing the last executed SKIP command.

1 : SKIP has been executed.

0 : SKIP has not been executed.

[Format]

Example) <Numeric Variable>=M_SKIPCQ [(<Numerical Expression>)]
--

[Terminology]

<Numeric Variable> Specifies the numerical variable to assign.

<Numerical Expression> 1 to 32, Specifies the task slot number. If this parameter is omitted, the current slot will be used as the default.

[Reference Program]

10 MOV P1 WTHIF M_IN(10)=1,SKIP	' If the input signal 10 is 1 when starting to move to P1, skip the MOV instruction.
20 IF M_SKIPCQ=1 THEN GOTO 1000	' If SKIP instruction has been executed, jump to line 1000.
.	
1000 END	

[Explanation]

(1) Checks if a SKIP instruction has been executed.

(2) This variable only reads the data.

(3) SKIP status will be cleared when a SKIP check is executed.

M_SPD/M_NSPD/M_RSPD**[Function]**

Returns the speed information during XYZ and JOINT interpolation.

M_SPD : Currently set speed.

M_NSPD : Initial value (optimum speed control).

M_RSPD : Directive speed.

[Format]

Example) <Numeric Variable>=M_SPD [(<Numerical Expression>)]

Example) <Numeric Variable>=M_NSPD [(<Numerical Expression>)]

Example) <Numeric Variable>=M_RSPD [(<Numerical Expression>)]

[Terminology]

<Numeric Variable> Specifies the numerical variable to assign.

<Numerical Expression> 1 to 32, Specifies the task slot number. If this parameter is omitted, the current slot will be used as the default.

[Reference Program]

10 M1=M_SPD ' M1 will contain the currently set speed.

20 SPD M_NSPD ' Reverts the speed to the optimum speed control mode.

[Explanation]

(1) M_RSPD returns the directive speed at which the robot is operating.

(2) This can be used in M_RSPD multitask programs or with WTH and WTHIF statements.

(3) This variable only reads the data.

M_SVO**[Function]**

Returns the current status of the servo power supply.

1 : Servo power ON

0 : Servo power OFF

[Format]

Example) <Numeric Variable>=M_SVO [(<Mechanism Number>)]

[Terminology]

<Numeric Variable> Specifies the numerical variable to assign.

<Mechanism Number> Enter the mechanism number. 1 to 3, If the argument is omitted, 1 is set as the default value.

[Reference Program]

10 M1=M_SVO(1) ' M1 will contain the current status of the servo power supply.

[Explanation]

(1) The status of the robot's servo can be checked.

(2) This variable only reads the data.

M_TIMER

[Function]

Time is measured in milliseconds. This can be used to measure the operation time of the robot or to measure time accurately.

[Format]

Example) <Numeric Variable>=M_TIMER (<Numerical Expression>)

[Terminology]

<Numeric Variable>	Specifies the numerical variable to assign.
<Numerical Expression>	Enter the number to 8 from 1. Parentheses are required.

[Reference Program]

```
10 M_TIMER(1)=0
20 MOV P1
30 MOV P2
40 M1=M_TIMER(1)
```

' M1 will contain the amount of time required to move from P1 to P2 (in ms).
Example) If the time is 5.346 sec. the value of M1 is 5346.

[Explanation]

- (1) A value may be assigned.
- (2) Since measurement can be made in milliseconds (ms), precise execution time measurement is possible.

M_UAR

[Function]

Returns whether the robot is in the user specified area.
Bits 0 through 7 correspond to areas 1 through 8.
1 : Within user specified area
0 : Outside user specified area

[Format]

Example) <Numeric Variable>=M_UAR [(<Mechanism Number>)]

[Terminology]

<Numeric Variable>	Specifies the numerical variable to assign.
<Mechanism Number>	Enter the mechanism number. 1 to 3, If the argument is omitted, 1 is set as the default value.

[Reference Program]

```
10 M1=M_UAR(1)
```

' M1 indicates whether the robot is within or outside the user specified area.

[Explanation]

- (1) For details on how to use user specified areas, refer to [Page 279, "About user-defined area"](#).
- (2) This variable only reads the data.

M_WAI

[Function]

Returns the standby status of the program for the specified task slot.

1 : Paused (The program has been paused.)

0 : Not paused (Either the program is running or is being stopped.)

[Format]

Example) <Numeric Variable>=M_WAI [(<Numerical Expression>)]

[Terminology]

<Numeric Variable> Specifies the numerical variable to assign.

<Numerical Expression> 1 to 32, Specifies the task slot number. If this parameter is omitted, the current slot will be used as the default.

[Reference Program]

10 M1=M_WAI(1) ' M1 will contain the standby status of slot 1.

[Explanation]

(1) This can be used to check whether the program has been paused.

(2) Combine M_RUN and M_WAI to determine if the program has stopped (in case the currently executed line is the top line).

(3) This variable only reads the data.

P_BASE/P_NBASE

[Function]

Returns information related to the base conversion data.

P_BASE : Returns the base conversion data that is currently being set.

P_NBASE : Returns the initial value (0, 0, 0, 0, 0, 0) (0, 0).

[Format]

Example) <Position Variables>=P_BASE [(<Mechanism Number>)]

Example) <Position Variables>=P_NBASE

[Terminology]

<Position Variables> Specifies the position variable to assign.

<Mechanism Number> Enter the mechanism number. 1 to 3, If the argument is omitted, 1 is set as the default value.

[Reference Program]

10 P1=P_BASE ' P1 will contain the base conversion data that is currently being set.

20 BASE P_NBASE ' Resets the base conversion data to the initial value.

[Explanation]

(1) P_NBASE will contain (0, 0, 0, 0, 0, 0) (0, 0).

(2) Be careful when using base conversion since it may affect the teaching data.

(3) Use the BASE instruction when changing the base position.

Erroneous value may cause the robot to take unexpected actions. Take extra caution when changing the setting.

(4) This variable only reads the data.

P_CURR

[Function]

Returns the current position (X, Y, Z, A, B, C) (FL1, FL2).

[Format]

Example) <Position Variables>=P_CURR [(<Mechanism Number>)]

[Terminology]

<Position Variables>	Specifies the position variable to assign.
<Mechanism Number>	Enter the mechanism number. 1 to 3, If the argument is omitted, 1 is set as the default value.

[Reference Program]

10 DEF ACT 1,M_IN(10)=1 GOTO 1000	' Defines interrupt.
20 ACT 1=1	' Enables interrupt.
30 MOV P1	
40 MOV P2	
50 ACT 1=0	' Disables interrupt.
1000 P100=P_CURR	' Loads the current position when an interrupt signal is received.
1010 MOV P100,-100	' Moves 100 mm above P100 (i.e, -100 mm in the Z direction of the tool).
1020 END	' Ends the program.

[Explanation]

- (1) This can be used to identify the current position.
- (2) This variable only reads the data.

[Reference]

[J_CURR](#)

P_FBC

[Function]

Returns the current position based on the feedback values from the servo.

[Format]

Example) <Position Variables>=P_FBC [(<Mechanism Number>)]

[Terminology]

<Position Variables>	Specifies the position variable to assign.
<Mechanism Number>	Enter the mechanism number. 1 to 3, If the argument is omitted, 1 is set as the default value.

[Reference Program]

10 P1=P_FBC	' P1 will contain the current position based on the feedback.
-------------	---

[Explanation]

- (1) Returns the current position based on the feedback values from the servo.
- (2) This variable only reads the data.

[Reference]

[J_FBC](#)

P_SAFE

[Function]

Returns the safe point (XYZ position of the JSAFE parameter).

[Format]

Example) <Position Variables>=P_SAFE [(<Mechanism Number>)]

[Terminology]

<Position Variables>	Specifies the position variable to assign.
<Mechanism Number>	Enter the mechanism number. 1 to 3, If the argument is omitted, 1 is set as the default value.

[Reference Program]

10 P1=P_SAFE	' P1 will contain the set safe point being set.
--------------	---

[Explanation]

- (1) Returns the XYZ position, which has been converted from the joint position registered in parameter JSAFE.
- (2) This variable only reads the data.

P TOOL/P NTOOL

[Function]

Returns tool conversion data.

P_TOOL: Returns the tool conversion data that is currently being set.

P_NTOOL: Returns the initial value (0,0,0,0,0,0,0)(0,0).

[Format]

Example) <Position Variables>=P_TOOL [(<Mechanism Number>)]

Example) <Position Variables>=P_NTOOL

[Terminology]

<Position Variables> Specifies the position variable to assign.

<Mechanism Number> Enter the mechanism number. 1 to 3, If the argument is omitted, 1 is set as the default value.

[Reference Program]

10 P1=P_TOOL ' P1 will contain the tool conversion data.

[Explanation]

(1) P_TOOL returns the tool conversion data set by the TOOL instruction or the MEXTL parameter.

(2) Use the TOOL instruction when changing tool data.

(3) This variable only reads the data.

P ZERO

[Function]

Always returns (0,0,0,0,0,0,0)(0,0).

[Format]

Example) <Position Variables>=P_ZERO

[Terminology]

<Position Variables> Specifies the position variable to assign.

[Reference Program]

10 P1=P_ZERO '(0,0,0,0,0,0,0)(0,0) is assigned to P1.

[Explanation]

(1) This can be used to initialize the P variable to zeros.

(2) This variable only reads the data.

4.13 Detailed Explanation of Functions

4.13.1 How to Read Described items

[Function]	: This indicates a function of a function.
[Format]	: This indicates how to input the function argument.
[Reference Program]	: An example program using function is shown.
[Terminology]	: This indicates the meaning and range of an argument.
[Explanation]	: This indicates detailed functions and precautions.
[Reference]	: This indicates related function.

4.13.2 Explanation of Each Function

Each variable is explained below in alphabetical order.

ABS

[Function]

Returns the absolute value of a given value.

[Format]

<Numeric Variable>=ABS (<Numerical Expression>)

[Reference Program]

```
10 P2.C=ABS(P1.C)      ' P2.C will contain the value of P1.C without the sign.  
20 MOV P2  
30 M2=-100  
40 M1=ABS(M2)         ' 100 is assigned to M1.
```

[Explanation]

(1) Returns the absolute value (Value with the positive sign) of a given value.

[Reference]

[SGN](#)

ALIGN

[Function]

Positional posture axes (A, B, and C axes) are converted to the closest XYZ postures (0 , ± 90 , and ± 180).

ALIGN outputs numerical values only. The actual operation will involve movement instructions such as the MOV instruction.

[Format]

$\langle \text{Position Variables} \rangle = \text{ALIGN} \ (\langle \text{Position} \rangle)$
--

[Reference Program]

```

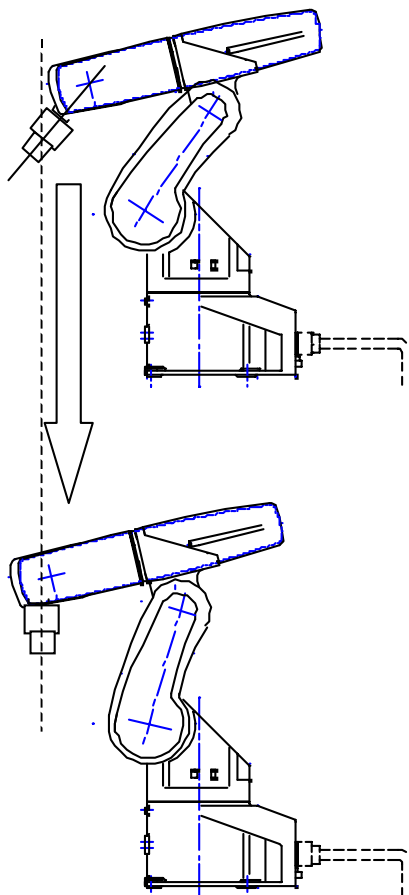
10 P1=P_CURR
20 P2=ALIGN(P1)
30 MOV P2

```

[Explanation]

- (1) Converts the A, B, and C components of the position data to the closest XYZ postures (0 , ± 90 , and ± 180).
- (2) Since the return value is of position data type, an error will be generated if the left-hand side is of joint variable type.
- (3) This function cannot be used in vertical multi-joint 5-axes robot.

The following shows a sample case for the axis B.



ASC

[Function]

Returns the character code of the first character in the string.

[Format]

<code><Numeric Variable>=ASC (<Character String Expression>)</code>

[Reference Program]

```
10 M1=ASC("A")      ' &H41 is assigned to M1.
```

[Explanation]

- (1) Returns the character code of the first character in the string.
- (2) An error will be generated if the string is a null string.

[Reference]

[CHR\\$](#), [VAL](#), [CVI](#), [CVS](#), [CVD](#)

ATN/ATN2

[Function]

Calculates the arc tangent.

[Format]

<code><Numeric Variable>=ATN (<Numerical Expression>)</code>
<code><Numeric Variable>=ATN2 (<Numerical Expression 1>, <Numerical Expression 2>)</code>

[Terminology]

<Numeric Variable>	Calculates the arc tangent with specified expression, and returns the result. The unit is radian.
<Numerical Expression>	Calculated value of $\Delta Y / \Delta X$.
<Numerical Expression 1>	ΔY
<Numerical Expression 2>	ΔX

[Reference Program]

```
10 M1=ATN(100/100)      '  $\pi/4$  is assigned to M1.  
20 M2=ATN2(-100,100)
```

[Explanation]

- (1) Calculates the arc tangent of a given numerical expression. Unit is in radians.
- (2) The range of the returned value for ATN is $-\pi/2 < \text{ATN} < \pi/2$.
- (3) The range of the returned value for ATN2 is $-\pi < \text{ATN} < \pi$.
- (4) If <Numerical Expression 2> evaluates to 0, ATN2 will return $\pi/2$ when <Numerical Expression 1> evaluates to a positive value and $-\pi/2$ when <Numerical Expression 1> evaluates to a negative value.
- (5) In the case of ATN2, it is not possible to describe a function that contains an argument in <numerical expression 1> and <numerical expression 2>. If such a function is described, an error will be generated during execution.

NG example `M1=ATN2 (MAX (MA, MB) , 100)`
 `M1=ATN2 (CINT (10. 2) , 100)`

[Reference]

[SIN](#), [COS](#), [TAN](#)

BIN\$

[Function]

Value is converted to a binary string.

[Format]

<Character String Variable >=BIN\$ (<Numerical Expression>)

[Reference Program]

```
10 M1=&B11111111
```

```
20 C1$=BIN$(M1)      ' C1$ will contain the character string of "11111111".
```

[Explanation]

- (1) Value is converted to a binary string.
- (2) If the equation does not evaluate to an integer, the integral value obtained by rounding the fraction will be converted to a binary string.
- (3) VAL is a command that performs the opposite of this function.

[Reference]

HEX\$, STR\$, VAL

CALARC

[Function]

Provides information regarding the arc that contains the three specified points.

[Format]

```
<Numeric Variable 4> = CALARC (<Position 1>, <Position 2>, <Position 3>,
                                <Numeric Variable 1>, <Numeric Variable 2>, <Numeric Variable 3>,
                                <Position Variables 1>)
```

[Terminology]

<Position 1>	Specifies the starting point of the arc.
<Position 2>	Specifies the passing point of the arc. Same as the three points in the MVR instruction.
<Position 3>	Specifies the endpoint of the arc.
<Numeric Variable 1>	Radius of the specified arc (in mm) will be calculated and returned.
<Numeric Variable 2>	Central angle of the specified arc (in radians) will be calculated and returned.
<Numeric Variable 3>	Length of the specified arc (in mm) will be calculated and returned.
<Position Variables 1>	The center coordinates of the specified arc (in mm) will be calculated and returned (as a position data type, ABC are all zeros).
<Numeric Variable 4>	Return value 1 : Calculation was performed normally. -1 : Of positions 1, 2, and 3, either two points had the exact same position or all three points were on a straight line. -2 : All three points are at approximately the same position.

[Reference Program]

```
10 M1=CALARC(P1,P2,P3,M10,M20,M30,P10)
20 IF M1<>1 THEN END      ' Ends if an error occurs.
30 MR=M10                 ' Radius.
40 MRD=M20                ' Circular arc angle.
50 MARCLEN=M30            ' Circular arc length.
60 PC=P10                 ' Coordinates of the center point.
```

[Explanation]

- (1) Provides information regarding the arc that is determined by the three specified points, position 1, position 2 and position 3.
- (2) If the arc generation and calculation of various values succeeded, 1 will be returned as the return value.
- (3) If some points have the exact same position or if all three points are on a straight line, -1 will be returned as the return value. In such cases, the distance between the starting point and the endpoint will be returned as the arc length, -1 as the radius, 0 as the central angle, and (0, 0, 0) as the center point.
- (4) If circular arc generation fails, -2 will be returned as the return value. If a circular arc cannot be generated, -1, 0, 0 and (0, 0, 0) are returned as the radius, central angle, arc length and center point, respectively.
- (5) It is not possible to describe a function that contains an argument in <position 1>, <position 2>, <position 3>, <numeric variable 1>, <numerical variable 2>, <numeric variable 3> and <position variable 1>. If such a function is described, an error will be generated during execution.

CHRSRCH

[Function]

Searches a string from a string array.

[Format]

<Numeric Variable>=CHRSRCH (<String array Variable>,<Character String>, <The Register Place Of Search Result>)

[Reference Program]

```

10 DIM CTBL$(10)           ' Array variables definition.
20 CTBL$(1)="ABC"          ' "ABC".
30 CTBL$(2)="123"          ' "123" .
40 CTBL$(3)="XYZ"          ' "XYZ".
50 CTBL$(4)=""             '
60 CHRSRCH CTBL$(1), "123", M1 ' The element number 2 is stored to M1.
```

[Explanation]

- (1) Returns the element number of the character strings which match.
- (2) If the two or three dimensions is specified in the array variables, an error occurs at execution.
- (3) The search distinguishes uppercase and lowercase.

[Reference]

[CHR\\$](#)

CHR\$

[Function]

Returns the character that has the character code obtained from the specified numerical expression.

[Format]

<Character String Variable >=CHR\$ (<Numerical Expression>)

[Reference Program]

```
10 M1=&H40
20 C1$=CHR$(M1+1)      ' "A" is assigned to C1$.
```

[Explanation]

- (1) Returns the character that has the character code obtained from the specified numerical expression.
- (2) If the numerical expression does not evaluate to an integer, the character will be returned whose character code corresponds to the integral value obtained by rounding the fraction.

[Reference]

[ASC](#)

CINT

[Function]

Rounds the fractional part of an numerical expression to convert the value into an integer.

[Format]

<Numeric Variable>=CINT (<Numerical Expression>)
--

[Reference Program]

```
10 M1=CINT(1.5)      ' 2 is assigned to M1.
20 M2=CINT(1.4)      ' 1 is assigned to M2.
30 M3=CINT(-1.4)     ' -1 is assigned to M3.
40 M4=CINT(-1.5)     ' -2 is assigned to M4.
```

[Explanation]

- (1) Returns the value obtained by rounding the fractional part of an equation.

[Reference]

[INT](#), [FIX](#)

CKSUM

[Function]

Calculates the checksum of the string.

[Format]

<Numeric Variable>=CKSUM (<Character String>, <Numerical Expression 1>, <Numerical Expression 2>)

[Terminology]

- | | |
|--------------------------|--|
| <Character String> | Specifies the string from which the checksum should be calculated. |
| <Numerical Expression 1> | Specifies the first character position from where the checksum calculation starts. |
| <Numerical Expression 2> | Specifies the first character position from where the checksum calculation ends. |

[Reference Program]

10 M1=CKSUM("ABCDEFGH",1,3) ' &H41("A")+&H42("B")+&H43("C")=&HC6 is assigned to M1.

[Explanation]

- (1) Adds the character codes of all characters in the string from the starting position to the end position and returns a value between 0 and 255.
- (2) If the starting position is outside the range of the string, an error will be generated.
- (3) If the end position exceeds the end of the string, checksum from the starting position to the last character in the string will be calculated.
- (4) If the result of addition exceeds 255, a degenerated value of 255 or less will be returned.
- (5) It is not possible to describe a function that contains an argument in <Character String>, <Numerical Expression 1> and <Numerical Expression 2>. If such a function is described, an error will be generated during execution.

COS

[Function]

Gives the cosine.

[Format]

<Numeric Variable>=COS (<Numerical Expression>)

[Reference Program]

10 M1=COS(RAD(60))

[Explanation]

- (1) Calculates the cosine of the numerical expression.
- (2) The range of arguments will be the entire range of values that are allowed.
- (3) The range of the return value will be from -1 to 1.
- (4) The unit of arguments is in radians.

[Reference]

SIN, TAN, ATN/ATN2

CVI

[Function]

Converts the character codes of the first two characters of a string into an integer.

[Format]

<Numeric Variable>=CVI (<Character String Expression>)
--

[Reference Program]

10 M1=CVI("10ABC") ' &H3031 is assigned to M1.

[Explanation]

- (1) Converts the character codes of the first two characters of a string into an integer.
- (2) An error will be generated if the string consists of one character or less.
- (3) MKI\$ can be used to convert numerical values into a string.
- (4) This can be used to reduce the amount of communication data when transmitting numerical data with external devices.

[Reference]

ASC, CVS, CVD, MKI\$, MKS\$, MKD\$

CVS

[Function]

Converts the character codes of the first four characters of a string into a single precision real number.

[Format]

<Numeric Variable>=CVS (<Character String Expression>)
--

[Reference Program]

10 M1=CVS("FFFF") ' 12689.6 is assigned to M1.

[Explanation]

- (1) Converts the character codes of the first four characters of a string into an single-precision real number.
- (2) An error will be generated if the string consists of three character or less.
- (3) MKS\$ can be used to convert numerical values into a string.

[Reference]

ASC, CVI, CVD, MKI\$, MKS\$, MKD\$

CVD

[Function]

Converts the character codes of the first eight characters of a string into a double precision real number.

[Format]

<Numeric Variable>=CVD (<Character String Expression>)
--

[Reference Program]

```
10 M1=CVD("FFFFFFFF")      ' +3.52954E+30 is assigned to M1.
```

[Explanation]

- (1) Converts the character codes of the first eight characters of a string into a double precision real number.
- (2) An error will be generated if the string consists of seven character or less.
- (3) MKD\$ can be used to convert numerical values into a string.

[Reference]

[ASC](#), [CVI](#), [CVS](#), [MKI\\$](#), [MKS\\$](#), [MKD\\$](#)

DEG

[Function]

Converts the unit of angle measurement from radians (rad) into degrees (deg).

[Format]

<Numeric Variable>=DEG (<Numerical Expression>)

[Reference Program]

```
10 P1=P_CURR
20 IF DEG(P1.C) < 170 OR DEG(P1.C) > -150 THEN *NOERR
30 ERROR(9100)
40 *NOERR
```

[Explanation]

- (1) Converts the radian value of an numerical expression into degree value.
- (2) When the posture angles of the position data are to be displayed using positional constants, the unit used for ((500, 0, 600, 180, 0, 180) (7, 0)) is DEG. As in the case of P1.C, the unit used will be in radians (rad) when the rotational element of the positional variable is to be referenced directly. Value of P1.C can be handled in DEG. In such case, set parameter "PRGMDEG" to 1.

[Reference]

[RAD](#)

DIST

[Function]

Calculates the distance between two points (position variables).

[Format]

<Numeric Variable>=DIST (<Position 1>, <Position 2>)
--

[Reference Program]

10 M1=DIST(P1,P2)

' M1 will contain the distance between positions 1 and 2.

[Explanation]

- (1) Returns the distance between positions 1 and 2 (in mm).
- (2) Posture angles of the position data will be ignored; only the X, Y, and Z data will be used for calculation.
- (3) The joint variables cannot be used. Trying to use it will result in an error during execution.
- (4) It is not possible to describe a function that contains an argument in <position 1> and <position 2>. If such a function is described, an error will be generated during execution.

EXP

[Function]

Calculates exponential functions. (an numerical expression that uses "e" as the base.)

[Format]

<Numeric Variable>=EXP (<Numerical Expression>)

[Reference Program]

10 M1=EXP(2)

' e² is assigned to M1.

[Explanation]

- (1) Returns the exponential function value of the numerical expression.

[Reference]

[LN](#)

FIX

[Function]

Returns the integral portion of the numerical expression.

[Format]

<Numeric Variable>=FIX (<Numerical Expression>)

[Reference Program]

10 M1=FIX(5.5) ' 5 is assigned to M1.

[Explanation]

- (1) Returns the integral portion of the numerical expression value.
- (2) If the numerical expression evaluates to a positive value, the same number as INT will be returned.
- (3) If the numerical expression evaluates to a negative value, then for instance $\text{FIX}(-2.3) = -2.0$ will be observed.

[Reference]

[CINT](#), [INT](#)

FRAM

[Function]

Calculates the position data that indicates a coordinate system (plane) specified by three position data. Normally, use DEF PLT and PLT instructions for pallet calculation.

[Format]

<Numeric Variable 4>=FRAM (<Numeric Variable 1>, <Numeric Variable 2>,
<Numeric Variable 3>)

[Terminology]

<Numeric Variable 1>	This will be the origin of X, Y, and Z of the plane to be specified by three positions. A variable or a constant.
<Numeric Variable 2>	A point on the X axis of the plane to be specified by three positions. A variable or a constant.
<Numeric Variable 3>	A point in the positive Y direction of the X-Y plane on the plane to be specified by three positions. A variable or a constant.
<Numeric Variable 4>	Variable to which the result is assigned.

[Reference Program]

10 P100=FRAM(P1,P2,P3)	' Create P100 coordinate system based on P1, P2 and P3 positions.
20 BASE P100	' Position of P100 will be used as the origin for robot.

[Explanation]

- (1) This can be used to define the base coordinate system.
- (2) This creates a plane from the three coordinates X, Y, and Z for the three positions to calculate the position of the origin and the inclination of the plane, and returns the result as a position variable. The X, Y, and Z coordinates of the position data will be identical to that of position variable 1, while A, B, and C will be the inclination of the plane to be specified by the three positions.
- (3) Joint variables cannot be used as the argument. When a joint variable is used, an error will be generated.
- (4) Since the return value is a position data, an error will be generated if a joint variable is used in the left-hand side.
- (5) It is not possible to describe a function that contains an argument in <position 1>, <position 2> and <position 3>. If such a function is described, an error will be generated during execution.
NG example P10=FRAM(FPRM(P01,P02,P03), P04, P05)

[Reference]

Relative conversion (* operator). Refer to [Page 279, "5.3 About user-defined area"](#).

HEX\$

[Function]

Converts the value of an numerical expression (Between -32768 to 32767) into hexadecimal string.

[Format]

<Character String Variable >=HEX\$ (<Numerical Expression> [, <Number of output characters>])

[Reference Program]

```
10 C1$=HEX$(&H41FF)      ' "41FF" is assigned to C1$.
20 C2$=HEX$(&H41FF,2)    ' "FF" is assigned to C2$.
```

[Explanation]

- (1) Converts the value of an numerical expression into hexadecimal string.
 - (2) If <Number of output characters> is specified, the right most part of the converted string is output for the specified length.
 - (3) If the numerical value is not an integer, the integer value obtained by rounding the fraction will be converted into hexadecimal string.
 - (4) VAL is a command that performs this procedure in reverse.
 - (5) If <number of output characters> is specified, it is not possible to describe a function that contains an argument in <numerical expression>. If such a function is described, an error will be generated during execution.
- NG example C1\$=HEX\$(ASC("a"),1)

[Reference]

[BIN\\$](#), [STR\\$](#), [VAL](#)

INT

[Function]

Returns the largest integer that does not exceed the value of the numerical expression.

[Format]

<Numeric Variable>=INT (<Numerical Expression>)

[Reference Program]

```
10 M1=INT(3.3)           ' 3 is assigned to M1.
```

[Explanation]

- (1) Returns the largest integer that does not exceed the value of the numerical expression.
- (2) If the numerical expression evaluates to a positive value, the same number as FIX will be returned.
- (3) If the numerical expression evaluates to a negative value, then for instance FIX(-2.3) = -3.0 will be observed.

[Reference]

[CINT](#), [FIX](#)

INV

[Function]

Obtains the position data of the inverse matrix of the position variable. This is used to perform relative calculation of the positions.

[Format]

<Position Variables>=INV (<Position Variables>)

[Reference Program]

10 P1=INV(P2) ' P1 will contain the inverse matrix of P2.

[Explanation]

- (1) Obtains the position data of the inverse matrix of the position variable.
- (2) Joint variables cannot be used as the argument. When a joint variable is used, an error will be generated.
- (3) Since the return value is a position data, an error will be generated if a joint variable is used in the left-hand side.

JTOP

[Function]

Given joint data will be converted into position data.

[Format]

<Position Variables>=JTOP (<Joint Variables>)

[Reference Program]

10 P1=JTOP(J1) ' The position that expresses the J1 (joint type) position using the XYZ type will be assigned to P1.

[Explanation]

- (1) Converts the joint data into the position data.
- (2) Position variables cannot be used as the argument. When a position variable is used, an error will be generated.
- (3) Since the return value is a position data, an error will be generated if a joint variable is used in the left-hand side.

[Reference]

[PTOJ](#)

LEFT\$

[Function]

Obtains a string of the specified length starting from the left end.

[Format]

<Character String Variable>=LEFT\$ (<Character String>, <Numerical Expression>)

[Reference Program]

10 C1\$=LEFT\$("ABC",2) ' "AB" is assigned to C1\$.

[Explanation]

- (1) Obtains a string of the specified length starting from the left end.
- (2) An error will be generated if the value is a negative value or is longer than the string.
- (3) It is not possible to describe a function that contains an argument in <Character String> and <Numerical Expression>. If such a function is described, an error will be generated during execution.

[Reference]

MID\$, RIGHT\$

LEN

[Function]

Returns the length of the string.

[Format]

<Numeric Variable>=LEN (<Character String>)

[Reference Program]

10 M1=LEN("ABCDEFG") ' 7 is assigned to M1.

[Explanation]

- (1) Returns the length of the argument string.

[Reference]

LEFT\$, MID\$, RIGHT\$

LN

[Function]

Returns the natural logarithm. (Base e.)

[Format]

<Numeric Variable>=LN (<Numerical Expression>)
--

[Reference Program]

10 M1=LN(2) ' 0.693147 is assigned to M1.

[Explanation]

- (1) Returns the natural logarithm of the value of the numerical expression.
- (2) An error will be generated if the numerical expression evaluates to a zero or a negative value.

[Reference]

[EXP](#), [LOG](#)

LOG

[Function]

Returns the common logarithm. (Base 10.)

[Format]

<Numeric Variable>=LOG (<Numerical Expression>)

[Reference Program]

10 M1=LOG(2) ' 0.301030 is assigned to M1.

[Explanation]

- (1) Returns the common logarithm of the value of the numerical expression.
- (2) An error will be generated if the numerical expression evaluates to a zero or a negative value.

[Reference]

[EXP](#), [LN](#)

MAX

[Function]

Obtains the maximum value.

[Format]

<Numeric Variable>=MAX (<Numerical Expression 1>, <Numerical Expression 2>, ...)

[Reference Program]

10 M1=MAX(2,1,3,4,10,100) ' 100 is assigned to M1.

[Explanation]

- (1) Returns the maximum value among the arbitrary number of arguments.
- (2) The length of this instruction can be up to the number of characters allowed in a single line (123 characters).
- (3) It is not possible to describe a function that contains an argument in <Numerical Expression 1>, <Numerical Expression 2> and If such a function is described, an error will be generated during execution.

[Reference]

MIN

MID\$

[Function]

Returns a string of the specified length starting from the specified position of the string.

[Format]

<Character String Variable >=MID\$ (<Character String>, <Numerical Expression 2>, <Numerical Expression 3>)

[Reference Program]

10 C1\$=MID\$("ABCDEFGH",3,2) ' "CD" is assigned to C1\$.

[Explanation]

- (1) A string of the length specified by argument 3 is extracted from the string specified by the first argument starting from the position specified by argument 2 and returned.
- (2) An error will be generated if numerical expression 2 or 3 evaluates to a zero or a negative value.
- (3) An error is generated if the position of the last character to be extracted is larger than the length of the string specified by the first argument.
- (4) It is not possible to describe a function that contains an argument in <Character String>, <Numerical Expression 2> and <Numerical Expression 3>. If such a function is described, an error will be generated during execution.

[Reference]

LEFT\$, RIGHT\$, LEN

MIN

[Function]

Obtains the minimum value.

[Format]

<Numeric Variable>=MIN (<Numerical Expression 1>, <Numerical Expression 2>, ...)
--

[Reference Program]

```
10 M1=MIN(2,1,3,4,10,100)          ' 1 is assigned to M1.
```

[Explanation]

- (1) Returns the minimum value among the arbitrary number of arguments.
- (2) The length of this instruction can be up to the number of characters allowed in a single line (123 characters).
- (3) It is not possible to describe a function that contains an argument in <Numerical Expression 1>, <Numerical Expression 2> and If such a function is described, an error will be generated during execution.

[Reference]

[MAX](#)

MIRROR\$

[Function]

Inverts the bit string representing each character code of the string in binary, and obtains the character-coded string.

[Format]

<Character String Variable >=MIRROR\$ (<Character String Expression>)

[Reference Program]

```
10 C1$=MIRROR$("BJ")              ' "RB" is assigned to C1$.
                                   ' "BJ" =&H42,&H4A=&B01000010,&B01001010.
                                   ' Inverted =&H52,&H42=&B01010010,&B01000010.
                                   ' Output ="RB".
```

[Explanation]

- (1) Inverts the bit string representing each character code of the string in binary, and obtains the character-coded string.

MKI\$**[Function]**

Converts the value of an numerical expression (integer) into a two-byte string.

[Format]

<Character String Variable >=MKI\$ (<Numerical Expression>)

[Reference Program]

```
10 C1$=MKI$(123)
20 M1=CVI(C1$)           ' 123 is assigned to M1.
```

[Explanation]

- (1) Converts the lowest two bytes of the value of an numerical expression (integer) into a strings.
- (2) Use CVI to convert the string to a value.
- (3) This can be used to reduce the amount of communication data when transmitting numerical data to external devices.

[Reference]

[ASC](#), [CVI](#), [CVS](#), [CVD](#), [MKS\\$](#), [MKD\\$](#)

MKS\$**[Function]**

Converts the value of an numerical expression (single-precision real number) into a four-byte string.

[Format]

<Character String Variable >=MKS\$ (<Numerical Expression>)

[Reference Program]

```
10 C1$=MKS$(100.1)
20 M1=CVS(C1$)           ' 100.1 is assigned to M1.
```

[Explanation]

- (1) Converts the lowest four bytes of the value of an numerical expression (single-precision real number) into the strings.
- (2) Use CVS to convert the string to a value.
- (3) This can be used to reduce the amount of communication data when transmitting numerical data to external devices.

[Reference]

[ASC](#), [CVI](#), [CVS](#), [CVD](#), [MKI\\$](#), [MKD\\$](#)

MKD\$

[Function]

Converts the value of an numerical expression (double-precision real number) into a eight-byte string.

[Format]

<Character String Variable >=MKD\$ (<Numerical Expression>)

[Reference Program]

```
10 C1$=MKD$(10000.1)      '
20 M1=CVD(C1$)           ' 10000.1 is assigned to M1.
```

[Explanation]

- (1) Converts the lowest eight bytes of the value of an numerical expression (single-precision real number) into the strings.
- (2) Use CVD to convert the string to a value.
- (3) This can be used to reduce the amount of communication data when transmitting numerical data to external devices.

[Reference]

[ASC](#), [CVI](#), [CVS](#), [CVD](#), [MKI\\$](#), [MKI\\$](#)

POSCQ

[Function]

Checks whether the given position is within the movement range.

[Format]

<Numeric Variable>=POSCQ (<Position Variables>)

[Reference Program]

10 M1=POSCQ(P1) ' M1 will contain 1 if the position P1 is within the movement range.

[Explanation]

- (1) Check whether the position data given by an argument is within the movement range of the robot. Value 1 will be returned if it is within the movement range of the robot; value 0 will be returned if it is outside the movement range of the robot.
- (2) Arguments must give either the position data type or joint data type.

POSMID

[Function]

Obtain the middle position data when a linear interpolation is performed between two given points.

[Format]

<Position Variables>=POSMID (<Position Variables 1>, <Position Variables 2>,<Numerical Expression 1>, <Numerical Expression 2>)

[Reference Program]

10 P1=POSMID(P2,P3,0,0) ' The position data (including posture) of the middle point between P2 and P3 will be assigned to P1.

[Explanation]

- (1) Obtain the position data of the middle point when a linear interpolation is performed between two position data.
- (2) The first argument gives the starting point of the linear interpolation, while the second argument gives the end-point of the linear interpolation.
- (3) The third and fourth arguments correspond to the two TYPE arguments of the MVS command.
- (4) The arguments for the starting and end points must be positions that allow linear interpolation with the specified interpolation type. For instance, an error will be generated if the structure flags of the starting and end points are different.
- (5) It is not possible to describe a function that contains an argument in <Position Variables 1>, <Position Variables 2>,<Numerical Expression 1> and <Numerical Expression 2>. If such a function is described, an error will be generated during execution.

PTOJ

[Function]

Converts the given position data into a joint data.

[Format]

<Joint Variable>=PTOJ (<Position Variables>)
--

[Reference Program]

```
10 J1=PTOJ(P1)
```

' J1 will contain the value of P1 (XYZ position variable) that has been converted into joint data type.

[Explanation]

- (1) Converts the position data into the joint data.
- (2) Joint variables(J variable) cannot be used as the argument. When a joint variable is used, an error will be generated.
- (3) Since the return value is a joint data, an error will be generated if a position variable is used in the left-hand side.

[Reference]

JTOP

RAD

[Function]

Converts the unit of angle measurement from degrees (deg) into radians (rad).

[Format]

<Numeric Variable>=RAD (<Numerical Expression>)

[Reference Program]

```
10 P1=P_CURR  
20 P1.C=RAD(90)  
30 MOV P1
```

' Moves to P1, which is obtained by changing the C axis of the current position to 90 degrees.

[Explanation]

- (1) Converts the degree value of an numerical expression into radian value.
- (2) This can be used to assign values to the posture components (ABC) of a position variable or to execute trigonometric functions.

[Reference]

DEG

RDFL 1

[Function]

Returns the structure flag of the specified position using character data "R"/"L", "A"/"B", and "N"/"F".

[Format]

<code><Character String Variable >=RDFL1 (<Position Variables>, <Numerical Expression>)</code>
--

[Terminology]

<Position Variables>	Specifies the position variable from which the structure flag will be extracted.
<Numerical Expression>	Specifies which structure flag is to be extracted. 0 = "R" / "L", 1 = "A" / "B", 2 = "N" / "F"

[Reference Program]

```
10 P1=(100,0,100,180,0,180)(7,0)   ' Since the structure flag 7 (&B111) is RAN,
20 C1$=RDFL1(P1,1)                 ' C1$ will contain "A".
```

[Explanation]

- (1) Of the structure flags in the position data specified by argument 1, the flag specified by argument 2 will be extracted.
- (2) This function extracts information from the FL1 element of position data.
- (3) It is not possible to describe a function that contains an argument in <Position Variables> and <Numerical Expression>. If such a function is described, an error will be generated during execution.

[Reference]

[RDFL 2](#), [SETFL 1](#), [SETFL 2](#)

RDFL 2

[Function]

Returns the multiple rotation information of the specified joint axis.

[Format]

<code><Numeric Variable>=RDFL2 (<Position Variables>, <Numerical Expression>)</code>
--

[Terminology]

- | | |
|---|---|
| <code><Position Variables></code> | Specifies the position variable from which the multiple rotation information is to be extracted. |
| <code><Numerical Expression></code> | Specifies the value for the joint axis from which the multiple rotation information is to be extracted. (1 through 8) |

[Reference Program]

```
10 P1=(100,0,100,180,0,180)(7,&H00100000)      '
20 M1=RDFL2(P1,6)                                ' 1 is assigned to M1.
```

[Explanation]

- (1) Of the multiple rotation information of the position data specified by argument 1, the value for the joint axis specified by argument 2 is extracted.
- (2) The range of the return value is between -8 and 7.
- (3) This function extracts information from the FL2 element of position data.
- (4) Structure flag 2 (multiple rotation information) has a 32-bit structure, which contains 4 bits of information per axis for 8 axes.
- (5) When displaying in T/B and the multiple rotation is a negative value, value of -1 to -8 is converted into F to 8 (4-bit signed hexadecimal notation) and displayed.

<Sample display of multiple rotation information in TB> 87654321 axis <Relationship between display and number of multiple rotations per axis>

When multiple rotation of axis J6 is +1:	FL2=00100000		-2 -1 0 +1 +2	
When multiple rotation of axis J6 is -1:	FL2=00F00000		E F 0 1 2	

- (6) It is not possible to describe a function that contains an argument in `<Position Variables>` and `<Numerical Expression>`. If such a function is described, an error will be generated during execution.

[Reference]

[RDFL 1](#), [SETFL 1](#), [SETFL 2](#), [JRC](#)

RND

[Function]

Generates a random number.

[Format]

<Numeric Variable>=RND (<Numerical Expression>)

[Terminology]

<Numerical Expression> Specifies the initial value of random numbers. If this value is set to 0, subsequent random numbers will be generated without setting the initial value of random numbers.

<Numeric Variable> A value in the range of 0.0 to 1.0 will be returned.

[Reference Program]

```

10 DIM MRND(10)
20 C1=RIGHT$(C_TIME,2)      ' Initializes random numbers using the clock.
30 MRNDBS=CVI(C1))          ' in order to obtain different sequence of numbers.
40 MRND(1)=RND(MRNDBS)      ' Sets the initial value of random numbers and extracts the first random
                             number.
50 FOR M1=2 TO 10           ' Obtain other nine random numbers.
60  MRND(M1)=RND(0)
70 NEXT M1

```

[Explanation]

- (1) Initializes random numbers using the value provided by the argument and extracts a random number.
- (2) If the numerical expression provided as the argument evaluates to 0, initialization of random numbers will not take place and the next random number will be extracted.
- (3) When the same value is used to perform initialization of random numbers, identical random number sequence will be obtained.

RIGHT\$

[Function]

Obtains a string of the specified length starting from the right end.

[Format]

<Character String Variable >=RIGHT\$ (<Character String>, <Numerical Expression>)

[Reference Program]

```

10 C1$=RIGHT$("ABCDEFGH",3)      ' "EFG" is assigned to C1$.

```

[Explanation]

- (1) Obtains a string of the specified length starting from the right end.
- (2) An error will be generated if the value of the second argument is a negative value or is longer than the first string.
- (3) It is not possible to describe a function that contains an argument in <Character String> and <Numerical Expression>. If such a function is described, an error will be generated during execution.

[Reference]

LEFT\$, MID\$, LEN

SETFL 1

[Function]

Changes the structure flag of the specified position using a string (such as "RAN").

[Format]

<Position Variables>=SETFL1 (<Position Variables>, <Character String>)

[Terminology]

<Position Variables> Specifies the position variable whose structure flag is to be changed.

<Character String> Specifies the structure flag to be changed. Multiple flags can be specified.

"R" or "L": Right/Left setting.

"A" or "B": Above/Below setting.

"N" or "F": Nonflip/Flip setting.

[Reference Program]

```
10 MOV P1
```

```
20 P2=SETFL1(P1,"LBF")
```

30 MOV P2

[Explanation]

- (1) Returns the position data obtained by changing the structure flags in the position data specified by argument 1 to flag values specified by argument 2.
- (2) This function changes information from the FL1 element of position data. The content of the position data given by the argument will remain unchanged.
- (3) The structure flag will be specified starting from the last character in the string. Therefore, for instance, if the string "LR" is specified, the resulting structure flag will be "L".
- (4) If the flags are changed using a numerical value, set P1.FL1=7.
- (5) Structure flags may have different meanings depending on the robot model. For details, please refer to "ROBOT ARM SETUP & MAINTENANCE" for each robot.

The structure flag corresponds to 7 in the position constant (100, 0, 300, 180, 0, 180) (7, 0). The actual position is a bit pattern.

7 = & B 0 0 0 0 0 1 1 1

$$\begin{array}{l} \text{└─ } 1/0=N/F \\ \text{└─ } 1/0=A/B \\ \quad 1/0=B/L \end{array}$$

- (6) It is not possible to describe a function that contains an argument in `<Position Variables>` and `<Character String>`. If such a function is described, an error will be generated during execution.

[Reference]

RDFL 1, RDFL 2, SETFL 2

SETFL 2

[Function]

Changes the multiple rotation data of the specified position.

[Format]

<Position Variables>=SETFL2 (<Position Variables>, <Numerical Expression 1>, <Numerical Expression 2>)

[Terminology]

- <Position Variables> Specifies the position variable whose multiple rotation data are to be changed.
- <Numerical Expression 1> Specifies the axis number for which the multiple rotation data are to be changed. (1 through 8).
- <Numerical Expression 2> Specifies the multiple rotation data value to be changed (−8 through 7).

[Reference Program]

```

10 MOV P1
20 P2=SETFL2(P1,6,1)
30 MOV P2

```

[Explanation]

- (1) Returns the position data obtained by changing the position data's multiple rotation information of the joint axis specified by numerical expression 1 to the value specified by numerical expression 2.
- (2) This function changes information from the FL2 element of position data.
- (3) The content of the position of position variables given by the argument (X, Y, Z, A, B, C, and FL1) will remain unchanged.

Value of multiple rotation data							
	−900	−540	−180	0	180	540	900
Angle of each axis							
Value of multiple rotation data	...	−2 (E)	−1 (F)	0	1	2	...

- (4) It is not possible to describe a function that contains an argument in <Position Variables>, <Numerical Expression 1> and <Numerical Expression 2>. If such a function is described, an error will be generated during execution.

[Reference]

[RDFL 1](#), [RDFL 2](#), [SETFL 1](#)

SGN

[Function]

Checks the sign of the numerical expression.

[Format]

<Numeric Variable>=SGN (<Numerical Expression>)

[Reference Program]

```
10 M1=-12
20 M2=SGN(M1)      ' -1 is assigned to M2.
```

[Explanation]

(1) Checks the sign of the numerical expression and returns the following value.

Positive value	1
0	0
Negative value	-1

SIN

[Function]

Calculates the sine.

[Format]

<Numeric Variable>=SIN (<Numerical Expression>)

[Reference Program]

```
10 M1=SIN(RAD(60))      ' 0.866025 is assigned to M1.
```

[Explanation]

- (1) Calculates the sine to which the given numerical expression evaluates.
- (2) The range of values will be the entire range that numerical values can take.
- (3) The range of the return value will be from -1 to 1.
- (4) The unit of arguments is in radians.

[Reference]

[COS](#), [TAN](#), [ATN/ATN2](#)

SQR

[Function]

Calculates the square root of an numerical expression value.

[Format]

<Numeric Variable>=SQR (<Numerical Expression>)

[Reference Program]

```
10 M1=SQR(2)           ' 1.414214 is assigned to M1.
```

[Explanation]

- (1) Calculates the square root of the value to which the given numerical expression evaluates.
- (2) An error will be generated if the numerical expression given by the argument evaluates to a negative value.

STRPOS

[Function]

Searches for a specified string in a string.

[Format]

<Numeric Variable>=STRPOS (<Character String 1>, <Character String 2>)
--

[Reference Program]

```
10 M1=STRPOS("ABCDEFGH","DEF")   ' 4 is assigned to M1.
```

[Explanation]

- (1) Returns the position of the first occurrence of the string specified by argument 2 from the string specified by argument 1.
- (2) An error will be generated if the length of the argument 2 is 0.
- (3) For instance, if argument 1 is "ABCDEFGH" and argument 2 is "DEF", 4 will be returned.
- (4) If the search string could not be found, 0 will be returned.
- (5) It is not possible to describe a function that contains an argument in <Character String 1> and <Character String 2>. If such a function is described, an error will be generated during execution.

STR\$

[Function]

Converts the value of the numerical expression into a decimal string.

[Format]

<Character String Variable >=STR\$ (<Numerical Expression>)

[Reference Program]

10 C1\$=STR\$(123) ' "123" is assigned to C1\$.

[Explanation]

- (1) Converts the value of the numerical expression into a decimal string.
- (2) VAL is a command that performs this procedure in reverse.

[Reference]

[BIN\\$](#), [HEX\\$](#), [VAL](#)

TAN

[Function]

Calculates the tangent.

[Format]

<Numeric Variable>=TAN (<Numerical Expression>)

[Reference Program]

```
10 M1=TAN(RAD(60))           ' 1.732051 is assigned to M1.
```

[Explanation]

- (1) Returns the tangent of the value to which the numerical expression evaluates.
- (2) The range of arguments will be the entire range of values that are allowed.
- (3) The range of return values will be the entire range that numerical values can take.
- (4) The unit of arguments is in radians.

[Reference]

[SIN](#), [COS](#), [ATN/ATN2](#)

VAL

[Function]

Converts the value in the string into a numerical value.

[Format]

<Numeric Variable>=VAL (<Character String Expression>)

[Reference Program]

```
10 M1=VAL("15")
20 M2=VAL("&B1111")
30 M3=VAL("&HF")
```

[Explanation]

- (1) Converts the given character string expression string into a numerical value.
- (2) Binary (&B), decimal, and hexadecimal (&H) notations can be used for the string.
- (3) In the example above, M1, M2 and M3 evaluate to the same value (15).

[Reference]

[BIN\\$](#), [HEX\\$](#), [STR\\$](#)

ZONE

[Function]

Checks if the specified position is within the specified area (a rectangular solid defined by two points).

[Format]

$\langle \text{Numeric Variable} \rangle = \text{ZONE} (\langle \text{Position 1} \rangle, \langle \text{Position 2} \rangle, \langle \text{Position 3} \rangle)$

[Terminology]

- $\langle \text{Position 1} \rangle$ The position to be checked.
- $\langle \text{Position 2} \rangle$ The position of the first point that specifies the area.
- $\langle \text{Position 3} \rangle$ The position of the second point that specifies the area. (diagonal point)

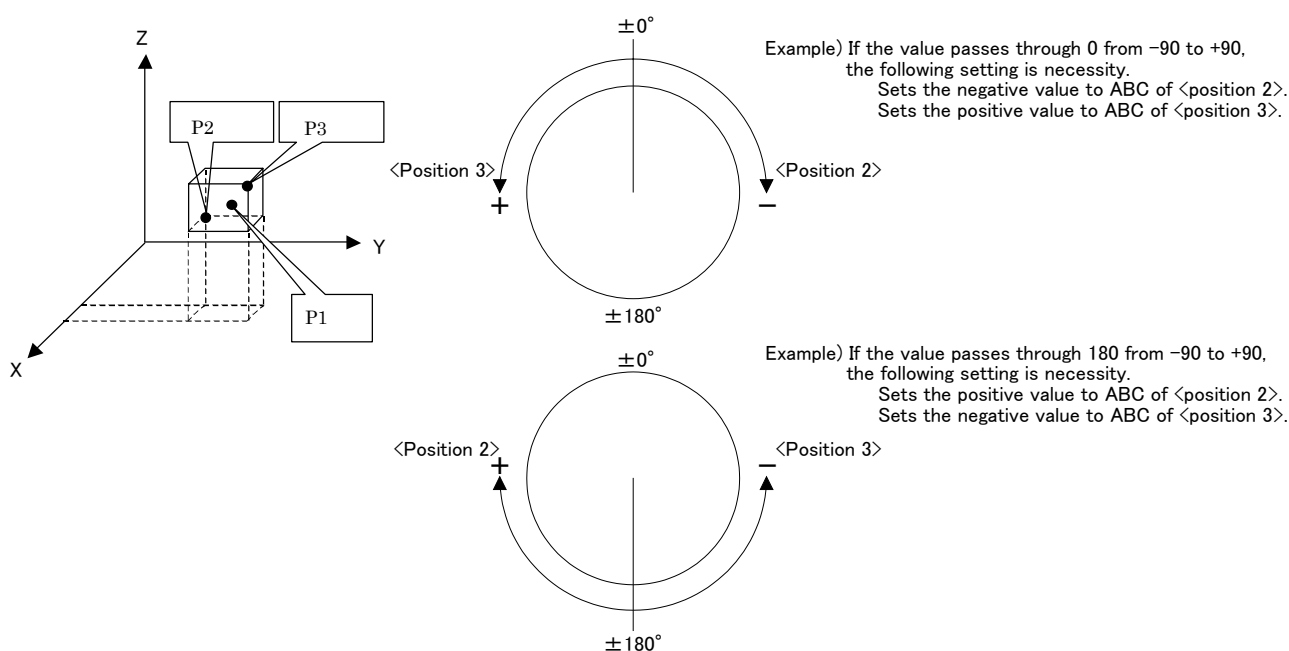
Positions 1 to 3 set the XYZ coordinates variable system (P variable X, Y, Z, A, B, and C).

[Reference Program]

```
10 M1=ZONE(P1,P2,P3)
20 IF M1=1 THEN MOV P_SAFE ELSE END
```

[Explanation]

- (1) This will check if position 1 is inside the rectangular solid defined by the two points, position 2 and position 3. (The two points will become the diagonal points of the rectangular solid.) If the point is inside the rectangular solid, 1 is returned; otherwise, 0 is returned.
- (2) To check whether position 1 is inside that area, each element of position 1 (X, Y, Z, A, B, and C) will be checked if it is between the values for position 2 and position 3.
- (3) As for the posture angles (A, B, and C), they are checked by rotating in the positive direction from the angle in position 2 to position 3 and by seeing if the target value is inside the swiped range.
Example) If P2.A is -100 and P3.A is $+100$, if P1.A is 50 , the value is within the range. Similar checking will be performed for B and C axes. (Refer to diagram below.)
- (4) For components that are not checked or do not exist, if the unit is in degrees, position 2 will be set to -360 and position 3 will be set to 360 . If the unit is in millimeters, position 2 will be set to -10000 and position 3 will be set to 10000 .
- (5) It is not possible to describe a function that contains an argument in $\langle \text{Position 1} \rangle$, $\langle \text{Position 2} \rangle$ and $\langle \text{Position 3} \rangle$. If such a function is described, an error will be generated during execution.



ZONE 2

[Function]

Checks if the specified position is within the specified area (Cylindrical area defined by two points).

[Format]

This function is available for controller software version G5 or later

<Numeric Variable>=ZONE2 (<Position 1>, <Position 2>, <Position 3>, <Numerical Expression>)

[Terminology]

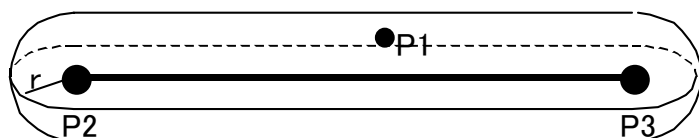
<Position 1>	The position to be checked.
<Position 2>	The position of the first point that specifies the area.
<Position 3>	The position of the second point that specifies the area.
<Numerical Expression>	Radius of the hemisphere on both ends.

[Reference Program]

```
10 M1=ZONE2(P1,P2,P3,50)
20 IF M1=1 THEN MOV P_SAFE ELSE END
```

[Explanation]

- (1) This will check if position 1 is inside the cylindrical area (Refer to diagram below) defined by the two points, position 2 and position 3, and the radius represented by the numerical expression. If the point is inside the space, 1 is returned; otherwise, 0 is returned.
- (2) This function checks whether the check position (X, Y, and Z coordinates) is within the specified area, but does not take the posture components into consideration.



- (3) It is not possible to describe a function that contains an argument in <Position 1>, <Position 2>, <Position 3> and <Numerical Expression>. If such a function is described, an error will be generated during execution.

5 Functions set with parameters

This controller has various parameters listed in [Table 5-1](#). It is possible to change various functions and default settings by changing the parameter settings.

For the parameters regarding dedicated I/O signals, refer to [Page 308](#), “6.3 Dedicated input/output”. After changing the parameters, make sure to turn the robot controller’s power OFF and then turn ON.

Parameter settings will not be in effect until the power is turned on again. For detailed operating method for parameters, refer to [Page 51](#), “(1) Setting the parameters”.



CAUTION

When changing parameters, check thoroughly the function and setting values first. Otherwise, the robot may move unexpectedly, which could result in personal injury or property damage.

Table 5-1 : List parameter

Parameter	Parameter name	No. of arrays No. of characters	Details explanation	Factory setting
Standard tool coordinates Refer to “5.1 Standard Tool Coordinates”.	MEXTL	Real value 6	Initial values will be set for the hand tip (control point) and the mechanical interface (hand mounting surface). The factory default setting is set to the mechanical interface as the control point. Change this value if a hand is installed and the control point needs to be changed to the hand tip. (This will allow posture control at the hand tip for XYZ or tool jog operation.) (X, Y, Z, A, B, C) Unit: mm, ABC deg.	(X,Y,Z,A,B,C) = 0.0,0.0,0.0,0.0,0.0,0.0
Tool base coordinates Refer to “5.1 Standard Tool Coordinates”	MEXBS	Real value 6	Sets the positional relationship between the base coordinate system and the robot coordinate system. The factory default setting is set so that the base coordinate system and the robot coordinate system are identical. This will be set when the coordinate system for the whole device is changed. This parameter does not need to be changed very often. This is set when the coordinate system for the whole device is to be identical. (X, Y, Z, A, B, C) Unit: mm, ABC deg.	(X,Y,Z,A,B,C) = 0.0,0.0,0.0,0.0,0.0,0.0
User area Refer to “5.3 About user-defined area”			Designate an area (rectangle defined with two XYZ coordinate points). A signal will be output if that area is outside the movement area (interference), or if the robot's current position is within that area. Up to eight limits can be set using the following four types of parameters. For components that are not checked or do not exist, if the unit is in degrees, set AREA*P1 to -360 and AREA*P2 to 360. If the unit is in mm, set AREA*P1 to -10000 and AREA*P2 to 10000 as the corresponding component.	
	AREA*P1 * is 1 to 8	Real value 8	Designate the first point of the area. (X, Y, Z, A, B, C) Unit: mm, ABC deg.	(X,Y,Z,A,B,C)= 0.0,0.0,0.0,-360.0,-360.0,-360.0
	AREA*P2 * is 1 to 8	Real value 8	Designate the second point of the area. (X, Y, Z, A, B, C) Unit: mm, ABC deg.	(X,Y,Z,A,B,C)= 0.0,0.0,0.0,+360.0,+360.0,+360.0
	AREA*ME * is 1 to 8	Integer 1	Designate the mechanism No. for which the user-defined area is to be validated. The mechanism No. is 1 to 4, but normally 1 is set.	0
	AREA*AT * is 1 to 8	Integer 1	Specify the behavior of the robot when the robot enters the user definition area. (Invalid / In-zone signal output / Error output = 0/1/2) Invalid : This function will be invalid. In-zone signal output : The dedicated output signal USRAREA will turn ON. Error output : An error is generated. Posture data will be ignored.	0 (Invalid)
	USRAREA		Defines the number of the signal that outputs the status. Refer to Page 308 , “6.3 Dedicated input/output”	-1,-1
Joint movement range	MEJAR	Real value 16	Set the overrun limit value for the joint coordinate system. Sets the movement range for each axis. Expanding of the movement range is not recommended, since there is possibility that the robot may strike the mechanical stopper. Set the minus and plus directions. (-J1,+J1,-J2,+J2,.....-J8,+J8) Unit : deg	Setting value for each mechanism
XYZ movement range	MEPAR	Real value 6	Set the overrun limit value for the XYZ coordinate system. The movement range of the robot will be limited based on XYZ coordinate system. This can be used to prevent the robot from striking peripheral devices during manual operation when the robot is installed within the device. Set the minus and plus directions. (-X,+X,-Y,+Y,-Z,+Z) Unit : mm	(-X,+X,-Y,+Y,-Z,+Z)= -10000,10000, -10000,10000, -10000,10000

Parameter	Parameter name	No. of arrays No. of characters	Details explanation	Factory setting
Free plane limit Refer to "5.4Free plane limit"			This is the overrun limit set on a free plane. Create a plane with three coordinate points, and set the area that does not include the origin as the outside-movement area. Up to eight limits can be set using the following three types of parameters.	
	SFC*P * is 1 to 8	Real value 9	Designate three points for creating the plane. X1,Y1,Z1 : Origin position in the plane X2,Y2,Z2 : Position on the X-axis in the plane X3,Y3,Z3 : Position in the positive Y direction of the X-Y plane in the plane	(X1,Y1,Z1, X2,Y2,Z2, X3,Y3,Z3)=0.0,0.0,0. 0.0,0.0,0.0,0.0,0.0, 0.0
	SFC*ME * is 1 to 8	Integer 1	Designate the mechanism No. for which the free plane limit is to be validated. The mechanism No. is 1 to 3, but normally 1 is set.	0
	SFC*AT * is 1 to 8	Integer 1	Designate the valid/Invalid of the set free plane limit. (Valid/Invalid = 1/0)	0 (Invalid)
Hand initial state Refer to "5.8About default hand status"	HANDINIT	Integer 8	Set the pneumatic hand I/F output for when the power is turned ON.	1,0,1,0,1,0,1,0
Hand type Refer to "5.7About the hand type"	HANDTYPE	Character string 8	Set the single/double solenoid hand type and output signal No. (D : double solenoid, S : single solenoid). Set the signal No. after the hand type. When D900 is set, the signal No. 900 and 901 will be output. In the case of D (double solenoid), please configure the setting so that the signals do not overlap	D900,D902,D904,D906,...
Program selection save	SLOTON	Integer 1	This parameter specifies whether or not to store the program name in the SLT1 parameter at program selection, as well as whether or not to maintain the program selection status at the end of cycle operation. (1) Enabling program name storage at program selection (Bit 0, enable/disable storage = 1/0) Enable storage: The name of the current program is stored in the SLT1 parameter at program selection for slot 1. Moreover, the program specified in the SLT1 parameter is selected when the power supply is turned on. Disable storage: The name of the current program is not stored in SLT1 parameter at program selection for slot 1. In the same way as when the storage is enabled, the program specified in the SLT1 parameter is selected when the power supply is turned on. (2) Maintaining program at the end of cycle operation (Bit 1, maintain/do not maintain = 1/0) Maintain: The status of program selection is maintained at the end of cycle operation. The parameter value does not become P.0000. Do not maintain: The status of program selection is not maintained at the end of cycle operation. The parameter value becomes P.0000. Setting values and operations 0: Disable storage, do not maintain 1: Enable storage, do not maintain (initial value) 2: Disable storage, maintain 3: Enable storage, maintain	1 (Valid)
Buzzer ON/OFF	BZR	Integer 1	Specifies the on/off of the buzzer sound that can be heard when an error occurs in the robot controller. (OFF/ON = 0/ 1)	1(ON)
Jog setting	JOGJSP	Real value 3	Designate the joint jog and step operation speed. (Inching H, inching L, maximum override.) Inching H: Feed amount when jog speed is set to High Unit: deg. Inching L: Feed amount when jog speed is set to Low Unit: deg. Maximum override: Operates at OP override I maximum override.	Setting value for each mechanism
	JOGPSP	Real value 3	Designate the XYZ jog and step operation speed. (Inching H, inching L, maximum override.) Inching H: Feed amount when jog speed is set to High Unit: deg. Inching L: Feed amount when jog speed is set to Low Unit: deg. Maximum override: Operates at OP override I maximum override. Operation exceeding the maximum speed 250 mm/s cannot be performed.	Setting value for each mechanism

Parameter	Parameter name	No. of arrays No. of characters	Details explanation	Factory setting
Jog speed limit value	JOGSPMX	Real value 1	Limit the robot movement speed during the teach mode. Unit: mm/s Even if a value larger than 250 is set, the maximum value will be limited to 250.	250.0
Automatic return setting after jog feed at pause Refer to "5.5Automatic return setting after jog feed at pause" *The "2: Return by XYZ interpolation" is available for controller software version H4 or later.	RETPATH	Integer 1	While running a program, if the program is paused by a stop and then the robot is moved by a jog feed for instance, at the time of restart, this setting makes the robot return to the position at which the program was halted before continuing. If this function is disabled, movement instructions will be carried out from the current position until the next point. The robot does not return to the position where the program was halted. 0: Invalid . 1: Return by JOINT interpolation. 2: Return by XYZ interpolation. Note) When returning by XYZ interpolation, carry out shorter circuit movement by 3 axis XYZ interpolation. Note) In the circle interpolation (MVC, MVR, MVR2, MVR3) command, this function is valid for H4 or later. Moreover, in the circle interpolation command and the MVA command, even if set up with 0, the operation is same as 1.	The RH-1000G and 1500G series and RH-15UHC are 2.. The type other than the above are 1.
Stop input B contact designation	INB	Integer 1	Change the dedicated input (stop) between the A contact and B contact. (A contact/B contact = 0/1)	0 (A contact)
Evasion point position	JSAFE	Real value 8	Specifies the evasion point position. Robot moves to the evasion point position if the robot program executes MOV P_SAFE instruction or receives input of the SAFEPOS signal, which is an external signal. (J1,J2,J3,J4,J5,J6,J7,J8) Unit : deg	Not allowed
User-designated origin	USERORG	Real value 8	Designate the user-designated origin position. This normally does not need to be set. (J1,J2,J3,J4,J5,J6,J7,J8) Unit : deg	(J1,J2,J3,J4,J5,J6,J7,J8)= 0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0
Slot table (Set during multi-task operation.)			Operation conditions for each task slot is set during multitask operations. These are set when the program is reset.	
	SLT* * is 1 to 32	Character string 4	Designate the 「program name」, 「operation mode」, 「starting conditions」, 「order of priority」 . Program name: Selected program name. Use uppercase letters when using alphabet. Lowercase characters are not recognized. Operation mode: Continuous/1 cycle = REP/CYC REP : The program will be executed repeatedly. CYC: The program ends after one cycle is completed. (The program does not end if it runs in an endless loop created by a GOTO instruction.) Starting conditions: Normal/Error/Always =START / ERROR / ALWAYS START : This is executed by the START button on the operation panel or by the start signal. ALWAYS: This is executed immediately after the controller's power is turned on. This program does not affect the status such as startup. To edit a program whose attribute is set to ALWAYS, first cancel the ALWAYS attribute. A program with the ALWAYS attribute is being executed continuously and therefore cannot be edited. Change ALWAYS to START and turn on the controller's power again to stop the constant execution. ERROR: This is executed when an error is generated. This program does not affect the status such as startup. Programs with ALWAYS or ERROR set as the starting condition cannot execute the following movement instructions. An error will be generated if any of them is executed. MOV,MVS,MVR,MVR2,MVR3,MVC,MVA, DRIVE,GETM,RELM,JRC Order of priority: 1 to 31 (31 is the maximum) This value shows the number of lines to be executed at a time. This has the same meaning as the number of lines in the PRIORITY instruction. For instance, when two slots are used during execution, if SLT1 is set to 1 and SLT2 is set to 2, after one line of program in SLT1 is executed, two lines of program in SLT2 is executed. Therefore, more SLT2 programs will be executed and as a result, priority of SLT2 is higher.	"" ,REP,START,1

Parameter	Parameter name	No. of arrays No. of characters	Details explanation	Factory setting
Setting that allows the execution of X** instructions and SERVO instruction in an ALWAYS program. Refer to "5.6Automatic execution of program at power up"	ALWENA	Integer 1	XRUN, XLOAD, XSTP, XRST, SERVO and RESET ERR instructions become available in a program whose SLT* parameter is set to "constantly execute" (startup condition is set to ALWAYS). Enable/Disable = 7/0	0 (Not allowed)
User base program Refer to "4.3.24User-defined external variables"	PRGUSR	Character string 1	User base program is a program that is set when user-defined external variables are to be used. In case of DEF number, variable declaration instructions such as INTE and DIM are described. If an array variable is declared in the user base program using the DIM instruction, the same variable name must be redefined using the DIM instruction in the program that uses the user base program. Variables need not be redefined if the variable is not an array.	"" (Non)
No. of multitasks	TASKMAX	Integer 1	Designate the number of programs to be executed simultaneously.	8
Communication setting Refer to "5.10About the communication setting"	COMDEV	Character string 8	Communication environment is set for RS-232C in the front of the robot controller. However, since this is used by the PC support software, this normally does not need to be changed. When connecting vision sensors, etc., use of optional expansion serial interface is recommended. This configures which lines will be assigned to COM1 and COM2 when using communication lines in the OPEN instruction in MELFA BASIC IV. This parameter must be set if data link (used by the OPEN instruction) is to be performed. This parameter specifies the device that corresponds to COMn specified in the OPEN statement in the program (n is between 1 and 8). Parameters are starting from the left COM1, COM2, ... , COM8 in that order. To use expansion serial interface as COM, If CH1 is installed in option slot 1, "OPT11" should be entered as the value for the parameter. If CH2 (RS232) is installed in slot 1, "OPT12" should be entered as the value for the parameter. If CH2 (RS422) is installed in slot 1, "OPT13" should be entered as the value for the parameter. If CH1 is installed in slot 2, "OPT21" should be entered as the value for the parameter. If CH2 (RS232) is installed in slot 2, "OPT22" should be entered as the value for the parameter. If CH2 (RS422) is installed in slot 2, "OPT23" should be entered as the value for the parameter. For instance, if CH1 is used in slot 1 and is to be assigned to COM2, use "RS232", "OPT11", . . . , . If CH2 is used in slot 2 and is to be assigned to COM3, use "RS232", . , "OPT22", . . . , . If Ethernet interface is to be used as COM and is installed in option slot 1 with NETPORT1, enter "OPT11" for the parameter value. NETPORT2, enter "OPT12" for the parameter value. NETPORT3, enter "OPT13" for the parameter value. NETPORT4, enter "OPT14" for the parameter value. NETPORT5, enter "OPT15" for the parameter value. NETPORT6, enter "OPT16" for the parameter value. NETPORT7, enter "OPT17" for the parameter value. NETPORT8, enter "OPT18" for the parameter value. NETPORT9, enter "OPT19" for the parameter value. COM1 is already assigned the standard RS232C in the front of the controller. The following restrictions apply when optional cards are installed. Option slot 1: Additional axis, expansion serial interface, Ethernet Option slot 2: Additional axis, expansion serial interface, CC-Link Option slot 3: Additional axis Additional axis cards are for CR1-only cases. For optional expansion serial interface, refer to "Expansion Serial Interface Instruction Manual". For optional expansion serial interface, refer to "Ethernet Interface Instruction Manual".	"RS232", . . . , .

Parameter	Parameter name	No. of arrays No. of characters	Details explanation	Factory setting
	CBAU232	Integer 1	Baud rate (9600,19200) For optional expansion serial interface, refer to "Expansion Serial Interface Instruction Manual".	9600
	CPRTY232	Integer 1	Parity bit(0: None, 1: Odd, 2: Even) For optional expansion serial interface, refer to "Expansion Serial Interface Instruction Manual".	2 (Even)
	CSTOP232	Integer 1	Stop bit (1,2) For optional expansion serial interface, refer to "Expansion Serial Interface Instruction Manual".	2 (Stop bit)
	CTERM232	Integer 1	End code (0:CR 1 :CR+LF) For optional expansion serial interface, refer to "Expansion Serial Interface Instruction Manual".	0 (CR)
	CPRC232	Integer 1	Communication method(protocol) 0: For support software (Non-procedure) If data link is to be performed (OPEN, PRINT and INPUT instructions are executed from the program) under this setting, the external device must attach three characters "PRN" at the beginning when transmitting data. 1: For support software (With procedure) PC side must also be changed. 2: For data link with the robot program (used when communicating with vision sensors, etc.) Be advised that under this setting, connection with the support software cannot be made. Use the optional expansion serial interface. For optional expansion serial interface, refer to "Expansion Serial Interface Instruction Manual".	0
Output signal reset pattern Refer to "5.9>About the output signal reset pattern"			Set the operation to be taken when the general-purpose output signal for the CLR command or dedicated input (OUTRESET) is reset. Set with a 32-bit unit for each signal using the following parameters.(OFF/ON/hold=0/1/*)	
	ORST0	Character string 4	Set the signal No. 0 to 31.	00000000,00000000 0,00000000,00000000
	ORST32 : ORST224	Character string 4	Set the signal No. 32 to 63. : Set the signal No. 224 to 255.	00000000,00000000 0,00000000,00000000 : :
Output reset at reset	SLRSTIO	Integer 1	Designate the function to carry out general-purpose output signal reset when the program is reset. (Invalid/Valid = 0/1)	0 (Invalid)
Program reset operation rights	PRSTENA	Integer 1	Whether or not the operation right is required for program reset operation (Required/Not required = 0/1) If operation rights are abandoned, program can be reset from anywhere. However, this is not possible under the teaching mode for safety reasons.	0 (Required)
Program reset when key switch is switched	MDRST	Integer 1	Program paused status is canceled when the key switch is operated. (Invalid/Valid = 0/1)	0(Invalid)
Program selection rights setting	OPPSL	Integer 1	Specifies the program selection operation rights when the key switch of the operation panel is in AUTO (OP) mode. (External/OP = 0/1)	1 (OP)
	RMTPSL	Integer 1	Designate the program selection operation rights for the automatic (Ext) mode. (External/OP = 0/1)	0 (External)
TB override operation rights	OVRDTB	Integer 1	Specifies whether the operation rights are required when changing override from T/B. (Not required/Required = 0/1)	0(Not required)
Override change operation rights	OVRDENA	Integer 1	Specifies whether operation rights is required to change override. (Not required/Required = 0/1) If this is set to "Not required," override change can be set from anywhere.	0 (Required)

Parameter	Parameter name	No. of arrays No. of characters	Details explanation	Factory setting
Maximum acceleration/deceleration setting			Set the hand conditions and work conditions for when OADL ON is set with the program. Up to eight conditions can be set. The condition combination is selected with the LOADSET command.	
Refer to "5.11About the optimum acceleration/deceleration setting"	HNDDAT* * is 1 to 8	Real value 7	Set the hand conditions. (Designate with the tool coordinate system.) (Weight, size X, size Y, Size Z, center of gravity X, center of gravity Y, center of gravity Z) Unit: Kg, mm	Standard load .0,0,0,0,0,0,0,0,0,0,0
	WRKDAT* * is 1 to 8	Real value 7	Set the work conditions. (Designate with the tool coordinate system.) (Weight, size X, size Y, Size Z, center of gravity X, center of gravity Y, center of gravity Z) Unit: Kg, mm	0,0,0,0,0,0,0,0,0,0,0
	HNDHOLD* * is 1 to 8	Integer 2	Set whether to grasp or not grasp the workpiece when HOPEN (or HCLOSE) is executed. (Setting for OPEN, setting for CLOSE) (No grasp/grasp = 0/1)	0,1
	ACCMODE	Integer 1	Sets the initial value and enables/disables the optimum acceleration/deceleration mode. (Invalid/Valid = 0/1) (* This can be used in software version G1 or later.)	Note1)
Reads the program number from the numerical input when the start signal is input.	PST	Integer 1	To select a program from the normal external input signal, set the numerical input signal (IODETA) to the program number, establish the number with the program select signal (PRGSEL), and start with the START signal. If this function is enabled, the program select signal becomes unnecessary, and when the START signal turns ON, the program number is read from the numerical input signal (IODETA). (Function invalid/Valid = 0/1)	0 (Invalid)
Continue function	CTN	Integer 1	For only the program execution slot 1, the state when the power is turned OFF is held, and the operation can be continued from the saved state when the power is turned ON next. The saved data is the program execution environment (override, execution step line, program variables, etc.), and the output signal state. When this function is valid, if the robot is operated when the power is turned OFF, the robot will start in the standby state when the power is turned ON next. To continue operation, turn the servo power ON, and input start. (Function invalid/Valid = 0/1) <Precautions> (1) This function stores the status of execution memory at the time of power off using a part of program memory area. Therefore, this function cannot be used unless there is free space of at least 100K bytes in the program memory area. Please follow the precautions listed below when using this function. The program may become destroyed if the precautions are not followed. 1) Back up all programs in a PC and delete all programs in the controller. 2) Set the "CTN" parameter to 1 and turn the power ON again. 3) Reload only the necessary programs onto the controller. If there is not sufficient empty space at this point, this function cannot be used. If there is not enough space, revert the parameters. (2) Do not change the parameters while the program is still left on, because the program will be destroyed. Although there are 210K bytes of standard memory, available space will be 110K bytes when this function is enabled. Reduction of 1100 points in position count conversion Reduction of 2200 steps in step count conversion With the standard specifications, the memory will drop by 56%. Number of points = 2500 → 1400 points/Prg. Number of steps = 5000 → 28000 steps/Prg. (3)As for robots with axes without brakes (J4 and J6 axes of RV-1A/2AJ, or J4 axis of RH-5AH), the arm may lower due to gravitational weight or rotate itself when the power is turned off. Thus, extra care is necessary when using this function. (4)Program that can continue using the Continuity function is the one loaded in task slot 1. Programs in task slot 2 or subsequent slots will not continue but will restart in program reset state. (5)The following parameters cannot be changed after this function is enabled. Be sure to change them, if necessary, prior to enabling this function. SLTn, SLOTON, TASKMAX. (6)If parameters in the slot table (SLT*) are changed after enabling this function, the changes are not reflected in the slot table. Disable the continue function once, turn the power supply off and then on, and then change parameters in the slot table.	0 (Invalid)

Parameter	Parameter name	No. of arrays No. of characters	Details explanation	Factory setting
JRC command (Multiple rotation function of axes)	JRCQTT	Integer 1	Set the execution status of the JRC instruction.	0 (Execution invalid)
*This parameter can be used for controller software version E4 or later.	JRCQTT	Real value 8	JSet the change amount to increment or decrement with the JRC command in the order of J1, J2, J3 to J8 axes from the head element. The setting is valid only for the user-defined axis, so the J7 and J8 axes will be valid for the robot's additional axis, and a random axis for the mechanism's additional axis. The unit relies on the parameter AXUNT.	JRC execution valid robot 0,0,0,0,0,360,0,0 JRC execution invalid robot 0,0,0,0,0,0,0
	JRCORG	Real value 8	Set the origin coordinate value for executing the JRC O command and setting the origin. This setting is valid only for the user-defined axis. The unit relies on the parameter AXUNT.	0,0,0,0,0,0,0
	JRCORG	Real value 8	Set the origin coordinate value for executing the JRC O command and setting the origin. This setting is valid only for the user-defined axis. The unit relies on the parameter AXUNT.	0,0,0,0,0,0,0
Setting of additional axis	AXUNT	Integer 16	Set the unit system for the additional axis. Angle(degree)/Length(mm) = 0/1	0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0
Speed setting during mode change	OVRDMD	Integer 2	Override is set automatically when the mode is changed. First element.....override value when the mode is automatically changed from teaching mode Second element.....Override value when the mode is changed from Auto to Teaching. Current status is maintained if changed to 0.	0,0
User error setting	UER1 to UER20	Integer 1, Character string 3	Sets the message, cause, and method of recovery for errors from the ERROR instruction. Maximum of 20 user errors can be set. First element ... error number to set (9000 to 9299 is the available range). The default value 9900 is not available. Change the value before proceeding. Second element...Error message Third element...Cause Fourth element...Method of recovery If a space character is included in the message, enclose the entire message in double quotation marks (""). Example) 9000,"Time Out","No Signal","Check Button"	9900,"message","cause","treat"
Unit setting for the rotational element of position data	PRGMDEG	Integer 1	Specifies the unit used for describing the rotational element of position data in the robot program. 0 : RAD 1 : DEG Example) M1=P1.A (Unit for this case is specified.) (Default unit for referencing data components is radian.) The default rotational element for the position constant (P1=(100, 0, 300, 0, 180, 0, 180) (7, 0)) is DEG. This parameter is irrelevant.	0 (RAD)
Dedicated I/O signal			For the parameters of the dedicated I/O signal, refer to Page 308 , "6.3 Dedicated input/output".	
Select the function of singular point adjacent alarm Refer to Page 295 , "5.12 About the singular point adjacent alarm"	MESNGLSW	Integer 1	Designate the valid/invalid of the singular point adjacent alarm. (Invalid/Valid = 0/1) When this parameter is set up "VALID", this warning sound is buzzing even if parameter: BZR (buzzer ON/OFF) is set up "OFF".	1 (Valid)
Robot language setting	RLING	Integer 1	Select the robot language 1 : MELFA-BASIC IV 0 : MOVEMASTER COMMAND Note) This function is only available for certain types of robots, such as RV-1A. Please verify that the type of robot that you are using is listed in the "Command List" of "Separate Volume: Standard Specification" before using this instruction.	1
Display language Note2)	LNG	Character string 1	Set up the display language. "JPN" : Japanese "ENG" : English The following language is changed. (1)The display LCD of teaching pendant. (2) Personal computer support software. *alarm message of the robot. *Parameter explanation list. (3)Alarm message that read from the robot with external communication. (Standard RS232C, Extended serial I/F, Ethernet I/F)	The "JPN" is Japanese specification. The "ENG" is English specification.

Parameter	Parameter name	No. of arrays No. of characters	Details explanation	Factory setting
This parameter switches the access target of a program. Refer to Page 296, "5.13 ROM Operation Function". *This parameter can be used for controller software version H7 or later.	ROMDRV	Integer 1	The access target of a program can be switched between RAM and ROM. 0: RAM mode. (Standard mode.) 1: ROM mode. (Special mode.)	0 (RAM mode.)
Copy the information on the RAM to the ROM. Refer to Page 296, "5.13 ROM Operation Function". *This parameter can be used for controller software version H7 or later.	BACKUP	Character string 1	Copy the program, the parameter, the common variable, and the error log to the ROM from the RAM. Do not change this parameter.	SRAM→FLROM (unchangeable)
Restore the information on the ROM to the RAM. Refer to Page 296, "5.13 ROM Operation Function". *This parameter can be used for controller software version H7 or later.	RESTORE	Character string 1	Restore the program, the parameter, the common variable, and the error log to the RAM from the ROM. Do not change this parameter.	FLROM→SRAM (unchangeable)
CC-Link error release permission. *This parameter can be used for controller software version H7 or later.	E7730	Integer 1	If the controller is used without connecting CC-Link even though it is equipped with the CC-Link option, error 7730 is generated and the controller becomes inoperable. This error cannot be canceled under normal circumstances, but it becomes possible to temporarily cancel the error by using this parameter. (Enable temporary error cancellation/disable error cancellation = 1/0) This parameter becomes valid immediately after the value is changed by the T/B or Personal Computer support software. It is not necessary to turn the power supply off and on again. Note, however, that the value of this parameter returns to 0 again (it is no longer possible to cancel the error) when the power supply is turned off and on because changes of the parameter value are not stored.	0 (disable error cancellation)
Speed optimization interpolation functional switch *This parameter can be used for controller software version H7 or later.	SPDOPT	Integer 1	Set enable/disable of speed optimization interpolation function just after the power supply turned on 1: Enable (Enable the speed optimization interpolation function at the power on) 0: Disable (Disable the speed optimization interpolation function at the power on) -1: Disable the SPDOPT command If the value of this parameter is 1 or 0, it is possible to switch between enabling and disabling the speed adjustment interpolation function using the SPDOPT instruction in a program. If the value is -1, the speed adjustment interpolation function is always disabled even if the SPDOPT instruction is used in a program. Note) This function is supported by limited models of RH-1000G systems, etc.	Only RH-1000G series is 1. Other type are -1.

Parameter	Parameter name	No. of arrays No. of characters	Details explanation	Factory setting												
Set the delay time of the GC/GO command and the moving command. *This parameter is valid for using the MOVEMASTER command only. *This parameter can be used for controller software version H7 or later.	HANDDLY	Integer 1	The delay time of hand open/close in MOVEMASTER command is the time specified by GP command. (Default value is 0.3 sec.) The delay time of hand open/close can be specified by this parameter. <table><tr><th>Parameter value</th><th>Motorized hand</th><th>Pneumatic hand</th></tr><tr><td>-1</td><td>When the status of the hand changes, the delay timer specified by GP command is taken.</td><td>The delay time specified by the GP instruction is stored in this parameter when opening/closing the hand, regardless of whether or not the hand status has changed.</td></tr><tr><td>0</td><td>No delay</td><td>No delay</td></tr><tr><td>Value Unit(msec)</td><td colspan="2">When the status of the hand changes, the delay timer specified with this parameter is taken.</td></tr></table> The units of the delay time specified by GP command are 1 / 10 seconds. The units of the delay time specified with this parameter are 1 / 1000 seconds (=msec).	Parameter value	Motorized hand	Pneumatic hand	-1	When the status of the hand changes, the delay timer specified by GP command is taken.	The delay time specified by the GP instruction is stored in this parameter when opening/closing the hand, regardless of whether or not the hand status has changed.	0	No delay	No delay	Value Unit(msec)	When the status of the hand changes, the delay timer specified with this parameter is taken.		-1
Parameter value	Motorized hand	Pneumatic hand														
-1	When the status of the hand changes, the delay timer specified by GP command is taken.	The delay time specified by the GP instruction is stored in this parameter when opening/closing the hand, regardless of whether or not the hand status has changed.														
0	No delay	No delay														
Value Unit(msec)	When the status of the hand changes, the delay timer specified with this parameter is taken.															
The gravity direction *This parameter can be used for controller software version H4 or later. Note) This function can be used in RV-1A/2AJ, RV-4A/5AJ series, and RV-20A.	MEGDIR	Real value 4	This parameter specifies the direction and magnitude of gravitational acceleration that acts on the robot according to the installation posture for the X, Y, and Z axes of the robot coordinate system, respectively (unit: mm/second²). There are four elements: installation posture, gravitational acceleration in the X axis direction, gravitational acceleration in the Y axis direction, and then gravitational acceleration in the Z axis direction, in this order from the left. <table><tr><th>Installation posture</th><th>Setting value (Installation posture, gravitational acceleration in the X axis direction, gravitational acceleration in the Y axis direction, and then gravitational acceleration in the Z axis direction)</th></tr><tr><td>On floor</td><td>(0.0, 0.0, 0.0, 0.0)</td></tr><tr><td>Against wall</td><td>(1.0, 0.0, 0.0, 0.0)</td></tr><tr><td>Hanging</td><td>(2.0, 0.0, 0.0, 0.0)</td></tr><tr><td>Optional posture*¹</td><td>(3.0, ***, ***, ***)</td></tr></table> *1: "***" indicates the numerical value Refer to the lower example about the set value of the optional postures. The example of the setting of gravity acceleration is shown below. Example: If the robot is tilted 30 degrees forward (see the figure below): The direction gravity acceleration of X axis (X_g) = $9.8 \times \sin(30 \text{ degrees}) = 4.9$. The direction gravity acceleration of the Z axis (Z_g) = $9.8 \times \cos(30 \text{ degrees}) = 8.5$. Note that the value is set to -8.5 because the direction is opposite to the Z axis of the robot coordinate system. The direction gravity acceleration of the Y axis (Y_g) = 0.0 Therefore, the set value is (3.0, 4.9, 0.0, and -8.5) 	Installation posture	Setting value (Installation posture, gravitational acceleration in the X axis direction, gravitational acceleration in the Y axis direction, and then gravitational acceleration in the Z axis direction)	On floor	(0.0, 0.0, 0.0, 0.0)	Against wall	(1.0, 0.0, 0.0, 0.0)	Hanging	(2.0, 0.0, 0.0, 0.0)	Optional posture* ¹	(3.0, ***, ***, ***)	0.0, 0.0, 0.0, 0.0		
Installation posture	Setting value (Installation posture, gravitational acceleration in the X axis direction, gravitational acceleration in the Y axis direction, and then gravitational acceleration in the Z axis direction)															
On floor	(0.0, 0.0, 0.0, 0.0)															
Against wall	(1.0, 0.0, 0.0, 0.0)															
Hanging	(2.0, 0.0, 0.0, 0.0)															
Optional posture* ¹	(3.0, ***, ***, ***)															

Note1)Shows the factory setting of each model in the following.

- RH-5AH/10AH/ 5AH series and RV-100TH/150TH/100THL/150THL1
- Except the above0

Note2)The parameter is set up based on the order specifications before shipment.

Order to dealer when the instruction manual of the other language is necessity.

More, the caution seals that stuck on the robot arm and the controller are made based on the language of the order specification. Use it carefully when selecting the other language.

5.1 Standard Tool Coordinates

Tools data must be set if the robot's control point is to be set at the hand tip when the hand is installed on the robot. The setting can be done in the following two manners.

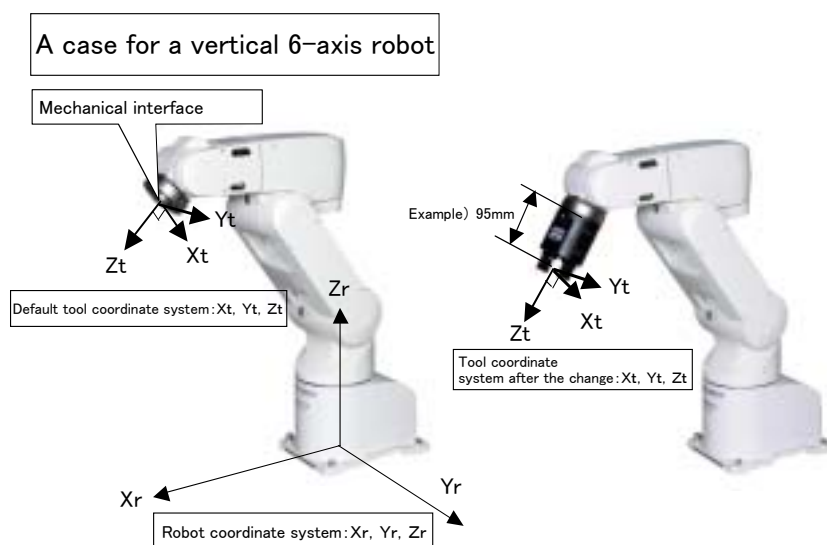
- 1) Set in the MEXTL parameter.
- 2) Set in the robot program using the TOOL instruction.

The default value at the factory default setting is set to zero, where the control point is set to the mechanical interface (flange plane).

Structure of tools data : X, Y, Z, A, B, C

X, Y and Z axis : Shift from the mechanical interface in the tool coordinate system

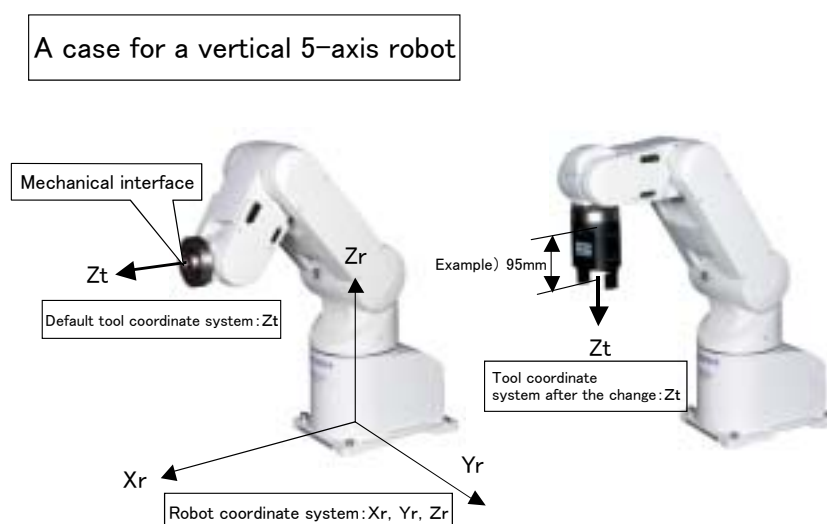
A axis : X-axis rotation in the tool coordinate system
 B axis : Y-axis rotation in the tool coordinate system
 C axis : Z-axis rotation in the tool coordinate system



<A case for a vertical 6-axis robot>

- 1) Sample parameter setting
 Parameter name: MEXTL
 Value: 0, 0, 95, 0, 0, 0
- 2) Sample TOOL instruction setting
 10 TOOL (0,0,95,0,0,0)

A 6-axis robot can take various postures within the movement range.

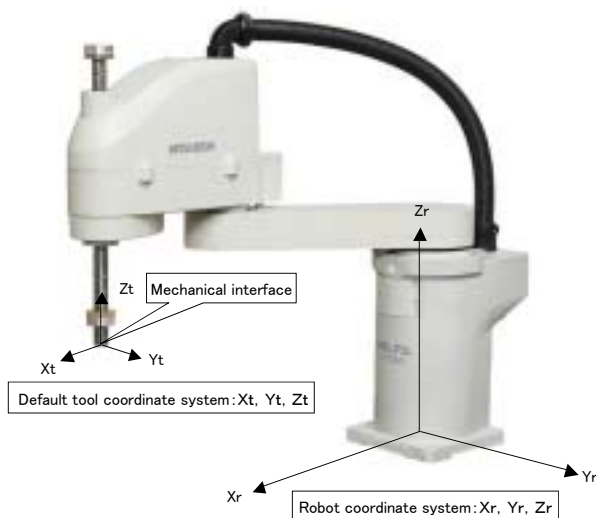


<A case for a vertical 5-axis robot>

- 1) Sample parameter setting
 Parameter name: MEXTL
 Value: 0, 0, 95, 0, 0, 0
- 2) Sample TOOL instruction setting
 10 TOOL (0,0,95,0,0,0)

Only the Z-axis component is valid for a 5-axis robot for movement range reasons. Data input to other axes will be ignored.

A case for a horizontal 4-axis robot



<A case for a horizontal 4-axis robot>

1) Sample parameter setting

Parameter name: MEXTL

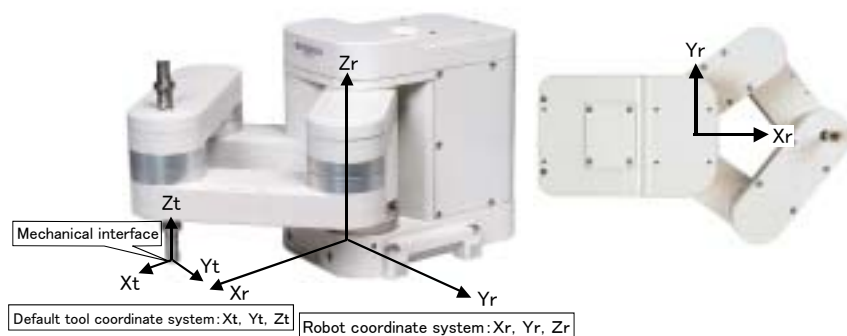
Value: 0, 0, -10, 0, 0, 0

2) Sample TOOL instruction setting

10 TOOL (0,0,-10,0,0,0)

Horizontal 4-axis robots can basically offset using parallel shifting. Note that the orientation of the tool coordinate system is set up differently from that of vertical robots.

A case for a RP series robot



<A case for a RP series robot>

1) Sample parameter setting

Parameter name: MEXTL

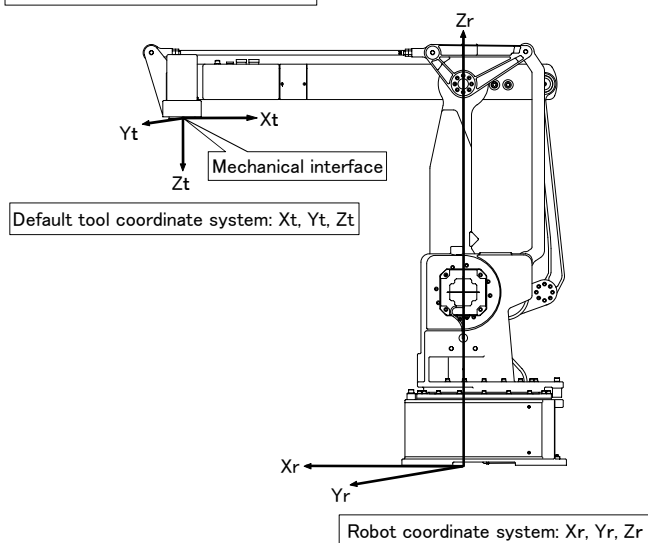
Value: 0, 0, -10, 0, 0, 0

2) Sample TOOL instruction setting

10 TOOL (0,0,-10,0,0,0)

RP-series robots use the same coordinate system as the horizontal 4-axis robots. Note that the orientation of the tool coordinate system is set up differently from that of vertical robots.

A case for a palletizing robot



<A case for a palletizing robot>

1) Sample parameter setting

Parameter name: MEXTL

Value: 0, 0, +10, 0, 0, 0

2) Sample TOOL instruction setting

10 TOOL (0,0,+10,0,0,0)

<Other robots>

The following models basically use the same coordinate system as the horizontal 4-axis robots.

Liquid crystal glass transportation robot: RH-1000G***, RH-1500G***, RC-1000GW***,
Palletizing robot : RV-100TH*, RV-150TH

An axis element of the tool conversion data may or may not be valid depending on the robot model.
See [Table 5-2](#) to set the appropriate data.

Table 5-2 : Valid axis elements of the tool conversion data depending on the robot model

Type	Number of axis	An axis element of the tool conversion data ^{Note1)}					
		X	Y	Z	A	B	C
RP-1AH/3AH/5AH	4	○	○	○	×	×	○
RV-1A, RV-2A RV-4A, RV-3AL	6	○	○	○	○	○	○
RV-2AJ, RV-3AJ RV-5AJ, RV-4AJL	5	△	△	○	△	△	△
RH-5AH/10AH/15AH	4	○	○	○	△	△	○
RH-15UHC		○	○	○	×	×	○
RH-1000GH Series	4						
RH-1000GJ Series	5	○	○	○	×	×	○
RH-1500GJ Series							
RH-1500G Series	6	○	○	○	○	○	○
RC-1000G Series	4	○	○	○	×	×	○ ^{Note2)}
RV-60TH RV-100TH/150TH RV-100THL/150THL	4	○	○	○	×	×	○
RS-30AG	4	○	○	○	○	×	×
RV-15AJ/15AP	5/6	○	△	○	△	△	△

Note1) ○ : Valid, △ : Invalid, × : The setting value is fixed to 0

Note2) All elements were added (This is not relative calculation)

5.2 About Standard Base Coordinates

When shifting the robot origin to a position other than the center position of the J1 axis of the robot, the conversion is performed using the base coordinate system. The setting will be done from the following two points. When base data is changed, the coordinates of teaching positions will be values based on the base coordinate system.

- 1) Set in the MEXTL parameter.
- 2) Set in the robot program using the BASE instruction.

The factory default setting value is set to zero at the base coordinate system position, which is identical to the robot origin.

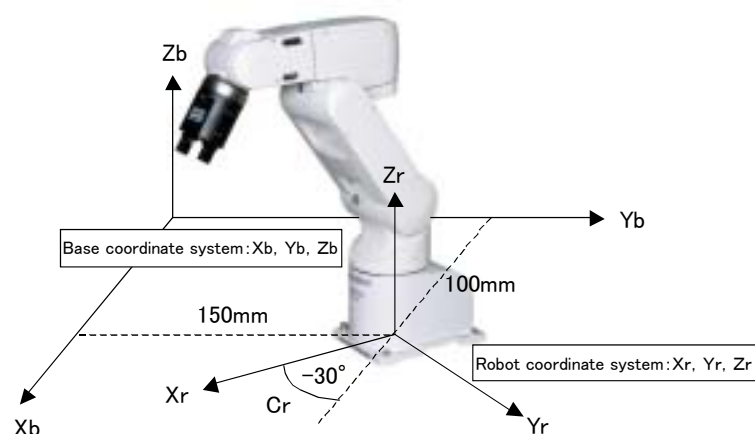
Structure of base coordinate system data: X, Y, Z, A, B, and C

X, Y and Z axis : The position of robot coordinate system from the base coordinate system origin

A axis : X-axis rotation in the base coordinate system

B axis : Y-axis rotation in the base coordinate system

C axis : Z-axis rotation in the base coordinate system



Example)

- 1) Sample parameter setting

Parameter name: MEXBS

Value : 150,100,0,0,0,-30

- 2) Sample BASE instruction setting

10 BASE (150,100,0,0,0,-30)

Normally, the base coordinate system need not be changed. If you wish to change it, see the sample above when configuring the system. Note that the BASE instruction within the robot program may shift the robot to an unexpected position. Exercise caution when executing the instruction.

An axis element of the base conversion data may or may not be valid depending on the robot model.

See [Table 5-3](#) to set the appropriate data.

Table 5-3 : Valid axis elements of the base conversion data depending on the robot model

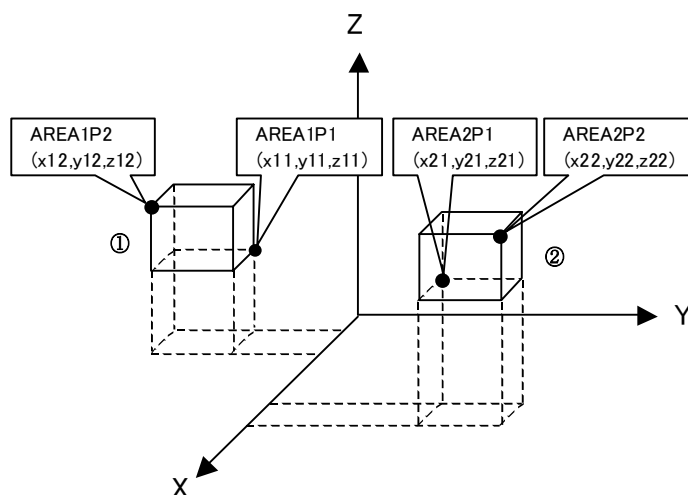
Type	Number of axis	An axis element of the base conversion data ^{Note1)}					
		X	Y	Z	A	B	C
RP-1AH/3AH/5AH	4	○	○	○	×	×	○
RV-1A, RV-2A RV-4A, RV-3AL	6	○	○	○	○	○	○
RV-2AJ, RV-3AJ RV-5AJ, RV-4AJL	5	○	○	○	△	△	△
RH-5AH/10AH/15AH	4	○	○	○	△	△	○
RH-15UHC		○	○	○	×	×	○
RH-1000GH Series	4	○	○	○	×	×	○
RH-1000GJ Series	5	○	○	○	×	×	○
RH-1500GJ Series		○	○	○	×	×	○
RH-1500G Series	6	○	○	○	○	○	○
RC-1000G Series	4	○	○	○	×	×	○ ^{Note2)}
RV-60TH RV-100TH/150TH RV-100THL/150THL	4	○	○	○	×	×	○
RS-30AG	4	○	○	○	○	×	×
RV-15AJ/15AP	5/6	△	△	△	△	△	○ ^{Note3)}

Note1) ○ : Valid, △ : Invalid, × : The setting value is fixed to 0

Note2) All elements were added (This is not relative calculation)

Note3) The setting is made in units of 45 degrees

5.3 About user-defined area



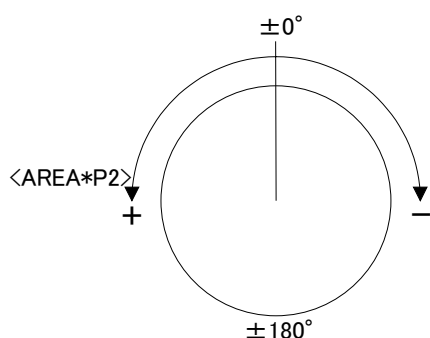
When operation is performed together with peripheral devices, work area may have to be shared. Under such circumstances, one device must let the other know when it is within the shared area. For this purpose, a robot can be configured to output a signal while it is in a certain area by setting parameters.

For instance, in the diagram to the left, the following parameter setting will output the signal 10 when operating in area (1) and output the signal 11 when operating in area (2).

Similar confirmation is possible using the M_UAR variable if checking within the program. Refer to [Page 226, "M_UAR"](#).

Parameter name	Meaning of the value	Value
AREA1P1	Position data for the first point: X,Y,Z,A,B,C	x 11, y 11, z 11, -360, -360, -360
AREA1P2	Position data for the second point: X,Y,Z,A,B,C	x 12, y 12, z 12, 360, 360, 360
AREA1ME	Target mechanism number: Usually 1	1
AREA1AT	Invalid/Output signal/Error: 0/1/2	1
AREA2P1	Position data for the first point: X,Y,Z,A,B,C	x 21, y 21, z 21, -360, -360, -360
AREA2P2	Position data for the second point: X,Y,Z,A,B,C	x 22, y 22, z 22, 360, 360, 360
AREA2ME	Target mechanism number: Usually 1	1
AREA2AT	Invalid/Output signal/Error: 0/1/2	1
USRAREA	Output signal: starting number, end number	10, 11 (Information regarding whether or not the robot is in AREA1 is output to the signal 10, and whether or not the robot is in AREA2 is output to the signal 11.)

- *1 Enter the coordinates (x, y, and z) for x11 to z22.
- *2 In the setting sample above, since the posture data (A, B, and C) are ignored, a signal will be output regardless of the posture. To set the posture data (A, B, and C), set the values in the AREA*P1 to AREA*P2 direction. (Example: In the case of $-10 \rightarrow +30$, evaluation as in-area occurs in the range from -10 to $+30$, but in the case of $+30 \rightarrow -10$ evaluation as in-area occurs in the range from $+30$ to -10 .)
- *3 For a non-existent axis of the posture data (for instance the A- and B-axes of horizontal multi-joint type robots), make sure that AREA*P1 is set to -360 and AREA*P2 is set to $+360$.
- *4 AREA*ME specifies to which mechanism will the area checking configuration apply. Under the standard configuration (one unit is connected), this is set to 1.
- *5 AREA*AT specifies the type of area checking. The meaning of the value is shown below.
 0 = Does not check, 1 = Outputs a signal, 2 = Generates an error.
 In the case of 2, the posture data will be ignored.



Example) If the value passes through 0 from -100 to $+100$, the following setting is necessary.

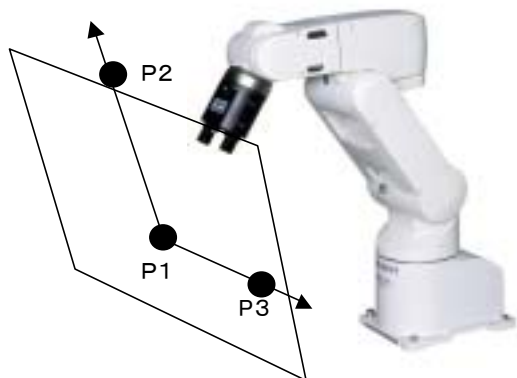
Sets the negative value to ABC of <AREA*P1>.
 Sets the positive value to ABC of <AREA*P2>.

Example) If the value passes through 180 from -100 to $+100$, the following setting is necessary.

Sets the positive value to ABC of <AREA*P1>.
 Sets the negative value to ABC of <AREA*P2>.

5.4 Free plane limit

Defines any plane in the robot coordinate system, determines the front or back of the plane, and generates a free plane limit error.



As can be seen in the diagram to the left, any plane can be defined by three points (P1, P2, and P3), after which an evaluation of which side of the plane it is in (the side that includes the robot origin or the other side) can be performed.

This function can be used to prevent collision with the floor or interference with peripheral devices.

Maximum of eight planes can be monitored.

There is no limit to the plane.

Parameter and value	Explanation
SFCnP (n=1 to 8)	Specifies the 3 points that define the plane. P1 coordinates X1, Y1, and Z1: The origin of the plane P2 coordinates X2,Y2,Z2 : A position on the X axis of the plane P3 coordinates X3,Y3,Z3 : A position in the positive Y direction of the X-Y plane in the plane
SFCnME (n=1 to 3)	Specifies the mechanism number to which the free plane limit applies. Sets 1. In the case of multiple mechanisms, the mechanism numbers are specified.
SFCnAT (n=1 to 8)	Specify 1 to enable free plane limit. (Enabled/Disabled = 1/0, default value is 0.)

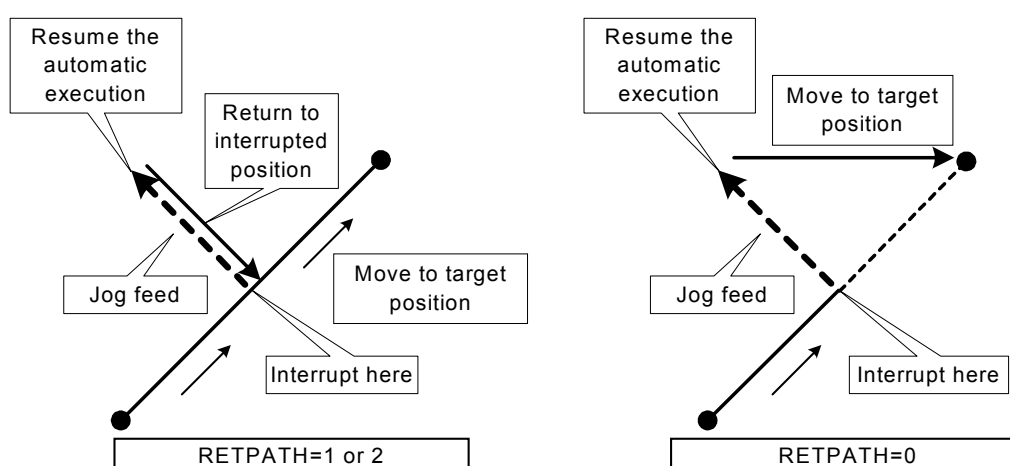
After setting the parameters above, turn the controller's power ON again. This will allow the generation of free plane limit error when it crosses the plane.

5.5 Automatic return setting after jog feed at pause

This specifies the path behavior that takes place when the robot is paused during automatic operation or during step feed operation, moved to a different position using a jog feed with T/B, and the automatic operation is resumed or the step feed operation is executed again. See the following diagram.

Parameter and value	Description of the operation
RETPATH=1 (Default)	1) Returns to the original position where the pause took place using joint interpolation. 2) Resumes from the line that was paused.
RETPATH=0	Resumes from the line that was paused from the position resulting after the jog operation. Therefore, movement will take place using the interpolation method of the instruction under execution from the current position to the next target position.
RETPATH=2 *1	1) Return by XYZ interpolation to the interrupted position. 2) Resume the interrupted line.

*1: The "RETPATH=2" is available for controller software version H4 or later.



The table below lists the values that can be specified for each interpolation instruction and controller software version.

Table 5-4 : Interpolation instruction and RETPATH setting value

Interpolation command	Is before than H4 edition *1	Is H4 edition or later
MOV	0	0
MVS	1	1 2
MVC	0	0 (The same operation as 1) *2
MVR	1	1
MVR2		2
MVR3		
MVA	0 (The same operation as 1) *3 1	0 (The same operation as 1) *3 1 2

*1: If "RETPATH=2" is set for versions earlier than H4, the same operation as when "RETPATH=1" is set is obtained. (Returns to interrupted position by JOINT interpolation)

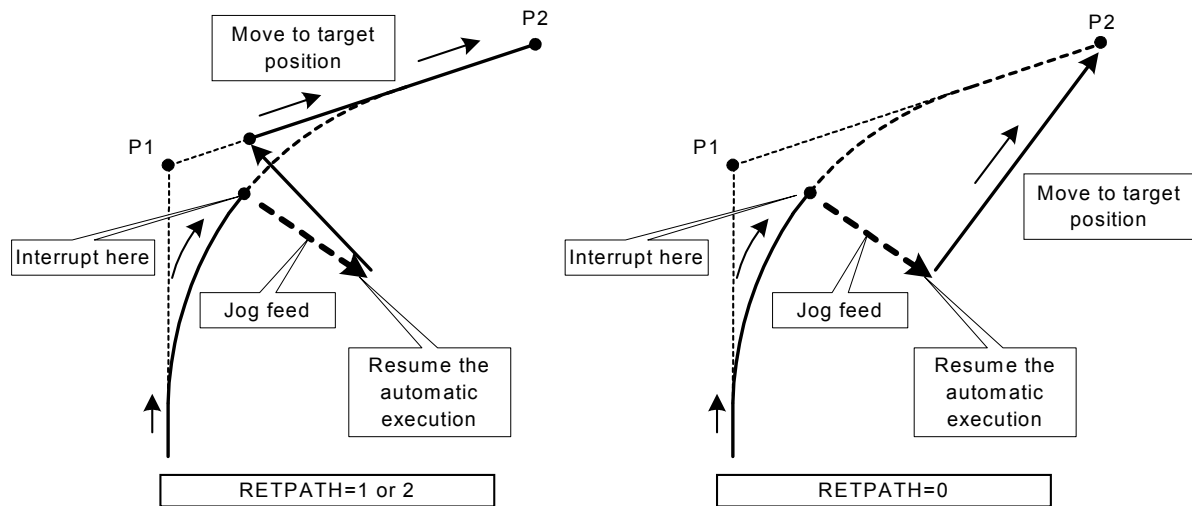
*2: Note that the operation when RETPATH=0 is set depends on the software version at circular interpolation (MVC, MVR, MVR2, MVR3). For versions later than H4, the same operation as when RETPATH=1 is set is obtained even if RETPATH=0 is set. (Returns to interrupted position by JOINT interpolation)

*3: In the MVA command, even if set up with RETPATH=0, the operation is same as RETPATH=1. (Returns to interrupted position by JOINT interpolation)

Caution) If movement other than a joint jog (XYZ, tools, cylindrical, etc.) has been used when the "RETPATH" parameter is set to 1, joint interpolation will be used to return to the original position at the time pause took place. Therefore, be careful not to interfere with peripheral devices.

Note) If the parameter "RETPATH" is set to 2 for a robot whose structure data is valid or with multiple rotations, and the robot is moved from a suspended position by joint jog, the robot is moved to a position different from the original structure data and/or multiple-rotation data and may become unable to return to the suspended position. In this case, adjust the position of the robot to the suspended position and resume moving the robot.

If "RETPATH=1 or 2" is set as shown in the figure below, and the robot is operated continuously (continuous path operation) using the CNT instruction, the robot returns to a position on the travel path from P1 to P2 instead of the suspended position. When "RETPATH=0" is set, the robot moves to the target position from the current position.



5.6 Automatic execution of program at power up

The following illustrates how to automatically run a robot program when the controller's power is turned on. However, since the robot starts operating simply by turning the power on, exercise caution upon using this function.

Related parameters

Parameter and value	Description of the operation
SLT*	Exmple) SLT2=2,ALWAYS,REP Specifies the program name, start condition, and operation status. The point here is the start condition.
ALWENA	0 → 7 In the ALWAYS program, it is possible to execute multitask-related instructions such as XRUN and XLOAD, and also the SERVO instruction.

- (1) First, create an ALWAYS program and an operating program.

<Program #2, ALWAYS program>

```

100 ' Auto Start Sample Program
110 '
120 ' Execute Program #1 if the key switch is AUTO (Ext.).
130 ' Stop the program and return the execution line to the beginning of the program if the key switch is not AUTO (Ext.).
140 '
150 IF M.MODE<>3 AND (M.RUN(1)=1 OR M.WAI(1)=1) THEN GOSUB *MTSTOP
160 IF M.MODE=3 AND M.RUN(1)=0 AND M.WAI(1)=0 THEN GOSUB *MTSTART
165 IF M.MODE=2 THEN HLT ' for DEBUG
170 END
180 '
190 *MTSTART
200 XRUN 1,"1"
210 RETURN
220 '
230 *MTSTOP
240 XSTP 1
250 XRST 1
260 RETURN

```

< Program #1, operating program > (this can be any program)

```

100 'Main Program (Position data is for RV-2AJ.)
110 SERVO ON
120 M_OUT(8)=0
130 MOV P1
140 M_OUT(8)=1
150 MOV P2
160 END
P1=(+300.00,-200.00,+200.00,+0.00,+180.00,+0.00)(6,0)
P2=(+300.00,+200.00,+200.00,+0.00,+180.00,+0.00)(6,0)

```

- (2) Set the parameter.

Parameter and value	Description of the operation
SLT2	SLT2=2,ALWAYS,REP ' Execute program #2 in ALWAYS mode.
ALWENA	0 → 7 In the ALWAYS program, it is possible to execute multitask-related instructions such as XRUN and XLOAD, and also the SERVO instruction.

After the setting is complete, turn the controller's power OFF.

- (3) Turn the power ON.

In the sample above, after the controller's power is turned on, when the key switch is turned to AUTO (Ext.), program #1 is executed and the robot starts its operation.

5.7 About the hand type

The factory default setting assumes that the double-solenoid type hand will be used. If the single-solenoid type is used or if a general-purpose signal is to be used to control the robot, the HANDTYPE parameter must be set as described below.

Factory default parameter settings

Parameter name	Value
HANDTYPE	D900,D902,D904,D906, , , ,

From the left, the values correspond to hand #1, #2, and so on. The default value is shown below.

Hand 1 = accesses signals #900 and #901

Hand 2 = accesses signals #902 and #903

Hand 3 = accesses signals #904 and #905

Hand 4 = accesses signals #906 and #907

The hand numbers 1 through 4 (or 8) will be used as the argument in the hand open/close instructions (HOPEN or HCLOSE).

<Setting method>

When a double-solenoid type is used, 'D' must be added in front of the signal number to specify the number.

In the case of double-solenoid type, hand number will be from 1 to 4.

When a single-solenoid type is used, 'S' must be added in front of the signal number to specify the number.

In the case of single-solenoid type, hand number will be from 1 to 8.

<Example>

- 1) To assign two hands of the double-solenoid type from the general-purpose signal #10
HANDTYPE=D10,D12, , , , ,
- 2) To assign three hands of the double-solenoid type from the general-purpose signal #10
HANDTYPE=S10,S11,S12, , , , ,
- 3) To assign hand 1 to the general-purpose signal #10 as the single-solenoid type while assigning hand 2 to the general-purpose signal #12 as the single-solenoid type
HANDTYPE=D10,S12, , , , ,

5.8 About default hand status

The factory default setting is shown below.

Hand type	Status	Status of output signal number		
		Mechanism #1	Mechanism #2	Mechanism #3
When pneumatic hand interface is installed (double-solenoid is assumed)	Hand 1 = Open	900=1 901=0	910=1 911=0	920=1 921=0
	Hand 2 =Open	902=1 903=0	912=1 913=0	922=1 923=0
	Hand 3 =Open	904=1 905=0	914=1 915=0	924=1 925=0
	Hand 4 =Open	906=1 907=0	916=1 917=0	926=1 927=0
When electric-powered hand interface is installed	Hand open	M_OUT (9*0) through M_OUT (9*7) are used by the system and therefore unavailable to the user. If used, normal opening and closing of the hand will not be possible.		
I/F before interface installation	–	M_OUT (9*0) through M_OUT (9*7) do not function.		

*A single controller can control multiple robots. If pneumatic hand interface is to be used for each robot, the hand output signal number is assigned in the following manner.

Mechanism #1 = #900 to #907 (This will be the case for standard configuration with one unit connected.)

Mechanism #2 = #910 to #917

Mechanism #3 = #920 to #927

*When electric-powered hand interface is used, the system will use the 900's using special controls. The users should not access the 900's directly but instead use the hand control instructions or the hand operation from T/B only. If you access the 900's, normal opening and closing of the hand will not be possible.

The default parameters are set as shown below so that all hands start as "Open" immediately after power up.

Parameter name	Signal number	Value
HANDINIT	900, 901, 902, 903, 904, 905, 906, 907	1, 0, 1, 0, 1, 0, 1, 0

*The above describes the situation for standard configuration (one unit is connected). When multiple mechanisms are used, specify the mechanism number to set the HANDINIT parameter.

If for instance hand 1 alone needs to be closed when the power is turned ON, the following should be set. Similarly, in the case of electric-powered hand (hand number is fixed to 1), the hand will be closed when the power is turned on if the following configuration is applied.

Parameter name	Signal number	Value
HANDINIT	900, 901, 902, 903, 904, 905, 906, 907	0, 1, 1, 0, 1, 0, 1, 0, 1

Caution) If you set the initial hand status to "Open," note that the workpiece may be dropped when the power is turned ON.

5.9 About the output signal reset pattern

The factory default setting sets all general-purpose output signals to OFF (0) at power up. The status of general-purpose output signals after power up can be changed by changing the following parameter. Note that this parameter also affects the general-purpose output signal reset operation (called by dedicated I/O signals) and the reset pattern after executing the CLR instruction.

	Parameter name	Value (Values are all set to 0 at the factory default setting.)
Remote I/O	ORST0	Signal number 0-----7 8-----15 16-----23 24-----31 00000000, 00000000, 00000000, 00000000
	ORST32	32-----40 41-----49 50-----57 58-----66 (Same as above) 00000000, 00000000, 00000000, 00000000
	ORST64	00000000, 00000000, 00000000, 00000000
	ORST96	00000000, 00000000, 00000000, 00000000
	ORST128	00000000, 00000000, 00000000, 00000000
	ORST160	00000000, 00000000, 00000000, 00000000
	ORST192	00000000, 00000000, 00000000, 00000000
	ORST224	00000000, 00000000, 00000000, 00000000
Option with signal function for slot 1	ORST3000	00000000, 00000000, 00000000, 00000000
	ORST3032	00000000, 00000000, 00000000, 00000000
	ORST3064	00000000, 00000000, 00000000, 00000000
	ORST3096	00000000, 00000000, 00000000, 00000000
	ORST3128	00000000, 00000000, 00000000, 00000000
	ORST3160	00000000, 00000000, 00000000, 00000000
	ORST3192	00000000, 00000000, 00000000, 00000000
	ORST3224	00000000, 00000000, 00000000, 00000000
	ORST3256	00000000, 00000000, 00000000, 00000000
	ORST3288	00000000, 00000000, 00000000, 00000000
	:	:
	:	:
	:	:
	:	:
Option with signal function for slot 2	ORST4984	00000000, 00000000, 00000000, 00000000
	ORST5016	00000000, 00000000, 00000000, 00000000
	ORST6000	00000000, 00000000, 00000000, 00000000
	ORST6032	00000000, 00000000, 00000000, 00000000
	ORST6064	00000000, 00000000, 00000000, 00000000
	ORST6096	00000000, 00000000, 00000000, 00000000
	ORST6128	00000000, 00000000, 00000000, 00000000
	ORST6160	00000000, 00000000, 00000000, 00000000
	ORST6192	00000000, 00000000, 00000000, 00000000
	ORST6224	00000000, 00000000, 00000000, 00000000
	ORST6256	00000000, 00000000, 00000000, 00000000
	ORST6288	00000000, 00000000, 00000000, 00000000
	:	:
	:	:
	:	:
	ORST7984	00000000, 00000000, 00000000, 00000000
	ORST8016	00000000, 00000000, 00000000, 00000000

The value corresponds to bits from the left.

Setting is "0", "1", or "*".

"0" = Set to off

"1" = Set to on

"*" = Maintain status with no change. (Set to off at power up.)

For instance, if you want to always turn ON immediately after power up the general-purpose signals 10, 11 and 12 of the standard I/O and 32, 33 and 40 of the expansion I/O, the robot should be set to the configuration shown below.

Parameter name	Value
ORST0	00000000, 00111000, 00000000, 00000000
ORST32	11000000, 10000000, 00000000, 00000000

In addition to the above, to make 20, 21 and 22 retain their individual on/off status upon a general-purpose output signal reset, the robot should be set to the configuration shown below.

Parameter name	Value
ORST0	00000000, 00111000, 0000***0, 00000000
ORST32	11000000, 10000000, 00000000, 00000000

In the case above, general-purpose signals 20, 21 and 22 will start up as 0 (off) after a power up. The setting cannot be made in such a way that will turn the signal to 1 (on) after power up and will retain the current status upon a general-purpose output signal reset.

Caution) When editing the parameters, do not enter an incorrect number of zeros. If the number of zeros is incorrect, an error is generated next time the power is turned on.

5.10 About the communication setting

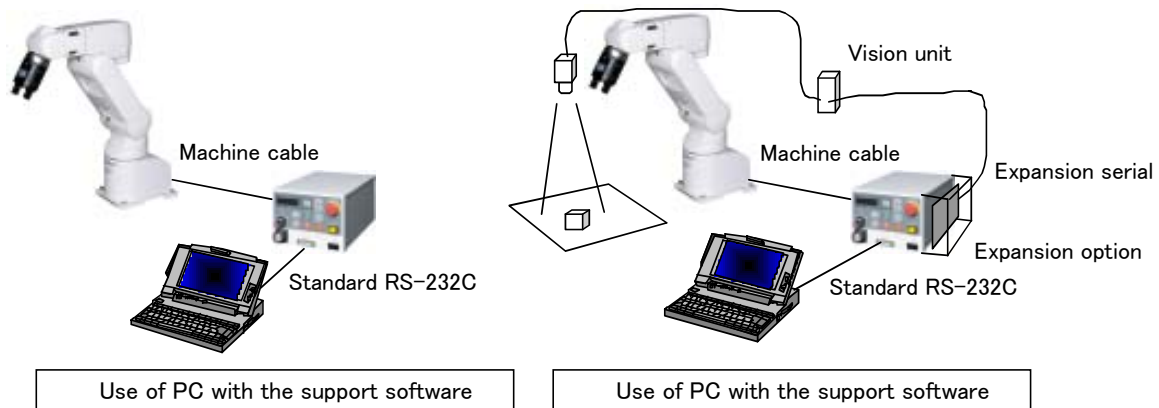
(1) Overview

The controller for the CRn series can support 1 standard RS-232C, 2 optional expansion serial cards (2 ports per card), totaling 5 ports. The optional expansion serial interface card has two ports, where one is RS-232C only and the other can select either RS-232C or RS-422. Up to two expansion serial cards can be used.

Standard RS-232C port normally connects to a PC for robot program transferring and debugging done with the PC support software. Optional cards can link with vision sensor and external devices for data communication. Communication is performed using the communication instructions (OPEN, CLOSE, PRINT, INPUT, etc.) in the robot program. This is referred to as data link. The controller cannot be controlled from external devices such as a PC (i.e., automatic execution or status monitoring). If this is necessary, contact the dealer or branch from where the robot has been purchased for further consultation.

Example of standard RS-232C port usage

Example of expansion RS-232C port usage



Caution)The example above is for robot controller CR1. Controllers other than the CR1 do not require expansion option box.

(2) Performing data link with the outside using RS-232C

Although the standard RS-232C can be used as the data link port, it is recommended that the standard RS-232C be reserved for robot program correction and monitoring at the time of start up and an optional expansion serial interface be provided as a dedicated data link port. The following sample program shows how to use an expansion serial interface.

<Parameter setting> The communication setting of the expansion serial interface is described below.

Parameter name	Default value	After change	Meaning
COMDEV	RS232,	RS232, OPT11,	When using CH1 in option slot 1
CBAUE11	9600	←	Baud rate = 9600bps
CPRTYE11	2	←	Parity = Even
CSTOPE11	2	←	Stop bit = 2
CTERME11	0(CR)	1(CRLF)	Termination code = CRLF (carriage return and line feed)
CPRCE11	0	2	Switch to data link This setting is for communicating with the outside using OPEN, PRINT, INPUT and CLOSE instructions in the robot program.
Communication example		OPEN "COM2:" AS #1 Robot External PRINT #1, "ABC" "ABC" CRLF → M1 = 10 PRINT #1, M1 "10" CRLF → INPUT #1, C1\$ ← "RUN" CRLF INPUT #1, M1 ← "20" CRLF	

For expansion serial interface, refer to "Expansion Serial Interface Instruction Manual."

<Sample robot program>

The following is a sample program which moves to the camera position, obtains correction data from RS-232C, moves to the position above the corrected position, moves to the target position, closes hand, and breaks away.

Command	Comment
10 OPEN "COM2:" AS #1	'Opens RS232C communication.
20 MOV P1,-50	'Moves to camera position.
30 M_OUT(10)=1 DLY 0.5	'Outputs camera start signal (start up the sensor).
40 PX=P1	'Obtains the camera origin.
50 INPUT #1,MX,MY,MC	'Inputs correction data.
60 PX.X=PX.X+MX	'Applies correction data in the X direction.
70 PX.Y=PX.Y+MY	'Applies correction data in the Y direction.
80 PX.C=PX.C+RAD(MC)	'Applies correction data in the C axis.
90 MOV PX,-50	'Moves to a position above the workpiece.
100 OVRD 30	'Slows down.
110 MVS PX	'Moves to the target position.
120 DLY 0.3	'Waits for positioning.
130 HCLOSE	'Closes hand.
140 DLY 0.3	'Waits for hand closing to complete.
150 MVS ,-50	'Breaks away.
:	:

*In this example, the camera origin position must be stored in robot coordinates through teaching in advance. The Z-axis also must allow the grasping of the workpiece.

<Communication data> (Data string sent by the external devices such as the sensor.)

Data sent to the robot is sent in the order of X, Y, and then C. CR (carriage return = 0d in hexadecimal) is sent as the termination code. The units are mm, mm, and deg (degree), respectively. In the following example, data sent is X = 12.34 mm, Y = 0.39 mm, and C = 56.78° .

```
"12.34, 0.39, 56.78(CR)"
```

(3) Notes on using both the PC support software and the data link software on standard port

If the standard port on the front face of the controller is to be used to connect to both the PC support software at power up and to an external device (such as a PC that is running specialized software) via the data link during operation, the data transmitted to the controller from external devices must have "PRN" at the beginning of each data string.

```
Example) "PRN12.34, 0.39, 56.78(CR)"
```

These letters are required to distinguish between PC support software protocol and data link communication. After "PRN", data should follow immediately; do not insert a space.

Therefore, if the communication protocol from the external device cannot be changed, then the device cannot be used under the default standard port setting. Although it is possible to set a parameter (CPRC232) so that the "PRN" is not required, changing of this parameter disallows the use of PC support software. Therefore, if "PRN" cannot be added to the transmission data from the external device, use the optional expansion serial interface.

Vision sensor AS50VS from Mitsubishi comes with the standard functions to support this "PRN", which allows them to be supported at the standard port. However, with only the standard port, PC support software and vision sensor cannot be used at the same time since there is only one port.

- (4) How to ensure stable communication between the personal computer support software and the robot controller.
- When communicating with the robot controller (hereinafter referred to as the R/C) using the Personal Computer Support Software (hereinafter referred to as the Software), depending on the personal computer model and settings the communication may become unstable in a batch backup or program upload/download where large amounts of data are transmitted. To ensure stable communication with the R/C, change the communication settings (communication protocol) of the R/C and the Software as shown below. If the communication settings of the R/C and the Software do not match, normal communication cannot be established. Be sure to change the settings on both the R/C and Software sides.

<The Robot Controller>

As for the R/C communication settings, change the following parameters:

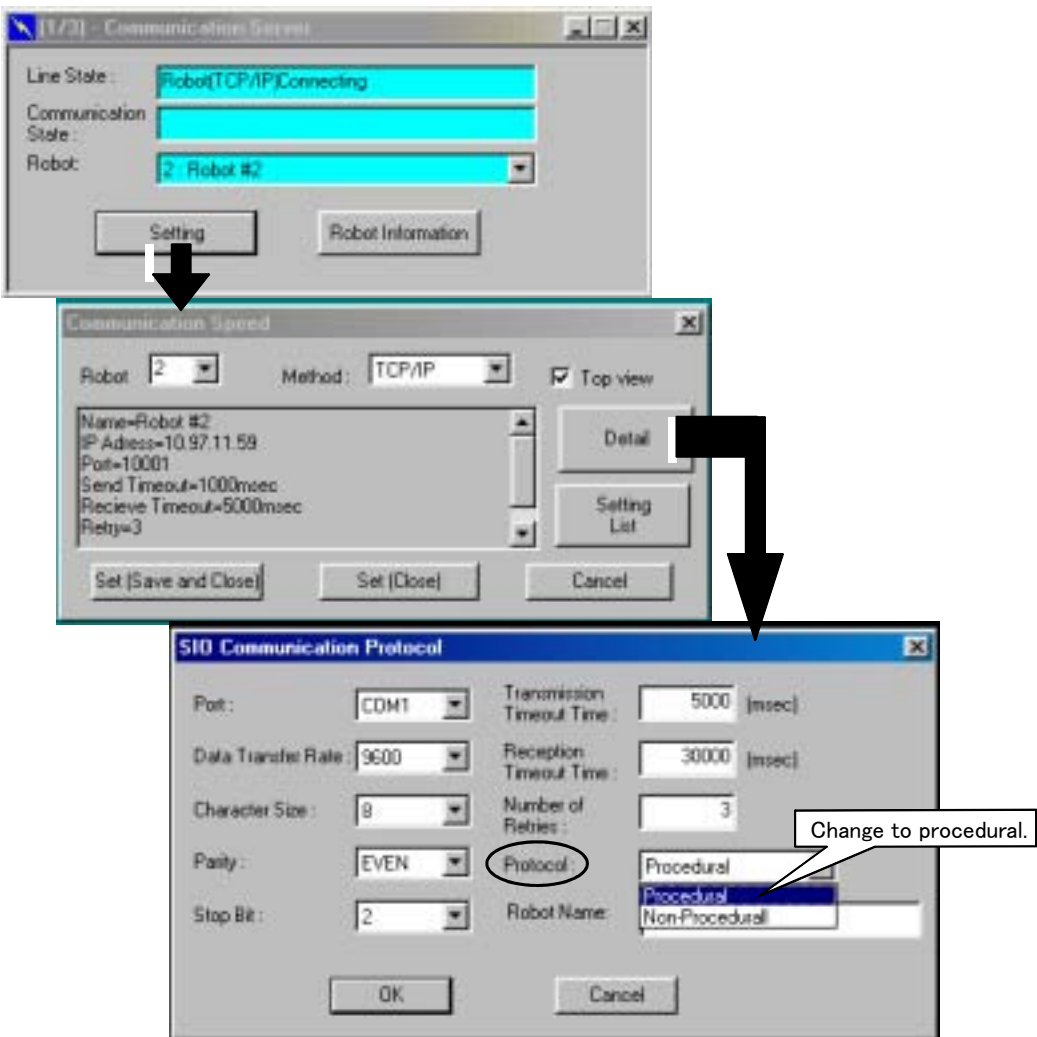
Parameter	Default	Change to
Communication protocol (CPRC232)	0 (Non-Procedural)	1 (Procedural)

- * To communicate with the personal computer via an extended RS-232C port by using an extension serial interface board, change the parameters of the extended RC-232C port.

<The Personal Computer Support Software>

Change the communication settings of the personal computer support software through the "communication server."

Parameter	Default
Protocol	Non-Procedural



- * When communicating with the R/C using your custom software, reset the R/C protocol setting to "Non-Procedural" in advance.

5.11 About the optimum acceleration/deceleration setting

Optimum acceleration/deceleration control allows the optimum acceleration/deceleration to be performed by LOADSET and OADL instructions automatically in response to the load at the robot tip. The following parameters must be set correctly in order to obtain the optimum acceleration/deceleration.

The factory default setting is as follows.

	Parameter	Value
setting the hand conditions	HNDDAT1	Maximum load, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0
	HNDDAT2	Maximum load, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0
	HNDDAT3	Maximum load, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0
	HNDDAT4	Maximum load, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0
	HNDDAT5	Maximum load, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0
	HNDDAT6	Maximum load, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0
	HNDDAT7	Maximum load, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0
	HNDDAT8	Maximum load, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0
setting the workpiece conditions	WRKDAT1	0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0
	WRKDAT2	0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0
	WRKDAT3	0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0
	WRKDAT4	0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0
	WRKDAT5	0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0
	WRKDAT6	0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0
	WRKDAT7	0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0
	WRKDAT8	0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0

Parameter values define, from the left in order, weight, size X, Y, and Z, and center of gravity X, Y, and Z. Up to eight hand conditions and eight workpiece conditions can be set. For the size of a hand, enter the length of a rectangular solid that can contain a hand. Optimal acceleration/deceleration will be calculated from the hand condition and the workpiece condition specified by a LOADSET instruction.

Parameter	Value (Factory default)
HNDHOLD1	0, 1
HNDHOLD2	0, 1
HNDHOLD3	0, 1
HNDHOLD4	0, 1
HNDHOLD5	0, 1
HNDHOLD6	0, 1
HNDHOLD7	0, 1
HNDHOLD8	0, 1

Parameter values that define grasping or not grasping is shown from the left for cases where the hand is open or closed.

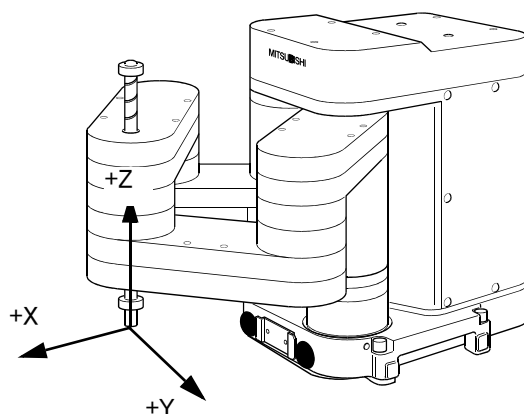
"0" = Set to not grasping

"1" = Set to grasping

- Depending on the hand's open/close status, optimum acceleration/deceleration calculation will be performed for either hand-alone condition or hand-and-workpiece condition.
- The hand's open/close status can be changed by executing the HOPEN/HCLOSE instruction.

The coordinate axes used as references when setting the hand and workpiece conditions are shown below for each robot model. The references of the coordinate axes are the same for both the hand and workpiece conditions. Note that all the sizes are set in positive values.

■ RP-1AH/3AH/5AH



Definitions of Coordinate Axes

In the coordinate system with the tip of the J4 axis as the origin:

Z axis: The upward direction is positive.

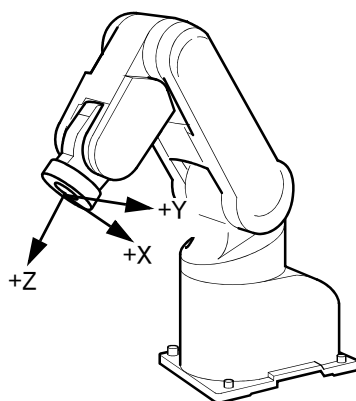
X axis: The direction of extension in the arm orientation is positive.

Y axis: A right hand coordinate system

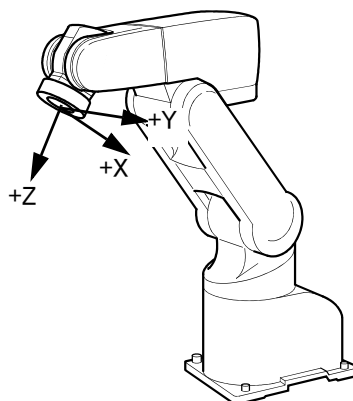
Axes that must be set:

Only the X and Y elements of the center of gravity and the X and Y elements of the size must be set.

■ RV-1A/2AJ, RV-2A/3AJ, RV-4A/5AJ, RV-20A



5-axis type



6-axis type

*Example of RV-1A/2AJ

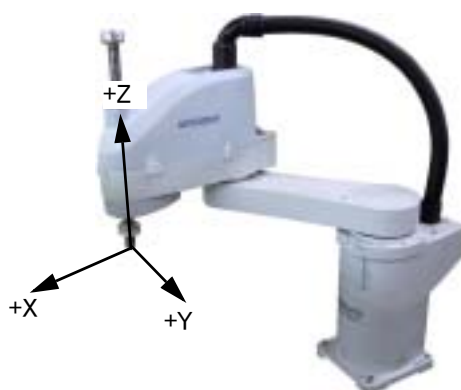
Definitions of Coordinate Axes

The tool coordinate is used for the coordinate axes.

Axes that must be set:

Only the X, Y and Z elements of the center of gravity and the X, Y and Z elements of the size must be set.

■ RH-5AH/10AH/15AH

Definitions of Coordinate Axes

In the coordinate system with the tip of the J4 axis as the origin:

Z axis: The upward direction is positive.

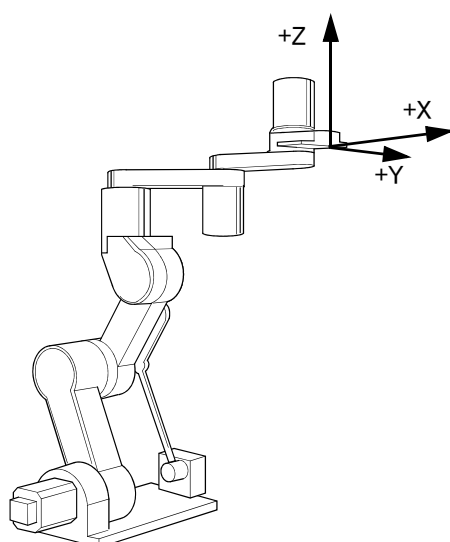
X axis: The direction of extension in the arm orientation is positive.

Y axis: A right hand coordinate system

Axes that must be set:

Only the X element of the center of gravity and the X and Y elements of the size must be set.

■ RV-1000G***

Definitions of Coordinate Axes

In the coordinate system with the center of the J5 axis as the origin for the 5-axis type and with the center of the J4 axis as the origin for the 4-axis type:

Z axis: The upward direction is positive.

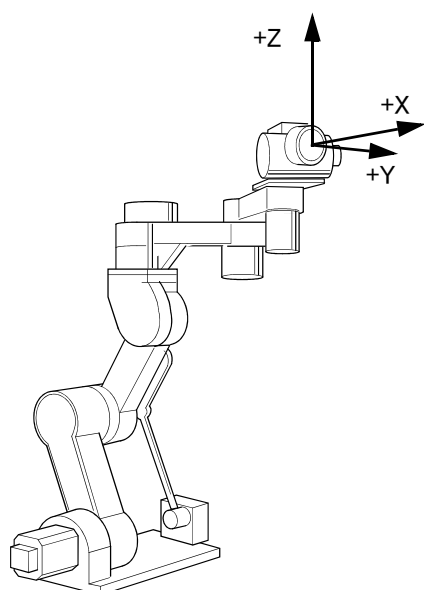
X axis: The direction of extension in the arm orientation is positive.

Y axis: A right hand coordinate system

Axes that must be set:

Only the X element of the center of gravity and the X and Y elements of the size must be set.

■ RV-1500G***

Definitions of Coordinate Axes

In the coordinate system with the center of the J6 axis as the origin for the 6-axis type and with the center of the J5 axis as the origin for the 5-axis type:

Z axis: The upward direction is positive.

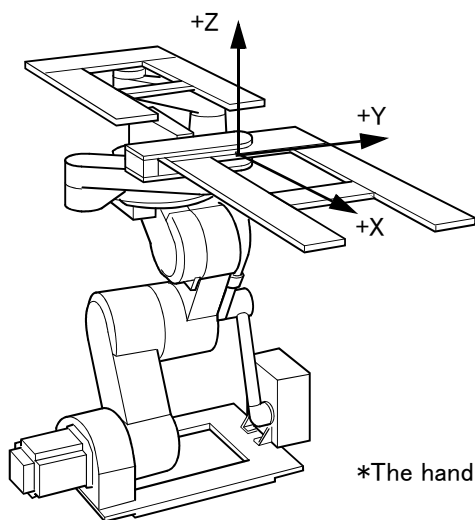
X axis: The direction of extension in the arm orientation is positive.

Y axis: A right hand coordinate system

Axes that must be set:

Only the X element of the center of gravity and the X and Y elements of the size must be set.

■ RC-1000GHWDC-SA/1000GHWLC-SA



Definitions of Coordinate Axes

In the coordinate system with the center of the flange as the origin:

Z axis: The upward direction is positive.

X axis: The direction of extension in the hand orientation is positive.

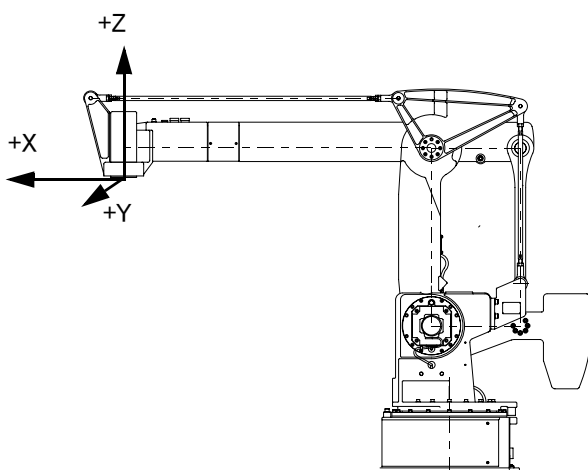
Y axis: A right hand coordinate system

Axes that must be set:

Only the X element of the center of gravity and the X and Y elements of the size must be set.

*The hand of the figure is an example.

■ RV-100TH/100THL/150TH/150THL/60TH



Definitions of Coordinate Axes

In the coordinate system with the center of the J4 axis as the origin:

Z axis: The upward direction is positive.

X axis: The direction of extension in the arm orientation is positive.

Y axis: A right hand coordinate system
(In the figure, the front direction)

Axes that must be set:

Only the X and Y elements of the center of gravity and the X and Y elements of the size must be set.

5.12 About the singular point adjacent alarm

When a robot having a singular point is being operated using a T/B, a singular point adjacent alarm is generated to warn the operators of the robot if the control point of the robot approaches a singular point.

Even if an alarm is generated, the robot continues to operate as long as it can perform operation unless operation is suspended. Also, an alarm is reset automatically when the robot moves away from a singular point. The following describes the details of the singular point adjacent alarm.

Note that this function is supported by the controller with the software version of G9 or later.

(1) Operations that generate an alarm

An alarm is generated if the control point of a robot approaches a singular point while a robot is performing any of the following operations using the T/B.

- 1) Jog operation (other than in joint jog mode)
- 2) Step feed and step return operations
- 3) MS position moving operation
- 4) Direct execution operation

If the robot approaches a singular point by any of the operations listed above, the buzzer of the controller keeps buzzing (continuous sound). However, the STATUS. NUMBER display on the operation panel does not change. Also, in the case of "[1] Jog operation (other than joint jog mode)" above, a warning is displayed on the T/B screen together with the sound of the buzzer.

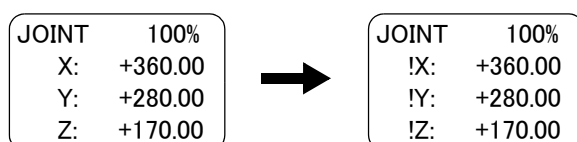


Fig.5-1 : Warning Display During Jog Operation

(2) Operations that do not generate an alarm

No alarm is generated when a robot is performing any of the operations listed below even if the control point of the robot approaches a singular point.

- 1) Additional axis jog operation initiated in joint jog mode using the T/B
- 2) When the joint interpolation instruction is executed even by an operation from the T/B
(Execution of the MOV instruction, MO position moving operation)
- 3) When the program is running automatically
- 4) Jog operation using dedicated input signals (such as JOGENA and JOGM)
- 5) When the robot is being operated using external force by releasing the brake
- 6) When the robot is stationary

5.13 ROM Operation Function

Because the ROM operation function has some restrictions on program operation and data retention, please use it after thoroughly understanding the specifications.

(1) Overview

Initially, the robot programs are saved in the RAM (SRAM) that is backed up in the battery.

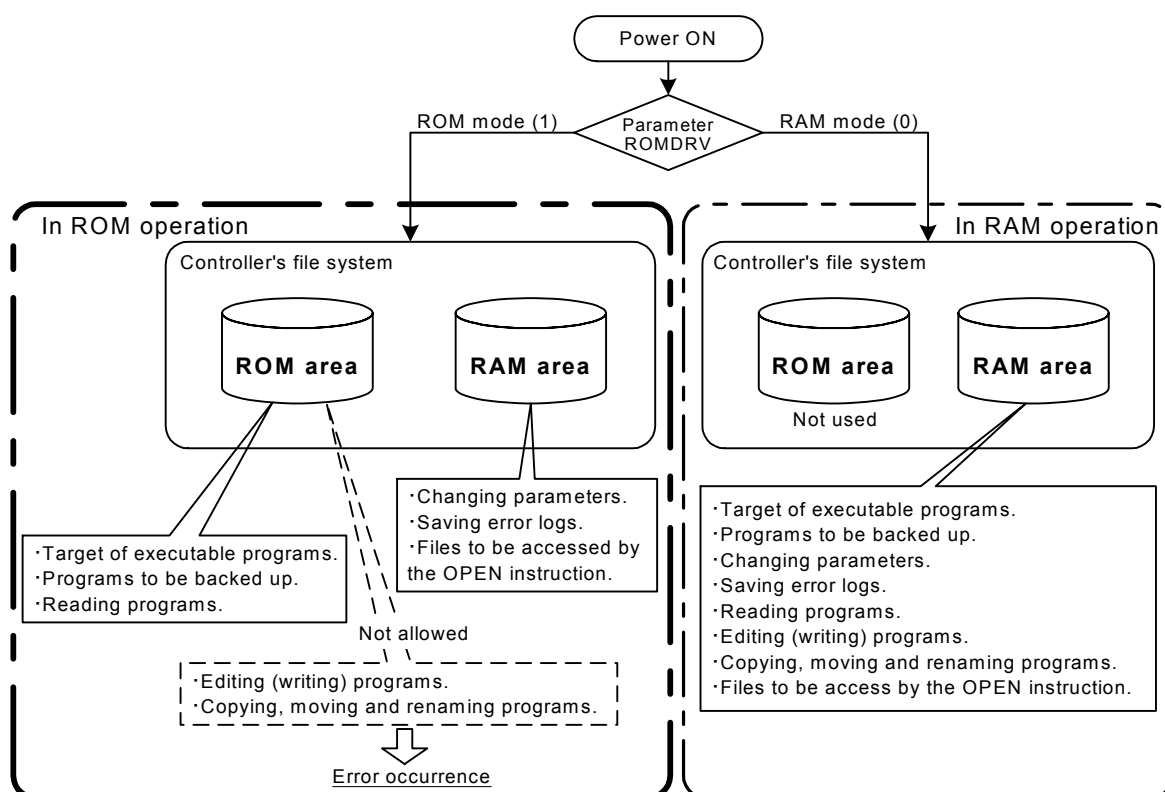
By saving the robot programs in ROM (FLROM), a loss of files due to the depletion of the backup battery, damage to the programs due to unexpected power shutoff (including momentary power failure) during a file access operation, or changes or deletion of the programs and position data due to an erroneous operation.

By changing the parameter values, the access target of the programs can be switched between ROM and RAM.

Once the access target of the programs is switched to ROM, it is referred to as in or during the ROM operation.

This function can be used with controller software version H7 or later.

Parameter	Description and value
ROMDRV	Switches the access target of the programs between ROM and RAM. 0 = RAM mode (initial value) 1 = ROM mode
BACKUP	Copies programs, parameters, common variables and error logs from the RAM area into the ROM area. SRAM → FLROM (fixed) * If this processing is canceled while being executed, "CANCEL" is displayed in the value field.
RESTORE	Rewrites programs, parameters, common variables and error logs in the ROM area into the RAM area. FLROM → SRAM (fixed) * If this processing is canceled while being executed, "CANCEL" is displayed in the value field.



While in the ROM operation, robot operations such as program execution and step operation can be performed in the same way as the RAM operation (when starting in RAM mode), but there are some restrictions. Refer to the following precautions.

◆◆◆ Precautions ◆◆◆

■ About variables

Variables may be changed by executing programs during the ROM operation; however, the changed values will be discarded when the controller power is turned off. The following lists the handling of variables during the ROM operation.

Variable *1	In ROM operation	In RAM operation
Local variable	The values of local variables are retained during program operation; however, they will be discarded when switching programs by the OP or external I/O signal, as well as when the power is turned off. The values of variables in the program called by the CALLP instruction will be discarded when they return to the called program.	The values of variables are retained even after the power is turned off. *2
Program external variable	The values of variables are retained until the power is turned off. (They will not be discarded by switching programs. The contents of changes will be discarded when the power is turned off.)	The values of variables are retained even after the power is turned off.
User-defined external variable		

*1 There are arithmetic variables, character string variables, position variables and joint variables.

*2 However, if a program is rewritten by using PC support software, the values of local variables used by programs will be discarded.

■ Changing variables during program execution

If the execution of a program is aborted during the ROM operation and "Variable Monitor" (refer to [Page 47, "3.12 Operating the monitor screen"](#)) of the T/B is used, the program cannot be resumed. Although the stop lamp stays lit, if the program is executed, the program will be executed from the first line. Be careful because peripheral devices may interfere with the robot.

The values of variables cannot be changed by using "Variable Monitor" (refer to [Page 47, "3.12 Operating the monitor screen"](#)) of the T/B during the ROM operation. Program Monitor (watch function) of PC support software can be used to change the values of variables in task slots other than the editing slot.

■ About programs

The target of program editing also becomes the ROM area. The programs in the controller is placed in the protected state (protect ON), and they cannot be canceled during the ROM operation. Once the ROM operation is switched to the RAM operation, the protect information reverts to the state set during the RAM operation. Programs may be read during the ROM operation, but they cannot be written. Similarly, programs can neither be copied nor renamed.

■ About parameters

Parameters and error log files are always saved in the RAM area regardless of switching between the ROM operation and the RAM operation. However, the RLNG parameter (for switching the robot language, refer to [Page 272, "RLING"](#)) cannot be changed during the ROM operation.

* Only a limited number of robot models can use the RLNG parameter.

■ About backup

During the ROM operation, programs are backed up from the ROM area, and parameters and error log files are backed up from the RAM area.

■ About direct executio

While in the ROM operation, local variables cannot be rewritten by direct execution.

■ About the continue function

While in the ROM operation, the continue function is disabled even if it is set.

* The continue function saves the execution status at the time of power OFF, and starts operating from the saved status the next time the power is turned on.

■ About extension memory

When extension memory is installed or removed during the ROM operation, an error will occur. Install or remove extension memory only after switching to the RAM operation.

■ About operating times

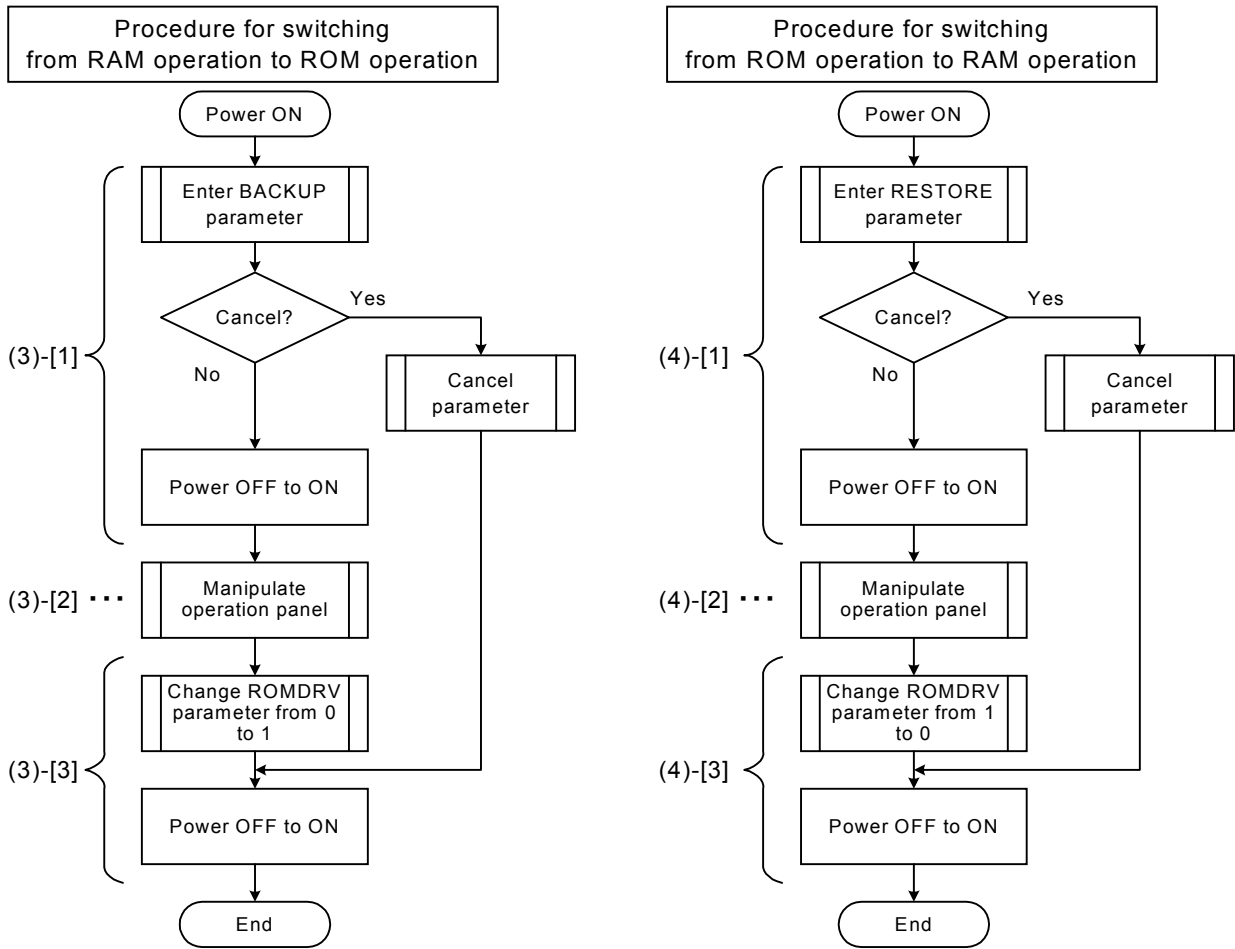
The operating times (power ON time and remaining battery time) are updated regardless of switching between the ROM and RAM operations.

■ About production information

The production information monitor (program operation count, cycle time, etc.) of Personal Computer support software is not added or updated during the ROM operation.

(2) Procedures for switching between ROM and RAM

The following flowcharts show the procedures for switching from the RAM operation to ROM operation, and vice versa.



For more information about the operating procedure of each of the above, see the following pages.

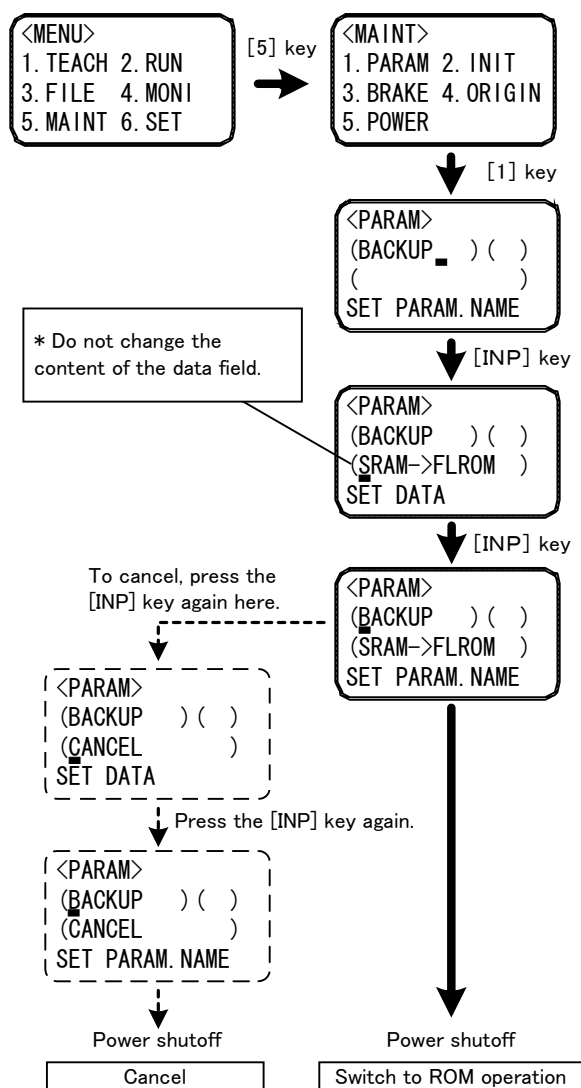
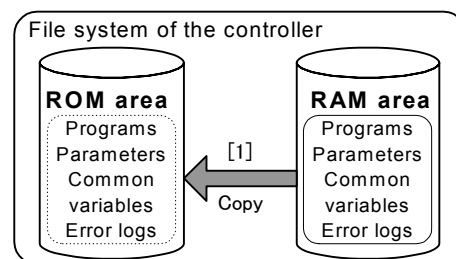
(3) Switching to the ROM operation

Use the following procedure (steps [1] to [3]) to switch to the ROM operation.

[1] Prepare to copy the information in the RAM area into the ROM area.

The programs created before the RAM operation was switched to the ROM operation are saved in the RAM area of the file system of the controller. First, copy these programs into the ROM area using the following procedure.

After the programs in the ROM area are cleared once, they are copied from the RAM area.



1) Display the parameter setting screen from the maintenance screen.

2) Enter "BACKUP" in the parameter name field, and press the [INP] key.

3) When "SRAM->FLROM" is displayed in the data field, press the [INP] key again as is.
* Do not change the content of the data field.

A self-check is performed to check whether or not the programs are normal prior to writing them into the ROM area. The ALWAYS program is automatically stopped during the self-check. When the program check is complete, the cursor moves to the parameter name field. If any abnormality is found during the program check, an error is output and the date of ROM write operation is registered in an error log.

[Cancel Operation]

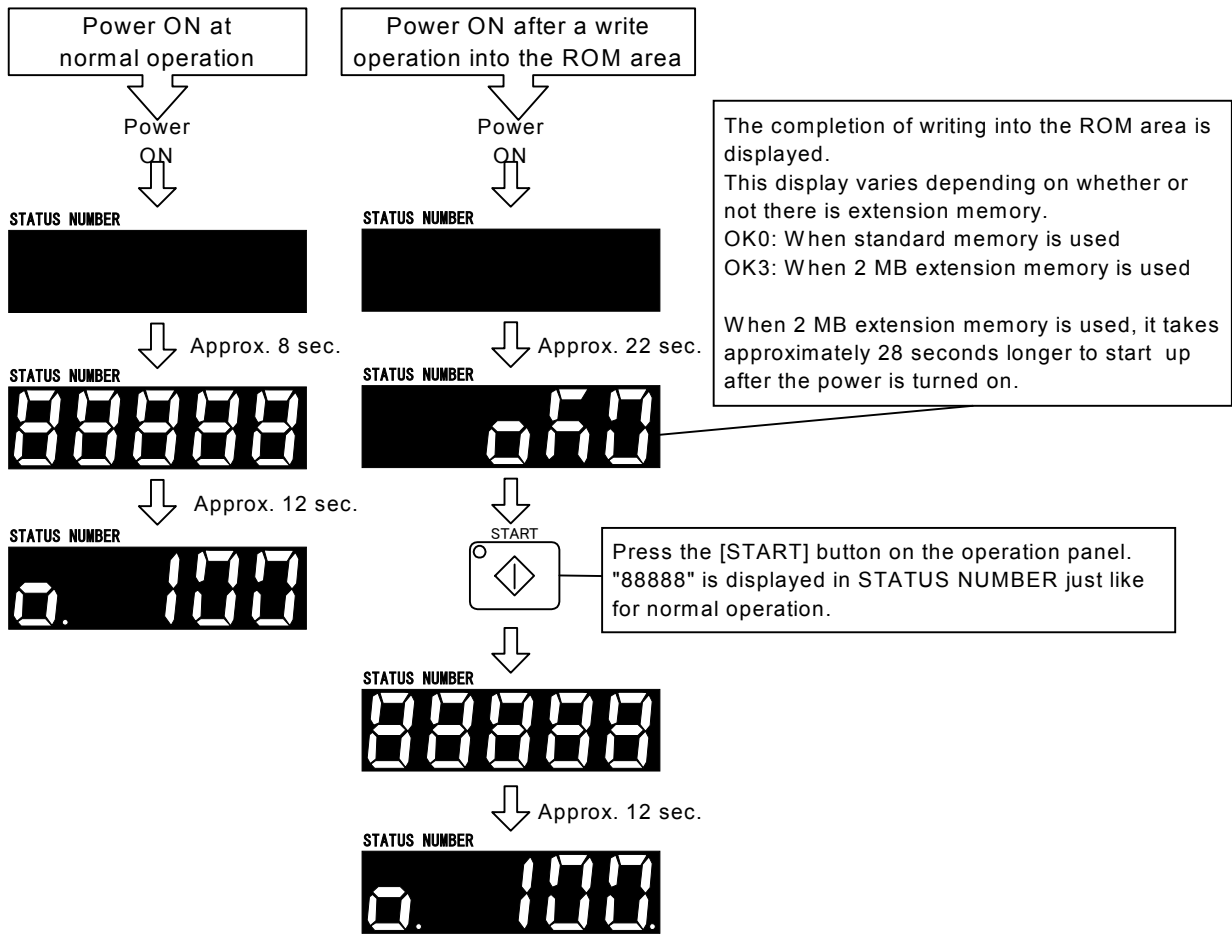
To cancel switching to the ROM operation, press the [INP] key again here. When "CANCEL" is displayed in the data field, press the [INP] key again.

4) Be sure to shut off the power here. Also, be sure to shut off the power during [Cancel Operation] in step 3) above. Copy to the ROM area is performed the next time the power is turned on. If the power is not shut off after the operation in step 3), data will not be properly copied into the ROM area.

5) Turn on the power to the controller.

After the power is turned on, "OK" is displayed in "STATUS NUMBER" on the operation panel. After verifying that "OK" is displayed, press the [START] button on the operation panel. (The following page describes the detail of this operation.)

[2] Execute a copy operation by manipulating the operation panel.

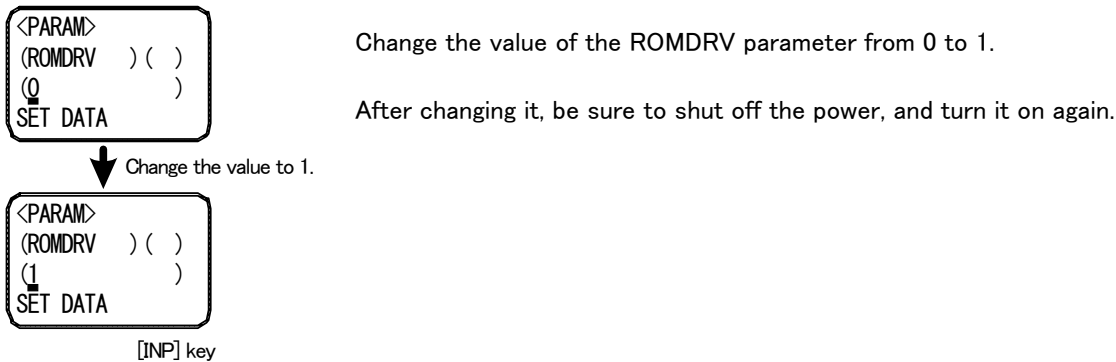


* The time required to start up after the power is turned on is based on controller software version H7. It varies depending on the version in use and how memory is used.

Caution

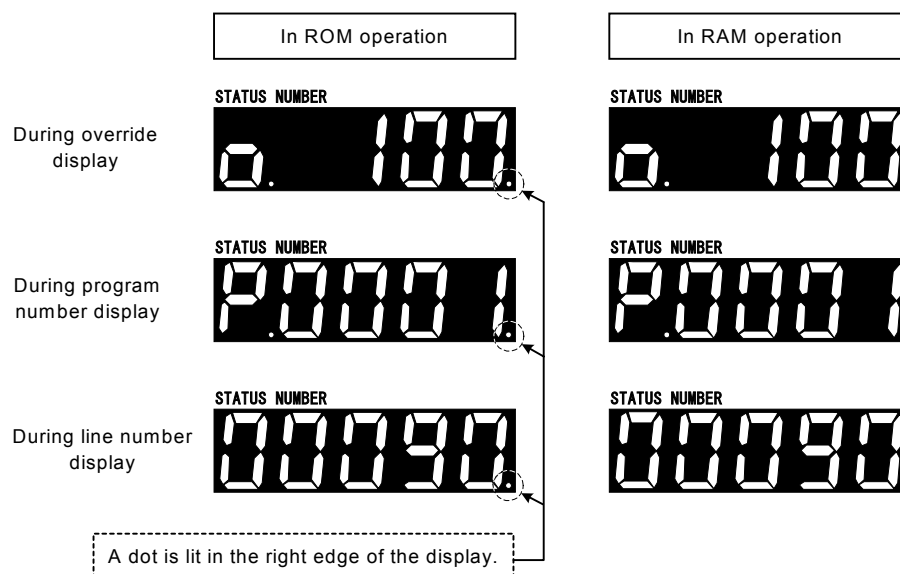
If switching from the RAM operation to the ROM operation is performed without copying any program into the ROM area, there would be no program to execute, or a program in which the corrected content has not been reflected would be executed. Therefore, be sure to perform the program copy operation described above.

[3] Change the parameter value and switch to the ROM operation.



(4) Display during the ROM operation

A dot is lit in the right edge of "STATUS NUMBER" on the operation panel during the ROM operation.



(5) Program editing during the ROM operation

It is possible to read the programs in the controller during the ROM operation; however, if a rewrite operation is performed, an error will occur. To edit a program, cancel the ROM operation (change the value of the ROMDRV parameter from 1 to 0, and reset the power to the controller) first, and then edit it. To switch back to the ROM operation after the completion of program editing, be sure to perform the operation "Copy Programs to ROM Area."

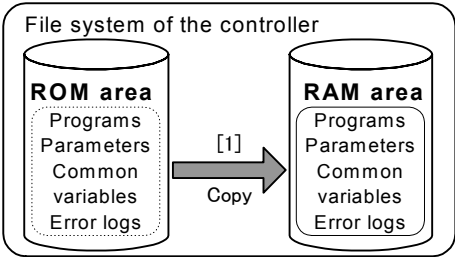
(6) Switching to the RAM operation

Use the following procedure (steps [1] to [3]) to switch to the RAM operation.

[1] Prepare to write the information in the ROM area back to the RAM area.

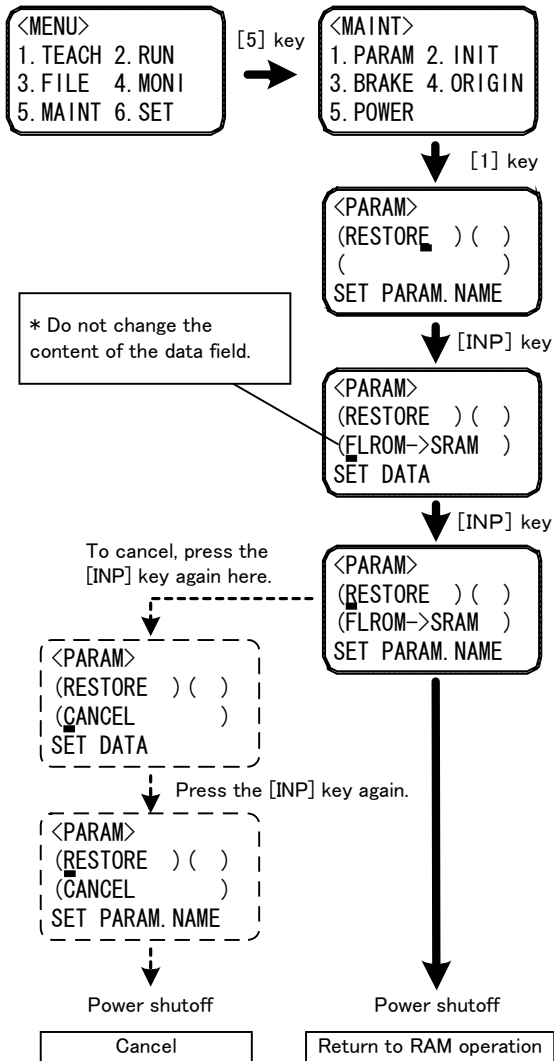
Write the programs and parameters written into the ROM area when the RAM operation was switched to the ROM operation back into the RAM area.

At this point, after the information (programs, parameters, values of common variables, and error logs) in the RAM area is cleared once, restore processing is performed from the ROM area.



Caution

If the information in the RAM area is restored into the ROM area, all the contents of the parameters changed during the ROM operation, the values of common variables, and logs of errors occurred are discarded. If any parameter was changed during the ROM operation, change the parameter again after switching to the RAM operation is complete.



1) Display the parameter setting screen from the maintenance screen.

2) Enter "RESTORE" in the parameter name field, and press the [INP] key.

3) When "FLROM->SRAM" is displayed in the data field, press the [INP] key again as is.
* Do not change the content of the data field.

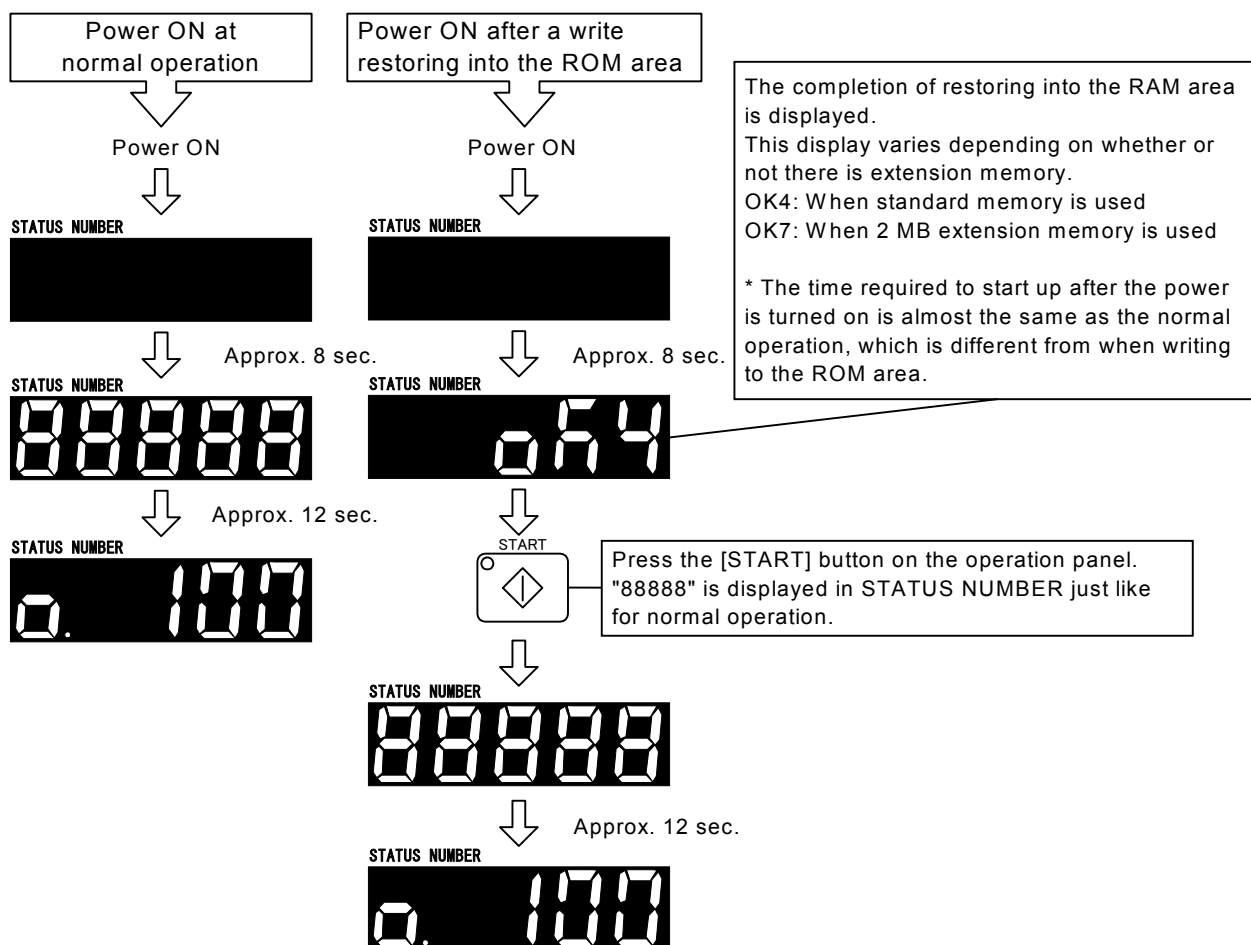
Prepare to restore the information back into the RAM area. At this point, the ALWAYS program does not stop. When the preparation is complete, the cursor moves to the parameter name field.

[Cancel Operation]
To cancel switching to the RAM operation, press the [INP] key again here. When "CANCEL" is displayed in the data field, press the [INP] key again. The cursor moves to the parameter name field.

4) Be sure to shut off the power here. Also, be sure to shut off the power during [Cancel Operation] in step 3) above. Copy to the RAM area is performed the next time the power is turned on. If the power is not shut off after the operation in step 3), data will not be properly copied into the RAM area.

5) Turn on the power to the controller.
After the power is turned on, "OK" is displayed in "STATUS NUMBER" on the operation panel. After verifying that "OK" is displayed, press the [START] button on the operation panel. (The following page describes the detail of this operation.)

[2] Execute a restore operation by manipulating the operation panel



[3] Change the parameter value and switch to the RAM operation.

<PARAM>
(ROMDRV) ()
(1)
SET DATA

Change the value of the ROMDRV parameter from 1 to 0.

↓ Change the value to 0.

<PARAM>
(ROMDRV) ()
(0)
SET DATA

After changing it, be sure to shut off the power, and turn it on again.

[INP] key

6 External input/output functions

6.1 Types

- (1)Dedicated input/output.....These I/O signals are used to indicate various statuses, such as the statuses of remote operations including the execution and stopping of robot programs, status information during execution, and status of the servo power supply. A desired function is assigned to each I/O signal. Functions are assigned in two ways: one is to set the applicable signal number in each dedicated parameter (Refer to [Page 308](#), “6.3 Dedicated input/output”), while the other is to use an emergency stop input (Refer to [Page 325](#), “6.6 Emergency stop input”). Frequently used functions have been assigned to signals in advance. The user may add new functions or modify the existing functions.
- (2)General-purpose input/output..These I/O signals are used to communicate with the PLC, etc., via robot programs. They are used to acquire positioning signals from peripheral devices or check the robot position. Usage is not limited.
- (3)Hand input/output.....These I/O signals are used in the control of robot hands, for example, to instruct the hand to open or close or to acquire information from the sensors installed in the hand. They can be controlled with user programs. Wiring is performed until near the tip of the robot hand. (Hand outputs are optional.)

Overall I/O Signal Map

	I/O Signal number	How to use
Standard remote I/Os	0 to 31 (15) ^{Note1)}	Refers/assigns with the M_IN,M_INB,M_INW,M_OUT,M_OUTB or M_OUTW variable Example) IF M_IN(0)=1 THEN M_OUT(0)=1
Expansion remote I/Os	32 to 255 (240) ^{Note1)}	Same as the above.
Hand input/output	900 to 907	Same as the above. May be substituted by the HOPEN and HCLOSE instructions. Example) IF M_IN(900) THEN M_OUT(900)=1 HOPEN 1 , HCLOSE 1
CC-LINK Bit ^{Note2)}	When one station is occupied : 6000 to 6031 When four station is occupied : 6000 to 6127 (This is the signal number for station number 1. The last 2 bits cannot be used.)	Refers/assigns with the M_IN, M_INB, M_INW, M_OUT, M_OUTB or M_OUTW variable Example) IF M_IN(6000)=1 THEN M_OUT(6000)=1
CC-LINK Register ^{Note2)}	When one station is occupied : 6000 to 6003 When four station is occupied : 6000 to 6015 (This is the signal number for station number 1.)	Referenced or assigned by the M_DIN and M_DOUT variables. Example) IF M_DIN(6000)=1000 THEN M_OUT(6000)=200

Note1)The descriptions in parentheses apply to the CR1 controller.

Note2)For details on CC-Link, refer to “CC-Link Interface Instruction Manual.”

6.2 Connection method

The robot and external input/output device are connected by connecting the optional external input/output cable to the parallel input/output unit connector in the controller and the external input/output device. One parallel input/output unit is mounted in the controller as a standard. However, up to eight units can be mounted using options.

The power supply (24VDC) for the remote input/output unit installed outside of the controller, and the power supply (12 to 24VDC) for the input/output circuit must be prepared by the user.

The standard input/output unit pin Nos. and signal assignment are shown in Table 6-1 and Table 6-2. The pin layout is shown in Fig. 6-1. Refer to the Table 6-3, for the CR1 controller source type.

*Refer to the separate "Standard Specifications" for details on the electrical specifications of the input/output circuit.

<Differences between Controller Models>

The number of standard I/O points provided by the CR1 controller is 16 for input points and 16 for output points. Controllers other than CR1 (CR2, CR4, CR7 and CR8) provide 32 input points and 32 output points as the standard.

Table 6-1 : Table of standard parallel input/output unit CN100 pin Nos. and signal assignments

Table 6-17: Table of standard pin assignments for pins 1 to 50							
Pin No.	2A-CBL wire color	Function name		Pin No.	2A-CBL wire color	Function name	
		General-purpose	Dedicated/power supply, common			General-purpose	Dedicated/power supply, common
1	Orange/red A	General-purpose output 0 General-purpose output 1	FG	26	Orange/blue A	General-purpose output 4 General-purpose output 5	FG
2	Gray/red A		0V: for pins 4-7	27	Gray/blue A		0V: for pins 29-32
3	White/red A		12/24V: for pins 4-7	28	White/blue A		12/24V: for pins 29-32
4	Yellow/red A		Running Note 2)	29	Yellow/blue A		
5	Pink/red A		During servo ON Note 2)	30	Pink/blue A		
6	Orange/red B	General-purpose output 2	During error occurrenceNote 2)	31	Orange/blue B	General-purpose output 6	0V: for pins 35-38 12/24V: for pins 35-38
7	Gray/red B	General-purpose output 3	Operation rights Note 2)	32	Gray/blue B	General-purpose output 7	
8	White/red B	General-purpose output 8	0V: for pins 10-13	33	White/blue B		
9	Yellow/red B		12/24V: for pins 10-13	34	Yellow/blue B		
10	Pink/red B			35	Pink/blue B	General-purpose output 12	
11	Orange/red C	General-purpose output 9	COM0(12V/24V(COM)) : for pins 15-22 Stop (all slots stop) Note 1)	36	Orange/blue C	General-purpose output 13	COM1(12V/24V(COM)) : for pins 40-47
12	Gray/red C	General-purpose output 10		37	Gray/blue C	General-purpose output 14	
13	White/red C	General-purpose output 11		38	White/blue C	General-purpose output 15	
14	Yellow/red C			39	Yellow/blue C		
15	Pink/red C	General-purpose input 0		40	Pink/blue C	General-purpose input 8	
16	Orange/red D	General-purpose input 1	Servo OFF Note 2)	41	Orange/blue D	General-purpose input 9	
17	Gray/red D	General-purpose input 2	Error reset Note 2)	42	Gray/blue D	General-purpose input 10	
18	White/red D	General-purpose input 3	Start Note 2)	43	White/blue D	General-purpose input 11	
19	Yellow/red D	General-purpose input 4	Servo ON Note 2)	44	Yellow/blue D	General-purpose input 12	
20	Pink/red D	General-purpose input 5	Operation rights Note 2)	45	Pink/blue D	General-purpose input 13	
21	Orange/red E	General-purpose input 6		46	Orange/blue E	General-purpose input 14	
22	Gray/red E	General-purpose input 7		47	Gray/blue E	General-purpose input 15	
23	White/red E			48	White/blue E		
24	Yellow/red E			49	Yellow/blue E		
25	Pink/red E			50	Pink/blue E		

[Caution] The wire colors indicate the identification of the optional external input/output cables.

Note 1) The stop input assignment is fixed to the input signal 0.

Note 2) These are assigned as the default. The assignment can be changed with the dedicated input/output parameter. Refer to Page 308, "6.3 Dedicated input/output" for details.

Table 6-2 : Table of standard parallel input/output unit CN300 pin Nos. and signal assignments

Pin No.	2A-CBL wire color	Function name		Pin No.	2A-CBL wire color	Function name	
		General-purpose	Dedicated/power supply, common			General-purpose	Dedicated/power supply, common
1	Orange/red A	General-purpose output 16	FG	26	Orange/blue A	General-purpose output 20	FG
2	Gray/red A		0V: for pins 4-7	27	Gray/blue A		0V: for pins 29-32
3	White/red A		12/24V: for pins 4-7	28	White/blue A		12/24V: for pins 29-32
4	Orange/red A			29	Yellow/blue A		
5	Pink/red A			30	Pink/blue A		
6	Orange/red B	General-purpose output 18	0V: for pins 10-13 12/24V: for pins 10-13	31	Orange/blue B	General-purpose output 22	0V: for pins 35-38 12/24V: for pins 35-38
7	Gray/red B	General-purpose output 19		32	Gray/blue B	General-purpose output 23	
8	White/red B			33	White/blue B		
9	Orange/red B			34	Yellow/blue B		
10	Pink/red B	General-purpose output 24		35	Pink/blue B	General-purpose output 28	
11	Orange/red C	General-purpose output 25	COM0(12V/24V(COM)) : for pins 15-22	36	Orange/blue C	General-purpose output 29	COM0(12V/24V(COM)) : for pins 40-47
12	Gray/red C	General-purpose output 26		37	Gray/blue C	General-purpose output 30	
13	White/red C	General-purpose output 27		38	White/blue C	General-purpose output 31	
14	Orange/red C			39	Yellow/blue C		
15	Pink/red C	General-purpose input 16		40	Pink/blue C	General-purpose input 24	
16	Orange/red D	General-purpose input 17		41	Orange/blue D	General-purpose input 25	
17	Gray/red D	General-purpose input 18		42	Gray/blue D	General-purpose input 26	
18	White/red D	General-purpose input 19		43	White/blue D	General-purpose input 27	
19	Orange/red D	General-purpose input 20		44	Yellow/blue D	General-purpose input 28	
20	Pink/red D	General-purpose input 21		45	Pink/blue D	General-purpose input 29	
21	Orange/red E	General-purpose input 22		46	Orange/blue E	General-purpose input 30	
22	Gray/red E	General-purpose input 23		47	Gray/blue E	General-purpose input 31	
23	White/red E			48	White/blue E		
24	Orange/red E			49	Yellow/blue E		
25	Pink/red E			50	Pink/blue E		

[Caution] The wire colors indicate the identification of the optional external input/output cables.

Table 6-3 : Standard parallel I/O interface CN100pin No. and signal assignment list<Source type of CR1 controller>

Pin No.	2A-CBL wire color	Function name		Pin No.	2A-CBL wire color	Function name	
		General-purpose	Dedicated/power supply, common			General-purpose	Dedicated/power supply, common
1	Orange/Red A		FG	26	Orange/Blue A		FG
2	Gray/Red A		0V:For pins 4-7, 10-13	27	Gray/Blue A		0V:For pins 29-32, 35-38
3	White/Red A		12V/24V:For pins 4-7, 10-13	28	White/Blue A		12V/24V:For pins 29-32, 35-38
4	Yellow/Red A	General-purpose output 0	Running	29	Yellow/Blue A	General-purpose output 4	
5	Pink/Red A	General-purpose output 1	Servo on	30	Pink/Blue A	General-purpose output 5	
6	Orange/Red B	General-purpose output 2	Error	31	Orange/Blue B	General-purpose output 6	Reserved
7	Gray/Red B	General-purpose output 3	Operation rights	32	Gray/Blue B	General-purpose output 7	
8	White/Red B		Reserved	33	White/Blue B		
9	Yellow/Red B		Reserved	34	Yellow/Blue B		
10	Pink/Red B	General-purpose output 8		35	Pink/Blue B	General-purpose output 12	
11	Orange/Red C	General-purpose output 9	COM0(12V/24V(COM)) :For pins 15-22	36	Orange/Blue C	General-purpose output 13	COM1(12V/24V(COM)) :For pins 40-47
12	Gray/Red C	General-purpose output 10		37	Gray/Blue C	General-purpose output 14	
13	White/Red C	General-purpose output 11		38	White/Blue C	General-purpose output 15	
14	Yellow/Red C			39	Yellow/Blue C		
15	Pink/Red C	General-purpose input 0	Stop(All slot) ^{Note1)}	40	Pink/Blue C	General-purpose input 8	
16	Orange/Red D	General-purpose input 1	Servo off	41	Orange/Blue D	General-purpose input 9	
17	Gray/Red D	General-purpose input 2	Error reset	42	Gray/Blue D	General-purpose input 10	
18	White/Red D	General-purpose input 3	Start	43	White/Blue D	General-purpose input 11	
19	Yellow/Red D	General-purpose input 4	Servo on	44	Yellow/Blue D	General-purpose input 12	
20	Pink/Red D	General-purpose input 5	Operation rights	45	Pink/Blue D	General-purpose input 13	
21	Orange/Red E	General-purpose input 6	Reserved	46	Orange/Blue E	General-purpose input 14	Reserved
22	Gray/Red E	General-purpose input 7		47	Gray/Blue E	General-purpose input 15	
23	White/Red E			48	White/Blue E		
24	Yellow/Red E			49	Yellow/Blue E		
25	Pink/Red E		Reserved	50	Pink/Blue E		Reserved

Note1)The assignment of the dedicated input signal "STOP" is fixed.

⚠ CAUTION

The signals assigned as a dedicated input can be used as general-purpose inputs during program execution. However, for safety purposes, these must not be shared with the general-purpose inputs except for inputting values. The signals assigned as dedicated outputs cannot be used in the program. If used, an error will occur during operation

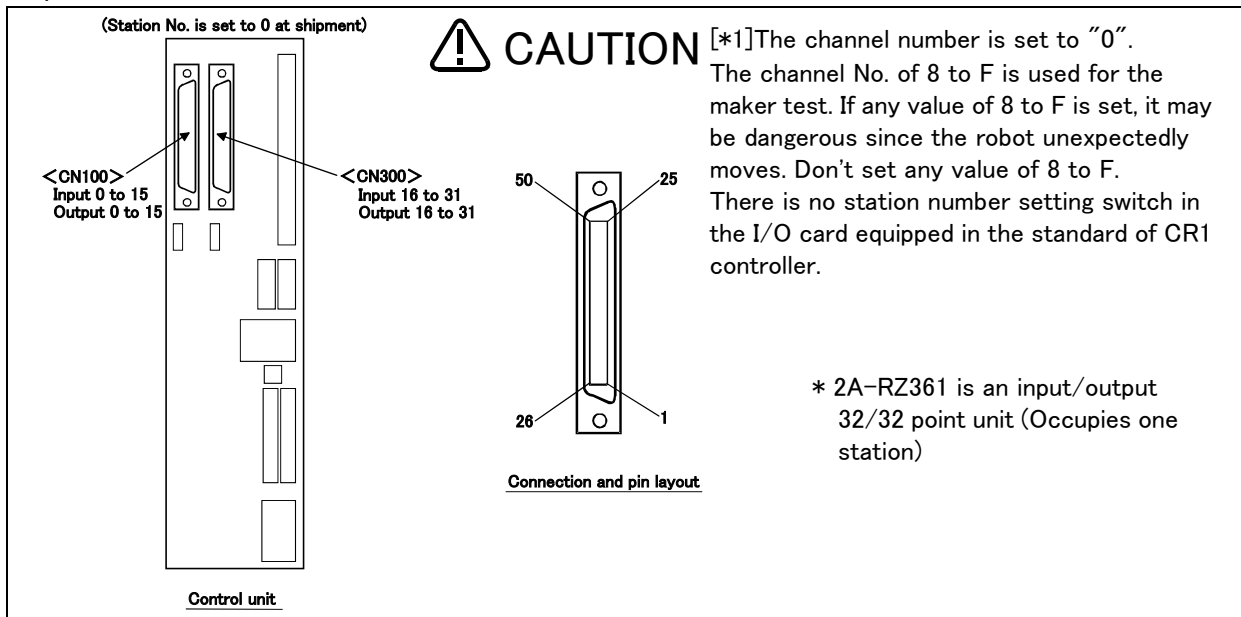


Fig.6-1 : Parallel input/output unit connection and pin layout

6.3 Dedicated input/output

The functions shown in Table 6-4 are available for the dedicated input/output signals. These are used by the parallel input/output unit by assigning the signal No. in the parameter.

The signal No. is assigned by the signal No. used in the order of "input signal" and "output signal" in each parameter. Refer to "(1)Setting the parameters" in Page 51, "3.13 Operation of maintenance screen" for details on setting the parameters. If a "-1" is designated for the assigned signal No., that signal will be invalidated.

The I/O parameters can be set on the T/B parameter screen or by using the maintenance tool of the PC support software (optional).

To use the dedicated I/O signals, set the key switch on the operation panel to AUTO (Ext.) beforehand.

Table 6-4 : Table of dedicated input/output

Parameter name	Class	Name	Function	Signal level Note5)	Factory shipment signal number. Input, output
RCREADY	Input	–	–		–1(No meaning),
	Output	Controller power ON ready	Outputs that the power has been turned ON and that the external input signal can be received.		–1
ATEXTMD	Input	–	–		–1(No meaning),
	Output	Remote mode output	This output indicates that the key switch on the operation panel is set to AUTO (Ext.), which is a remote operation mode. This signal must be turned ON before any control tasks using I/O signals can be performed.		–1
TEACHMD	Input	–	–		–1(No meaning),
	Output	Teaching mode output	This output indicates that the key switch on the operation panel is set to Teaching mode.		1
ATOPMD	Input	–	–		–1(No meaning),
	Output	Automatic mode output	This output indicates that the key switch on the operation panel is set to AUTO (OP).		1
IOENA	Input	Operation rights input signal	Sets the validity of the operation rights for the external signal control.	Level	5,
	Output	Operation rights output signal	Outputs the operation rights valid state for the external signal control. The operation right is given when the operation right input signal is ON, the mode switch is set to AUTO (Ext.), and there is no other device that currently has the operation right.		3
START (Operation right required)	Input	Start input	This input starts a program. To start a specific program, select the program using the program selection signal "PRGSEL" and numerical input "IODATA," and then input the start signal. Note that when the parameter "PST" is enabled, the system reads the program number from the numerical input (IODATA) and starts the corresponding program (i.e., program selection becomes no longer necessary). All task slots are executed during multitask operation. However, slots whose starting condition is set to ALWAYS or ERROR via a parameter "SLT**" will not be executed.	Edge	3,
	Output	Operating output	This output indicates that a program is being executed. During multitask operation, this signal turns ON when at least one task slot is operating. However, slots whose starting condition is set to ALWAYS or ERROR via a parameter "SLT**" will not be executed.		0
STOP	Input	Stop input	This input stops the program being executed. (This does not apply to slots whose starting condition is set to ALWAYS or ERROR.) The stop input signal No. is fixed to 0, and cannot be changed. All task slots are stopped during multitask operation. However, slots whose starting condition is set to ALWAYS or ERROR via a parameter "SLT**" will not be executed. Contacts A and B may be changed using the parameter INB.	Level	0(Cannot change),
	Output	Pausing output	This output indicates that the program is paused. Turns ON when there is not slot multitask running, and at least one slot is pausing. However, slots whose starting condition is set to ALWAYS or ERROR via a parameter "SLT**" will not be executed.		–1
STOPSTS	Input	–	–		–1(No meaning),
	Output	Stop signal input	Outputs that the stop is being input. (Logical ADD of all devices.)		–1

Parameter name	Class	Name	Function	Signal level Note5)	Factory shipment signal number. Input, output
SLOTINIT (Operation right required)	Input	Program reset	This input cancels the paused status of the program and brings the executing line to the top. Executing a program reset makes it possible to select a program. In the multitask mode, the program reset is applied to all task slots. However, slots whose starting condition is set to ALWAYS or ERROR via a parameter "SLT**" will not be executed.	Edge	-1,
	Output	Program selection enabled output	Outputs that in the program selection enabled state. Turns ON when program are not running or pausing. In multitask operation, this output turns ON when all task slots are neither operating nor paused. However, slots whose starting condition is set to ALWAYS or ERROR via a parameter "SLT**" will not be executed.		-1
ERRRESET	Input	Error reset input signal	Releases the error state.	Edge	2,
	Output	Error occurring output signal	Outputs that an error has occurred.		2
SRVON (Operation right required)	Input	Servo ON input signal	This input turns ON the servo power supply for the robot. With a multi-mechanism configuration, the servo power supplies for all mechanisms will be turned ON.	Edge	4,
	Output	In servo ON output signal	This output turns ON when the servo power supply for the robot is ON. If the servo power supply is OFF, this output also remains OFF. With a multi-mechanism configuration, this output turns ON when the servo of at least one mechanism is ON.		1
SRVOFF	Input	Servo OFF input signal	This input turns OFF the servo power supply for the robot.(Applicable to all mechanisms) The servo cannot be turned ON while this signal is being input.	Level	1,
	Output	Servo ON disable output signal	This output indicates a status where the servo power supply cannot be turned ON. (Echo back)		-1
AUTOENA	Input	Automatic operation enabled input	Disables automatic operation when inactive. If this signal is inactive, and the AUTO mode is entered, E5010 will occur. This input is used to interlock the operations via the operation panel with the I/O signals. Use of this input is not a requirement.	Level	-1,
	Output	Automatic operation enabled output	Outputs the automatic operation enabled state.		-1
CYCLE	Input	Cycle stop input signal	Starts the cycle stop.	Edge	-1,
	Output	In cycle stop operation output signal	Outputs that the cycle stop is operating. Turns OFF when the cycle stop is completed.		-1
MELOCK (Operation right required)	Input	Machine lock input signal	Sets/releases the machine lock state for all mechanisms. This can be set or released when all slots are in the program selection state. Signal level will be set to Level when program selection is enabled.	Level	-1,
	Output	In machine lock state output signal	Outputs the machine lock state. This turns On when at least one mechanism is in the machine lock state. During the machine lock state, the robot will not move, and program operation will be enabled.		-1
SAFEPOS (Operation right required)	Input	Safe point return input signal	Requests the safe point return operation. This signal initiates a joint interpolation movement to the position set by the parameter "JSAFE." The speed is determined by the override setting. Be careful not to interfere with peripheral devices.	Edge	-1,
	Output	In evasion point return output signal	Outputs that the evasion point return is taking place.		-1
BATERR	Input	—	—		-1,
	Output	Battery voltage drop	Outputs that the battery voltage has dropped.		-1
OUTRESET (Operation right required)	Input	General-purpose output signal reset	Resets the general-purpose output signal. The operation at the input is set with parameters ORST0 to ORST224.	Edge	-1,
	Output	—	—		-1(No meaning)
HLVLERR	Input	—	—		-1(No meaning),
	Output	High level error output signal	Outputs that a high level error is occurring.		-1
LLVLERR	Input	—	—		-1(No meaning),
	Output	Low level error output signal	Outputs that a low level error is occurring.		-1
CLVLERR	Input	—	—		-1(No meaning),
	Output	Warning level error output signal	Outputs that a warning level error is occurring.		-1

Parameter name	Class	Name	Function	Signal level Note5)	Factory shipment signal number. Input, output
EMGERR	Input	—	—		-1(No meaning),
	Output	Emergency stop output signal	Outputs that an emergency stop is occurring.		-1
SnSTART (n=1 to 32) (Operation right required)	Input	Slot n start input	Starts each slot. n=1 to 32	Edge	-1,
	Output	Slot n in operation output	Outputs the operating state for each slot. n=1 to 32		-1
SnSTOP (n=1 to 32)	Input	Slot n stop input	Outputs the operating state for each slot. n=1 to 32	Level	-1,
	Output	Slot n in pausing output	Outputs that each slot and program is temporarily stopped. n=1 to 32		-1
MnSRVOFF (n=1 to 3)	Input	Mechanism n servo OFF input signal	This signal turns OFF the servo for each mechanism. n=1 to 3 The servo cannot be turned ON while this signal is being input.	Level	-1,
	Output	Mechanism n servo ON disabled output signal	Outputs the servo ON disabled state. (Echo back)		-1
MnSRVON (n=1 to 3) (Operation right required)	Input	Mechanism n servo ON input signal	Turns the servo for each mechanism ON. n=1 to 3	Edge	-1,
	Output	Mechanism n in servo ON output signal.	Turns the servo for each mechanism ON. n=1 to 3		-1
MnMELOCK (n=1 to 3) (Operation right required)	Input	Mechanism n machine lock input signal	Sets/releases the machine lock state for each mechanism. n=1 to 3	Level	-1,
	Output	Mechanism n in machine lock output signal	Outputs that the machine lock state is entered. n=1 to 3		-1
PRGSEL (Operation right required)	Input	Program selection input signal	Designates the setting value for the program No. with numeric value input signals. The program for slot 1 is selected. Output this signal when at least 30 ms has elapsed following the start of output to the numerical input (IODATA). This signal should also be output to the robot for at least 30 ms.	Edge	-1, Note5)
	Output	—	—		-1(No meaning)
OVRDSEL (Operation right required)	Input	Override selection input signal	Designates the setting value for the override with the numeric value input signals. Output this signal when at least 30 ms has elapsed following the start of output to the numerical input (IODATA). This signal should also be output to the robot for at least 30 ms.	Edge	-1, Note5)
	Output	—	—		-1(No meaning)
IODATA	Input	Numeric value input (Start bit number, end bit number)	Numerical values are read as binary values. • Program number (Read by the PRGSEL) If the parameter "PST" is enabled, it is read by the start signal. • Override (Read by the OVRDSEL) The bit width can be set arbitrarily. However, the accuracy of output values cannot be guaranteed when they exceed the set bit width. Output this input to the robot for at least 30 ms before inputting the PRGSEL or other setting signals.	Level	-1(Start bit), -1(End bit), Note1) Note5)
	Output	Numeric value output (Start bit number, end bit number)	Numerical values are output as binary values. • Program number (Output by the PRGOUT), • Override (Output by the OVRDOUT), • Outputs the line number (output by the LINEOUT) • Error number (output by the ERROUT). The bit width can be set arbitrarily. However, the accuracy of output values cannot be guaranteed when they exceed the set bit width. Read this signal when at least 30 ms has elapsed following the start of input of a program number (PRGOUT) or other signal to the robot.		-1(Start bit), -1(End bit)
PRGOUT	Input	Program No. output request	The program number for task slot 1 is output to the numerical output (IODATA). After the start of inputting this signal to the robot, wait at least 30 ms before reading the numerical output (IODATA) signal.	Edge	-1, Note5)
	Output	Program No. output signal	The "program number output in progress" status is output to the numerical output.		-1
LINEOUT	Input	Line No. output request	The line number for task slot 1 is output to the numerical output (IODATA). After the start of inputting this signal to the robot, wait at least 30 ms before reading the numerical output (IODATA) signal.	Edge	-1, Note5)
	Output	Line No. output request	The "line number output in progress" status is output to the numerical output.		-1

Parameter name	Class	Name	Function	Signal level Note5)	Factory shipment signal number. Input, output
OVRDOUT	Input	Override value request	The OP override is output to the numerical output (IODATA). After the start of inputting this signal to the robot, wait at least 30 ms before reading the numerical output (IODATA) signal.	Edge	-1, Note5)
	Output	Override value output signal	The "override output in progress" status is output to the numerical output.		-1
ERROUT	Input	Error No. output request	The error number is output to the numerical output (IODATA). After the start of inputting this signal to the robot, wait at least 30 ms before reading the numerical output (IODATA) signal.	Edge	-1, Note5)
	Output	Error No. output signal	The "error number output in progress" status is output to the numerical output.		-1
JOGENA (Operation right required)	Input	Jog valid input signal	Jogs the designated axis in the designated mode. Operation takes place while this signal is ON.	Level	-1,
	Output	Jog valid output signal	Outputs that the jog operation is entered.		-1
JOGM	Input	Jog mode input (start No., end No.)	Designates the jog mode. 0/1/2/3/4 = Joint, XYZ, cylindrical, 3-axis XYZ, tool	Level	-1(Start bit), -1(End bit),
	Output	Jog mode output (start No., end No.)	Outputs the current jog mode.		-1(Start bit), -1(End bit) Note2)
JOG+	Input	Jog feed plus side for 8- axes (start No., end No.)	Designates the jog operation axis. JOINT jog mode: J1, J2, J3, J4, J5, J6, J7 and J8 axes from the start number. XYZ jog mode: X, Y, Z, A, B, C, L1 and L2 axes from the start number. CYLINDER jog mode: X, θ , Z, A, B, C, L1 and L2 axes from the start number. 3-axis XYZ jog mode: X, Y, Z, J4, J5 and J6 axes from the start number. TOOL jog mode: X, Y, Z, A, B and C axes from the start number.	jiku	-1, Note3) -1
	Output	—	—		
JOG-	Input	Jog feed minus side for 8- axes (start No., end No.)	Designates the jog operation axis. JOINT jog mode: J1, J2, J3, J4, J5, J6, J7 and J8 axes from the start number. XYZ jog mode: X, Y, Z, A, B, C, L1 and L2 axes from the start number. CYLINDER jog mode: X, θ , Z, A, B, C, L1 and L2 axes from the start number. 3-axis XYZ jog mode: X, Y, Z, J4, J5 and J6 axes from the start number. TOOL jog mode: X, Y, Z, A, B and C axes from the start number.	Level	-1, Note3) -1
	Output	—	—		
HNDCTRLn (n=1 to 3)	Input	—	—		
	Output	Mechanism n hand output signal state (start No., end No.)	Outputs the general-purpose output(n=1) 900 to 907 state. Outputs the general-purpose output(n=2) 910 to 917 state. Outputs the general-purpose output(n=3) 920 to 927 state. Example) To output the four points from 900 through 903 to general-purpose output signals 3, 4, 5 and 6, set the HNDCTRL1 to (3, 6).		-1(Start bit), -1(End bit)
HNDSTSn (n=1 to 3)	Input	—	—		
	Output	Mechanism n hand input signal state (start No., end No.)	Outputs the hand input(n=1) 900 to 907 state. Outputs the hand input(n=2) 910 to 917 state. Outputs the hand input(n=3) 920 to 927 state. Example) To output the four points from 900 through 903 to general-purpose output signals 3, 4, 5 and 6, set the HNDCTRL1 to (3, 6).		-1(Start bit), -1(End bit)
HNDERRn (n=1 to 3)	Input	Mechanism n hand error input signal	Requests the hand error occurrence. A LOW level error (error number 30) will be generated.	Level	-1,
	Output	Mechanism n hand error output signal	Outputs that a hand error is occurring.		-1
AIRERRn (n=1 to 5)	Input	Mechanism n pneumatic pressure error input signal	Request the pneumatic pressure error occurrence. A LOW level error (error number 31) will be generated.	Level	-1,
	Output	Mechanism n pneumatic error output signal	Outputs that a pneumatic pressure error is occurring.		-1
USRAREA	Input	—	—		
	Output	User-designated area 8- points (start No., end No.)	Outputs that the robot is in the user-designated area. The output is made sequentially for areas 1, 2 and 3, as designed from the one closest to the start number. The area is set with parameters AREA1P1, AREA1P2 to AREA8P1 and AREA8P2.		-1(Start bit), -1(End bit) Note4)

Note 1) Set in the order of input start No., input end No., output start No. and output end No.

When using as the input or output of an actual value, use from the start No. to the end No., and express as a binary. The start No. indicates the low-order bit, and the end No. indicates the high-order bit. Set only the numbers required to express the value.

For example, when using for program selection and only programs 1 to 6 are available, the expression can be created by setting 3 bits. Up to 16 bits can be set.

Assignment examples are shown below.

Example) To set the start input signal in general-purpose input 16, and the operating output signal in general-purpose output 25.

Parameter START = {16, 25}

Example) When setting 4 bits of numerical input to general-purpose inputs 6 to 9, and 5 bits of numerical output to general-purpose outputs 6 to 10.

Parameter IODATA = {6, 9, 6, 10}

Note 2) Set in the order of input start No., input end No., output start No. and output end No.

When using as the actual jog mode, use from the start No. to the end No., and express as a binary. The start No. indicates the low-order bit, and the end No. indicates the high-order bit. Set only the numbers required to express the value.

For example, when using only the joint mode and XYZ mode, the expression can be created by setting 2 bits. Up to 3 bits can be set.

Note 3) Set in the order of input start No. and input end No.. The start No. designates the J1/X axis, and the end No. can designate up to the J8/L2 axis.

For example, when using a 6-axis robot, only 6 bits need to be set.

Even if using a 4-axis robot, when using the XYZ mode, the C axis is required, so 6 bits must be set. Up to 8 bits can be set.

Note 4) Set in the order of output start No. and output end No. The start number specifies area 1, while the end number specifies area 8 in the largest configuration.

For example, setting 2 bits will suffice if only two areas are used. A maximum of 8 bits can be set.

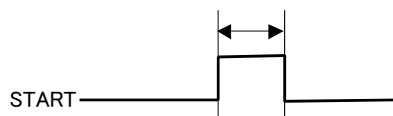
Note 5) The meanings of the signal level are explained below.

Level: → The designated function is validated when the signal is ON, and the function is invalidated when the signal is OFF. Make sure the signal is turned ON for at least 30 ms.

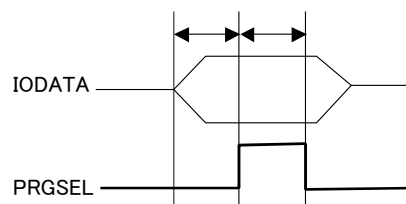
Edge: → The designated function is validated when the signal changes from the OFF to ON state, and the function maintains the original state even when the signal returns to the OFF state.

Example)

Set an interval of at least 300 ms



Set an interval of at least 300 ms



6.4 Enable/disable status of signals

Note that depending on the input signal type, the function may not occur even if the target signal is input depending on the robot state at that time, such as during operation or when stop is input.

The relation of the robot status to the input signal validity is shown below.

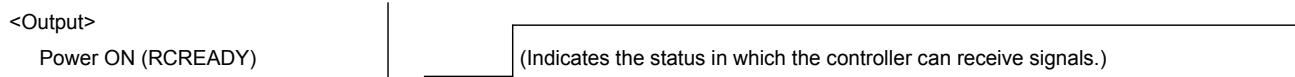
Table 6-5 : Validity state of dedicated input signals

Parameter name	Name	Validity of symbol on left according to robot states.
SLOTINIT	Program reset	These do not function in the operation state (when START output is ON).
SAFEPOS	Evasion point return input	
OUTRESET	General-purpose output signal reset	
PRGSEL	Program selection input	
START SnSTART (n=1 to 32)	Start input	These function only when the external input/output has the operation rights (when IOENA output is ON).
SLOTINIT	Program reset	
SRVON MnSRVON (n=1 to 3)	Servo ON input	
MELOCK MnMELOCK (n=1 to 3)	Machine lock input	
SAFEPOS	Evasion point return input	
PRGSEL	Program selection input	
OVRDSEL	Override selection input	
JOGENA	Jog enable input	
START	Start input	These do not function in the stop input state (when STOPSTS is ON).
SLOTINIT	Program reset	
SAFEPOS	Evasion point return input	
JOGENA	Jog enable input	
SRVON	Servo ON input	This does not function in the servo OFF input state.
MELOCK	Machine lock input	This functions only in the program selection state (when SLOTINIT output is ON).

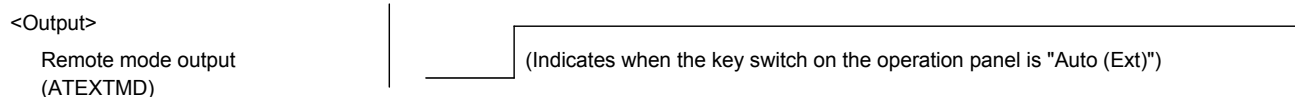
6.5 External signal timing chart

6.5.1 Individual timing chart of each signal

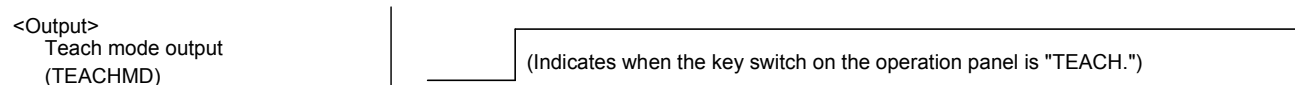
(1) RCREADY (Controller's power ON completion output)



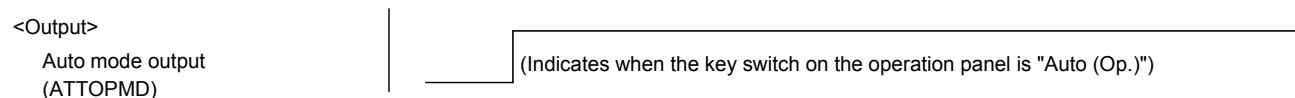
(2) ATEXTMD (Remote mode output)



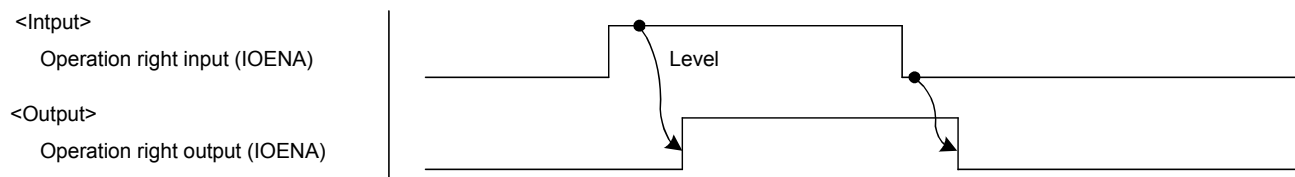
(3) TEACHMD (Teach mode output)



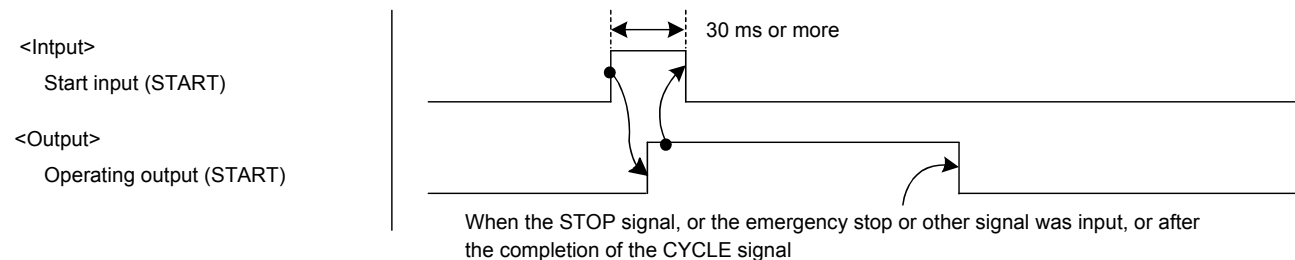
(4) ATTOPMD (Auto mode output)



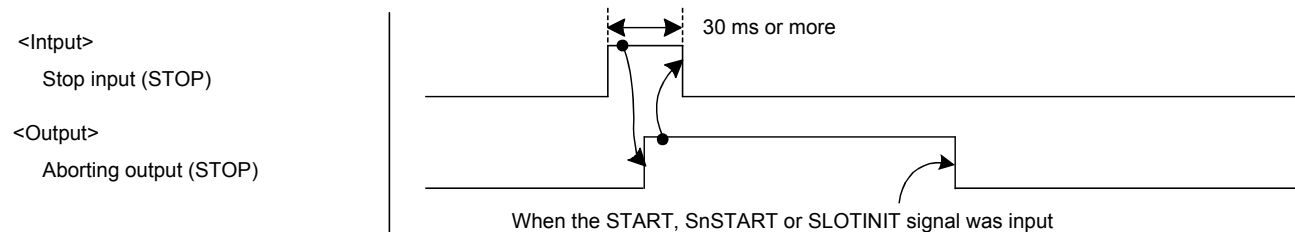
(5) IOENA (Operation right input signal/operation right output signal)



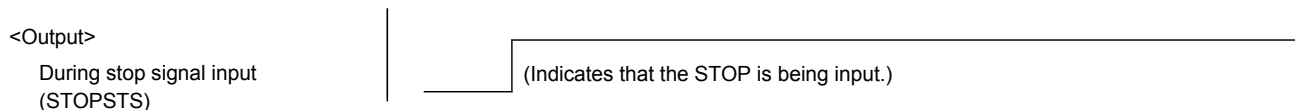
(6) START (Start input/operating output)



(7) STOP (Stop input/aborting output)



(8) STOPSTS (Output during stop signal input)

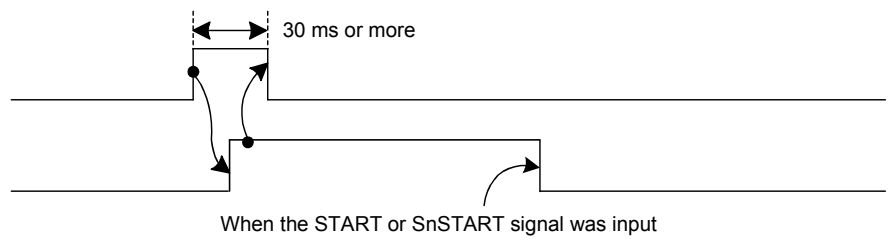


(9) SLOTINIT (Program reset input/program selectable output)

<Input>

Program reset (SLOTINIT)

<Output>

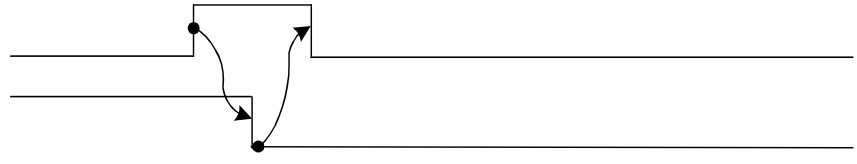
Program selectable output
(SLOTINIT)

(10) ERRRESET (Error reset input/output during error occurrence)

<Input>

Error reset input (ERRRESET)

<Output>

Output during error occurrence
(ERRRESET)

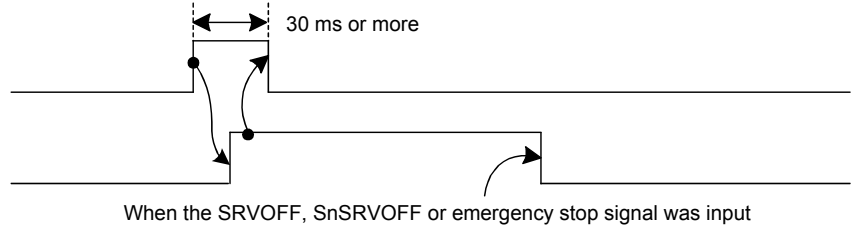
(11) SRVON (Servo ON input/output during servo ON))

<Input>

Servo ON input (SRVON)

<Output>

Output during servo ON (SRVON)



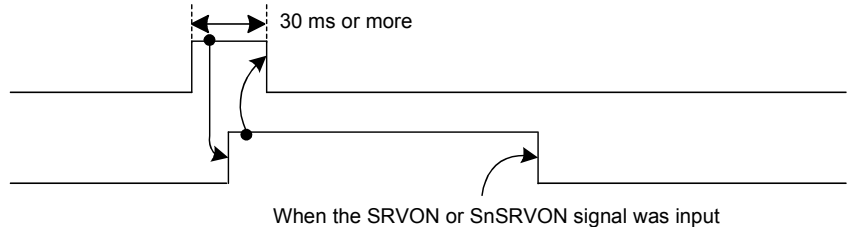
(12) SRVOFF (Servo OFF input/servo ON disable output)

<Input>

Servo OFF input (SRVOFF)

<Output>

Servo ON disable output (SRVOFF)

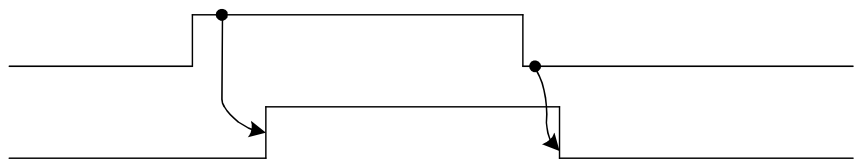


(13) AUTOENA (Auto operation input/auto operation enable output)

<Input>

Auto operation enable input
(AUTOENA)

<Output>

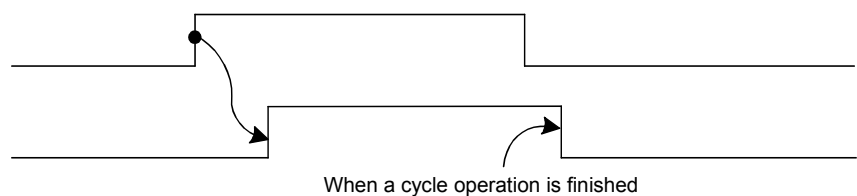
Auto operation enable output
(AUTOENA)

(14) CYCLE (Cycle stop input/output during cycle stop operation)

<Input>

Cycle stop input (CYCLE)

<Output>

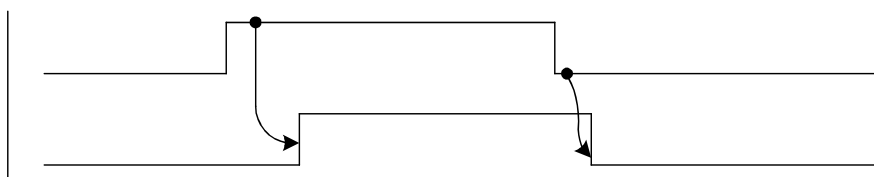
Output during cycle stop operation
(CYCLE)

(15) MELOCK (Machine lock input/output during machine lock)

<Input>

Machine lock input (MELOCK)

<Output>

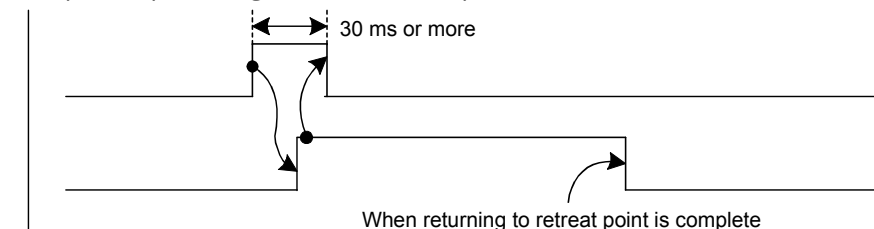
Output during machine lock
(MELOCK)

(16) SAFEPOS (Return to retreat point input/output during return to retreat point)

<Input>

Return to retreat point input
(SAFEPOS)

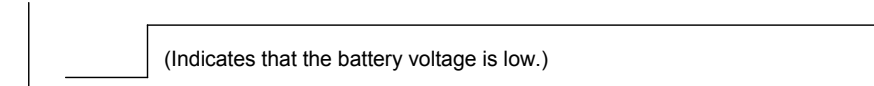
<Output>

Output during return to retreat point
(SAFEPOS)

(17) BATERR (Low battery voltage output)

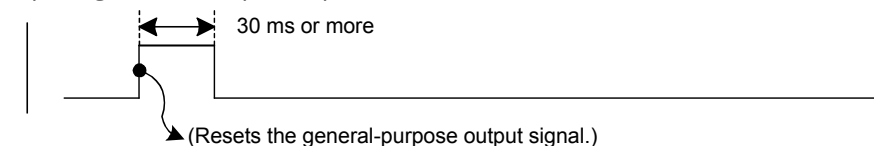
<Output>

Low battery voltage (BATERR)



(18) OUTRESET (General-purpose output signal reset request input)

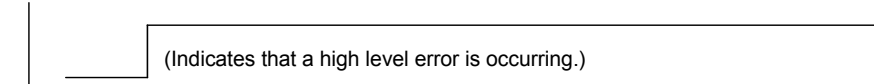
<Input>

General-purpose output signal reset
(OUTRESET)

(19) HVLERR (Output during high level error occurrence)

<Output>

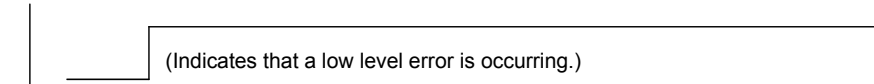
High level error output (HVLERR)



(20) LLVLERR (Output during low level error occurrence)

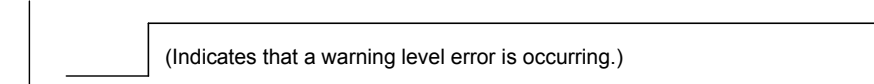
<Output>

Low level error output (LLVLERR)



(21) CLVLERR (Output during warning level error occurrence)

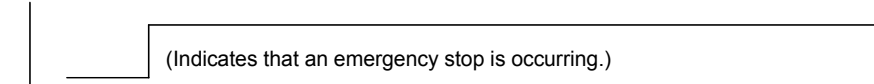
<Output>

Warning level error output
(CLVLERR)

(22) EMGERR (Output during emergency stop)

<Output>

Emergency stop output (EMGERR)

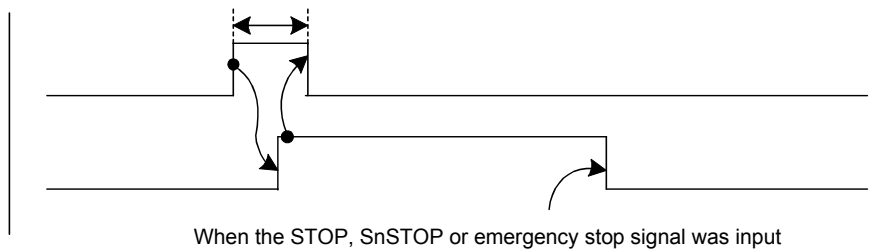


(23) SnSTART (Slot n start input/output during slot n operation)

<Input>

Slot n start input (SnSTART)

<Output>

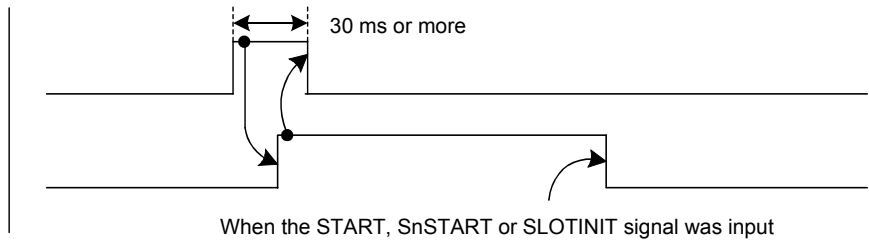
Output during slot n operation
(SnSTART)

(24) SnSTOP (Slot n stop input/output during slot n aborting)

<Input>

Slot n stop input (SnSTOP)

<Output>

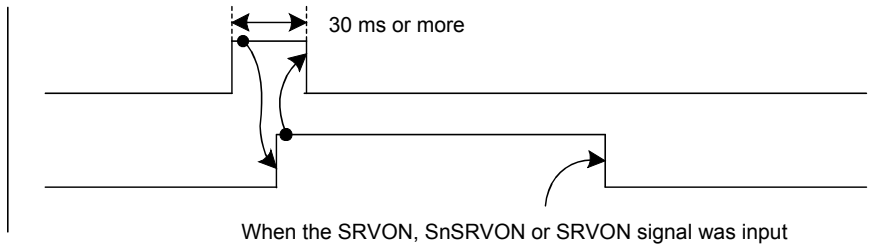
Output during slot n aborting
(SnSTOP)

(25) MnSRVOFF (Mechanical n servo OFF input/mechanical n servo ON disable output)

<Input>

Mechanical n servo OFF input
(MnSRVOFF)

<Output>

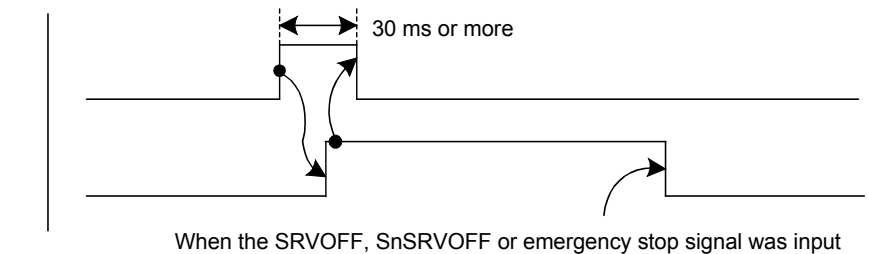
Mechanical n servo ON disable output
(MnSRVOFF)

(26) MnSRVON (Mechanical n servo ON input/output during mechanical n servo ON)

<Input>

Mechanical n servo ON input
(MnSRVON)

<Output>

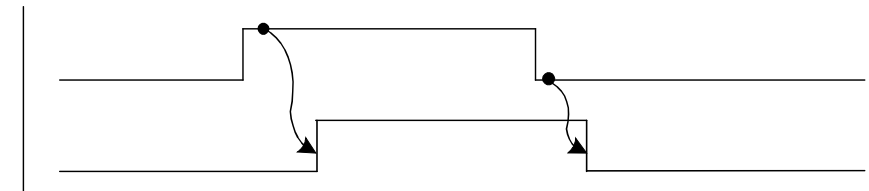
Output during mechanical n servo ON
(MnSRVON)

(27) MnMELOCK (Mechanical n machine lock input/output during mechanical n machine lock)

<Input>

Mechanical n machine lock input
(MnMELOCK)

<Output>

Output during mechanical n machine
lock (MnMELOCK)

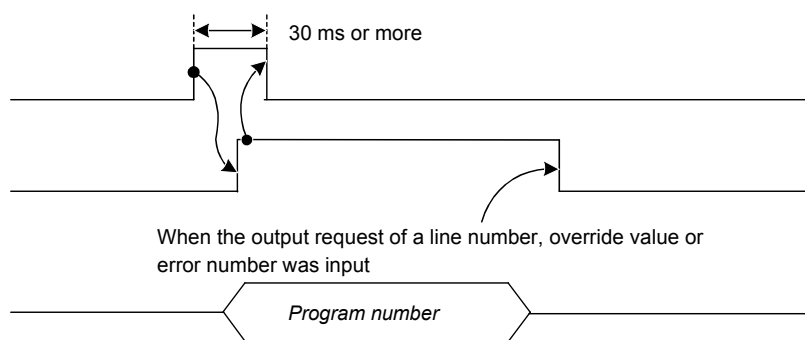
(28) PRGSEL (Program selection input)

* This is used together with the numeric value input (IODATA).

<Input>
Program number output request
(PRGOUT)

<Output>
Outputting program number (PRGOUT)

Numeric value output (IODATA)



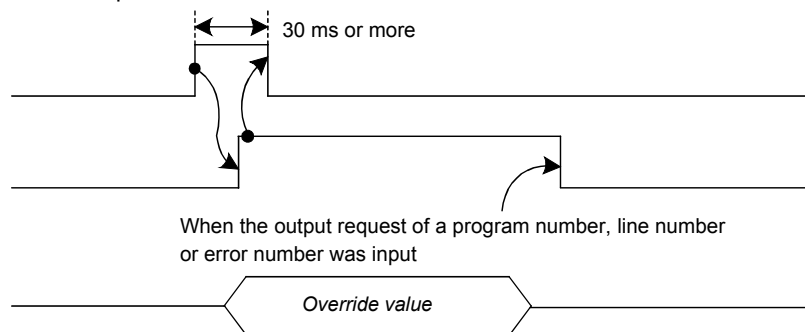
OVRDSEL (Override selection input)

* This is used together with the numeric value input (IODATA).

<Input>
Override value output request
(OVRDOUT)

<Output>
Override value output request
(OVRDOUT)

Numeric value output (IODATA)



(29) IODATA (Numeric value input/numeric value output)

* This is used together with PRGSEL, OVRDSEL, PRGOUT, LINEOUT, OVRDOUT or ERRROUT.

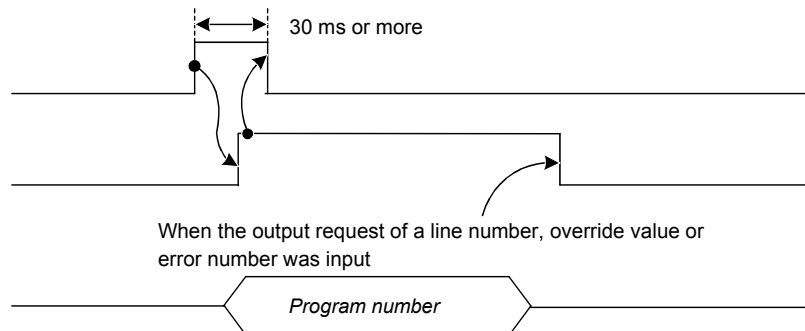
(30) PRGOUT (Program number output request input/outputting program number)

* This is used together with the numeric value output (IODATA).

<Input>
Program number output request
(PRGOUT)

<Output>
Outputting program number (PRGOUT)

Numeric value output (IODATA)



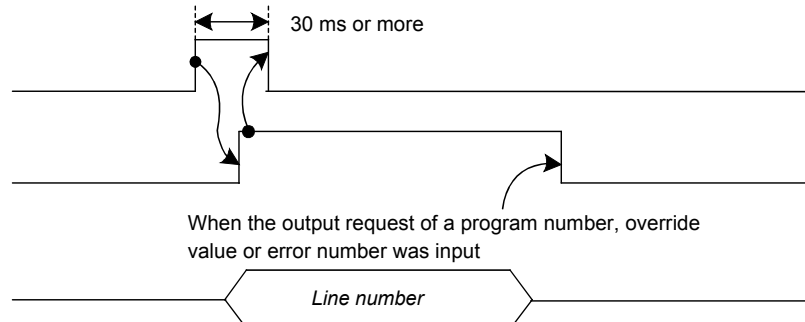
(31) LINEOUT (Line number output request input/outputting line number)

* This is used together with the numeric value output (IODATA).

<Input>
Line number output request (LINEOUT)

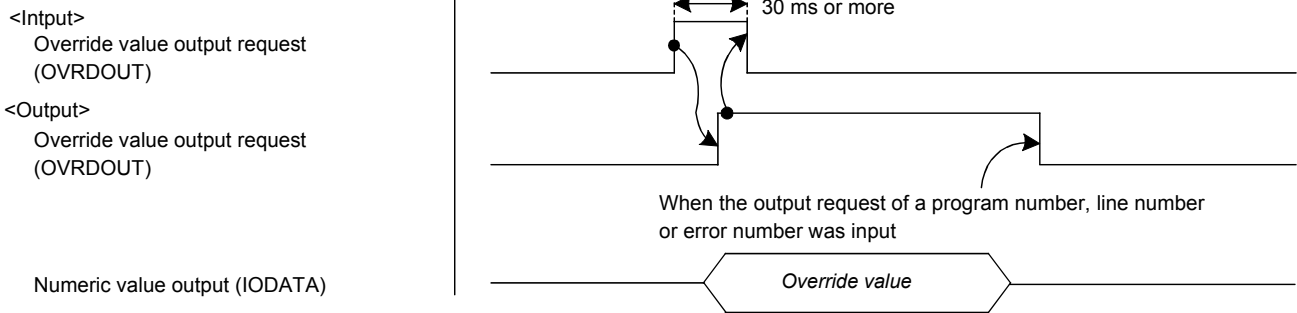
<Output>
Outputting line number (LINEOUT)

Numeric value output (IODATA)



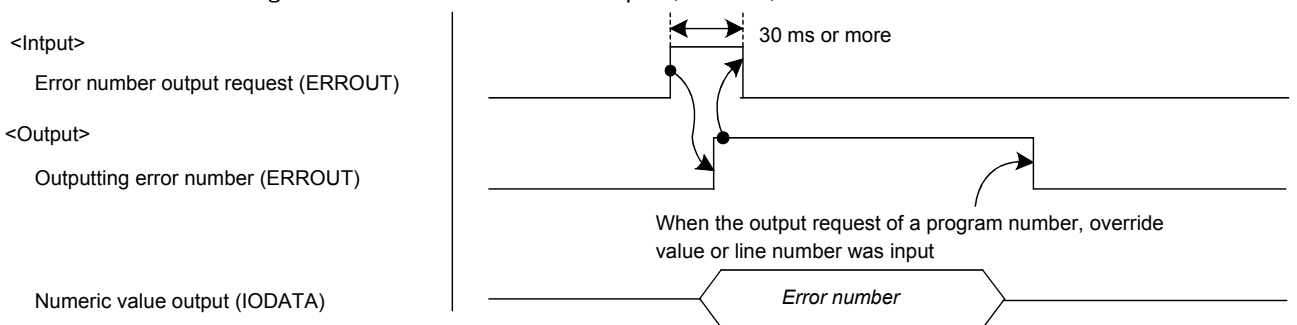
(32) OVRDOUT (Override value output request/outputting override value)

* This is used together with the numeric value output (IODATA).

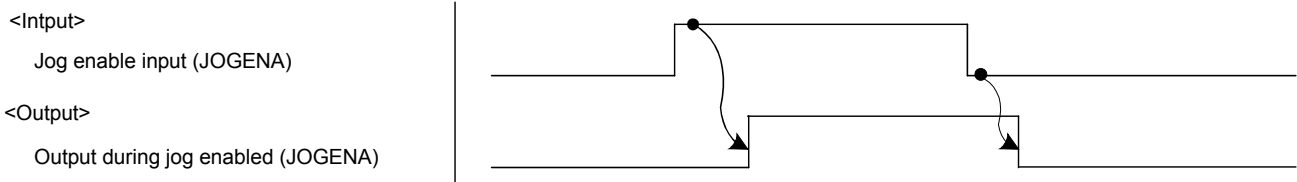


(33) ERROUT (Error number output request/outputting error number)

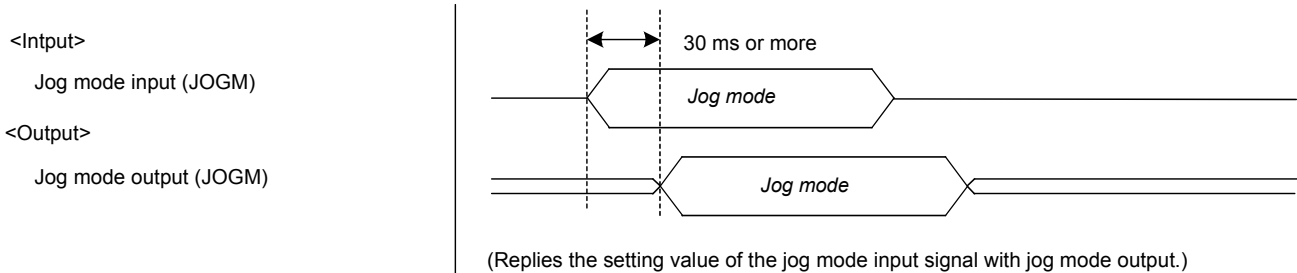
* This is used together with the numeric value input (IODATA).



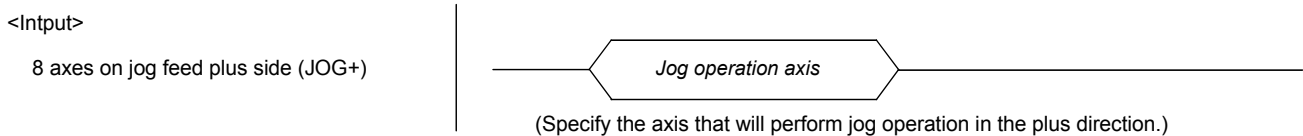
(34) JOGENA (Jog enable input/output during jog enabled)



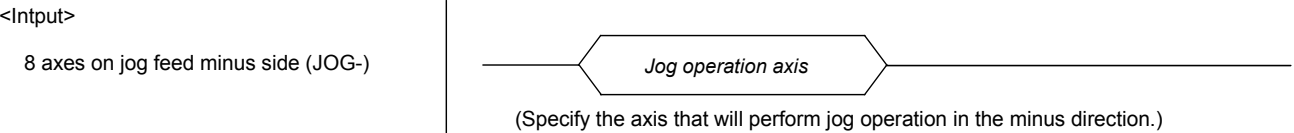
(35) JOGM (Jog mode input/jog mode output)



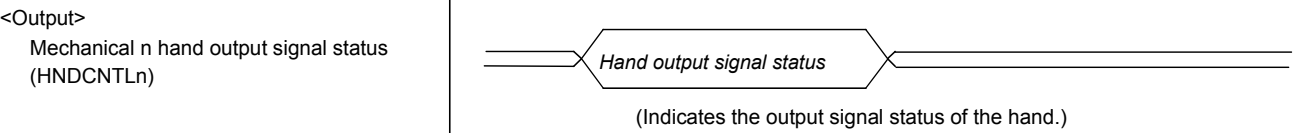
(36) JOG+ (Input for 8 axes on jog feed plus side)



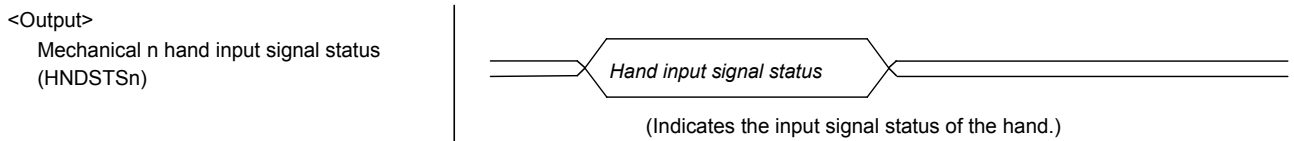
(37) JOG- (Input for 8 axes on jog feed minus side)



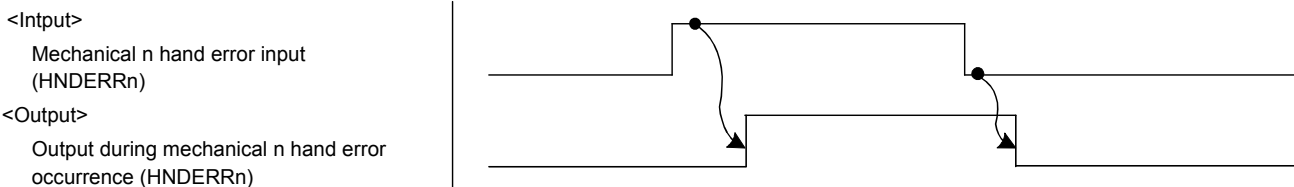
(38) HNDCTLn (Mechanical n hand output signal status)



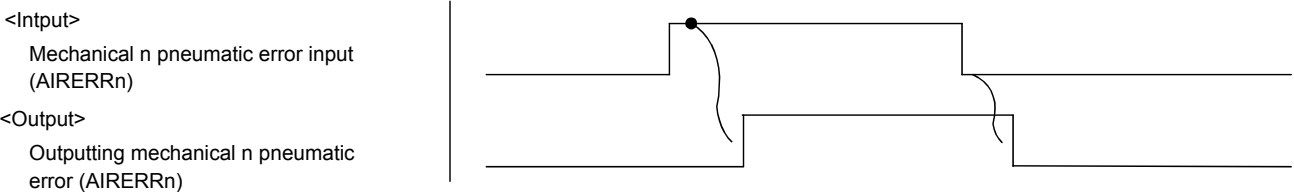
(39) HNDSTSn (Mechanical n hand input signal status)



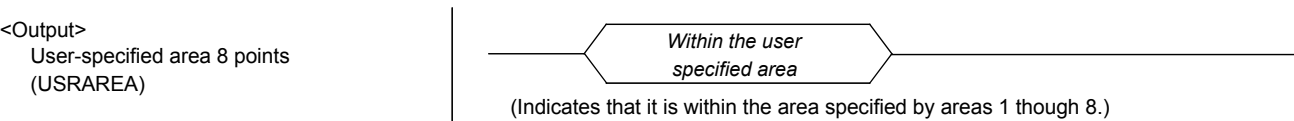
(40) HNDERRn (Mechanical n hand error input signal/output during mechanical n hand error occurrence)



(41) AIRERRn (Mechanical n pneumatic error input signal/outputting mechanical n pneumatic error)



(42) USRAREA (User-specified area 8 points output)



6.5.2 Timing chart example

(1) External signal operation timing chart (Part 1)

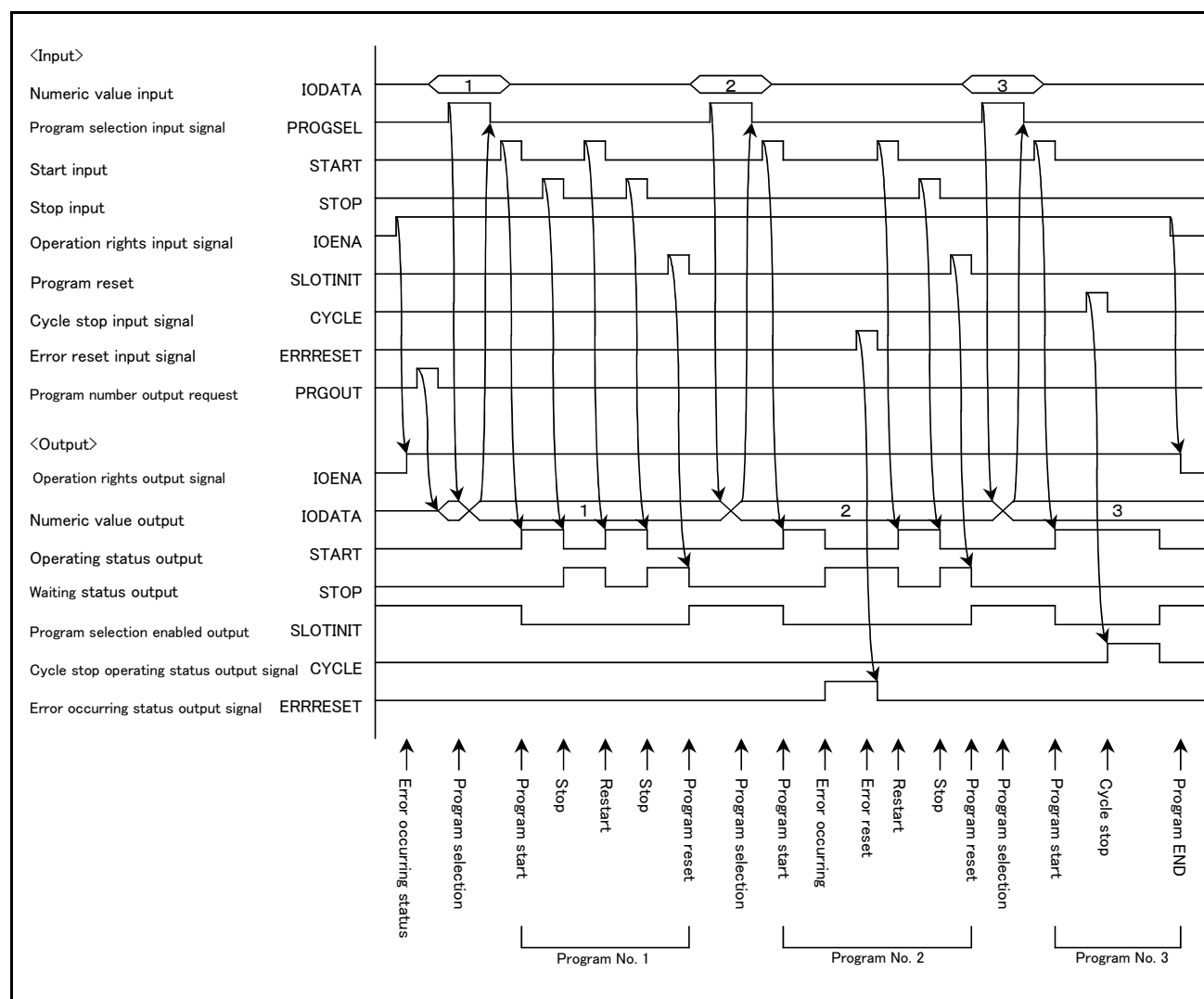


Fig.6-2 : Example of external operation timing chart (Part 1)

(2) External signal operation timing chart (Part 2)

An example of timing chart the servo ON/OFF, selecting the program, selecting the override, starting and outputting the line No., etc., with external signals is shown in [Fig. 6-3](#).

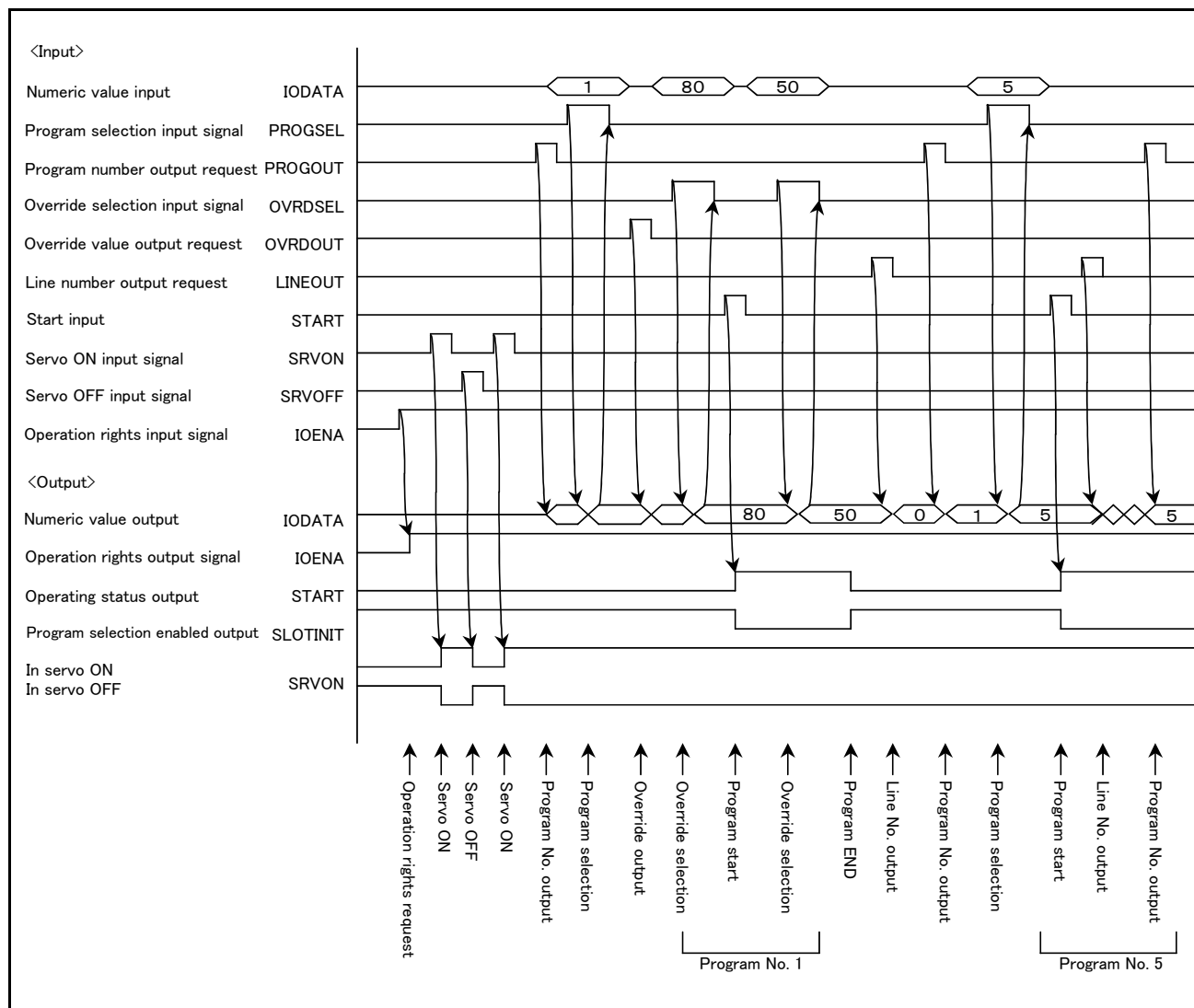


Fig.6-3 : Example of external operation timing chart (Part 2)

(3) Example of external operation timing chart (Part 3)

An example of the timing chart for error reset, general-purpose output reset and program reset, etc., with external signals is shown output in Fig. 6-4.

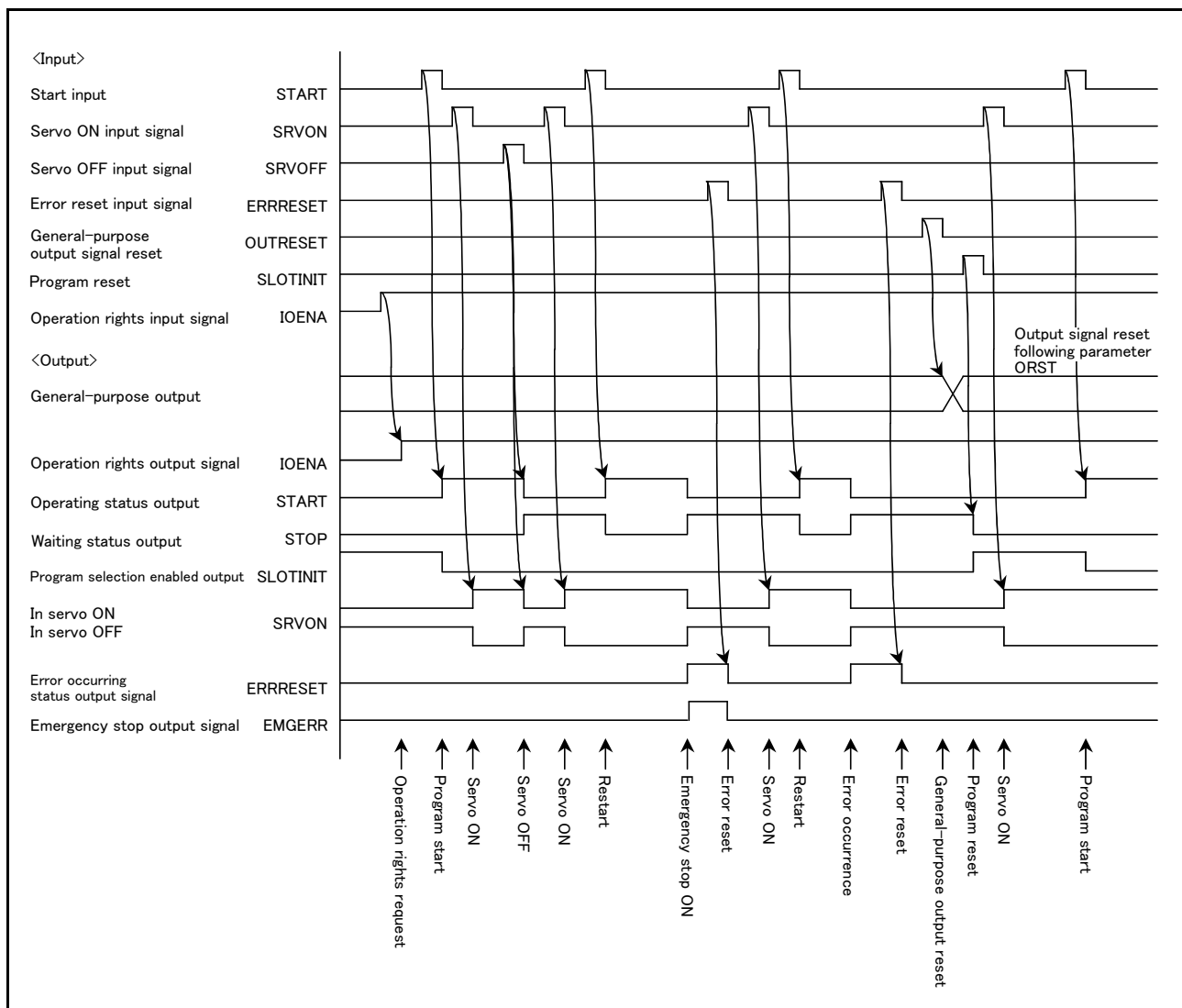


Fig.6-4 : Example of external operation timing chart (Part 3)

(4) Example of external operation timing chart (Part 4)

An example of the timing chart for jog operation, evasion point return and program reset, etc., with external signals is shown in Fig. 6-5.

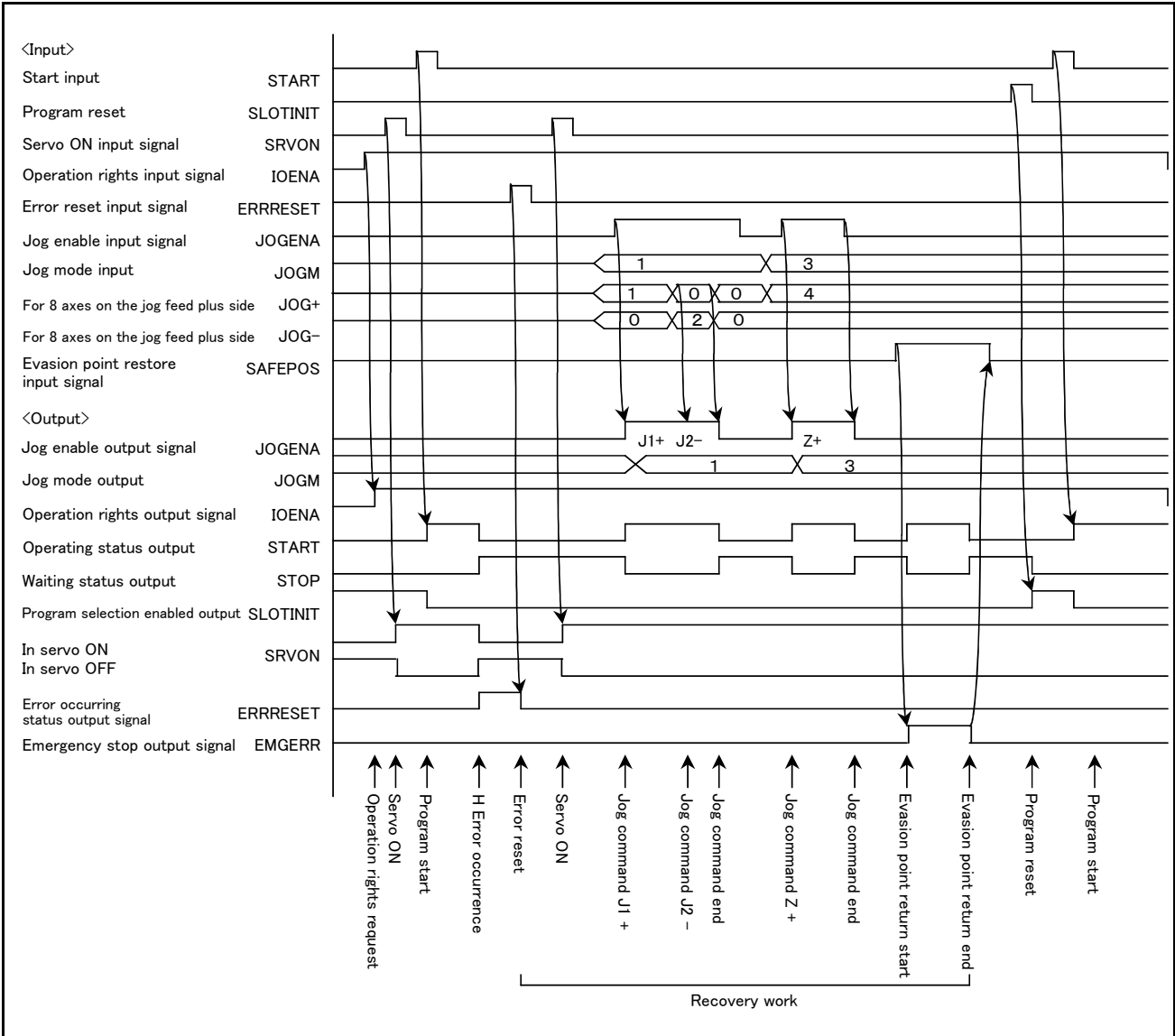


Fig.6-5 : Example of external operation timing chart (Part 4)

6.6 Emergency stop input

For wiring and other aspects of the emergency stop input, refer to the separate document entitled "Controller setup, basic operation, and maintenance."

6.6.1 Robot Behavior upon Emergency Stop Input

When an emergency stop signal is input while the robot is operating, the servo power supply is cut off by means of hardware control. The robot's tip path and stopping position after the input of an emergency stop signal cannot be specified. An overrun may occur depending on the robot speed or load condition of the tool.

7 Q & A

The following lists Q & A.

7.1 Movement

Q	A	Reference page
What are the features of this robot?	The features include the optimum acceleration/deceleration control, optimum speed control, compliance function, and multitask function.	Page 11, "2.3 Functions Related to Movement and Control"
I want to operate the robot tip in linear motion.	It is possible using the MVS instruction.	Page 60, "(2) Linear interpolation movement" Page 168, "MVS"
I want to operate the robot tip in circular motion.	It is possible using the MVR, MVR2, MVR3 and MVC instruction.	Page 165, "MVR" Page 166, "MVR2" Page 167, "MVR3" Page 164, "MVC"
I want to operate the robot tip in arch motion with ease.	It is possible using the DEF ARCH and MVA instruction.	Page 134, "DEF ARCH" Page 163, "MVA"
I want to operate the robot in accordance with the direction of the hand. I want to move the robot tip based on relative value.	It is possible by setting a proximity/departure distance using the MVS instruction. It can be done via multiplication or addition of position variables.	Page 168, "MVS" Page 81, "(2) Relative calculation of position data (multiplication)" Page 81, "(3) Relative calculation of position data (Addition)"
I want to skip the robot's singular points via linear interpolation.	It can be done via an 3-axes XYZ specification using the TYPE option of the MVS instruction. Although the 3-axes XYZ specification gives a linear path, the tip posture becomes unstable.	Page 168, "MVS"
I want to control the robot in a pliable manner.	It can be done using the CMP TOOL instruction, etc. Pliableness can be specified by the hand orientation, in applicable robot axis units, etc.	Page 124, "CMP TOOL" Page 120, "CMP JNT" Page 122, "CMP POS" Page 127, "CMPG" Page 126, "CMP OFF"
I want to turn the robot's tip axis multiple times.	It is possible using the JRC instruction. It is possible. However, the tip axis cannot be turned continuously. The axis stops after each turn.	Page 159, "JRC"
I want to set a control point at the hand tip.	It is possible to set using the TOOL instruction or the parameter "MEXTL." If only one hand is used, setting via the parameter is recommended. Perform the necessary setting before performing teaching.	Page 190, "TOOL" Page 275, "5.1 Standard Tool Coordinates"
I want to stop the robot for a specified period.	It is possible using the DLY instruction.	Page 144, "DLY"
I want to verify the positioning of the robot.	The stopping pulse range can be specified using the FINE instruction. Positioning can also be performed easily with the DLY instruction.	Page 147, "FINE" Page 144, "DLY"
I want to increase the robot's tact time.	The execution time can be improved by using the following methods. 1) Perform continuous path operation using the CNT instruction. 2) Perform optimum acceleration/deceleration control using the OADL instruction. 3) Perform optimum speed control using the SPD instruction.	Page 128, "CNT" Page 170, "OADL" Page 187, "SPD"
I want to operate the robot without stopping at each point.	It is possible using the CNT instruction.	Page 128, "CNT"
I want to change the acceleration time and deceleration time.	Using the ACCEL instruction is possible for set.	Page 112, "ACCEL"

Q	A	Reference page
I want to execute an automatic return to the evasion point.	A simple evasion-point return can be executed using the parameter "JSAFE" and the SAFEPS input signal. The robot may be moved along a specific evasion path while controlling the current position, using the user-defined area function and ZONE instruction.	Refer to "JSAFE" in Page 266, "5 Functions set with parameters" Page 279, "5.3 About user-defined area" Page 264, "ZONE" Page 265, "ZONE 2"
I want to improve the path accuracy.	It is possible using the PREC instruction.	Page 177, "PREC (Precision)"
I want to reduce the robot vibration.	Vibration can be suppressed by setting a smaller acceleration/deceleration rate with the ACCEL instruction. Vibration can also be reduced with the PREC instruction.	Page 112, "ACCEL" Page 177, "PREC (Precision)"
I want to limit the movement range of the robot.	It can be done using the joint movement range parameter "MEJAR," XYZ movement range parameter "MEPAR," free plane limit parameter, etc.	Refer to "MEJAR" and "MEPAR", etc. in Page 266, "5 Functions set with parameters"
How to open and close the hand?	The hand can be controlled using the HOPEN and HCLOSE instructions or M_IN (90n) and M_OUT (90n) signals. (n=0 to 7)	Page 154, "HOPEN / HCLOSE" Page 216, "M_IN/M_INB/M_INW" Page 221, "M_OUT/M_OUTB/M_OUTW"
I want to limit the movement range on an arbitrary plane.	It can be done using the free plane limit function.	Page 280, "5.4 Free plane limit"

7.2 Program

Q	A	Reference page
I want to execute two or more robot programs simultaneously.	It can be done using the multitask function.	Page 83, "4.2 Multitask function"
What are the precautions when using the multitask function?	See the reference pages.	Page 86, "4.2.4 Precautions for creating multitask program" Page 87, "4.2.5 Precautions for using a multitask program"
I want to call a program from within another program.	It is possible using the CALLP instruction.	Page 115, "CALLP"
I want to reference common positions and variables from separate programs.	External system variables can be used. Alternatively, a user base program can be created to use user-defined external variables.	Page 100, "4.3.23 Program external variables" Page 101, "4.3.24 User-defined external variables"
I want to calculate the palette position.	It is possible using the DEF PLT and PLT instruction.	Page 136, "DEF PLT" Page 176, "PLT"
I want to output and monitor signals, etc., while the robot is operating.	It is possible to use the DEF ACT or ACT instruction when multiple paths are monitored, or using the WTH or WTHIF instruction when a single path is monitored. The WTHIF allows for conditional judgment.	Page 132, "DEF ACT" Page 113, "ACT" Page 194, "WTH" Page 195, "WTHIF"
I want to monitor the robot to see if it is at a specified position and generate an error accordingly.	It can be done using the user-defined area function or ZONE instruction. A user error can be generated using the ERROR instruction.	Page 226, "M_UAR" Page 264, "ZONE" Page 265, "ZONE 2" Page 145, "ERROR"
I want to perform RS-232C communication with an external PC, etc.	It is possible.	Page 288, "5.10 About the communication setting" Page 174, "OPEN" Page 118, "CLOSE" Page 157, "INPUT" Page 178, "PRINT" Page 171, "ON COM GOSUB"
I want to acquire the current position of the robot.	The data can be referenced using the P_CURR or J_CURR variable.	Page 228, "P_CURR" Page 207, "J_CURR"
I want to measure the execution time.	It can be measured in milliseconds using the M_TIMER variable. 10 M_TIMER (1) = 0 20 MOV P1 30 MVS P2 40 M1 = M_TIMER (1) 50 HLT To check the time in milliseconds, monitor the M1 variable after the program is stopped.	Page 226, "M_TIMER"
I want the program to generate an error and stop.	A user error can be generated using the ERROR instruction.	Page 145, "ERROR"
I want to reset an error generated by another task in a multitask program.	It is possible to set the start condition for the task slot to ALWAYS and issue a RESET ERR instruction.	Page 84, "4.2.3 Operation state of each slot" Page 182, "RESET ERR"

7.3 Operation

Q	A	Reference page
I want to edit programs that are constantly executed.	To edit programs whose execution attribute is set to ALWAYS via the SLTn parameter, do so after canceling the ALWAYS attribute. ALWAYS programs cannot be edited since they are constantly executed. Change ALWAYS to START in the SLTn parameter, then turn off the controller's power and turn it back on, to stop the program from being constantly executed.	Page 21, "3.5.1 Creating a program" Refer to "SLTn" in Page 266, "5 Functions set with parameters"
I want to numerically correct the position learned via teaching.	It can be corrected on the T/B position screen.	Page 29, "(8) Correcting the MDI (Manual Data Input)"
I want to cancel the paused status of the program.	It can be done via a program-reset operation. This operation can be executed on the T/B operation panel or using an I/O signal.	Page 38, "(6) Resetting the program"
I want to check the program line by line.	It is possible to execute a step feed from the T/B.	Page 32, "(1) Step feed"
I want to start with automatic operation and switch to step feed in the middle to check the operation.	Use the HLT instruction to perform automatic operation, then switch to step feed via a T/B operation.	Page 35, "3.7 Automatic operation" Page 153, "HLT" Page 32, "(1) Step feed"
I want to move the robot to the position learned via teaching.	Position jump can be executed from the T/B.	Page 27, "(6) Confirming the position data (Position jump)"
I want to copy, delete or rename the program.	They can be done on the T/B management screen.	Page 44, "(3) Copying programs" Page 46, "(5) Deleting a program" Page 45, "(4) Changing the program name (Renaming)"
I want to write-protect the program.	They can be done on the T/B management screen. They can be done on the T/B management screen or Personal Computer Support Software.	Page 43, "(2) Program protection function"
I want to write-protect only the position data.	They can be done on the T/B management screen. They can be done on the T/B management screen or Personal Computer Support Software.	Page 43, "(2) Program protection function"
I want to check the available memory space.	It can be confirmed on the T/B directory screen.	Refer to T/B directory screen in Page 42, "3.11 Operating the program control screen"
I want to monitor the I/O statuses.	It can be done on the T/B monitor screen. They can be done on the T/B management screen or Personal Computer Support Software.	Page 47, "(1) Input signal monitor" Page 48, "(2) Output signal monitor"
I want to monitor the contents of variables.	It can be done on the T/B monitor screen. They can be done on the T/B management screen or Personal Computer Support Software.	Page 49, "(3) Variable monitor"
I want to view the error history.	It can be done on the T/B monitor screen. They can be done on the T/B management screen or Personal Computer Support Software.	Page 50, "(4) Error history"
I want to release the brake of the robot.	It can be done on the T/B brake screen.	Page 54, "(4) Releasing the brakes"
I want to view the current position of the robot.	It can be done on the T/B current position screen.	Page 26, "(5) Registering the current position data"
I want to change the amount of fixed-dimension feed in jog operation.	It can be set using the parameters "JOGJSP," "JOGPSP" and "JOGSPMX."	Refer to "JOGJSP", "JOGPSP" and "JOGSPMX" in Page 266, "5 Functions set with parameters"
I want the program to operate automatically upon turning on the power.	It can be done using the multitask function.	Page 283, "5.6 Automatic execution of program at power up"
I want to turn ON the servo forcibly when the robot is outside the movement range.	It can be done by canceling the LS.	Page 41, "3.9 Error reset operation"

7.4 External input/output signal

Q	A	Reference page
I want to check if the robot controller has started	It can be checked using the dedicated signal "RCREADY."	Refer to "RCREADY" in Page 308, "6.3 Dedicated input/output"
Is the operation right also needed to operate external signals?	The operation right is also needed to control the robot using external I/O signals. It is also necessary when operating the command signals, such as those used to "turn the servo ON" and "start." Safety-related signals, such as those used for stopping the operation and turning the servo OFF can be executed without operation right.	
I want to monitor the robot via external signals to see if it is at a specified position and generate an error accordingly.	It can be done using the user-defined area function.	Page 279, "5.3 About user-defined area" Refer to "USRAREA" in Page 308, "6.3 Dedicated input/output"
I want to select programs using external signals.	It can be done using the dedicated signals "PRGSEL" and "IODETA." Set the program number as a binary value in the IODETA signal (multiple bits), and read it via the PRGSEL signal.	Refer to "PRGSEL" and "IODETA" in Page 308, "6.3 Dedicated input/output"
I want to monitor a low battery.	It can be done using the dedicated signals "BATERR". Replace the battery without delay when this signal is turned ON.	Refer to "BATERR" in Page 308, "6.3 Dedicated input/output"
I want to execute a jog feed using external signals.	It can be done using the dedicated signals "JOGENA", "JOGM", "JOG+" and "JOG-".	Refer to "JOGENA", "JOGM", "JOG+" and "JOG-" in Page 308, "6.3 Dedicated input/output"
I want to check the operation of a program or signal without operating the robot.	It can be done using the dedicated signals "MLOCK". When the operation is started after this signal turned ON, the robot will not move physically. Only the program will run.	Refer to "MLOCK" in Page 308, "6.3 Dedicated input/output"
I want to cancel the paused status of the program.	It can be done using the dedicated signals "SLOTINIT".	Refer to "SLOTINIT" in Page 308, "6.3 Dedicated input/output"
I want to use robot programs to implement the PLC functions.	It can be done using the multitask function. Use one program to perform operation-related programs. Use another program to operate signal-processing programs. Use common system variables or user-defined external variables to exchange information between the programs.	Page 88, "4.2.6 Example of using multitask" Page 100, "4.3.23 Program external variables" Page 101, "4.3.24 User-defined external variables"
I want to reset the L7730 error temporarily. (Abnormalities in the cc-Link data link)	It is possible with the parameter E7730.	Refer to "E7730" in Page 266, "Table 5-1 : List parameter" .

7.5 Parameter

Q	A	Reference page
I want to operate the robot continuously from the power-OFF state.	It can be done using the Continuity function.	Refer to "CTN" in Page 266, "5 Functions set with parameters"
I want to set the hand mode that becomes effective when the power is turned ON.	It can be done using the parameters "HANDINIT" and "HANDTYPE."	Refer to "HANDINIT" and "HANDTYPE" in Page 266, "5 Functions set with parameters" Page 284, "5.7 About the hand type" Page 285, "5.8 About default hand status"
I want to share the RS-232C port located on the front side of the controller between an external device such as a PC or sensor on one hand and PC support software on the other.	Some protocol-related limitations apply. See the reference page.	Page 288, "5.10 About the communication setting"
I want to cancel the triggered buzzer.	It can be done using the parameters "BZR".	Refer to "BZR" in Page 266, "5 Functions set with parameters"

8 Appendix

8.1 Reference Material

8.1.1 About sink/source type of the standard external input and output

There are two types of external input/output circuit specifications: sink type and source type. The circuit specification type specified by the customers at ordering is built into the product (sink type for general Japanese/international products, and source type for CE mark products for Europe). The external input/output circuit built into the controller as standard is explained below, together with the two types of circuit specifications mentioned above.

(1) Electrical specifications of input/output circuit

The external input circuit specification is shown in Table 8-1. The external output circuit specification is shown in Table 8-2.

Table 8-1 : Electrical specifications of input circuit

Item		Specifications	Internal circuit
Type		DC input	<Sink type> <Source type>
No. of input points		32 or 16 ^{Note1)}	
Insulation method		Photo-coupler insulation	
Rated input voltage		DC12V/DC24V	
Rated input current		Approx. 3mA/approx. 7mA	
Working voltage range		1 0.2VDC to 26.4VDC(ripple rate within 5%)	
ON voltage/ON current		8VDC or more/2mA or more	
OFF voltage/OFF current		4VDC or less/1 mA or less	
Input resistance		Approx. 3.3k Ω	
Response time	OFF-ON	1 0ms or less(DC24V)	
	ON-OFF	1 0ms or less(DC24V)	
Common method		8 points per common	
External wire connection method		Connector	

Note1) The number of input points differ with robot type. Refer to the separate manual "Standard specifications".

Table 8-2 : Electrical specifications of output circuit

Item		Specifications	Internal circuit
Type		Transistor output	<Sink type> <Source type>
No. of output points		32 or 16 ^{Note1)}	
Insulation method		Photo-coupler insulation	
Rated load voltage		DC12V/DC24V	
Rated load voltage range		DC 1 0.2 ~ 30V(peak voltage 30VDC)	
Max. load current		0.1 A/point (1 00 %)	
Leakage current at OFF		0.1 mA or less	
Max. voltage drop at ON		DC0.9V(TYP.)	
Response time	OFF-ON	2ms or less (hardware response time)	
	ON-OFF	2ms or less (Resistance load) (hardware response time)	
Fuse rating		Fuse 3.2A (one per common) Replacement not possible	
Common method		4 points per common (common terminal: 4 points)	
External wire connection method		Connector	
External power supply	Voltage	DC12/24V(DC10.2 ~ 30V)	
	Current	60mA (TYP. 24VDC per common) (base drive current)	

Note1) The number of output points differ with robot type. Refer to the separate manual "Standard specifications".

(2) Connection example

Shows the connection example with a Mitsubishi PLC. The Fig. 8-1 is the sink type example, and the Fig. 8-2 is the source type example.

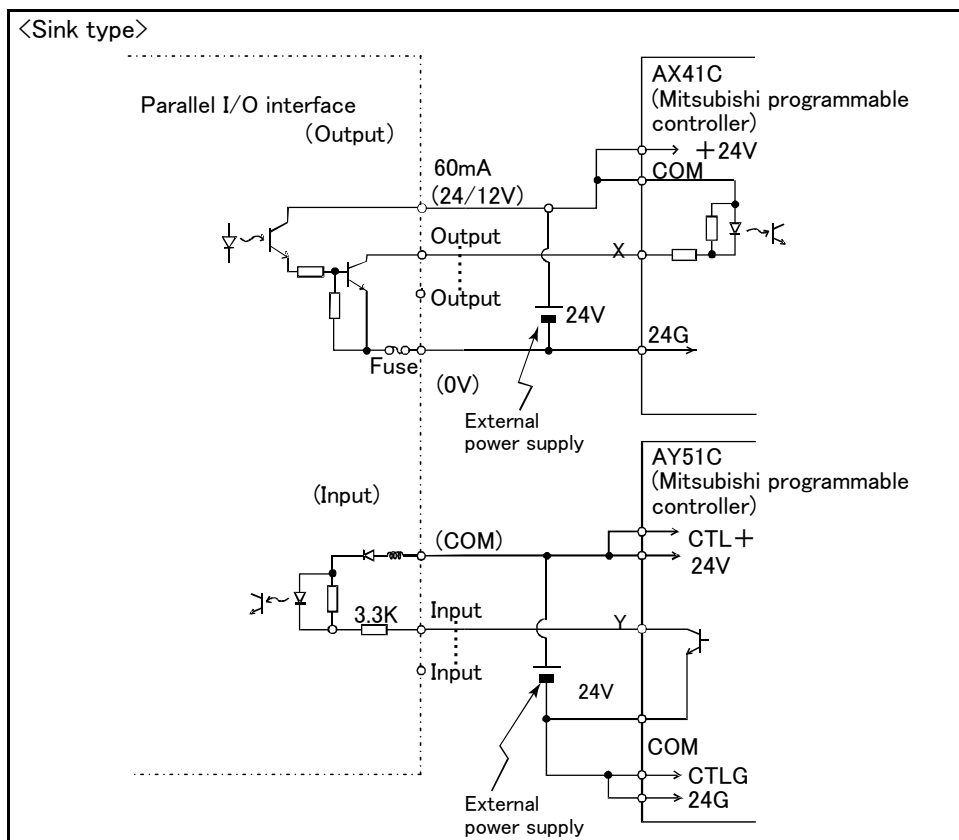


Fig.8-1 : Connection with a Mitsubishi PLC (Example of sink type)

*The input/output circuit external power supply (24 VDC) must be prepared by the customer.

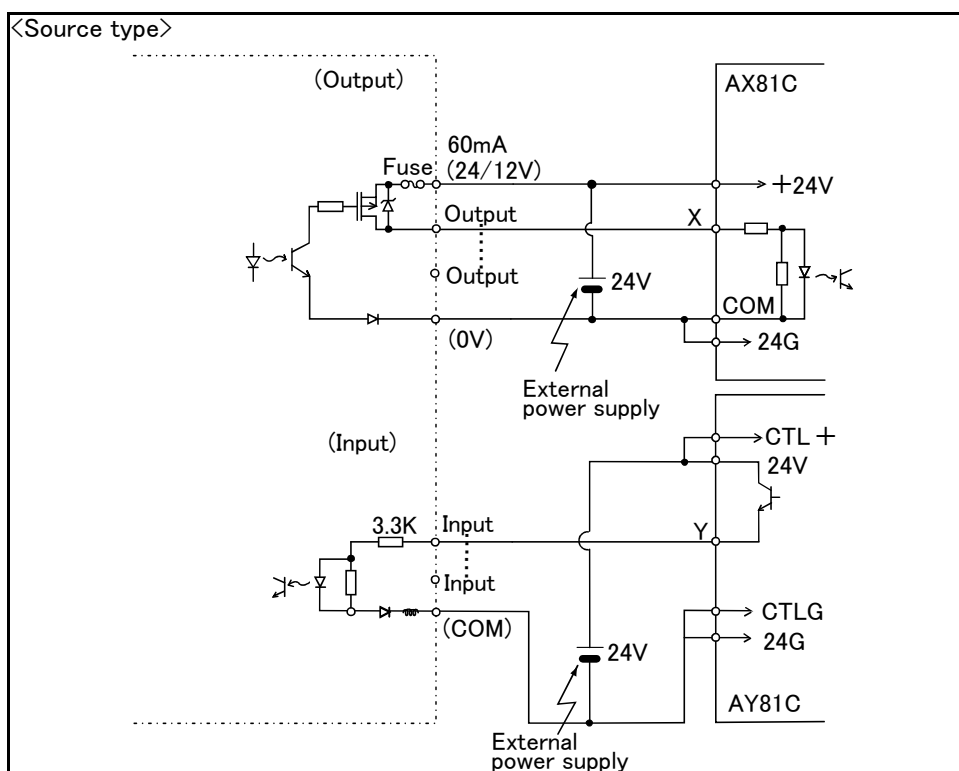


Fig.8-2 : Connection with a Mitsubishi PLC (Example of source type)

*The input/output circuit external power supply (24 VDC) must be prepared by the customer.

(3) Connector pin assignment

A list of connector pin numbers and signal assignments of the external input/output card is shown below.

In case of the CR1-571 controller, the assignments of pin numbers differ between the sink type and source type.

Table 8-3 and Table 8-4 show the connector pin assignments of the sink type and source type, respectively.

For other controllers, the pin assignments are the same for the sink type and source type. Table 8-5 and Table 8-6 show the connector pin assignments of CN100 and CN300, respectively.

■ For CR1 controller

Table 8-3 : Connector pin No. and signal assignment list (CR1 controller: sink type)

Pin No.	Line color Note1)	Function name		Pin No.	Line color Note1)	Function name	
		General-purpose	Dedicated/power supply, common			General-purpose	Dedicated/power supply, common
1	Orange/Red A		FG	26	Orange/Blue A		FG
2	Gray/Red A		0V:For pins 4-7	27	Gray/Blue A		0V:For pins 29-32
3	White/Red A		12V/24V:For pins 4-7	28	White/Blue A		12V/24V:For pins 29-32
4	Yellow/Red A	General-purpose output 0	Running	29	Yellow/Blue A	General-purpose output 4	
5	Pink/Red A	General-purpose output 1	Servo on	30	Pink/Blue A	General-purpose output 5	
6	Orange/Red B	General-purpose output 2	Error	31	Orange/Blue B	General-purpose output 6	
7	Gray/Red B	General-purpose output 3	Operation rights	32	Gray/Blue B	General-purpose output 7	
8	White/Red B		0V:For pins 10-13	33	White/Blue B		0V:For pins 35-38
9	Yellow/Red B		12V/24V:For pins 10-13	34	Yellow/Blue B		12V/24V:For pins 35-38
10	Pink/Red B	General-purpose output 8		35	Pink/Blue B	General-purpose output 12	
11	Orange/Red C	General-purpose output 9		36	Orange/Blue C	General-purpose output 13	
12	Gray/Red C	General-purpose output 10		37	Gray/Blue C	General-purpose output 14	
13	White/Red C	General-purpose output 11		38	White/Blue C	General-purpose output 15	
14	Yellow/Red C		COM0:For pins 15-22 Note2)	39	Yellow/Blue C		COM1:For pins 40-47 Note1)
15	Pink/Red C	General-purpose input 0	Stop(All slot) Note3)	40	Pink/Blue C	General-purpose input 8	
16	Orange/Red D	General-purpose input 1	Servo off	41	Orange/Blue D	General-purpose input 9	
17	Gray/Red D	General-purpose input 2	Error reset	42	Gray/Blue D	General-purpose input 10	
18	White/Red D	General-purpose input 3	Start	43	White/Blue D	General-purpose input 11	
19	Yellow/Red D	General-purpose input 4	Servo on	44	Yellow/Blue D	General-purpose input 12	
20	Pink/Red D	General-purpose input 5	Operation rights	45	Pink/Blue D	General-purpose input 13	
21	Orange/Red E	General-purpose input 6		46	Orange/Blue E	General-purpose input 14	
22	Gray/Red E	General-purpose input 7		47	Gray/Blue E	General-purpose input 15	
23	White/Red E		Reserved	48	White/Blue E		Reserved
24	Yellow/Red E		Reserved	49	Yellow/Blue E		Reserved
25	Pink/Red E		Reserved	50	Pink/Blue E		Reserved

Note1) "Line color" shows the line color of the external input/output cable 2A-CBL □□.

Note2) Sink type:24V/12V(COM), Source type:0V(COM)

Note3) The assignment of the dedicated input signal "STOP" is fixed.

Table 8-4 : Connector pin No. and signal assignment list (CR1 controller: source type type)

Pin No.	Line color ^{Note1)}	Function name		Pin No.	Line color ^{Note1)}	Function name	
		General-purpose	Dedicated/power supply, common			General-purpose	Dedicated/power supply, common
1	Orange/Red A		FG	26	Orange/Blue A		FG
2	Gray/Red A		0V:For pins 4-7, 10-13	27	Gray/Blue A		0V:For pins 29-32, 35-38
3	White/Red A		12V/24V:For pins 4-7, 10-13	28	White/Blue A		12V/24V:For pins 29-32, 35-38
4	Yellow/Red A	General-purpose output 0	Running	29	Yellow/Blue A	General-purpose output 4	
5	Pink/Red A	General-purpose output 1	Servo on	30	Pink/Blue A	General-purpose output 5	
6	Orange/Red B	General-purpose output 2	Error	31	Orange/Blue B	General-purpose output 6	
7	Gray/Red B	General-purpose output 3	Operation rights	32	Gray/Blue B	General-purpose output 7	
8	White/Red B		Reserved	33	White/Blue B		Reserved
9	Yellow/Red B		Reserved	34	Yellow/Blue B		Reserved
10	Pink/Red B	General-purpose output 8		35	Pink/Blue B	General-purpose output 12	
11	Orange/Red C	General-purpose output 9		36	Orange/Blue C	General-purpose output 13	
12	Gray/Red C	General-purpose output 10		37	Gray/Blue C	General-purpose output 14	
13	White/Red C	General-purpose output 11		38	White/Blue C	General-purpose output 15	
14	Yellow/Red C		COM0:For pins 15-22 ^{Note2)}	39	Yellow/Blue C		COM1:For pins 40-47 ^{Note1)}
15	Pink/Red C	General-purpose input 0	Stop(All slot) ^{Note3)}	40	Pink/Blue C	General-purpose input 8	
16	Orange/Red D	General-purpose input 1	Servo off	41	Orange/Blue D	General-purpose input 9	
17	Gray/Red D	General-purpose input 2	Error reset	42	Gray/Blue D	General-purpose input 10	
18	White/Red D	General-purpose input 3	Start	43	White/Blue D	General-purpose input 11	
19	Yellow/Red D	General-purpose input 4	Servo on	44	Yellow/Blue D	General-purpose input 12	
20	Pink/Red D	General-purpose input 5	Operation rights	45	Pink/Blue D	General-purpose input 13	
21	Orange/Red E	General-purpose input 6		46	Orange/Blue E	General-purpose input 14	
22	Gray/Red E	General-purpose input 7		47	Gray/Blue E	General-purpose input 15	
23	White/Red E		Reserved	48	White/Blue E		Reserved
24	Yellow/Red E		Reserved	49	Yellow/Blue E		Reserved
25	Pink/Red E		Reserved	50	Pink/Blue E		Reserved

Note1) "Line color" shows the line color of the external input/output cable 2A-CBL □□.

Note2) Sink type:24V/12V(COM), Source type:0V(COM)

Note3) The assignment of the dedicated input signal "STOP" is fixed.

■ For CR2/CR2A/CR4/CR7/CR8 controller

Table 8-5 : Connector CN100pin No. and signal assignment list (CR2/CR2A/CR4/CR7/CR8 controller: sink and source common)

Pin No.	Line color ^{Note1)}	Function name		Pin No.	Line color ^{Note1)}	Function name		
		General-purpose	Dedicated/power supply, common			General-purpose	Dedicated/power supply, common	
1	Orange/Red A	General-purpose output 0	FG	26	Orange/Blue A	General-purpose output 4	FG	
2	Gray/Red A		0V:For pins 4-7	27	Gray/Blue A		0V:For pins 29-32	
3	White/Red A		12V/24V:For pins 4-7	28	White/Blue A		12V/24V:For pins 29-32	
4	Yellow/Red A		Running	29	Yellow/Blue A			
5	Pink/Red A		Servo on	30	Pink/Blue A			
6	Orange/Red B	General-purpose output 2	Error	31	Orange/Blue B	General-purpose output 6	0V:For pins 35-38 12V/24V:For pins 35-38	
7	Gray/Red B	General-purpose output 3	Operation rights	32	Gray/Blue B	General-purpose output 7		
8	White/Red B	General-purpose output 8	0V:For pins 10-13	33	White/Blue B			
9	Yellow/Red B		12V/24V:For pins 10-13	34	Yellow/Blue B			
10	Pink/Red B			35	Pink/Blue B	General-purpose output 12		
11	Orange/Red C	General-purpose output 9	COM0:For pins 15-22 ^{Note2)} Stop(All slot) ^{Note3)}	36	Orange/Blue C	General-purpose output 13	COM1:For pins 40-47 ^{Note1)}	
12	Gray/Red C	General-purpose output 10		37	Gray/Blue C	General-purpose output 14		
13	White/Red C	General-purpose output 11		38	White/Blue C	General-purpose output 15		
14	Yellow/Red C	General-purpose input 0		39	Yellow/Blue C	General-purpose input 8		
15	Pink/Red C				40			Pink/Blue C
16	Orange/Red D	General-purpose input 1	Servo off	41	Orange/Blue D	General-purpose input 9		
17	Gray/Red D	General-purpose input 2	Error reset	42	Gray/Blue D	General-purpose input 10		
18	White/Red D	General-purpose input 3	Start	43	White/Blue D	General-purpose input 11		
19	Yellow/Red D	General-purpose input 4	Servo on	44	Yellow/Blue D	General-purpose input 12		
20	Pink/Red D	General-purpose input 5	Operation rights	45	Pink/Blue D	General-purpose input 13		
21	Orange/Red E	General-purpose input 6	Reserved Reserved Reserved	46	Orange/Blue E	General-purpose input 14	Reserved Reserved Reserved	
22	Gray/Red E	General-purpose input 7		47	Gray/Blue E	General-purpose input 15		
23	White/Red E			48	White/Blue E			
24	Yellow/Red E			49	Yellow/Blue E			
25	Pink/Red E			50	Pink/Blue E			

Note1) "Line color" shows the line color of the external input/output cable 2A-CBL □□.

Note2) Sink type:24V/12V(COM), Source type:0V(COM)

Note3) The assignment of the dedicated input signal "STOP" is fixed.

Table 8-6 : Connector CN300pin No. and signal assignment list (CR2/CR2A/CR4/CR7/CR8 controller: sink and source common)

Table 3-6-1: Connector CN50pin No. and signal assignment list (CN1~CN7: CN50 connector, sink and source common)

Pin No.	Line color Note1)	Function name		Pin No.	Line color Note1)	Function name	
		General-purpose	Dedicated/power supply, common			General-purpose	Dedicated/power supply, common
1	Orange/Red A	General-purpose output 16	FG	26	Orange/Blue A	General-purpose output 20	FG
2	Gray/Red A		0V:For pins 4-7	27	Gray/Blue A		0V:For pins 29-32
3	White/Red A		12V/24V:For pins 4-7	28	White/Blue A		12V/24V:For pins 29-32
4	Yellow/Red A			29	Yellow/Blue A		
5	Pink/Red A			30	Pink/Blue A		
6	Orange/Red B	General-purpose output 18	0V:For pins 10-13 12V/24V:For pins 10-13	31	Orange/Blue B	General-purpose output 22	0V:For pins 35-38 12V/24V:For pins 35-38
7	Gray/Red B	General-purpose output 19		32	Gray/Blue B	General-purpose output 23	
8	White/Red B			33	White/Blue B		
9	Yellow/Red B			34	Yellow/Blue B		
10	Pink/Red B	General-purpose output 24		35	Pink/Blue B	General-purpose output 28	
11	Orange/Red C	General-purpose output 25	COM0:For pins 15-22 Note2)	36	Orange/Blue C	General-purpose output 29	COM1:For pins 40-47 Note1)
12	Gray/Red C	General-purpose output 26		37	Gray/Blue C	General-purpose output 30	
13	White/Red C	General-purpose output 27		38	White/Blue C	General-purpose output 31	
14	Yellow/Red C			39	Yellow/Blue C		
15	Pink/Red C	General-purpose input 16		40	Pink/Blue C	General-purpose input 24	
16	Orange/Red D	General-purpose input 17		41	Orange/Blue D	General-purpose input 25	
17	Gray/Red D	General-purpose input 18		42	Gray/Blue D	General-purpose input 26	
18	White/Red D	General-purpose input 19		43	White/Blue D	General-purpose input 27	
19	Yellow/Red D	General-purpose input 20		44	Yellow/Blue D	General-purpose input 28	
20	Pink/Red D	General-purpose input 21		45	Pink/Blue D	General-purpose input 29	
21	Orange/Red E	General-purpose input 22	Reserved Reserved Reserved	46	Orange/Blue E	General-purpose input 30	Reserved Reserved Reserved
22	Gray/Red E	General-purpose input 23		47	Gray/Blue E	General-purpose input 31	
23	White/Red E			48	White/Blue E		
24	Yellow/Red E			49	Yellow/Blue E		
25	Pink/Red E			50	Pink/Blue E		

Note1) "Line color" shows the line color of the external input/output cable 2A-CBL □□.

Note2) Sink type:24V/12V(COM), Source type:0V(COM)



HEAD OFFICE : MITSUBISHI DENKI BLDG MARUNOUCHI TOKYO 100-8310 TELEX : J24532 CABLE MELCO TOKYO
NAGOYA WORKS : 1-14, YADA-MINAMI 5, HIGASHI-KU, NAGOYA, JAPAN